

IBM Information Management **software**



**IBM® InfoSphere™
DataStage
Fundamentals
Boot Camp
Lab Workbook**

Table of Contents

<u>Lab 01: Verify Information Server Services</u>	5
How to Bring Up Information Server Services (when InfoServer is down).....	7
<u>Lab 02: DataStage Administration</u>	8
Task: Open the Administration Console.....	8
Task: Specify property values in DataStage Administrator.....	11
<u>Lab 03: DataStage Designer</u>	16
Task: Log onto DataStage Designer	16
Task: Create a simple parallel job.....	17
Task: Compile, run, and monitor the job	26
Task: Create and use a job parameter	29
Task: Using the Parallel Debugger to test and debug your job.....	34
<u>Lab 04: Browsing the Repository</u>	39
Task: Execute a Quick Find	39
Task: Execute an Advanced Find	39
Task: Generate a report.....	40
Task: Perform an impact analysis	43
Task: Find the differences between two jobs.....	45
Task: Find the differences between two Table Definitions	48
<u>Lab 05: DataStage Parallel Architecture</u>	49
Task: Using a two-node configuration file	49
Task: Using data partitioning and collecting	50
Task: Experiment with different partitioning methods	53
<u>Lab 06: Sequential Data Access</u>	56
Task: Write data to and read data from a sequential file	56
Task: Reject link of a Sequential File stage.....	60
Task: Handling NULL values in a Sequential File stage.....	63
Task: Read data from multiple sequential files using File Pattern	69
Task: Read data with multiple readers.....	72
Task: Write data to a Data Set	73
<u>Lab 07: Relational (RDBMS) Data Access</u>	79
Task: Read data from a DB2 UDB table using a DB2 Connector stage.....	79
Task: Write data to a DB2 UDB Table using a DB2 Connector stage	81
Task: Import table definition of a relational table using Connector Import Wizard	85
Task: Read data from a DB2 UDB Table using an ODBC Connector stage.....	89
Task: Using ODBC Connector stage and the SQL Query Builder	91
<u>Lab 08: Combining Data</u>	97
Task: Lookup Stage with Equality Match	97
Task for extra credit: Examining the lookup failure in detail using the parallel debugger.....	103

Task: Handling lookup failure using lookup failure actions.....	106
Task: Range lookup on reference link	107
Task: Using the Join stage	111
Task: Using the Merge stage.....	115
Task: Using Funnel stage.....	118
<u>Lab 09: Sorting and Aggregating Data</u>	122
Task: Using Sort stage	122
Task: Using Remove Duplicates stage	123
Task: Using Aggregator stage.....	125
<u>Lab 10: Transforming Data</u>	128
Task: Create a parameter set	128
Task: Add a Transformer stage to a job and define a constraint	129
Task: Define an Otherwise link	133
Task: Define derivations	135
Task: Using a Transformer's loop function	140
<u>Lab 11: Job Control</u>	148
Task: Build a Job Sequence	148
Task: Add a user variable	155
Task: Add a Wait For File stage	157
Task: Add exception handling	158

LAB Notes:

1. List of userids and passwords used in the labs:

ENVIRONMENT	USER	PASSWORD
RHEL user	root	inf0sphere
IS admin ¹	isadmin	inf0server
DataStage user	dsuser	inf0server
WAS admin ²	wasdmin	inf0server
DB2 admin	db2admin	inf0server
DataStage admin	dsadm	inf0server

Note: the passwords contain a zero, not the letter o.

For DataStage Designer, please use user ID “dsuser”.

For DataStage Administrator, please use user ID “dsadm”.

2. In the labs, we will use the term “VM Machine” to refer to the VMWare environment that we use to run our IBM InfoSphere Information Server, and the term “Host Machine” to refer to the machine that we use VMWare Player or Workstation to load and host the VMWare image.
3. All the required data files are located at: /DS_Fundamentals/Labs. You will be using the DataStage project called “dstage1”.

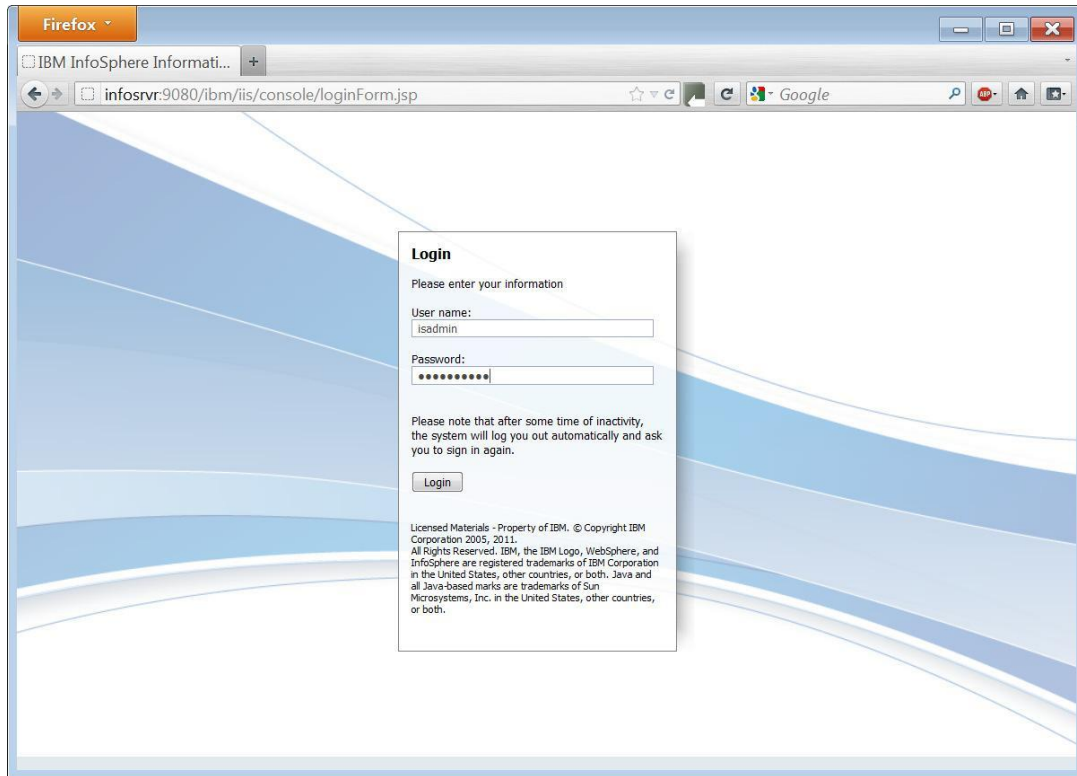
¹ IS admin: InfoSphere Information Server administrator

² WAS admin: WebSphere Application Server administrator

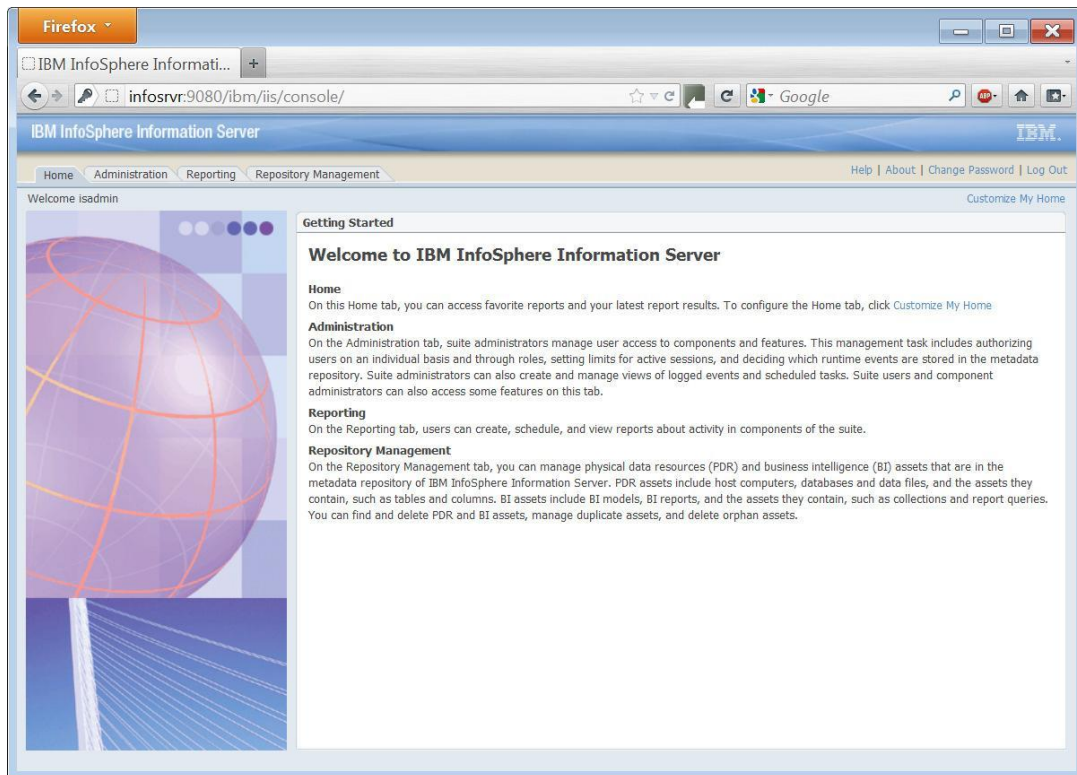
Lab 01: Verify Information Server Services

Task: Log onto the Information Server Web Console

1. On your Host Machine, open a new browser (IE or Firefox) and go to this URL: <https://is-server.ibm.com:9445/ibm/iis/console/> and the InfoSphere Information Server Web Console login page will be displayed. Enter the isadmin/inf0server user ID and password then click Login.



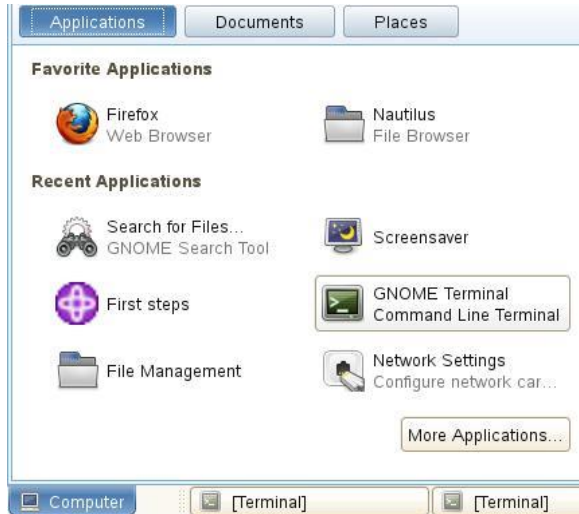
2. If you see the following window, Information Server is up and running.



3. If the browser does **not** display the web page, ask your instructor for help and follow these instructions together.

How to Bring Up Information Server Services (when InfoServer is down)

4. Ensure the VMware image is running.
5. Login as root with the password “inf0sphere”.
6. Click on Computer (bottom left corner), and open GNOME Terminal



7. To change the user to db2inst1, type the following command “su db2inst1”
8. To start the DB2 Database, type “db2start”.
9. Type ‘exit’ to go back to the root user.
10. In order to Start the WebSphere App server, execute this command (This is a time consuming process that takes more than a few minutes to complete):

```
/opt/IBM/WebSphere/AppServer/bin/startServer.sh server1
```

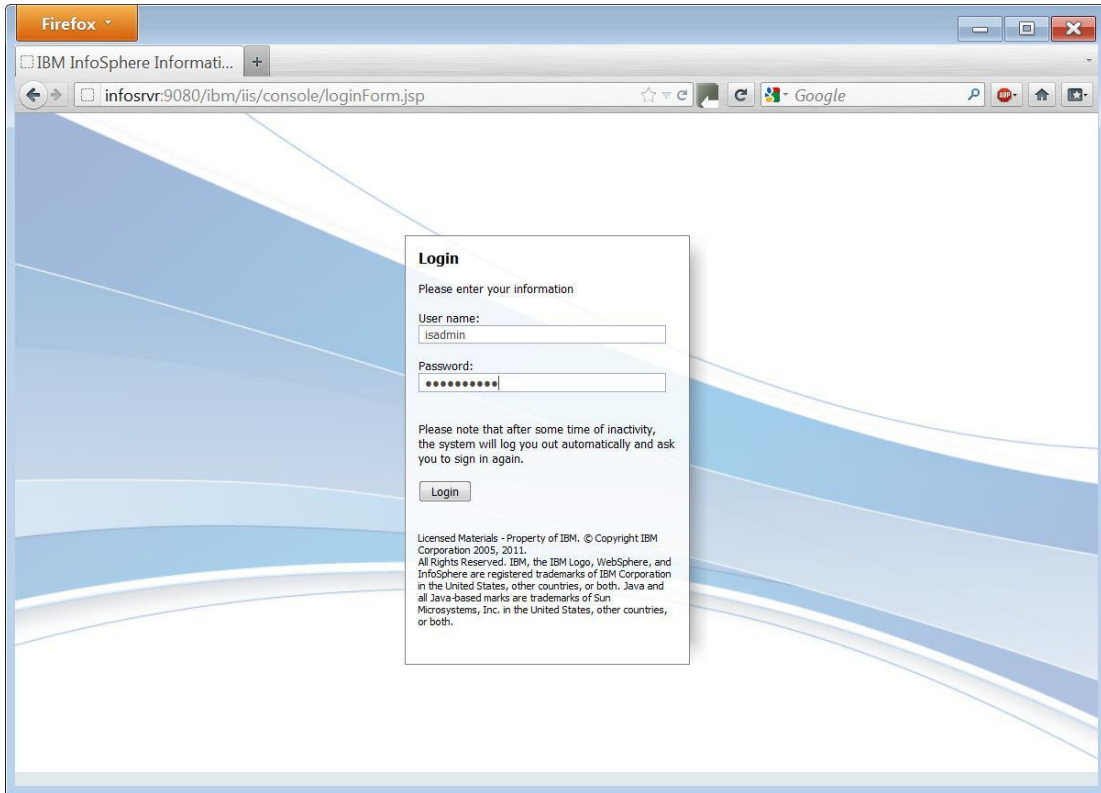
11. Finally, execute the following command to start the ASBNode:

```
/opt/IBM/InformationServer/ASBNode/bin/NodeAgents.sh Start
```

Lab 02: DataStage Administration

Task: Open the Administration Console

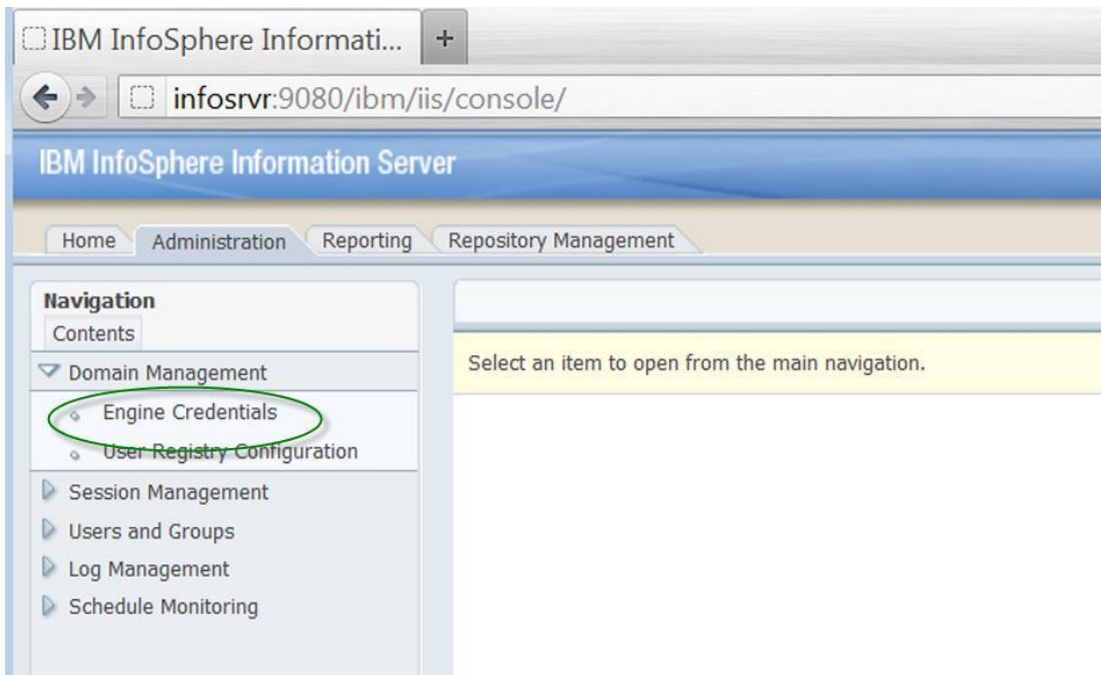
1. If you logoff from the last lab, then log onto the IBM Information Server Web Console.



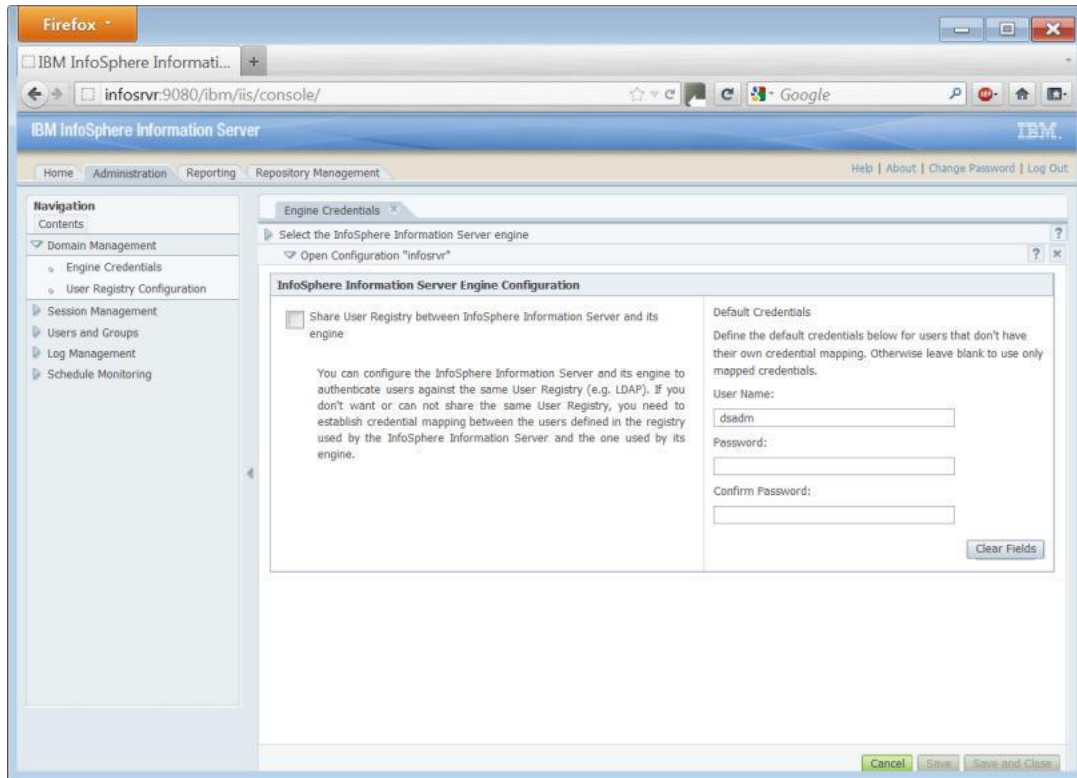
2. Click the Administration tab.



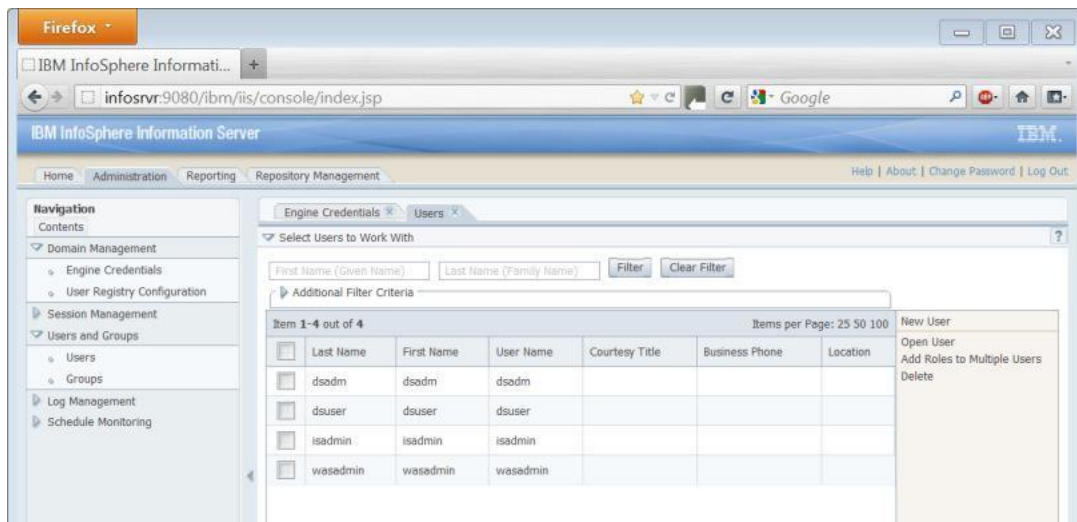
3. Expand Domain Management. Click Engine Credentials.



4. Select infosrvr and then click Open Configuration. There should be a user ID in the Default Credentials area and the password is not shown. Do not change anything here. Otherwise, you will not be able to login to any client. Click Cancel to exit (you may have to scroll down in order to see the buttons).

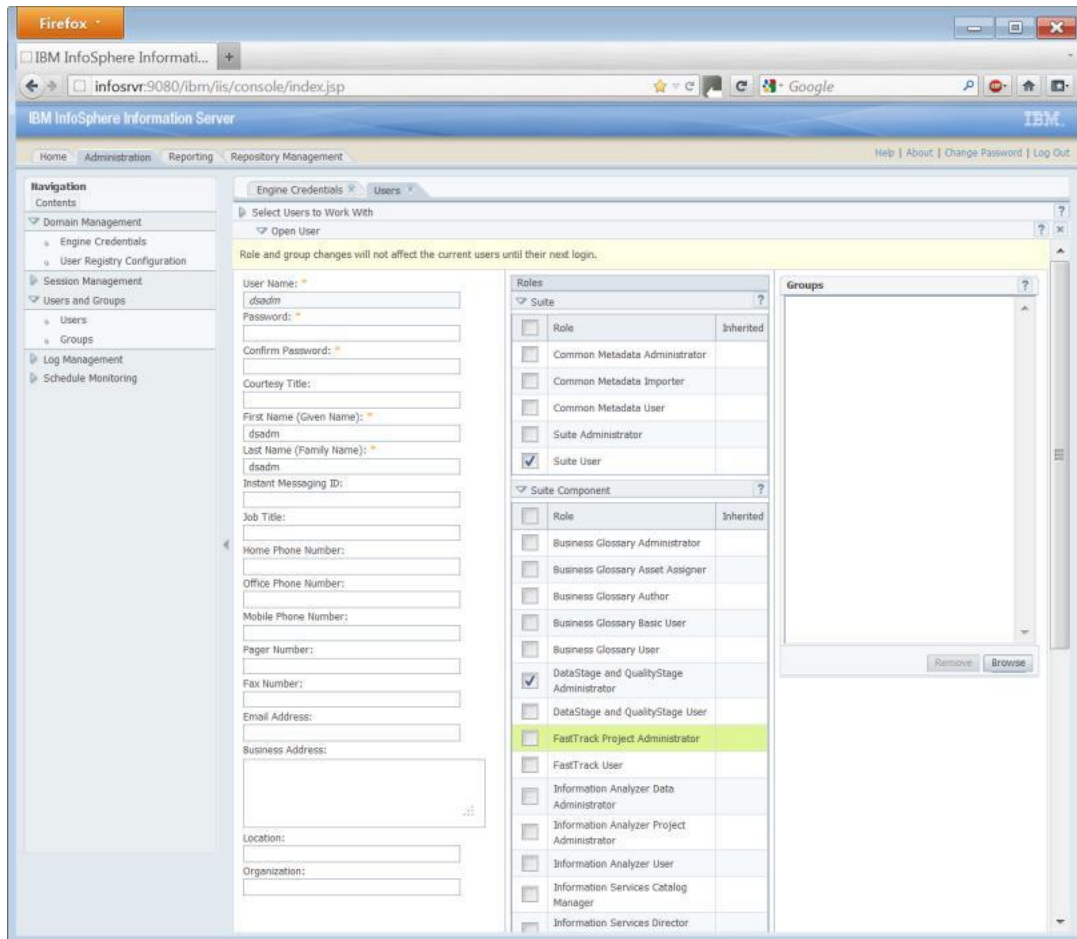


5. Now expand Users and Groups and then click Users. Here, the Information Server Suite Administrator user ID, isadmin, is displayed. Also the WebSphere Application Server administrator user ID, wasadmin, the DataStage Administrator dsadm and the DataStage user dsuser is displayed. There might be other users as well.



6. Select any user and then click Open User.

- Note the properties of this user. Expand the Suite Component. Note the Suite Roles and the Component Roles that have been assigned to this user.



- Return to the Users main window by clicking on the Cancel button (you might have to scroll down in order to see it).
- Click Log Out on the upper right corner of the screen and then close the browser.

Task: Specify property values in DataStage Administrator

- On your host system, open the DataStage Administrator from the desktop icon or Start->Programs-> IBM InfoSphere Information Server -> IBM InfoSphere DataStage and QualityStage Administrator.

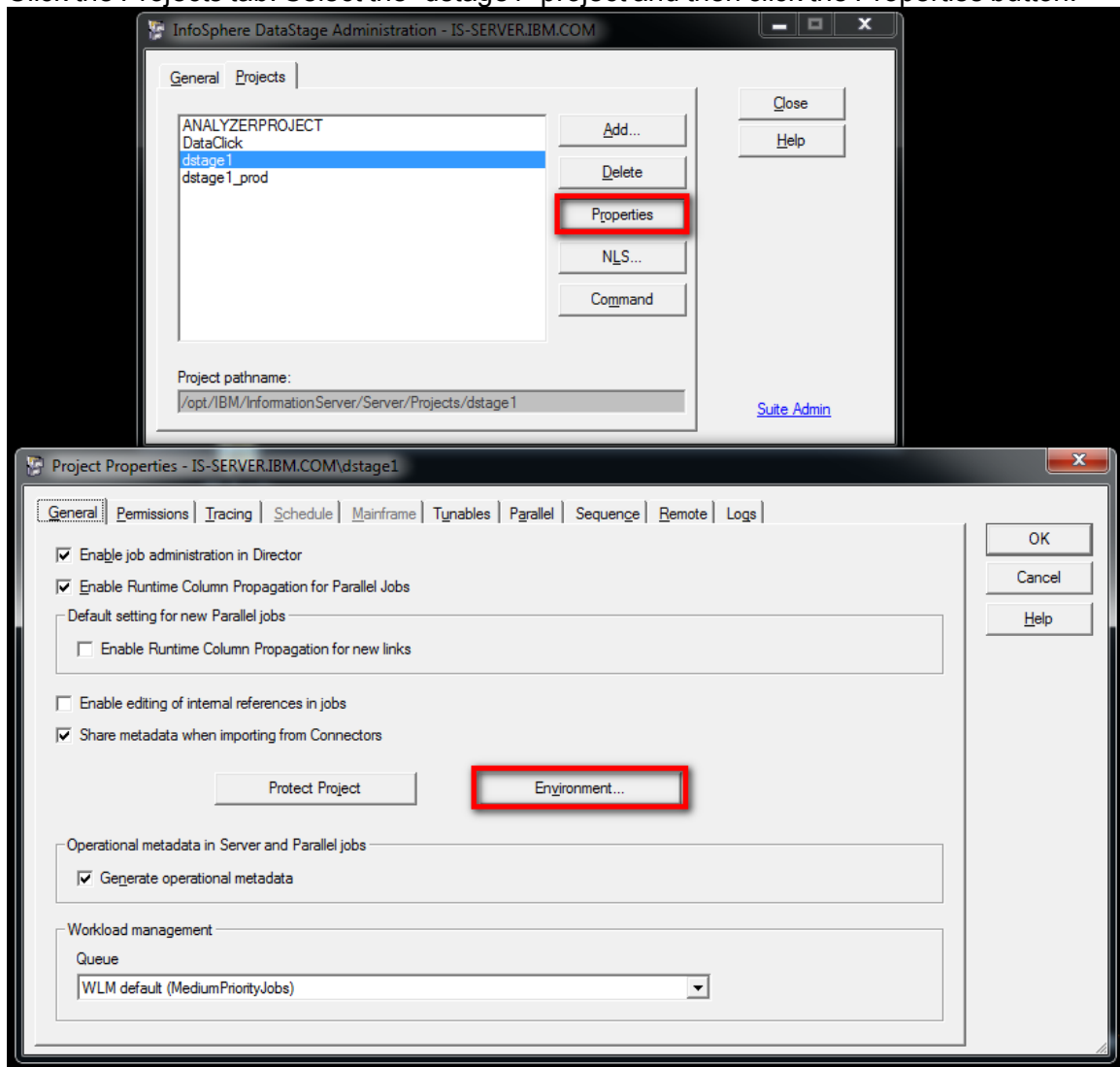
2. Specify the host name of the Information Server services tier computer (infosrvr), followed by a colon, followed by the port number (9445) to connect. Use dsadm as the User name and inf0server as the password to attach to the DataStage server. In our case the host name of the Information Server engine is the same as the one for the services tier. Click Login.

The screenshot shows a Windows-style dialog box titled "Attach to DataStage". The background features the IBM logo and a blue wavy graphic. The text "IBM® InfoSphere™ DataStage® and QualityStage" and "Version 11.3" is displayed. Below this, there are four input fields:

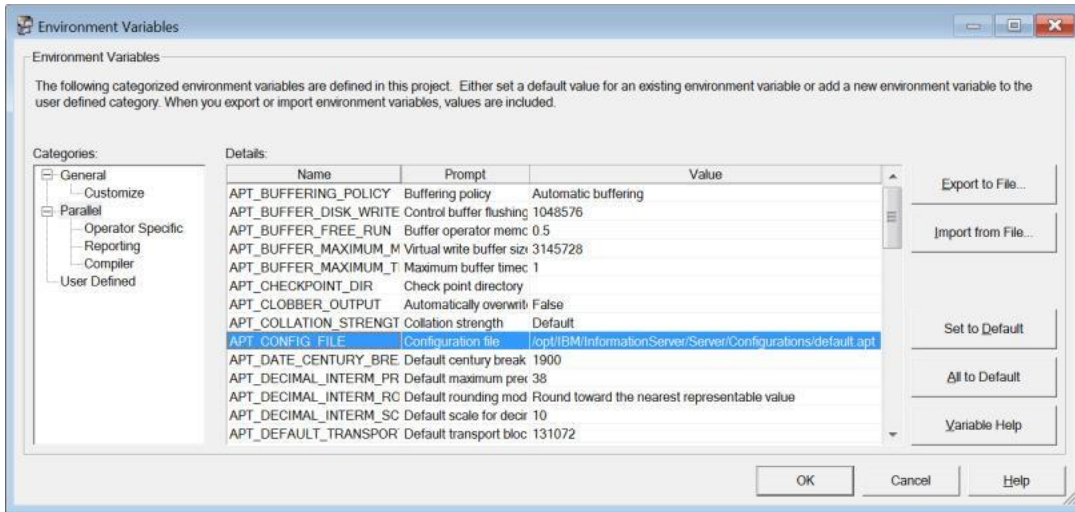
- Host name of the services tier:** A dropdown menu showing "is-server.ibm.com:9445".
- User name:** A text box containing "isadmin".
- Password:** A text box with masked characters (dots).
- Host name of the Information Server engine:** A dropdown menu showing "IS-SERVER.IBM.COM".

At the bottom of the dialog box are two buttons: "Login" and "Cancel".

3. Click the Projects tab. Select the “dstage1” project and then click the Properties button.

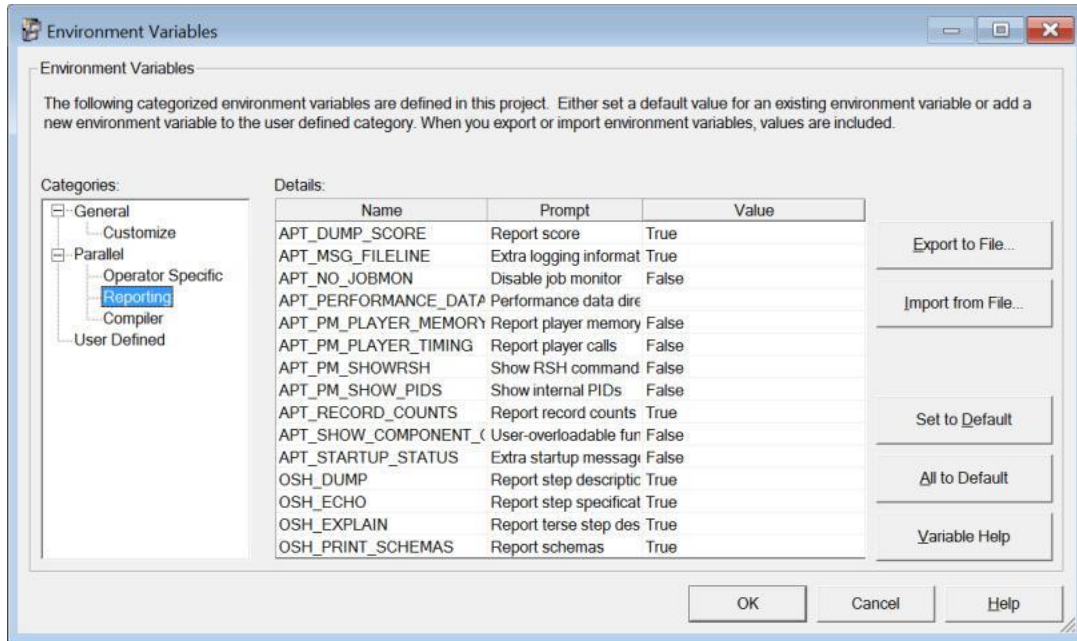


- Click the Environment button to open up the Environment variables window. In the Parallel folder, examine the APT_CONFIG_FILE parameter and its default (The configuration file is discussed in a later module).

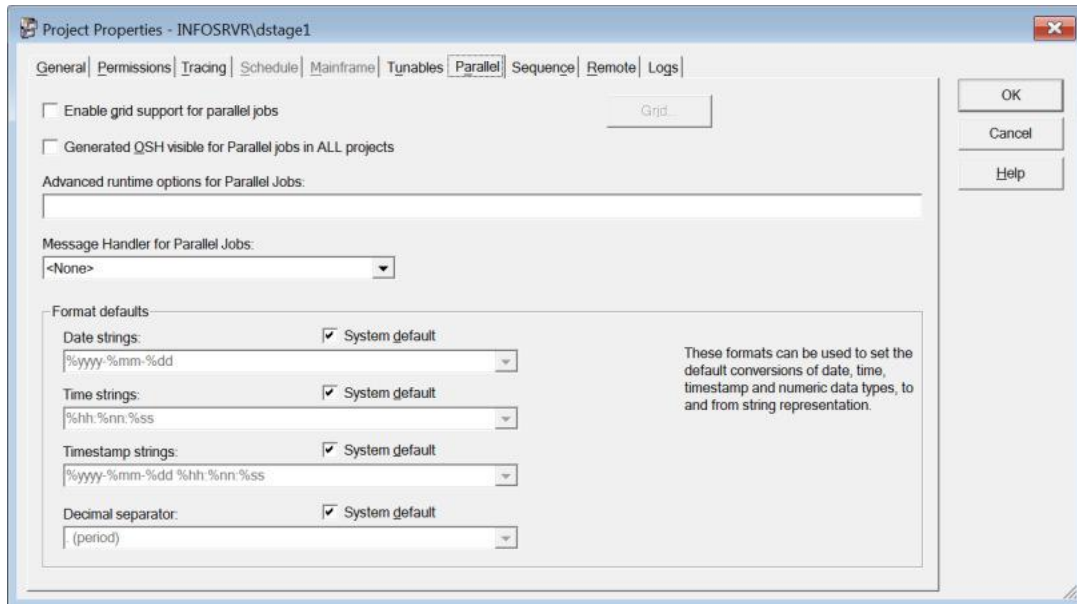


- In the Reporting folder, set the variables shown below to true as the screen snapshot:

APT_DUMP_SCORE	True
APT_MSG_FILELINE	True
APT_RECORD_COUNTS	True
OSH_DUMP	True
OSH_ECHO	True
OSH_EXPLAIN	True
OSH_PRINT_SCHEMAS	True



- Click OK.
- Go to the tab Parallel and browse the parameters and available settings. Do the same for each of other tabs. Click OK when done.

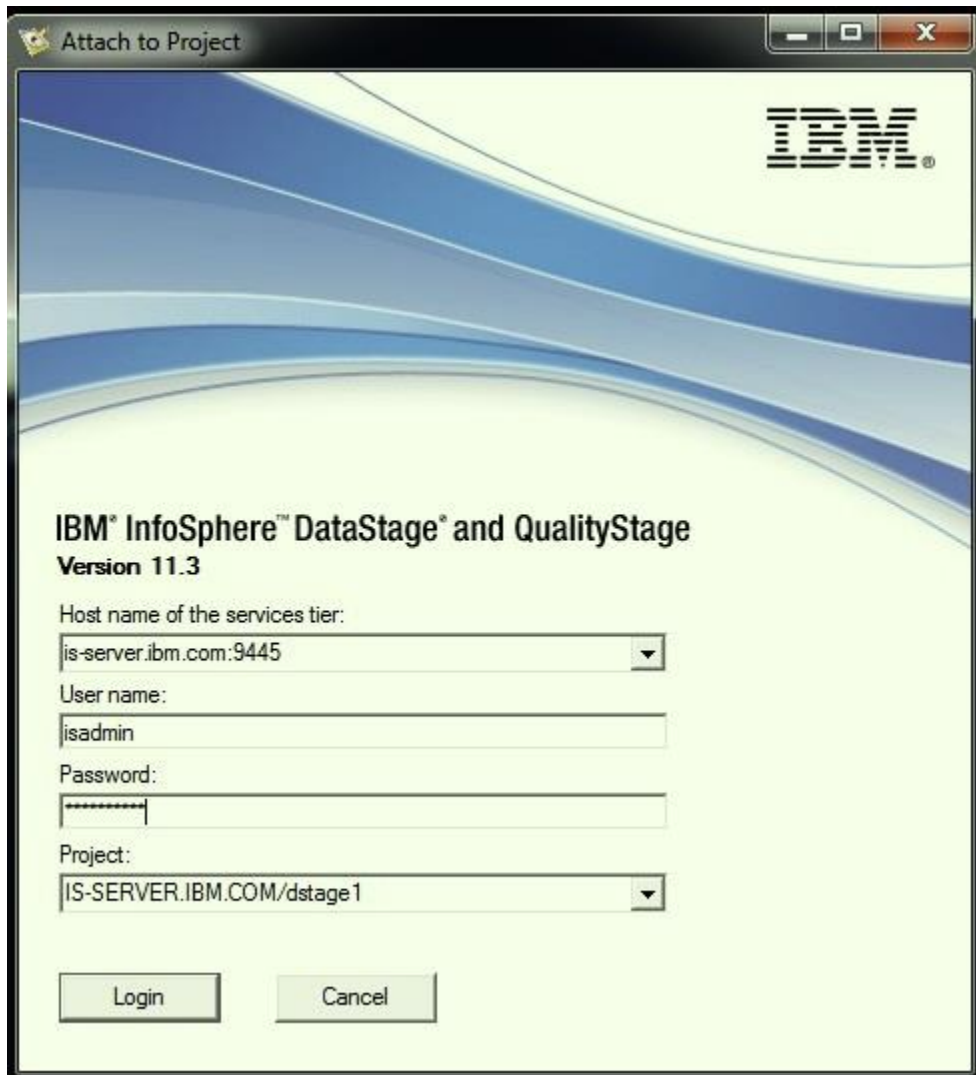


- Close DataStage Administrator by clicking Close.

Lab 03: DataStage Designer

Task: Log onto DataStage Designer

1. Open the DataStage Designer client program from the host system and type the following information to log into your DataStage project using the isadmin ID.



Attach to Project

IBM®

IBM® InfoSphere™ DataStage® and QualityStage
Version 11.3

Host name of the services tier:
is-server.ibm.com:9445

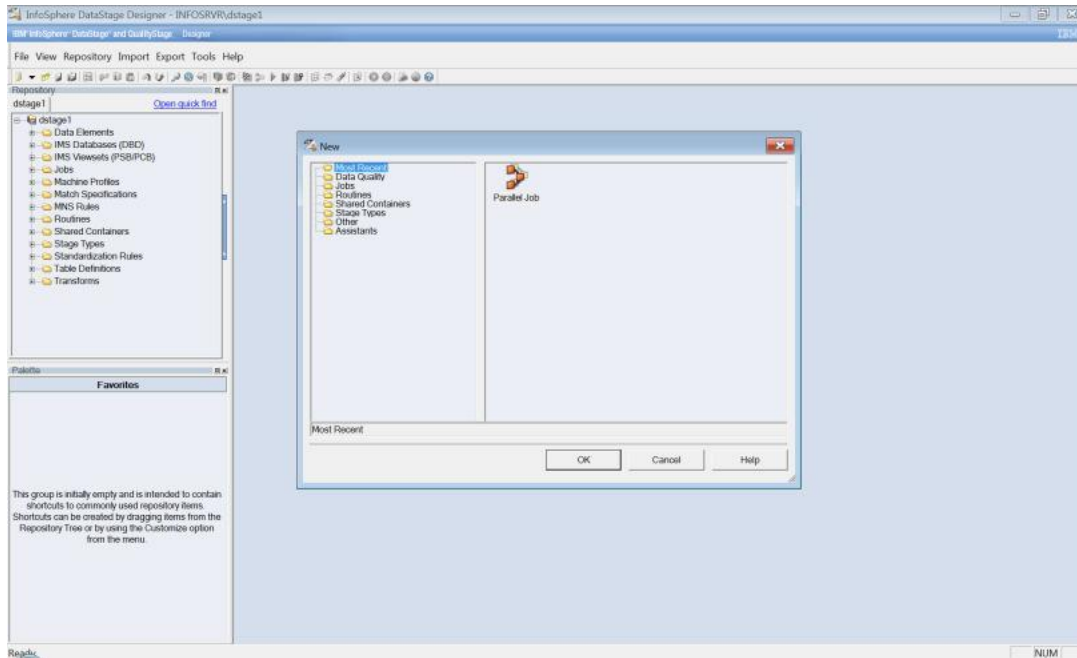
User name:
isadmin

Password:

Project:
IS-SERVER.IBM.COM/dstage1

Login Cancel

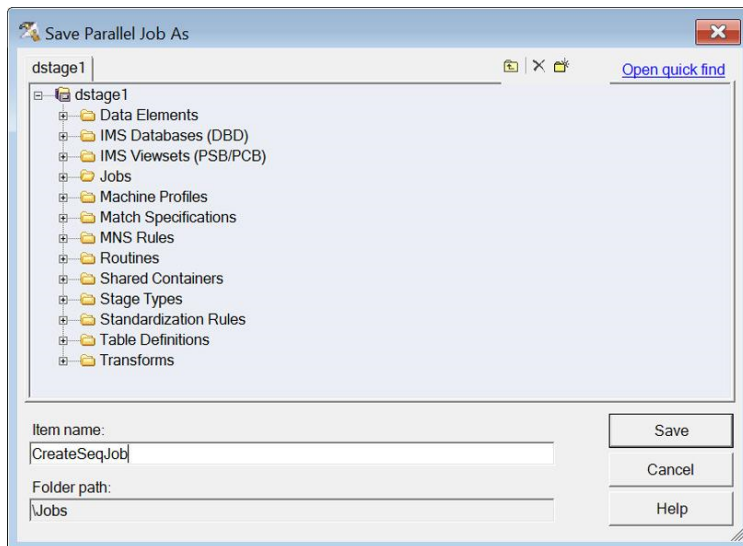
- Once you log on to the Designer client, you will see a screen as below:



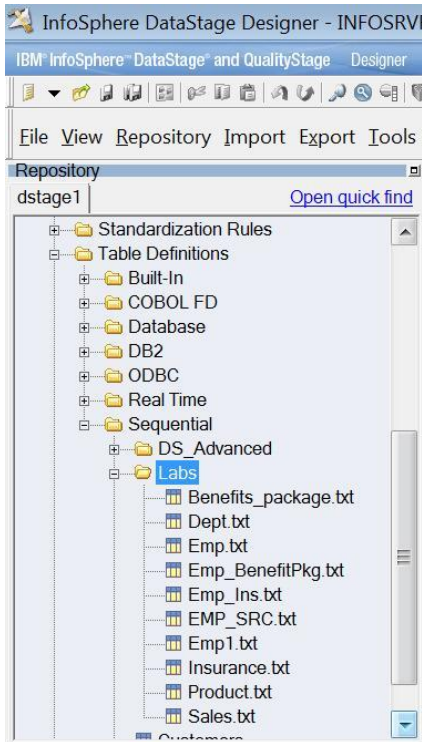
Task: Create a simple parallel job

In this task, you will design a job that reads data from the Selling_Group_Mapping.txt file, copies it through a Copy stage, and then writes the data to a new file named Selling_Group_Mapping_Copy.txt.

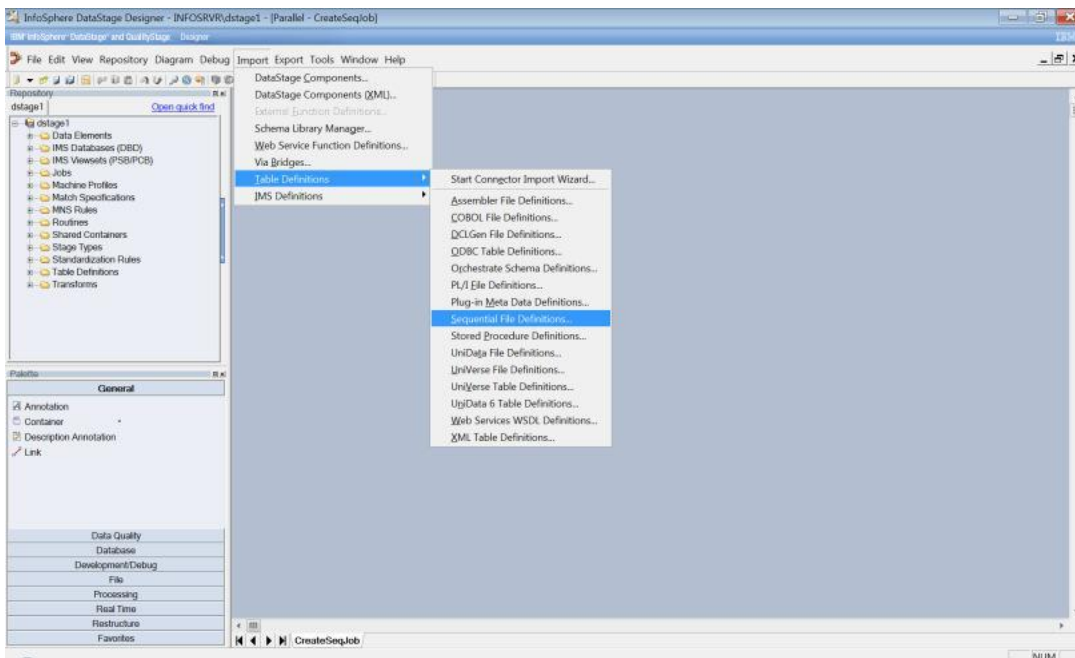
- If you still have the first window open as displayed in the screenshot, select Parallel Job and click OK. Otherwise, open a new Parallel job by either clicking on the “New” icon (first one from left) or from the menu File New.
- Save the job now with the name CreateSeqJob into the Jobs folder in the repository by doing File Save As...



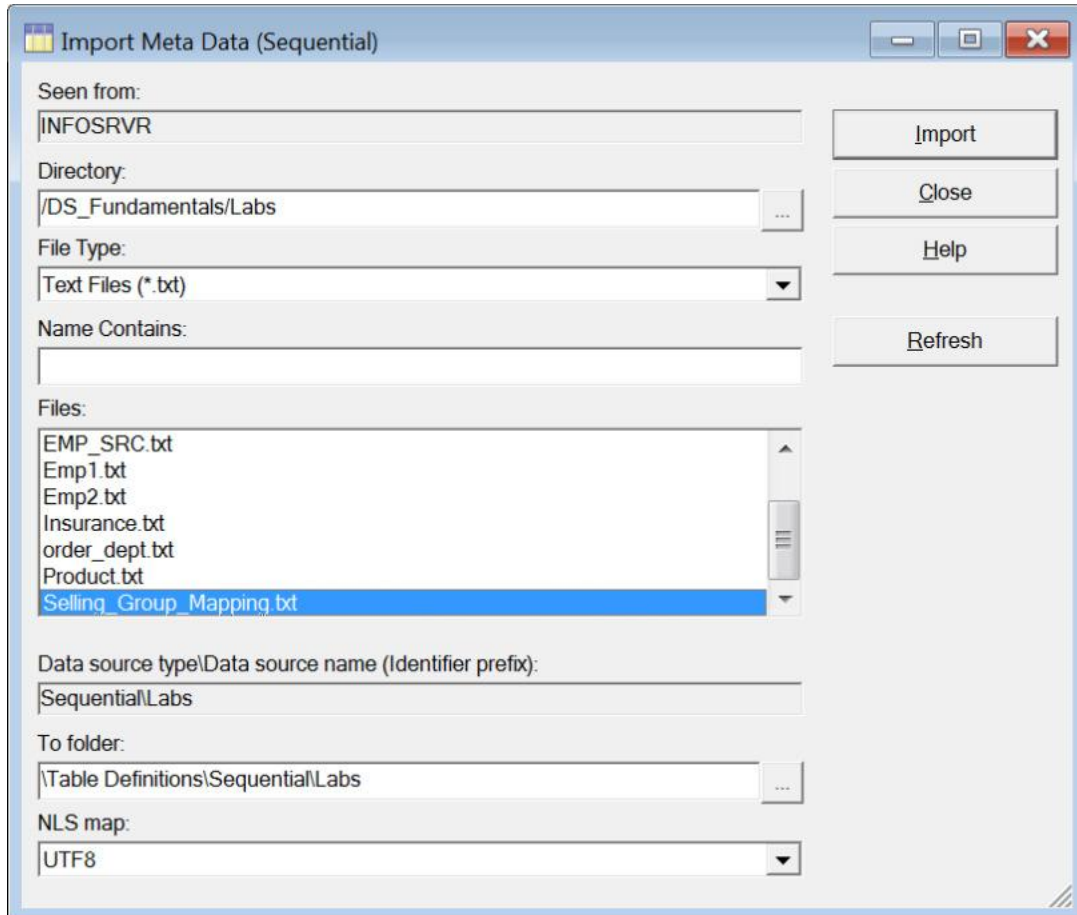
3. Browse Repository window for a Table Definition called Selling_Group_Mapping.txt. The Table Definitions are located in the “Table Definitions” Folder Sequential in the Labs.



4. It appears that it's not yet there. We must therefore import the table definition of the sequential file Selling_Group_Mapping.txt. Click on Import Table Definitions Sequential File Definition.



5. Choose /DS_Fundamentals/Labs Directory by clicking the button to the right of the Directory field. Note that the files will not be displayed because you are just selecting the directory. After you click OK to the directory browser, the files will be displayed in the Files area. Select the file Selling_Group_Mapping.txt and click Import.



6. Check the box First line is column names and then go to the Define tab.

Define Sequential Meta Data

Filename: /DS_Fundamentals/Labs/Selling_Group_Mapping.txt (Map=UTF8)

Format Define

☐ Fixed-width columns ☒ First line is column names

Fixed-width

Column widths:

Spaces between columns:

Delimited

☐ Tab ☐ Space ☒ Comma

Other Delimiter: ,

Quote Character: "

Data Preview

Selling_Gr...	Selling_Gr...	Special_Ha...	Distr_Chan...
150000	SG015 FREE...	6	Other
530000	SG053 TRUC...	6	Other
550000	SG055 LIVE...	6	Other
860000	SG086 TRUC...	6	Other
870000	SG087 FREE...	6	Other
1120000	SG112 N.W....	6	Other

Preview OK Cancel Help

7. Verify you have the four fields as shown on the next image, and click OK.

Define Sequential Meta Data

Filename: /DS_Fundamentals/Labs/Selling_Group_Mapping.txt (Map=UTF8)

Format Define

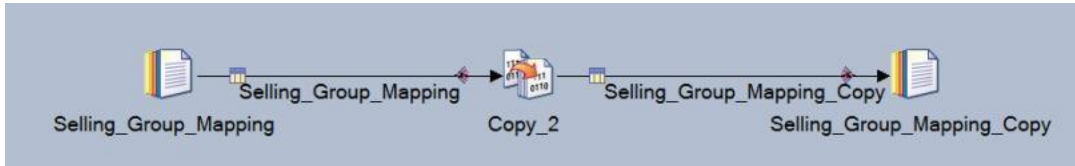
	Column name	Key	SQL type	Extended	Length	Scale	Nullable	Display	Data
1	Selling_Group_Code	☒	Integer		10		No	7	
2	Selling_Group_Desc	☐	VarChar		255		No	25	
3	Special_Handling_Code	☐	Integer		10		No	1	
4	Distr_Chann_Desc	☐	VarChar		255		No	12	

Data Preview

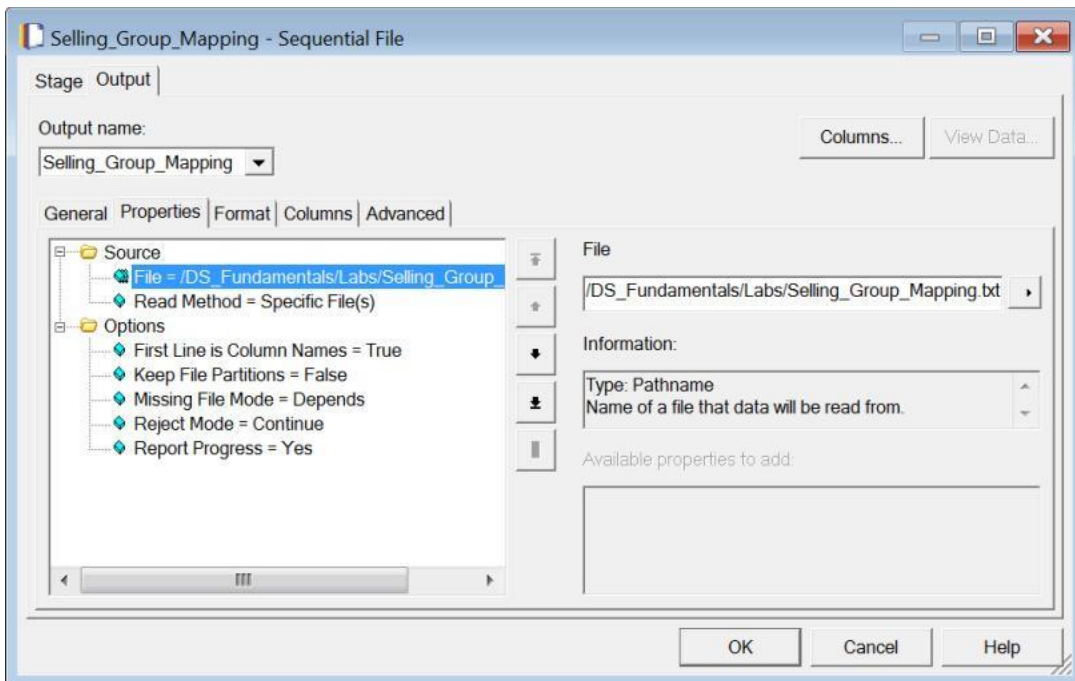
Selling_Gr...	Selling_Gr...	Special_Ha...	Distr_Chan...
150000	SG015 FREE...	6	Other
530000	SG053 TRUC...	6	Other
550000	SG055 LIVE...	6	Other
860000	SG086 TRUC...	6	Other
870000	SG087 FREE...	6	Other
1120000	SG112 N.W....	6	Other

Preview OK Cancel Help

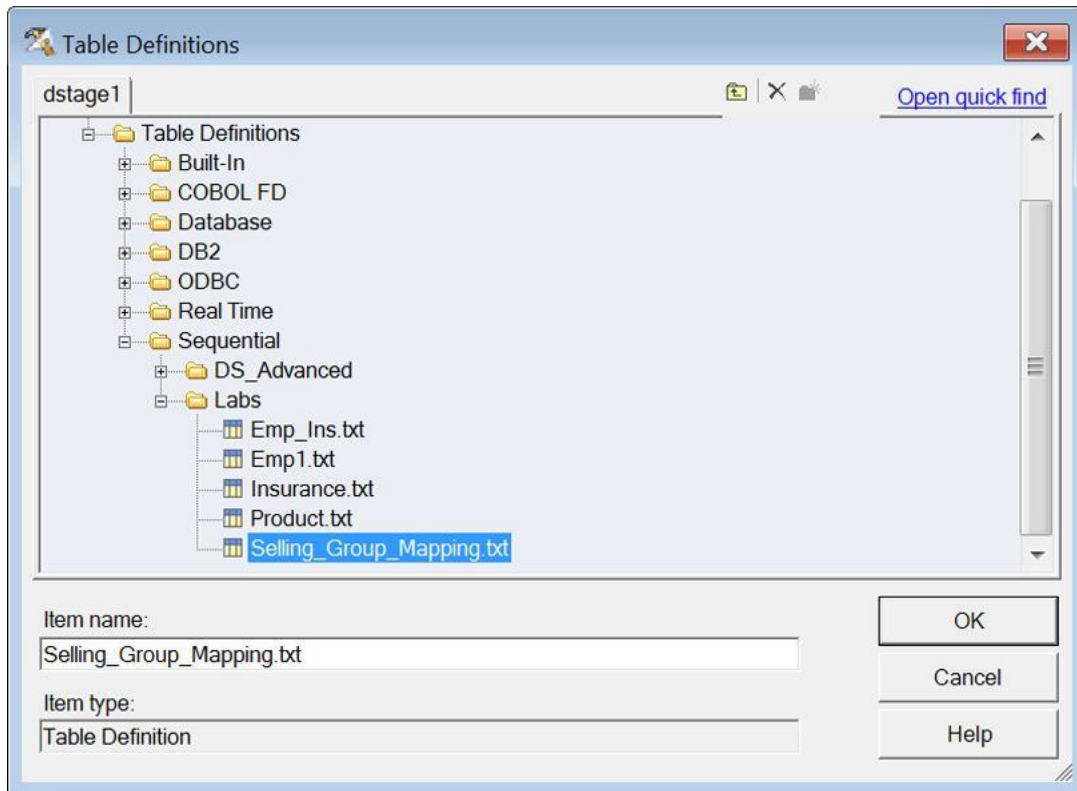
8. Close the import window.
9. Add a Sequential File stage ('File' category in the Palette), a Copy stage ('Processing' category in the Palette), and a second Sequential File stage. Draw links between them and name the stages and links as shown. You can select the name and type over it or select the object and right click to rename.



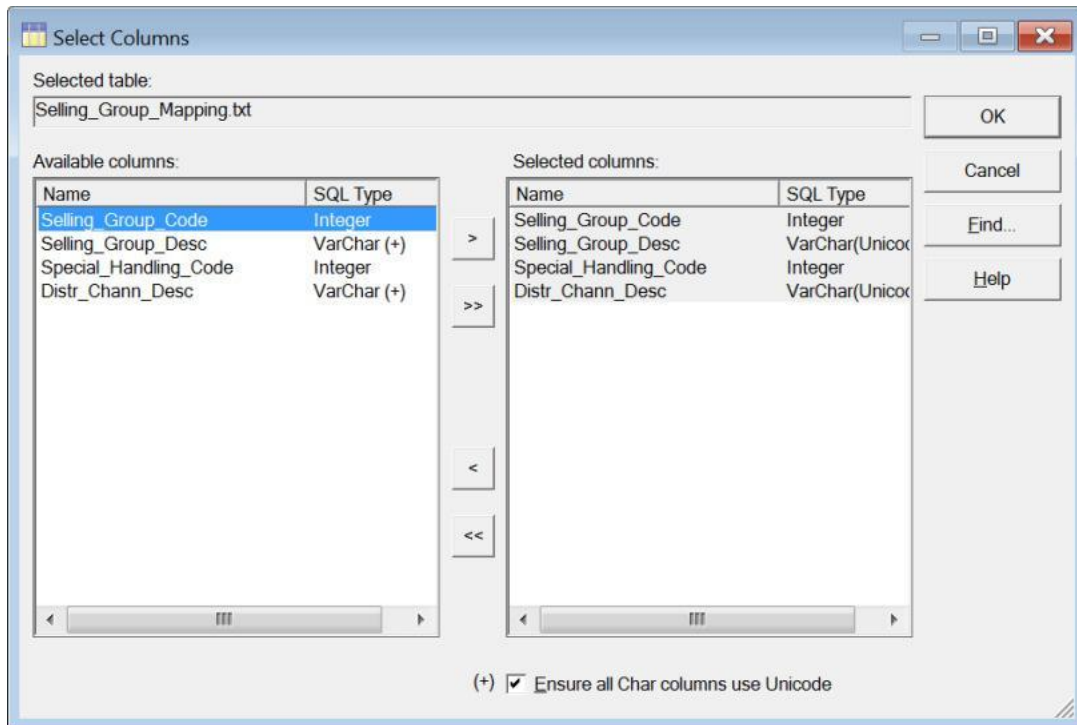
10. Double click on the source Sequential File stage. We need to specify the file to read in the Properties tab. Select the File property and then use the right arrow to browse for file to find the Selling_Group_Mapping.txt file. Hit the Enter key after you selected file to set it into the File property. Here be sure to set the First Line is Column Names to True. If you don't, your job will have trouble reading the first row and issue a warning message in the log.



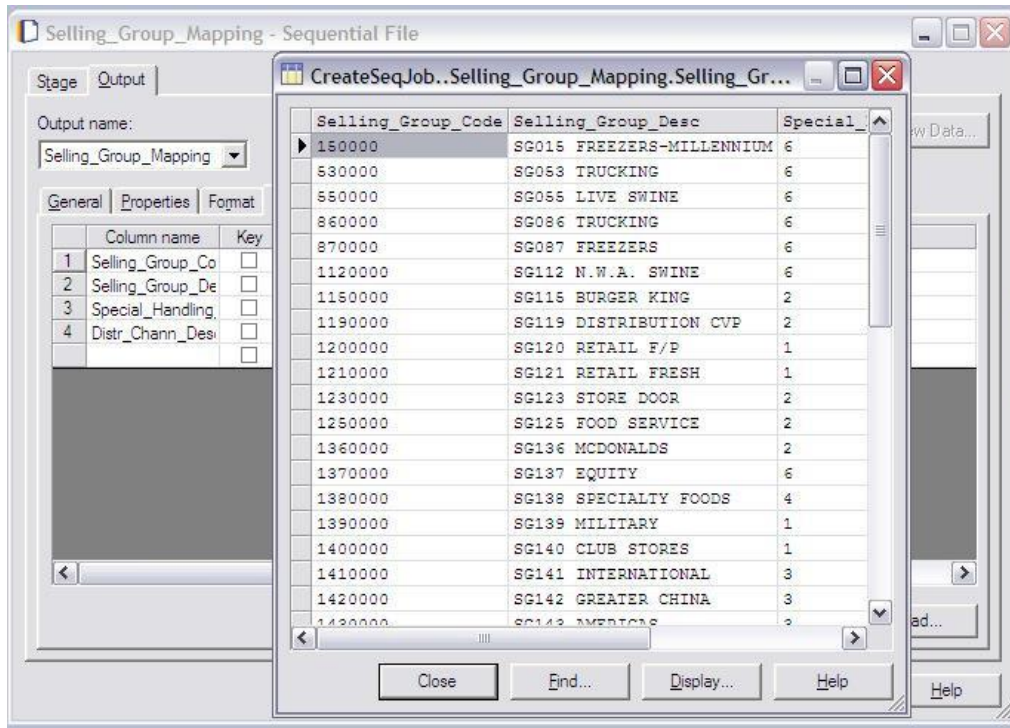
11. Next go to the Format tab and click the Load button to load the format from the Selling_Group_Mapping.txt table definition under folder /Table Definitions/Sequential/Labs.



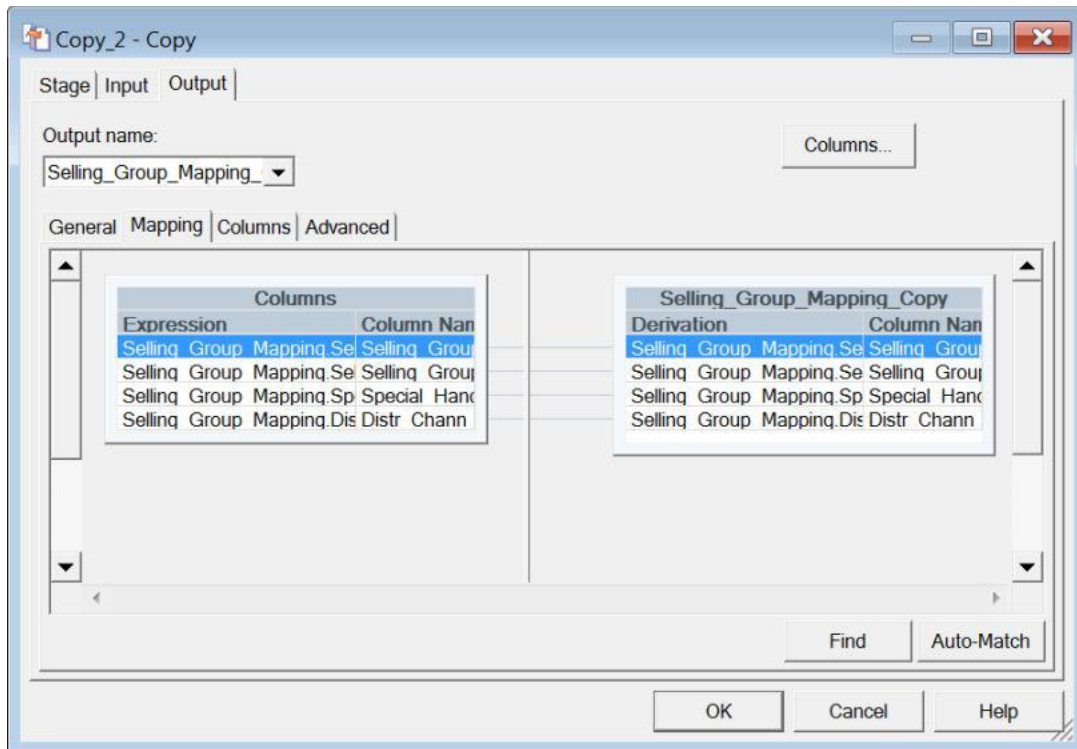
12. Next go to the Columns tab and load the columns from the same table definition in the repository. Click OK to accept the columns.



13. Click View Data and then OK to verify that the metadata has been specified properly. This is true when you can see the data window. Otherwise you will get an error message. Close the View Data window and click OK to close the Sequential File stage editor.

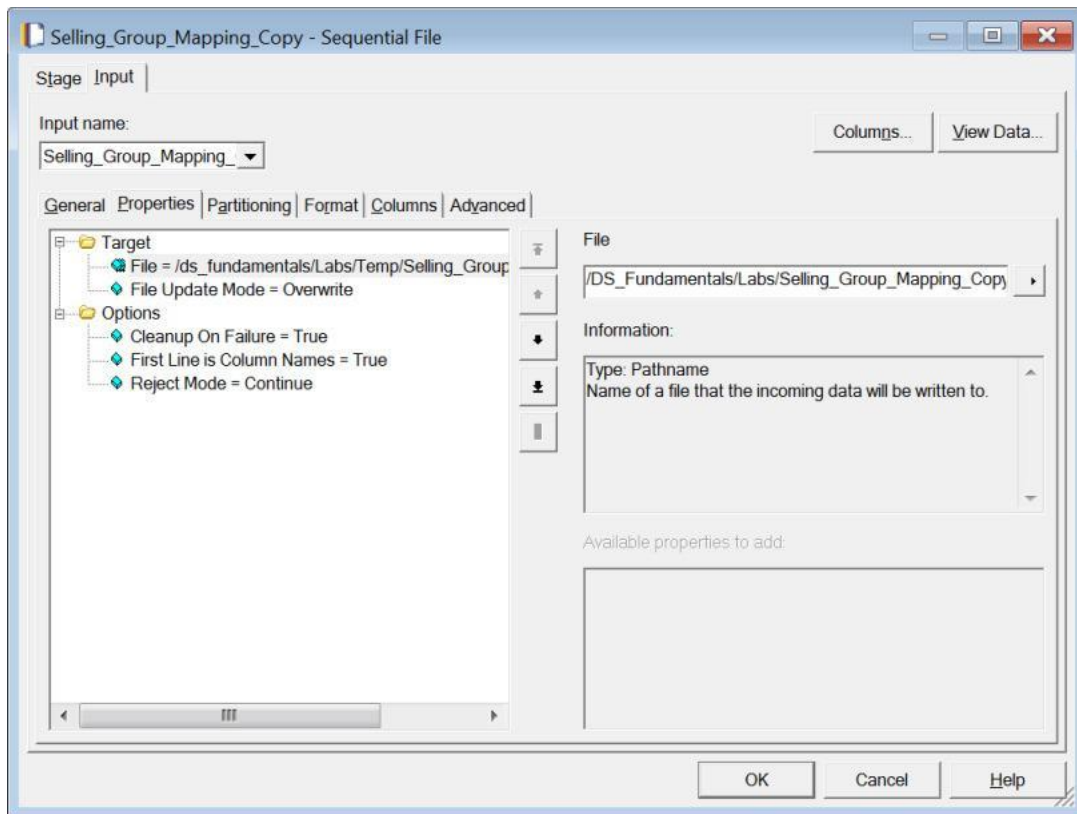


14. In the Copy stage Output tab > Mapping tab, drag the columns across from the source to the target.



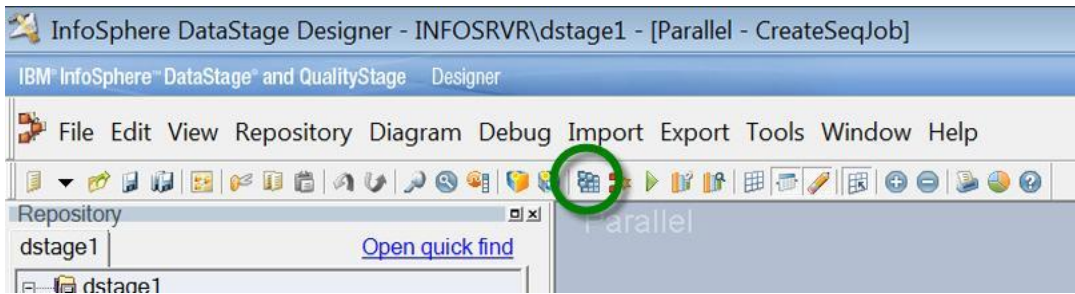
15. Click OK to save the settings.

16. Open the target Sequential File stage
17. Ensure that in the Format tab, the Delimiter setting in the Field defaults folder is set to comma delimited.
18. In the Properties tab in the File property, type the directory name /DS_Fundamentals/Labs/ and name the file Selling_Group_Mapping_Copy.txt. Instead of typing, you can use the right arrow button to 'browse for file'. Then pick the Selling_Group_Mapping.txt file and come back to correct it to "_Copy.txt".
19. Set option 'First Line is Column Names' to true. The File Update Mode should be 'Overwrite'. Click OK to save your settings.

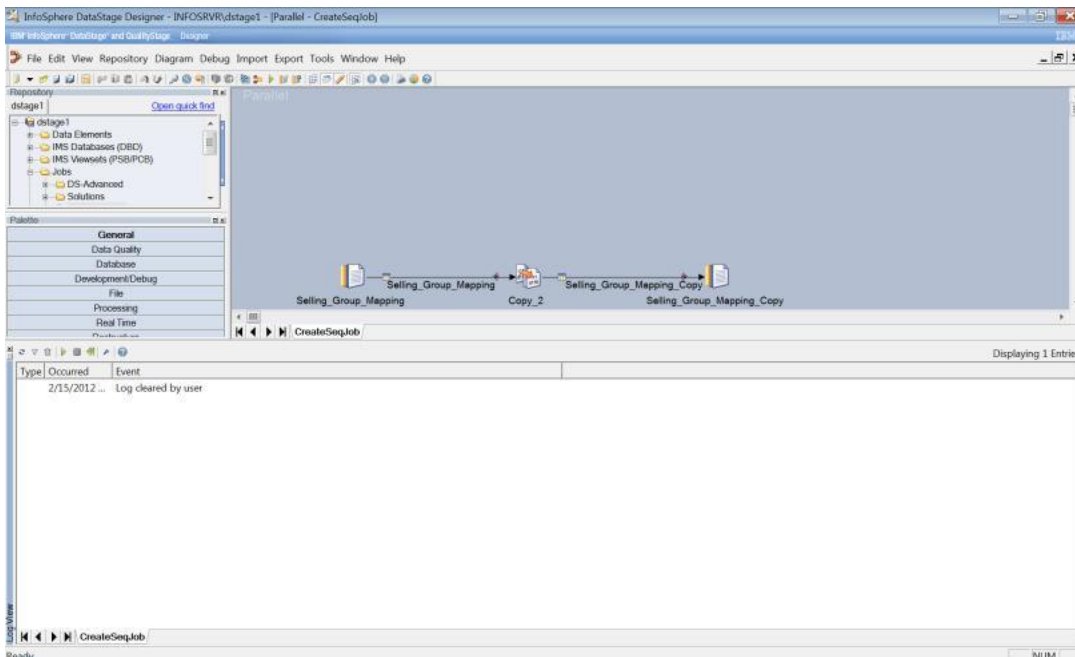


Task: Compile, run, and monitor the job

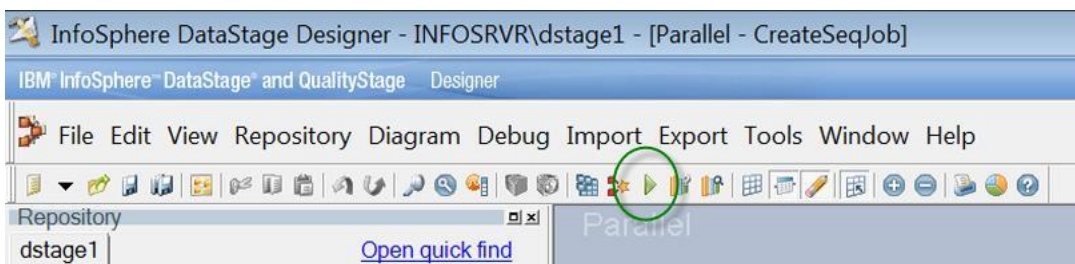
1. Save your job
2. Click the Compile button.



3. After the compilation is finished, click your right mouse button over an empty part of the canvas. Select or verify that “Show performance statistics” is enabled (a checkmark should be present in front of “Show performance statistics”). This will show, for each link, how many rows were processed and the throughput per second.
4. Click on the menu View > Job Log. Enlarge the job log window.

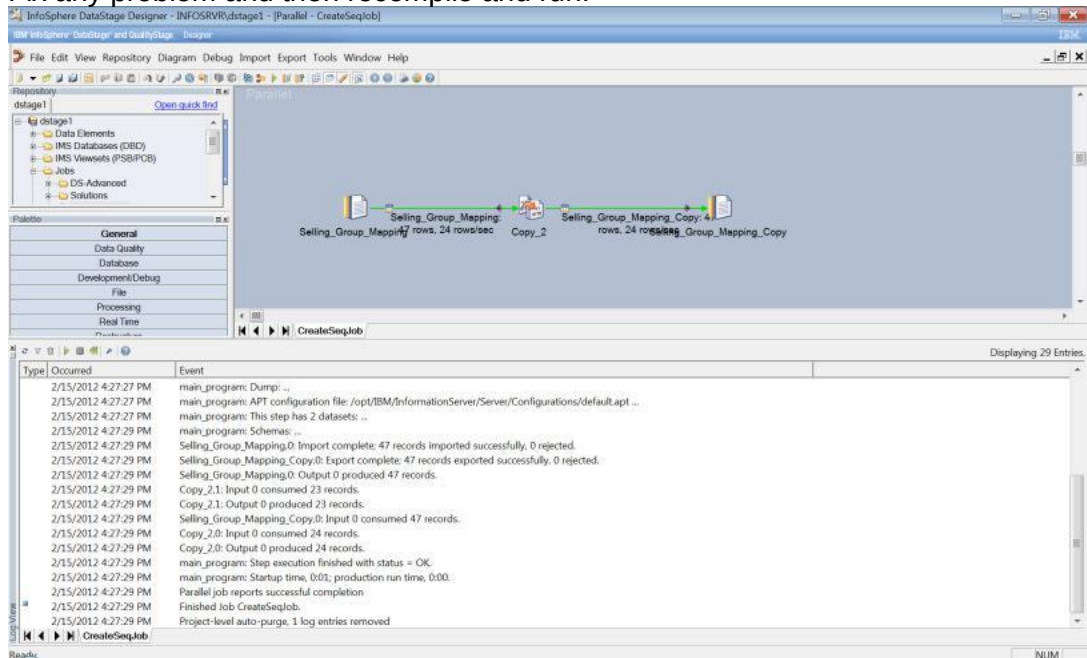


5. Run the job by pressing the green arrow icon.



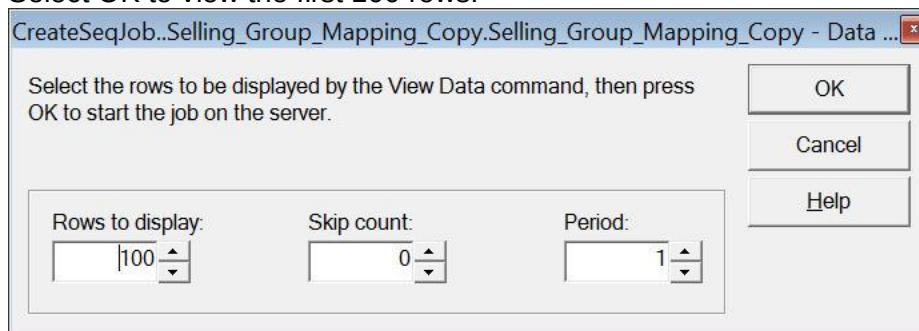
6. Select ‘Run’

7. Scroll through the messages in the log. There should be no warnings (yellow) or errors (red). If there are, double-click on the messages to examine their contents. Fix any problem and then recompile and run.



8. Rearrange the job view window to make more canvas space available.
9. You can view the result data by right clicking on the target sequential file and choosing 'View Selling_Group_Mapping_Copy data...'.

10. Select OK to view the first 100 rows.



11. The target file content will be shown in DataStage.

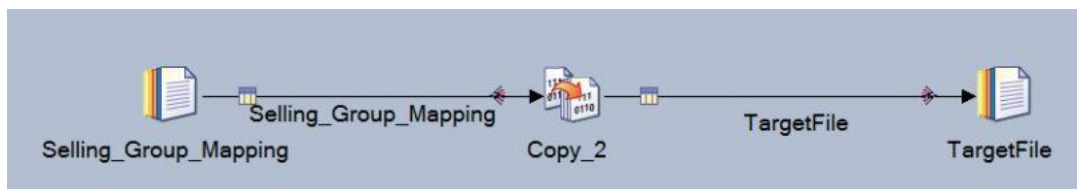
CreateSeqJob..Selling_Group_Mapping_Copy.Selling_Group_Mapping_Copy - Data Browser

Selling_Group_Code	Selling_Group_Desc	Special_Handling_Code	Distr_Chann_Desc
150000	SG015 FREEZERS-MILLENNIUM	6	Other
550000	SG055 LIVE SWINE	6	Other
870000	SG087 FREEZERS	6	Other
1150000	SG115 BURGER KING	2	Food Service
1200000	SG120 RETAIL F/P	1	Retail
1230000	SG123 STORE DOOR	2	Food Service
1360000	SG136 MCDONALDS	2	Food Service
1380000	SG138 SPECIALTY FOODS	4	Specialty Product
1400000	SG140 CLUB STORES	1	Retail
1420000	SG142 GREATER CHINA	3	International
1440000	SG144 JAPAN	3	International
1460000	SG146 R.E.M.A.	3	International
3110000	SG311 INTERNATIONAL SEAFOOD	6	Other
4010000	SG401 KOCH IND/HEFLIN	2	Food Service
4060000	SG406 HOSPITALITY XFER PR	6	Other
4230000	SG423 MEXICAN ORIGINAL SA	6	Other
4430000	SG443 MALLARD'S SALES	6	Other
5230000	SG523 ENTREE'	1	Retail
5550000	SG555 COBB VANTRESS	6	Other
6110000	SG611 REFRIGERATED PROCES	1	Retail
6130000	SG613 IOWA HAM	1	Retail
9130000	SG913 DISTRIBUTION XFER PH	6	Other
9230000	SG923 NEW COMMODITY XFER	16	Other

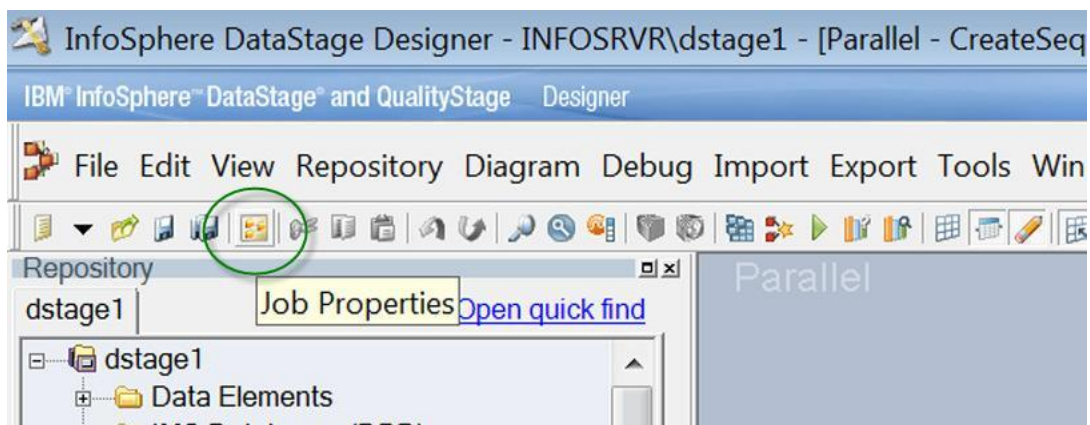
Close Find... Display... Help

Task: Create and use a job parameter

1. Save the job CreateSeqJob as CreateSeqJobParam. Rename the last link and the target Sequential File stage to "TargetFile".

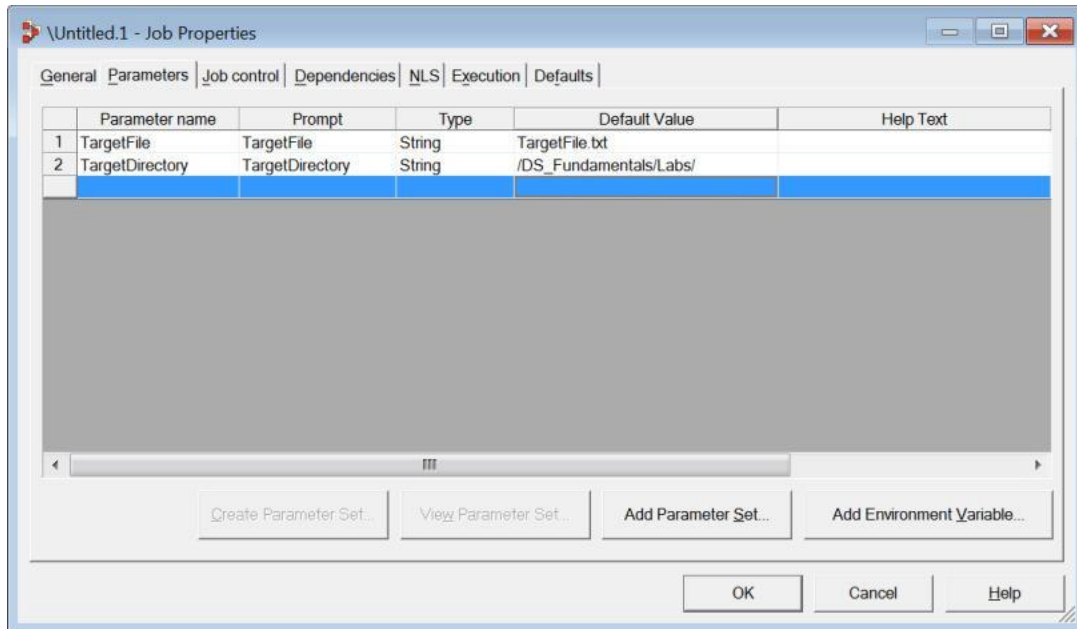


2. Open up the job properties window by clicking the 5th icon on the tool bar.

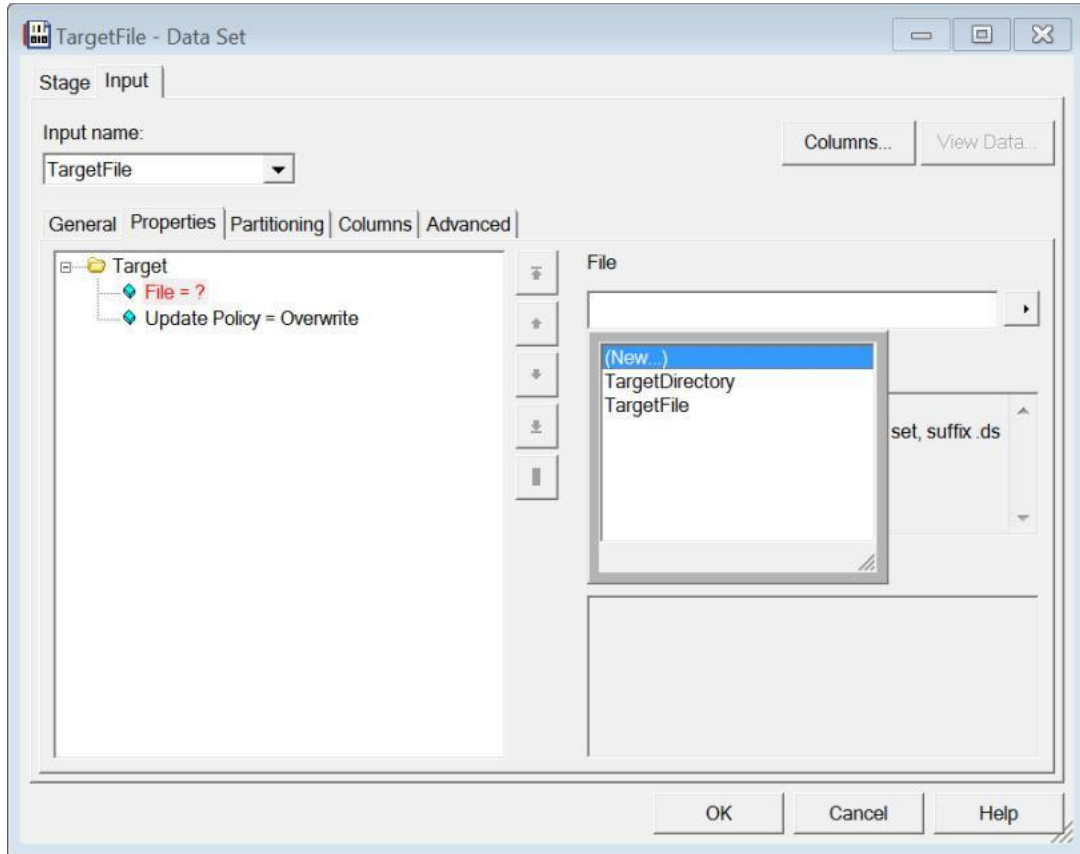


- On the Parameters tab, define the first job parameter named TargetFile of type string. You double click on the Parameter name field and simply type into it and then fill out the other fields. Create an appropriate default filename, e.g., TargetFile.txt. The second job parameter will contain the target directory called /DS_Fundamentals/Labs/. Make sure to also add the final forward slash.

Hit the Enter key to retain the changes. Click OK to close the window.

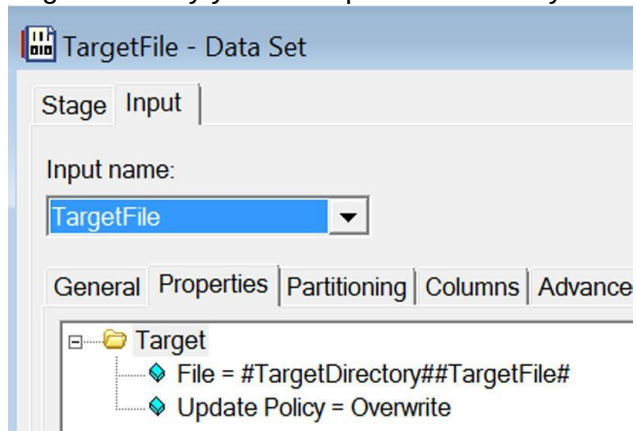


4. Open up your target Sequential File stage to the Properties tab. Select the File property. Delete the content of the File property. Click on the black arrow button on the right side of the text box.
5. Select 'Insert job parameter ...'.



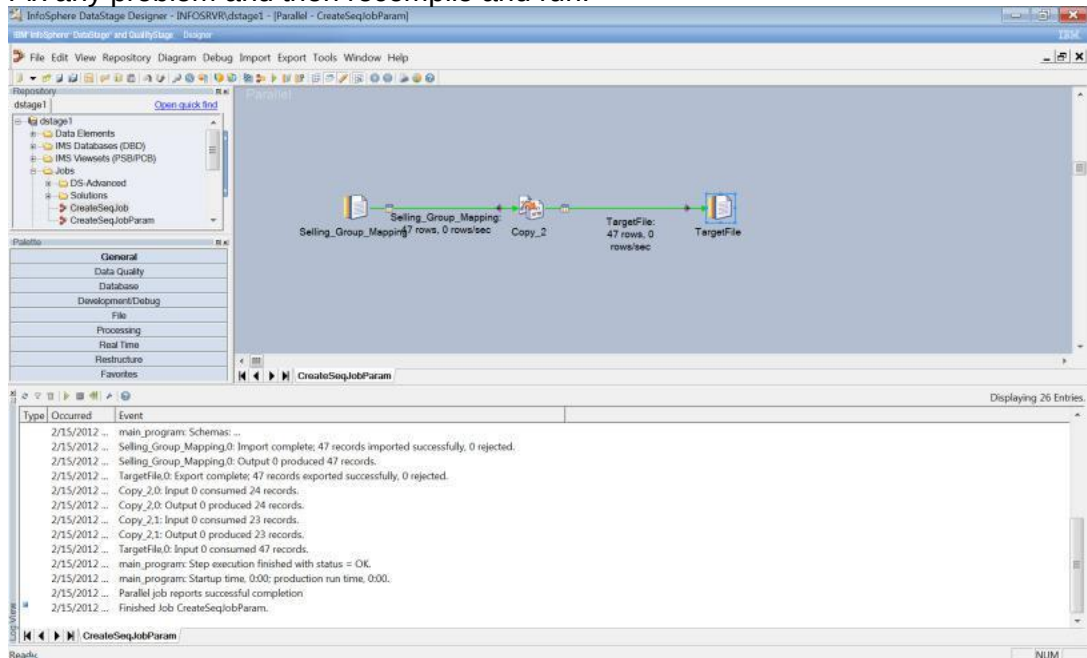
6. Select the TargetDirectory parameter first. Place the cursor at the end of the inserted #TargetDirectory# string. Repeat the step with the TargetFile parameter.
7. Your final File box string should look like: #TargetDirectory##TargetFile#

8. Note that the parameters are enclosed in # signs. If you did not add a final / for your target directory you could place it manually in between the parameters.

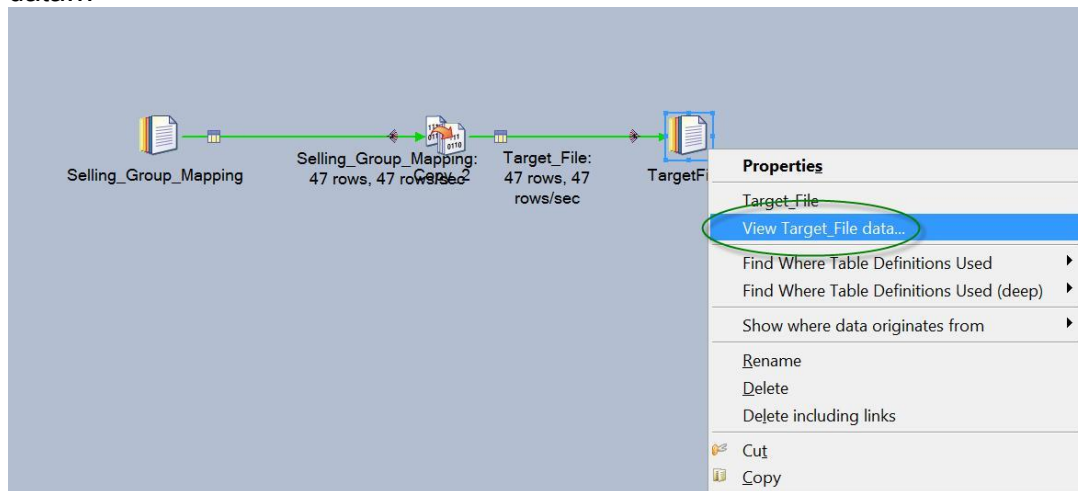


9. Hit return and click OK to save the changes.
10. Compile your job.
11. Run your job.
12. Enlarge the job log view

13. Scroll through the messages in the log. There should be no warnings (yellow) or errors (red). If there are, double-click on the messages to examine their contents. Fix any problem and then recompile and run.



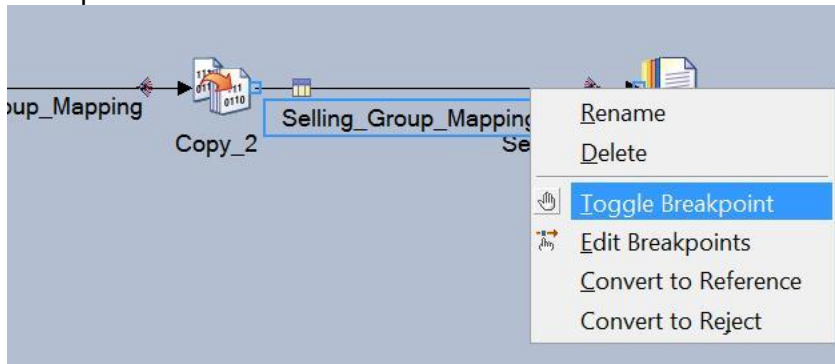
14. Right click on the final sequential stage 'Target File'. Select 'View Target_File data...'



15. Select OK to confirm the default parameter set value 'TargetFile.txt'.
16. Select OK to display the number of rows to display.

Task: Using the Parallel Debugger to test and debug your job

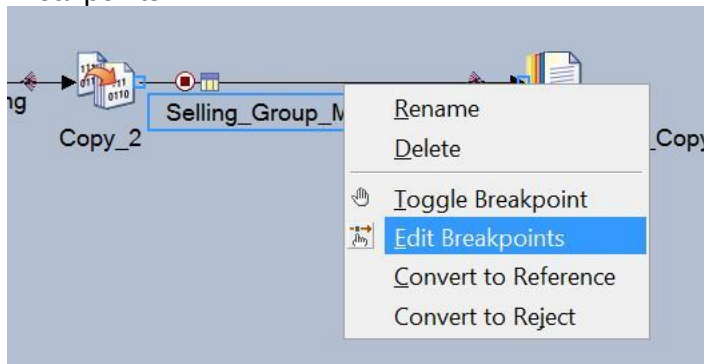
1. You can continue using the CreateSeqJobParam job. The debugger enables you to set breakpoints on the links in your job. When you run the job in debug mode, the job will stop when it reaches a breakpoint. To set a breakpoint, right click on the Selling_Group_Mapping_Copy link. Select Toggle Breakpoint. This will switch the breakpoint on and off.



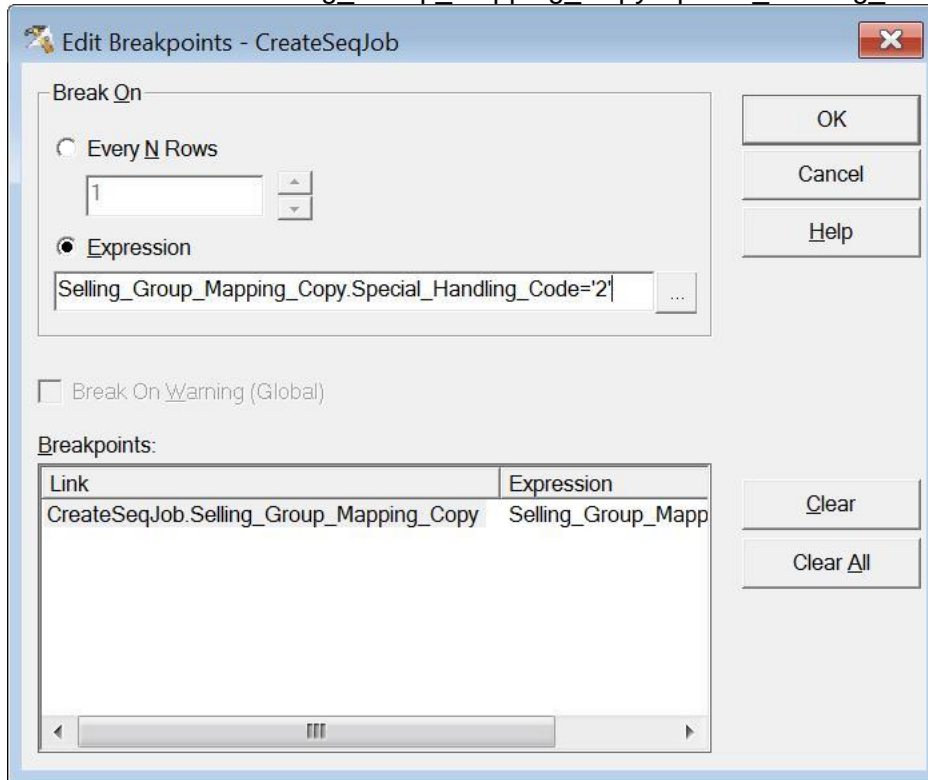
2. Note that the red breakpoint indicator will appear.



3. Right click on the Selling_Group_Mapping_Copy link again and select Edit Breakpoints.



- The Edit Breakpoints window will open. By default, the job will break on every row that is passing through the link. Change the settings to break when the Special_Handling_Code column data matches the value 2. Check the Expression field and select the Special_Handling_Code column. The expression field value should read `Selling_Group_Mapping_Copy.Special_Hanling_Code = '2'`.

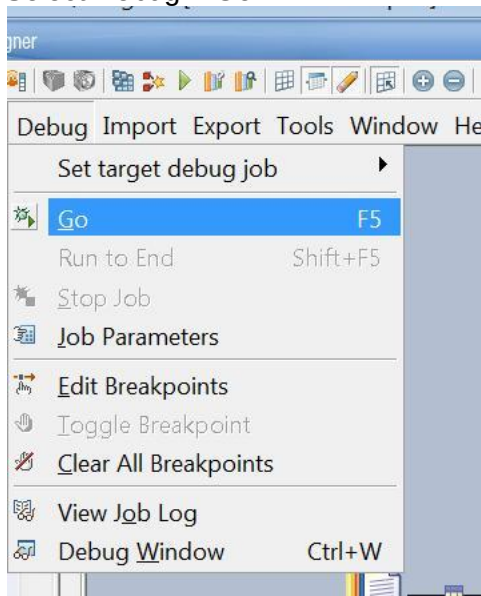


- Click OK.
- Right click on the Selling_Group_Mapping source stage and select 'View Selling_Group_Mapping data...'. Note that the first Special_Handling_Code values that match our criteria come in 7th and 8th place.

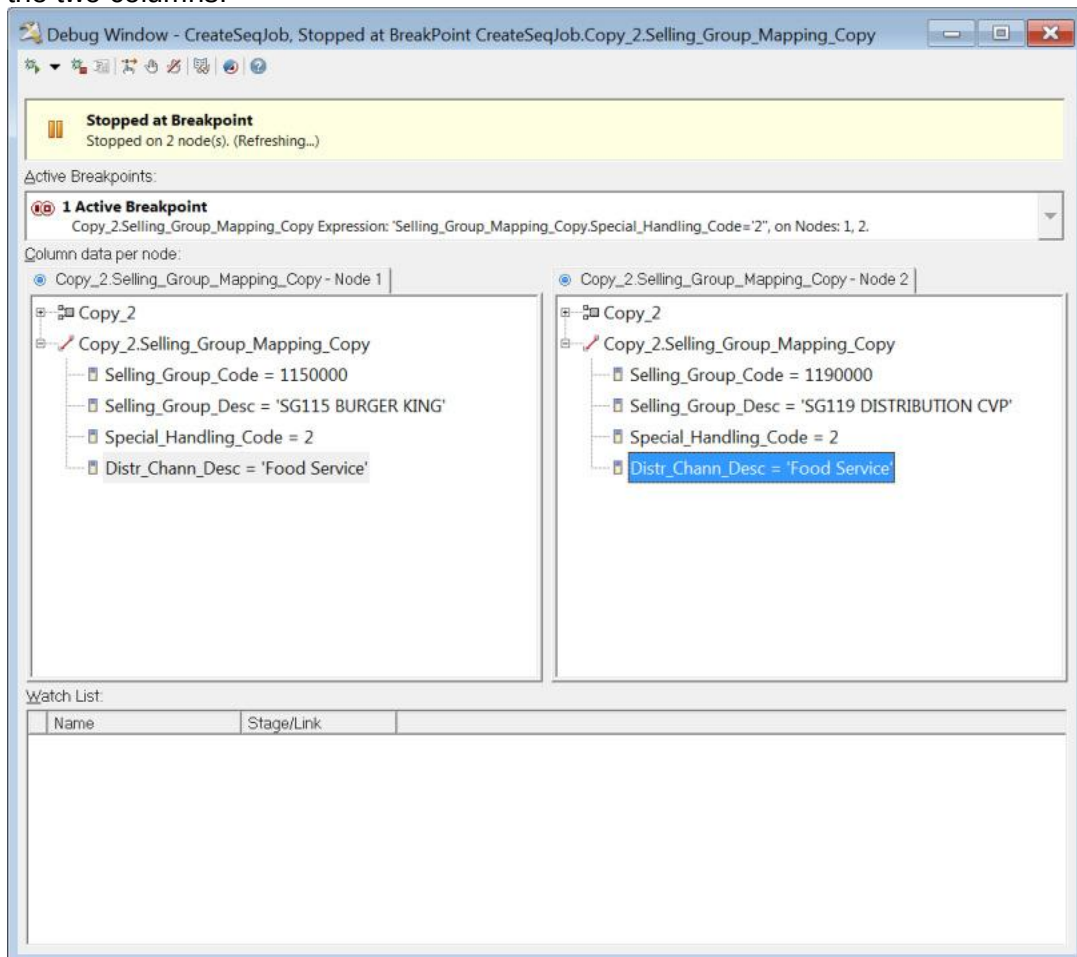
Selling_Group_Code	Selling_Group_Desc	Special_Handling_Code	Distr_Chann_Desc
150000	SG015 FREEZERS-MILLENNIUM	6	Other
530000	SG053 TRUCKING	6	Other
550000	SG055 LIVE SWINE	6	Other
860000	SG086 TRUCKING	6	Other
870000	SG087 FREEZERS	6	Other
1120000	SG112 N.W.A. SWINE	6	Other
1150000	SG115 BURGER KING	2	Food Service
1190000	SG119 DISTRIBUTION CVP	2	Food Service
1200000	SG120 RETAIL F/P	1	Retail
1210000	SG121 RETAIL FRESH	1	Retail
1230000	SG123 STORE DOOR	2	Food Service
1250000	SG125 FOOD SERVICE	2	Food Service
1360000	SG136 MCDONALDS	2	Food Service

- Recompile the job.

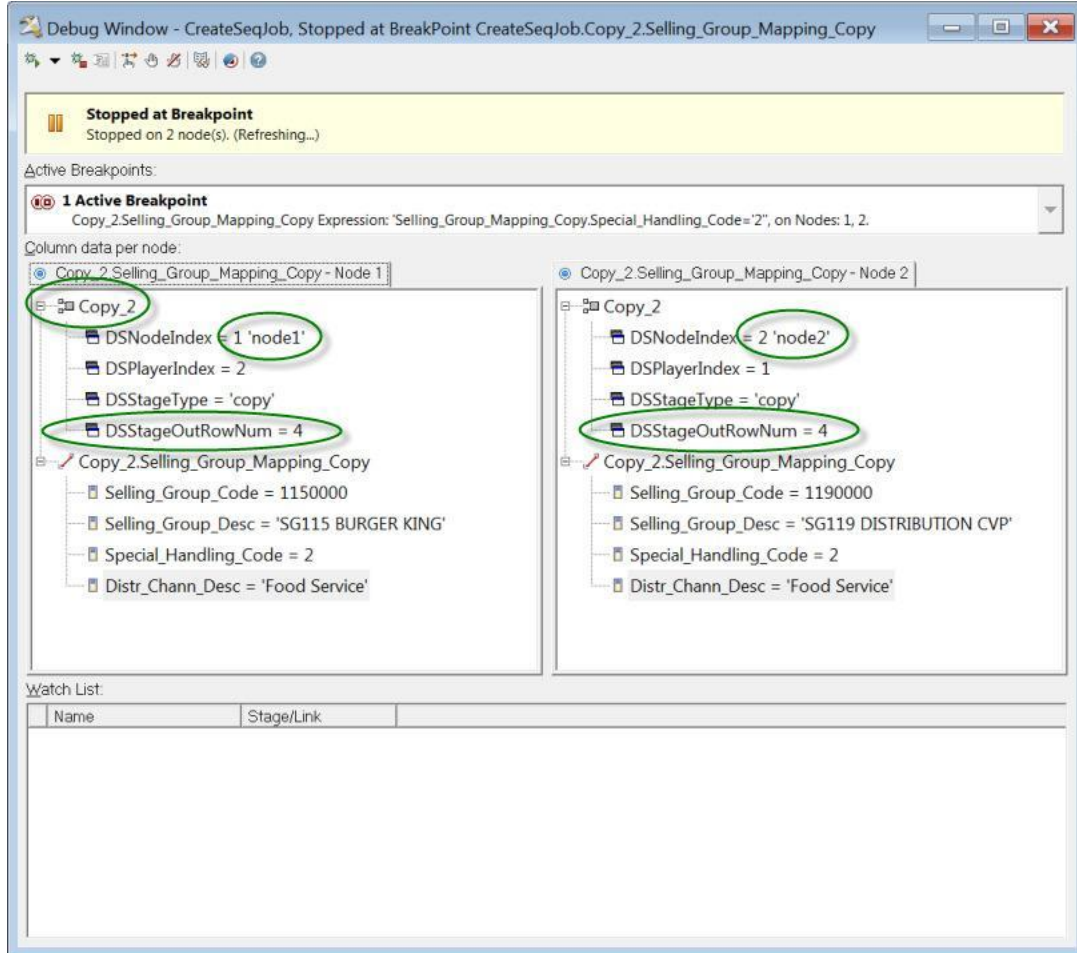
8. Select Debug > Go



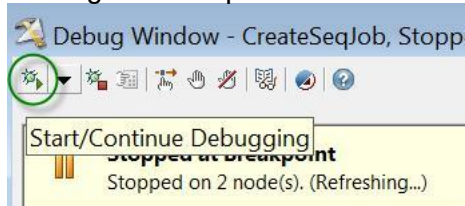
9. Click OK to run the job. The Debug Window should open up. Resize the window to fit the two columns.



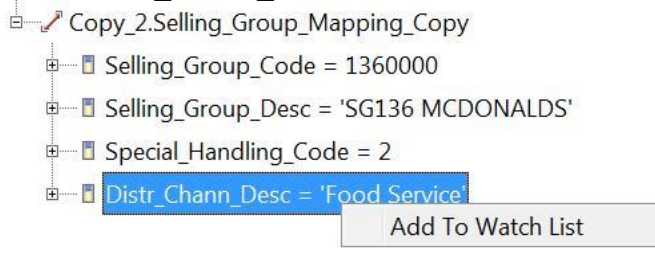
10. You will see two Column data windows since we are currently using a two node configuration. We will learn more about this configuration in a later module. Expand the Copy_2 details that are currently minimized. Note that three Special_Handling_Code rows that carry the value 6 have already passed through each processing node and that the breakpoint expression has now become active in the forth row of each node.



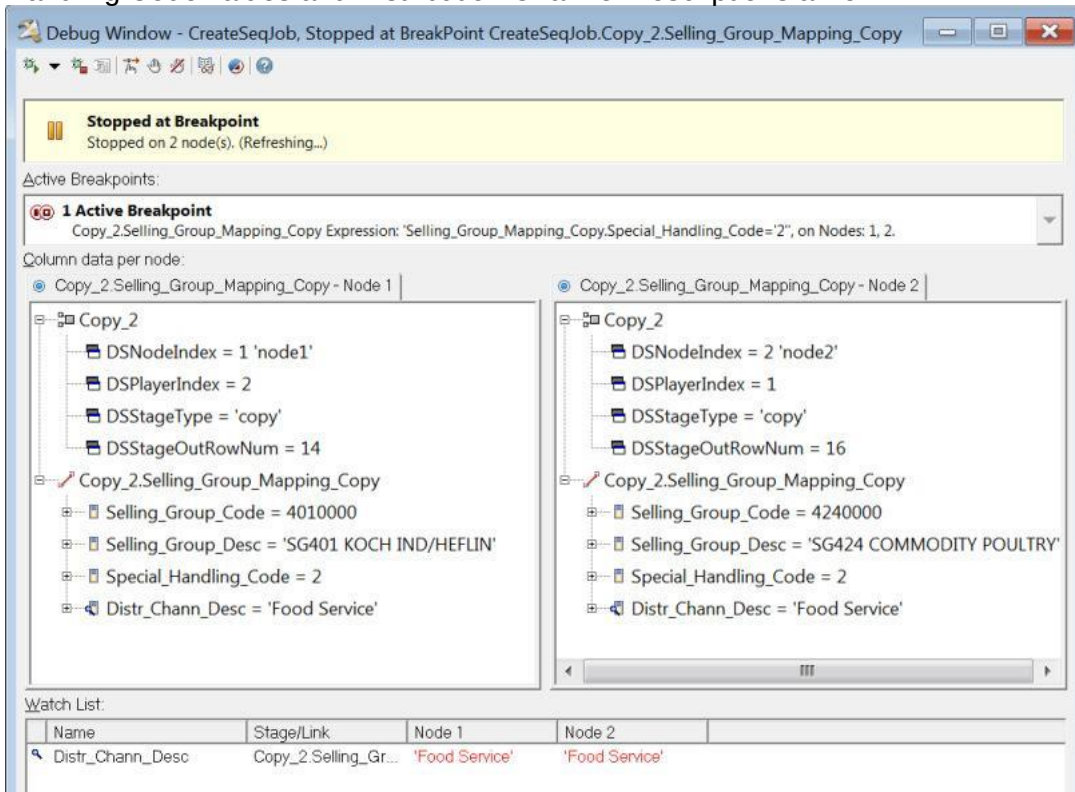
11. Step to the processing of the next row of data that matches our expression by clicking on the top left icon.



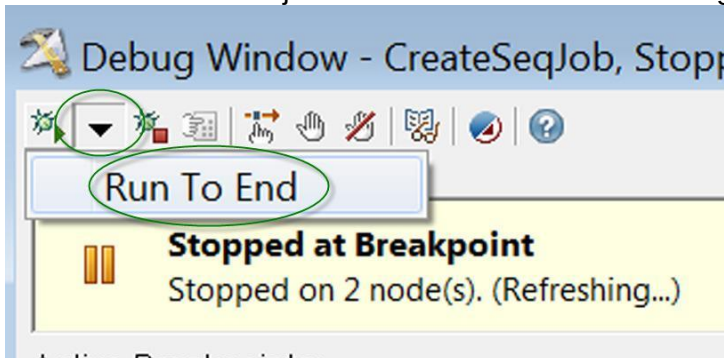
12. The next special handling code values will break the job run again. You can also dynamically define break criteria by adding values to your watch list. Right click on the Distr_Chann_Desc field and define 'Food Service' as a watch list item.



13. Continue to run the job debugger. The debugger will now watch out for Special Handling Code values and Distribution Channel Descriptions alike.



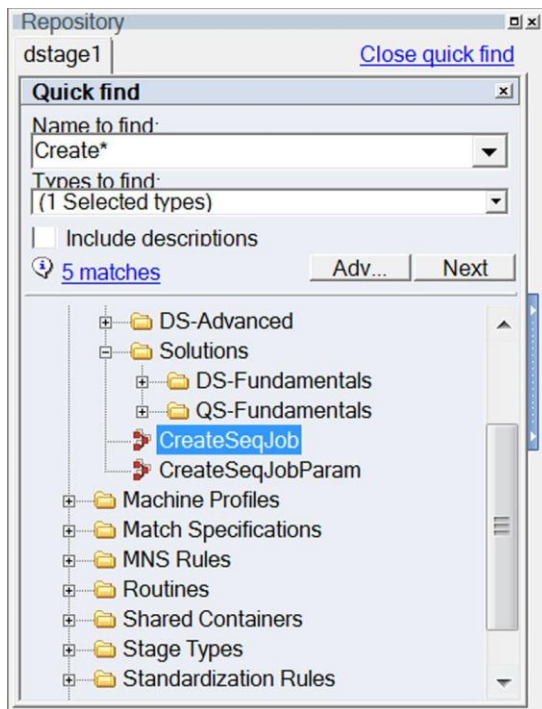
14. You can now run the job to the end and close the debug window.



Lab 04: Browsing the Repository

Task: Execute a Quick Find

1. Open 'Quick Find' by clicking the link at the top of the Repository window.
2. In the 'Name to find' box, type "Create*", and, in the 'Types to find' list, select 'Parallel Jobs'.
3. Click the Find button to run the search.
4. The first found item will be highlighted.

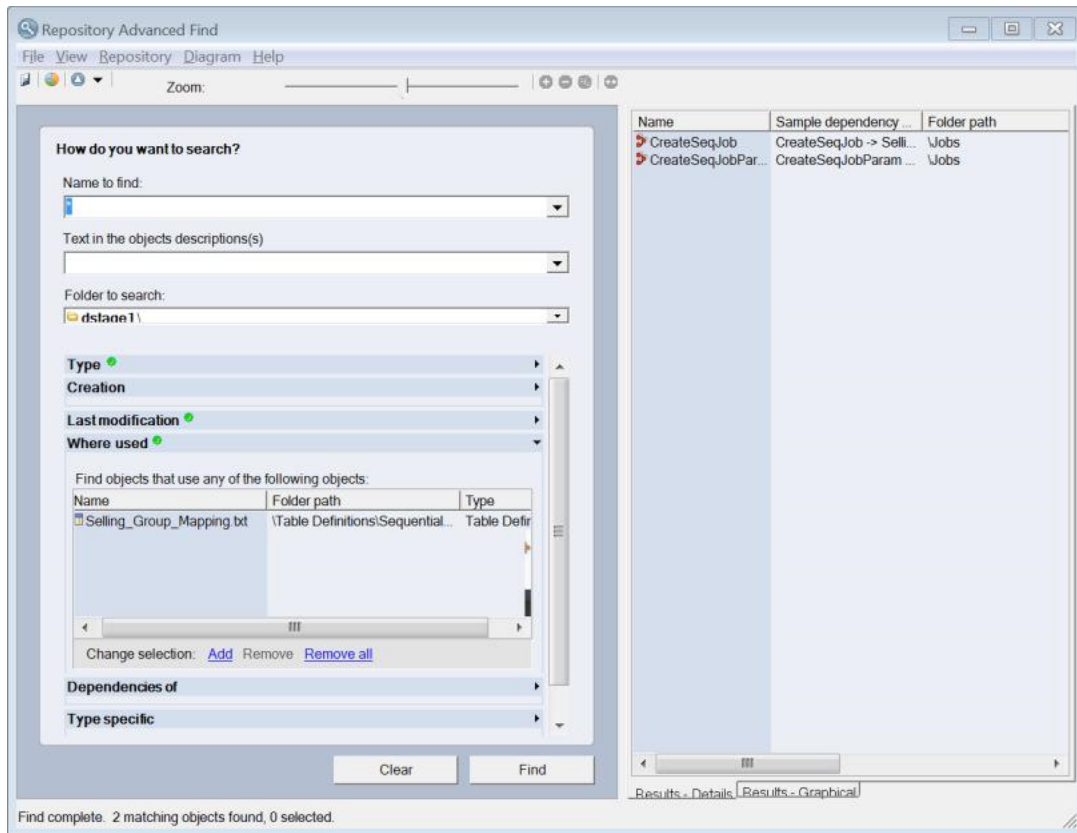


5. Click Next to highlight the next item.

Task: Execute an Advanced Find

1. Click on the link that displays the number of matches. This opens the Advanced Find window and displays the items found so far in the right pane.
2. Open the Last modification folder. Specify objects modified within the last week and run the search again.

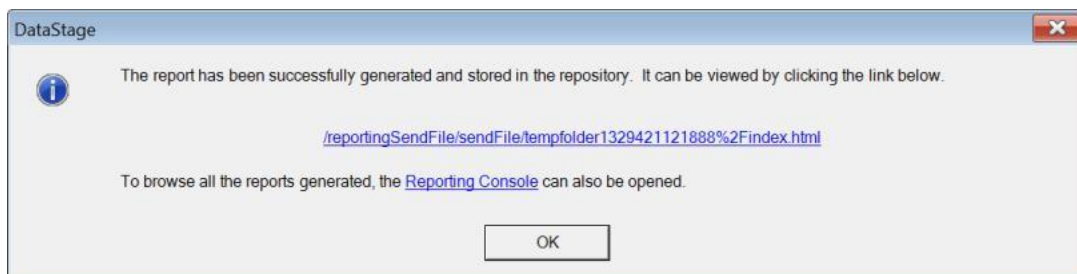
- You can also specify additional criteria, e.g. find jobs that use a certain table definition. Open the 'Where Used' folder. Add the Selling_Group_Mapping.txt Table Definition from Table Definitions > Sequential > Labs. Change Name to find to an asterisk (*). Click Find. This will show you the jobs that use this Table Definition.



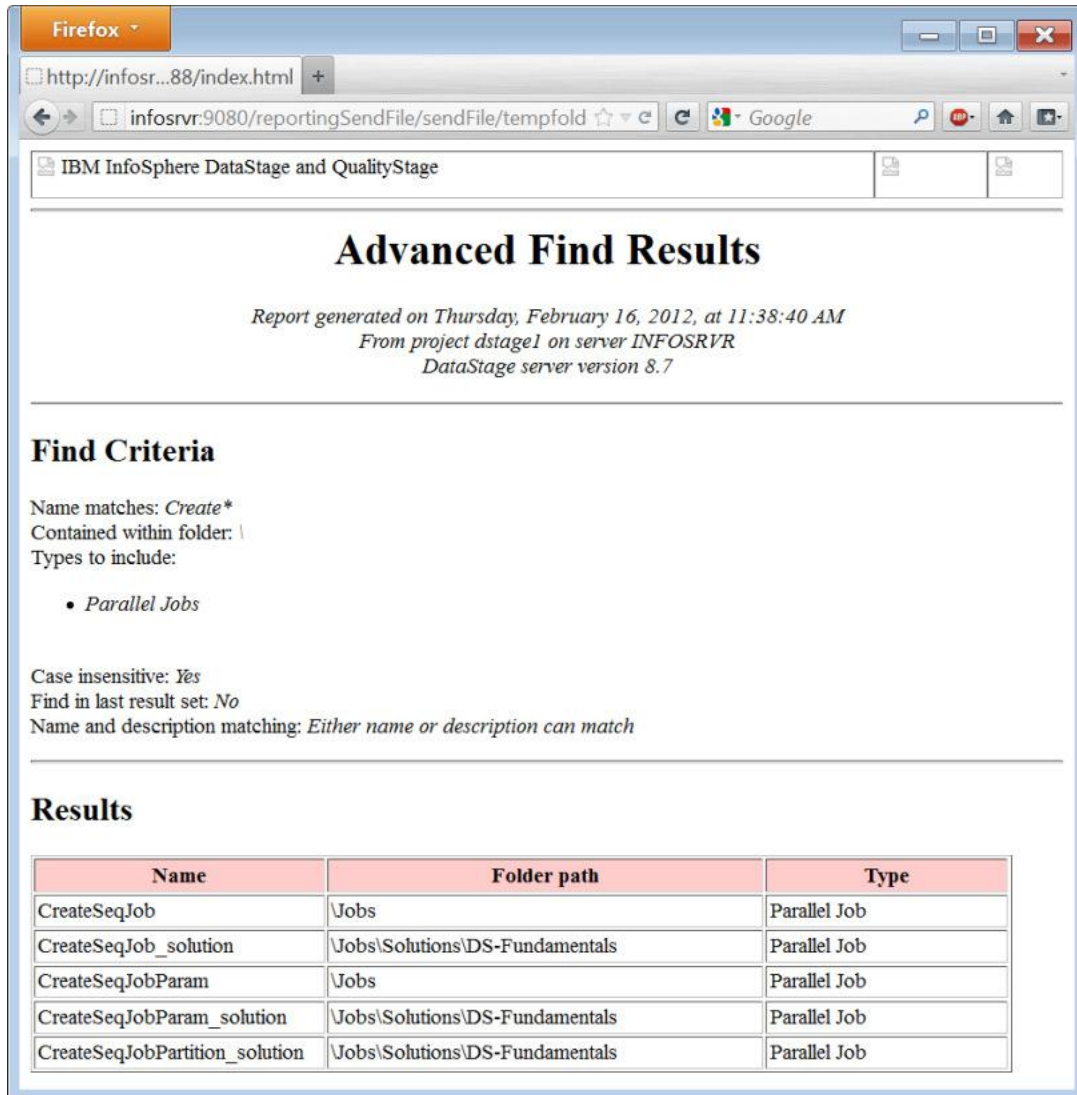
- Close the Advanced Find window.

Task: Generate a report

- Click the number of matches to get the search result window again.
- Click File > Generate Report to open a window from which you can generate a report describing the results of your Find. Click OK in the "Enter report details" window.



- Click on the top link to view the report. This report is saved in the Repository where it can be viewed by logging onto the Reporting Console.



Firefox

http://infosr...88/index.html

infosrvr:9080/reportingSendFile/sendFile/tempfold

IBM InfoSphere DataStage and QualityStage

Advanced Find Results

Report generated on Thursday, February 16, 2012, at 11:38:40 AM
From project dstage1 on server INFOSRV
DataStage server version 8.7

Find Criteria

Name matches: *Create**
Contained within folder: **
Types to include:

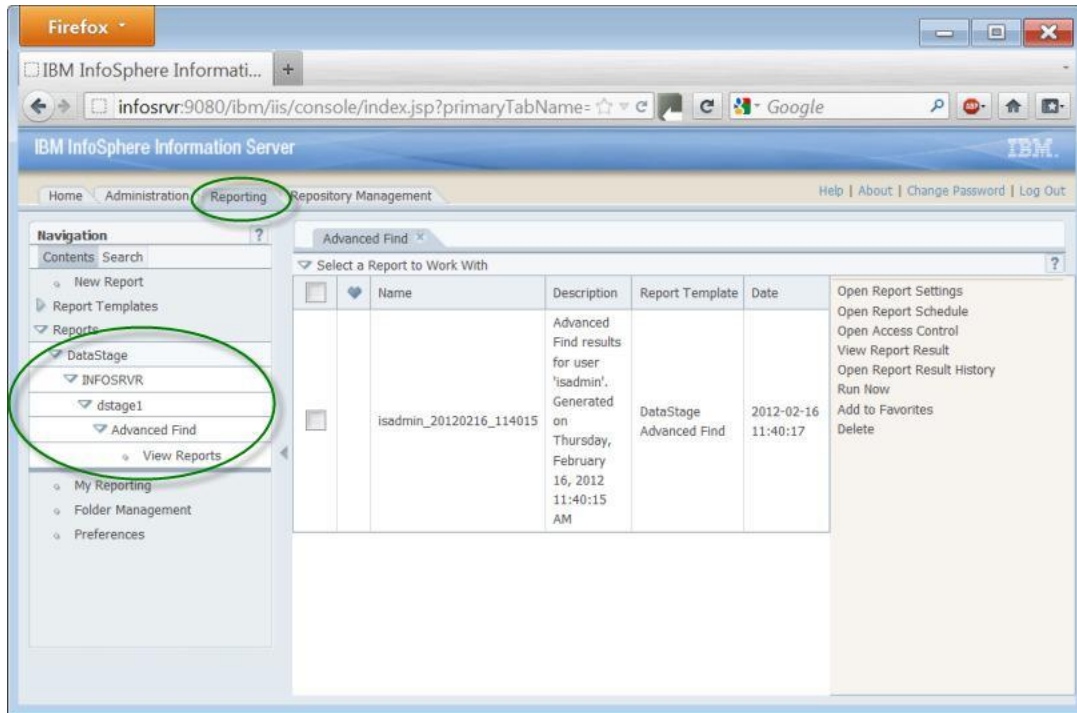
- Parallel Jobs*

Case insensitive: *Yes*
Find in last result set: *No*
Name and description matching: *Either name or description can match*

Results

Name	Folder path	Type
CreateSeqJob	\Jobs	Parallel Job
CreateSeqJob_solution	\Jobs\Solutions\DS-Fundamentals	Parallel Job
CreateSeqJobParam	\Jobs	Parallel Job
CreateSeqJobParam_solution	\Jobs\Solutions\DS-Fundamentals	Parallel Job
CreateSeqJobPartition_solution	\Jobs\Solutions\DS-Fundamentals	Parallel Job

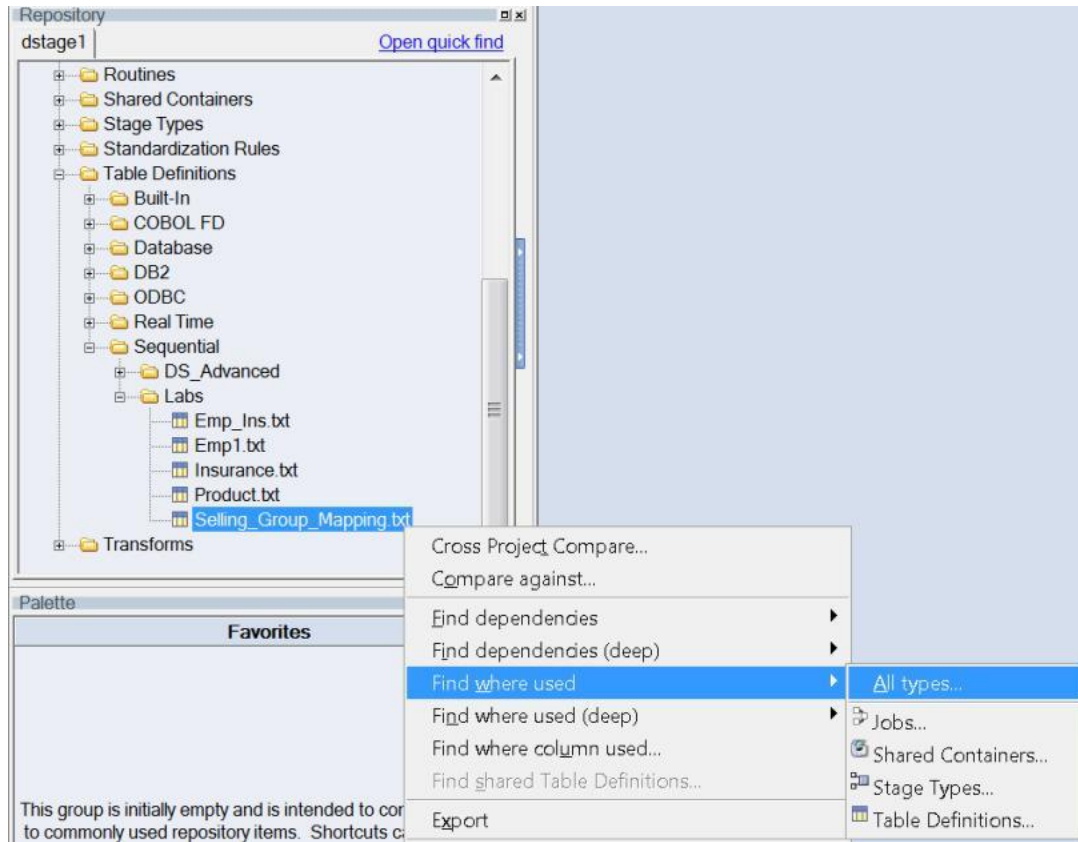
- After closing this window, click on the Reporting Console link. Switch to the Reporting tab, expand the Reports folder as shown. Click View Reports.



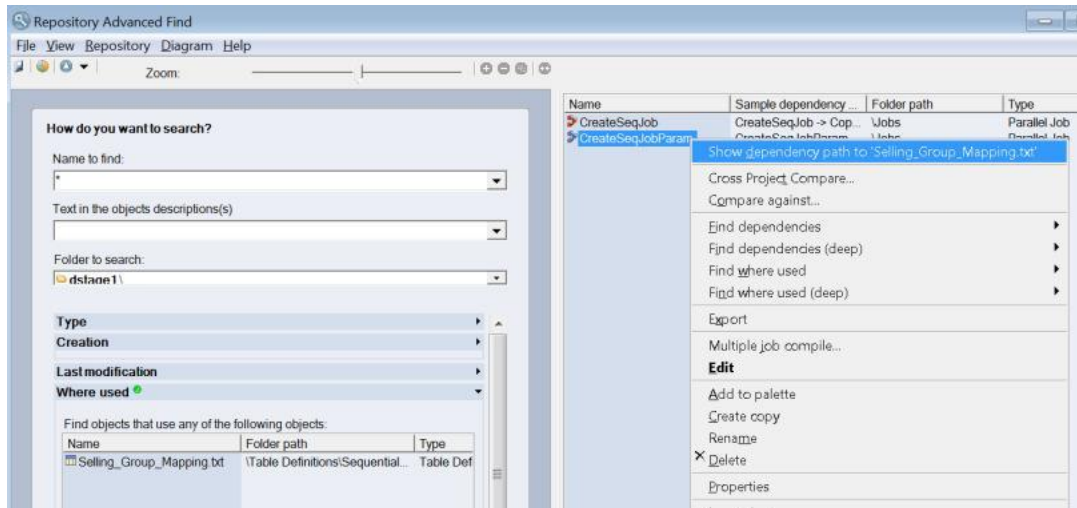
- Select your report and then click View Report Result. This displays the report you viewed earlier from Designer. By default, a Suite user only has permission to view the report. A Suite administrator can give additional administrative functions to a Suite user, including the ability to alter report properties, such as format.
- Close all windows and then close the Quick Find.

Task: Perform an impact analysis

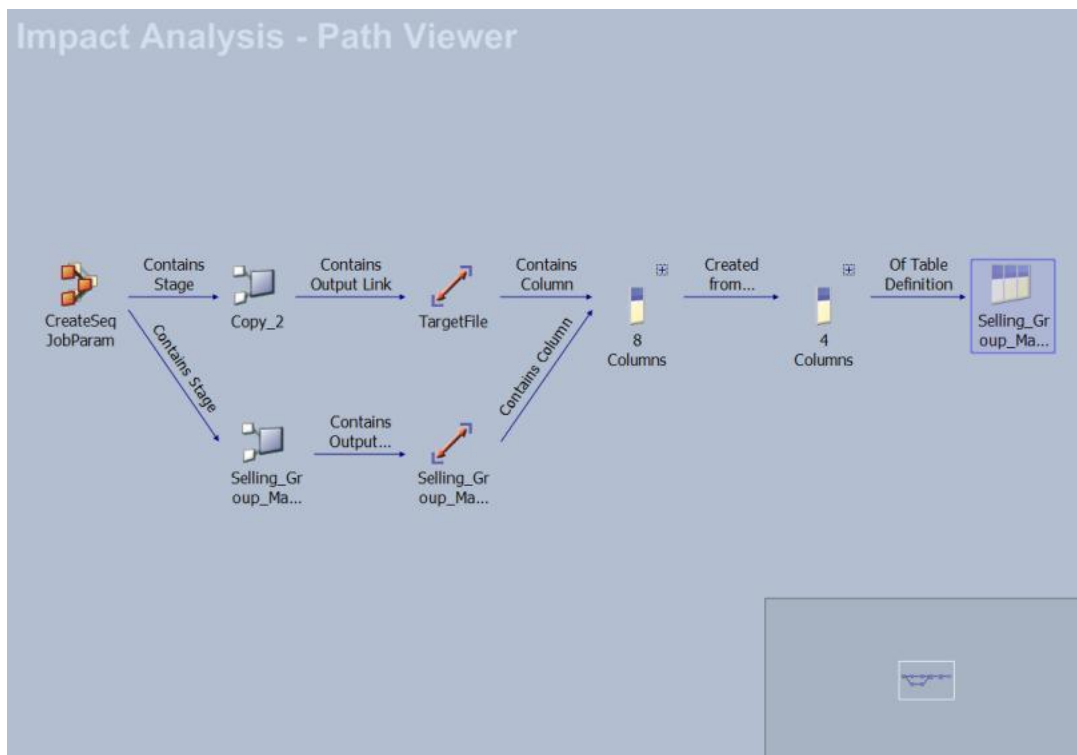
1. Impact analysis can help you identify related assets that are affected when you are about to change an asset like a Table Definition. In the Repository window, select your `Selling_Group_Mapping.txt` Table Definition. Click the right mouse button and then select `Find Where Used > All Types`.



- The result shows all the jobs that use this Table Definition. Click the right mouse button over the CreateSeqJobParam job listed and then click “Show dependency path to...”



- Use the Zoom button to adjust the size of the dependency path so that it fits into the window.

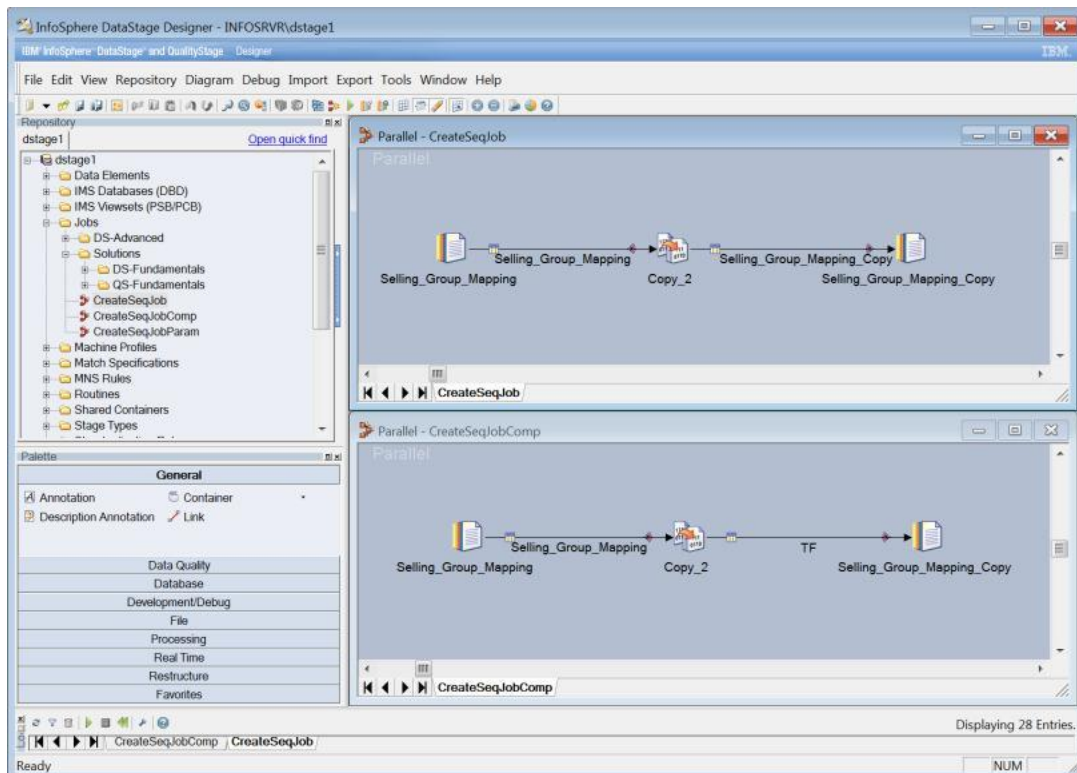


- Hold right mouse button over a graphical object and move the path around. The graph shows you in detail which stages require attention when you are about to change the Table Definition.

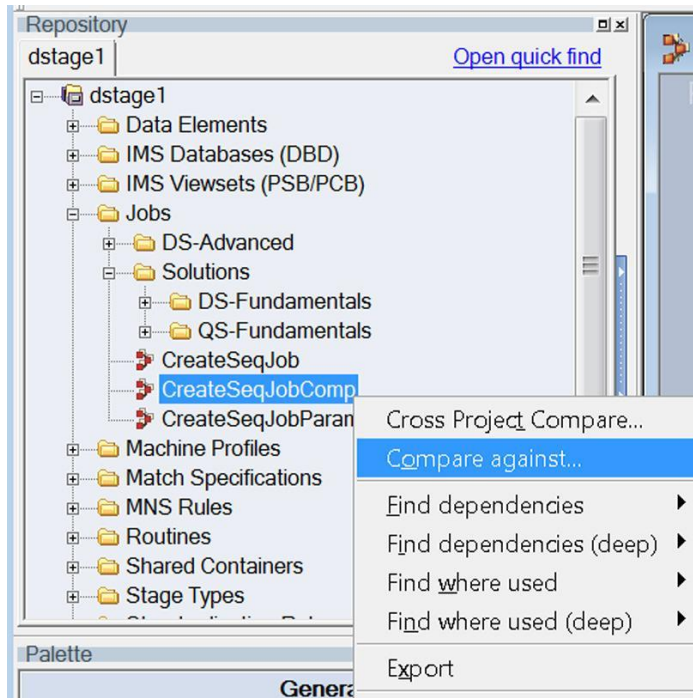
5. Notice the “birds-eye” view box in the lower right hand corner. This shows how the path is situated on the canvas. You can move the path around by clicking to one side of the image in the birds-eye view window and by holding the right mouse button down over the image and moving the image around.
6. Close the window.

Task: Find the differences between two jobs

1. Open your CreateSeqJob. Save it as CreateSeqJobComp. Make the following changes to the CreateSeqJobPartitonComp job.
2. Open up the Selling_Group_Mapping Sequential File source stage. On the Columns tab, change the length of the first column (Selling_Group_Code) to 111. On the Properties tab, change the First Line is Column Names to False.
3. Change the name of the output link from the Copy stage to TF (from Selling_Group_Mapping_Copy).
4. Save the changes to your job.
5. Open up both the CreateSeqJob and the CreateSeqJobComp jobs. Select Window > Tile to display both jobs in a tiled manner.



6. Right-click over your CreateSeqJobComp job name in the Repository window and then select Compare against.



7. In the Compare window select your CreateSeqJob job on the Item Selection window.

8. Click OK to display the Comparison Results window.

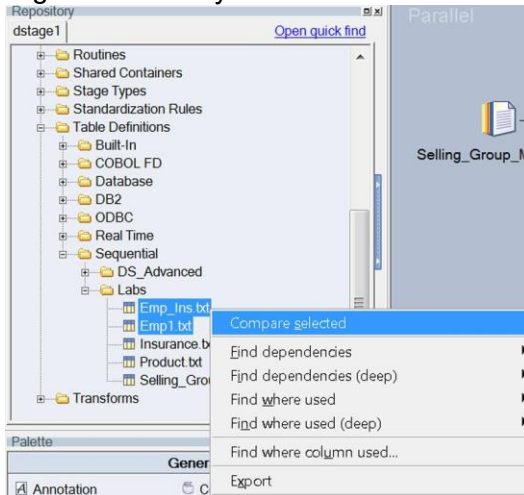
Comparing [CreateSeqJobComp](#) against [CreateSeqJob](#)

- Job Properties (1 Change)
 - Property **Name** was **Changed** from [CreateSeqJob](#) to [CreateSeqJobComp](#)
- Stages (7 Changes)
 - Selling_Group_Mapping_Copy (2 Changes)
 - Inputs (2 Changes)
 - Selling_Group_Mapping_Copy (1 Change)
 - was **Removed**
 - TF (1 Change)
 - was **Added**
 - Selling_Group_Mapping (3 Changes)
 - Outputs (3 Changes)
 - Selling_Group_Mapping (3 Changes)
 - Properties (1 Change)
 - Property **First Line is Column Names** was **Changed** from [firstLineColumnNames](#) to [False](#)
 - Column Changes (2 Changes)
 - Selling_Group_Code (2 Changes)
 - Property **Precision** was **Changed** from [10](#) to [111](#)
 - Property **Length/Type Info** was **Changed** from [Integer\(10\)](#) to [Integer\(111\)](#)
 - Copy_2 (2 Changes)
 - Outputs (2 Changes)
 - Selling_Group_Mapping_Copy (1 Change)
 - was **Removed**
 - TF (1 Change)
 - was **Added**

9. Click on the active link where there are differences and let the link take you to the item where the change was made.

Task: Find the differences between two Table Definitions

1. Select the Emp_Ins.txt and the Emp1.txt Table Definitions from the folder Sequential > Labs.
2. Right-click over your Table Definitions and then select 'Compare selected'.



3. The Comparison Results window is displayed.

Comparing Emp1.txt against Emp_Ins.txt

- Properties (3 Changes)
 - Property **Long description** was **Changed** from **Imported from: /DS_Fundamentals/Labs/Emp_Ins.txt (Map=UTF8) (as seen from INFOSRV) - 10/27/2011 10:56:18 AM** to **Imported from: /DS_Fundamentals/Labs/Emp1.txt (Map=UTF8) (as seen from INFOSRV) - 10/27/2011 10:57:01 AM**
 - Property **Locator** was **Changed** from
TableType="Sequential" | Computer="infosrv" | SoftwareProduct="FileSystem" | DataStore(Directory)="/DS_Fundamentals/Labs" | DataSchema(File)="Emp_Ins.txt" | DataCollection="Emp_Ins.txt" to
TableType="Sequential" | Computer="infosrv" | SoftwareProduct="FileSystem" | DataStore(Directory)="/DS_Fundamentals/Labs" | DataSchema(File)="Emp1.txt" | DataCollection="Emp1.txt"
 - Property **Name** was **Changed** from **Sequential\Labs\Emp_Ins.txt** to **Sequential\Labs\Emp1.txt**
- Columns (7 Changes)
 - EmpName (1 Change)
 - Property **Display** was **Changed** from **20** to **7**
 - Location (1 Change)
 - Property **Position** was **Changed** from **3** to **5**
 - DeptID (1 Change)
 - was **Removed**
 - DOB (1 Change)
 - was **Removed**
 - DOJ (1 Change)
 - was **Removed**
 - Company (1 Change)
 - was **Added**
 - Role (1 Change)
 - was **Added**

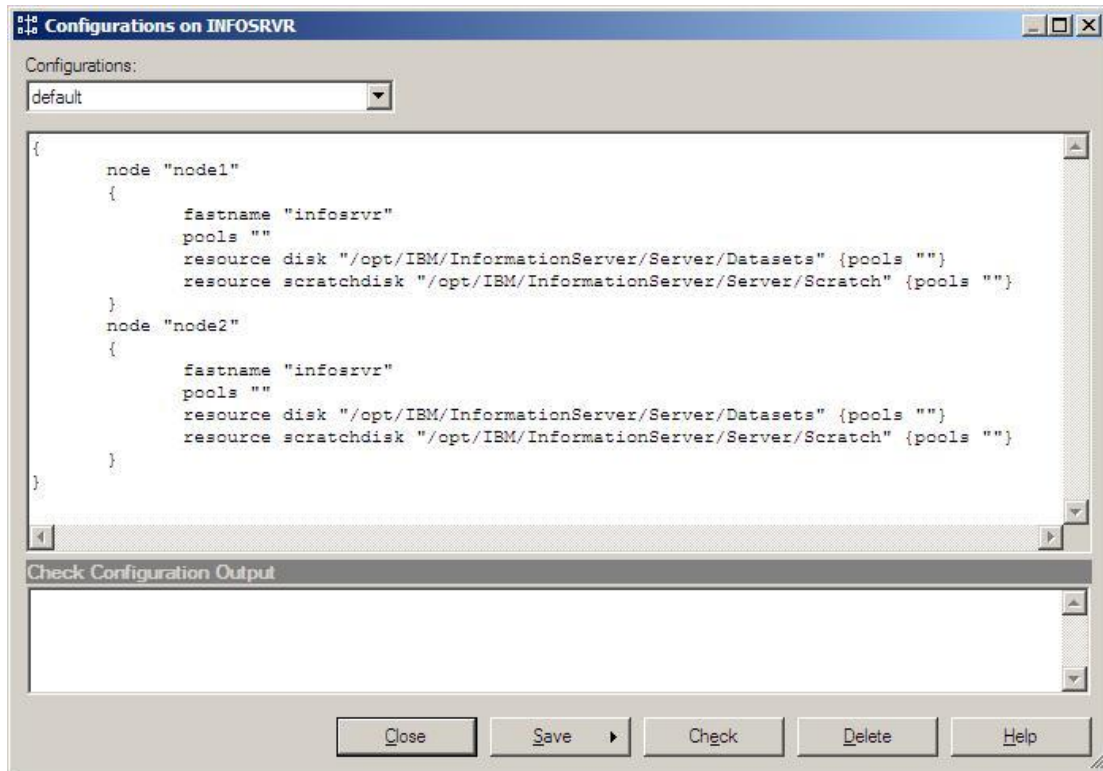
4. Click on the active link where there are differences and let the link take you to the item where the change was made.

Lab 05: DataStage Parallel Architecture

Task: Using a two-node configuration file

The lab exercises are more instructive when the jobs are executed with a two-node (or more) configuration file.

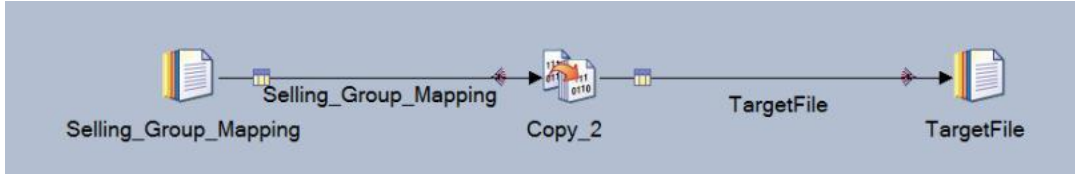
1. Click Tools > Configurations.
2. In the Configurations box, select the default configuration. You might want to expand the window so that the lines do not wrap to make them easier to understand.
3. Your file should look like the picture below with two nodes already defined. If only one node is listed, make a copy of the node definition through the curly braces, i.e. text from the 1st "node" to the first "}", paste it right after the end of the definition section for node1, and change the name of the new node to "node2". Be careful you only have a total of 3 pairs of the curly brackets; one encloses all the nodes, one encloses the node1 definitions, and one encloses the node2 definitions.



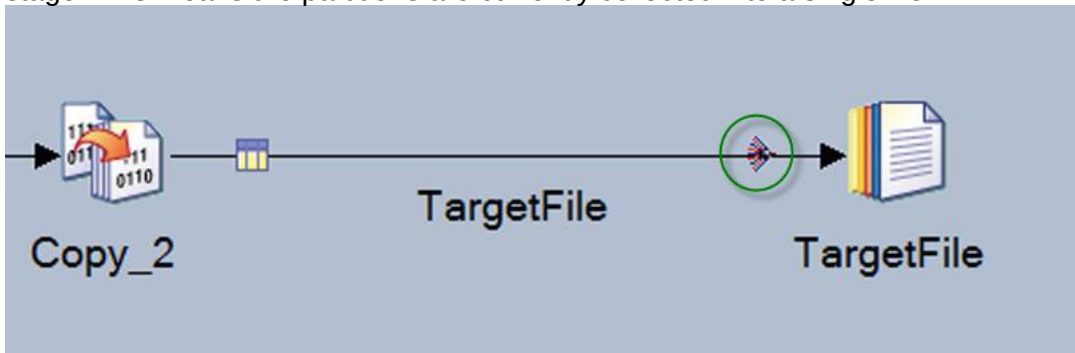
4. Save only if you have made the changes. Click Close.

Task: Using data partitioning and collecting

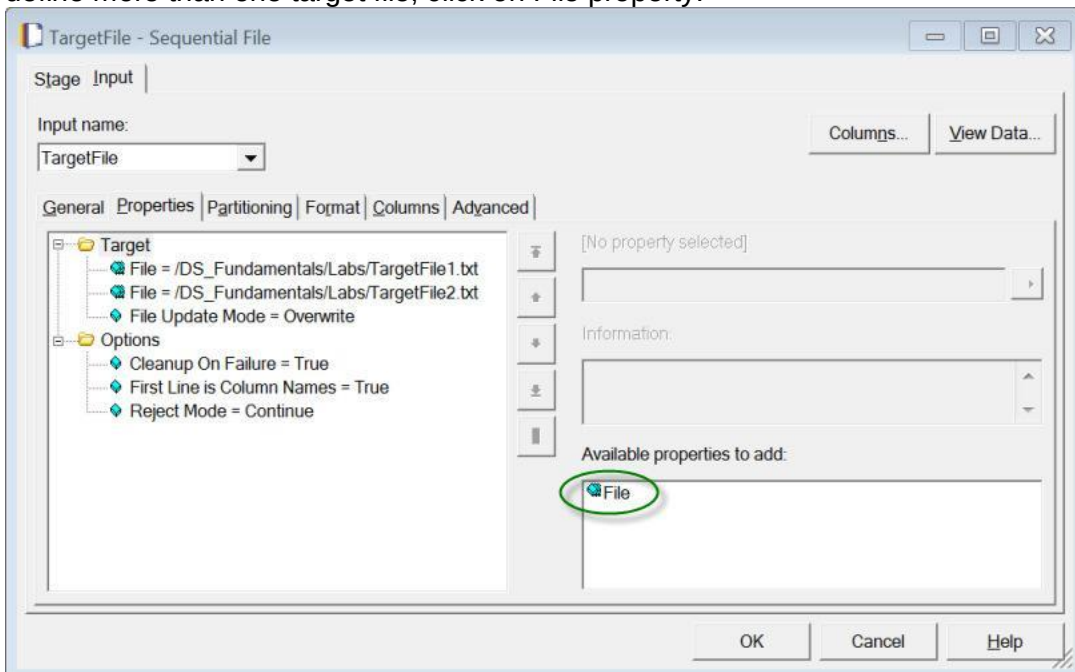
1. Open the CreateSeqJob and save it as CreateSeqJobPartition. Rename the target sequential file stage and input link to 'TargetFile'.



2. Note how the partitioning indicator is showing the 'fan in' symbol before the target stage. This means the partitions are currently collected into a single file.



3. In the target Sequential File stage, define two files, TargetFile1.txt and TargetFile2.txt, in order to see how DataStage data partitioning works. To define more than one target file, click on File property.

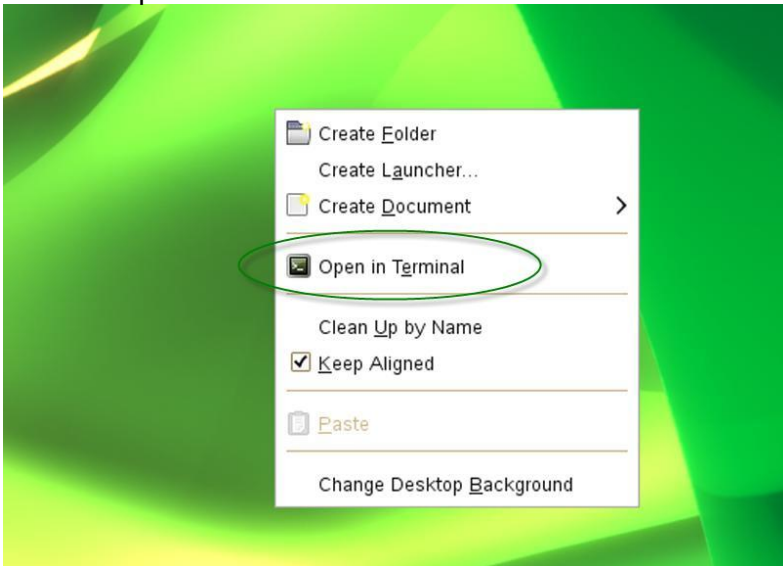


4. Note how the partitioning indicator on the link changed to 'Auto'



5. Compile and run your job.
6. View the job log. Notice how the data is exported to the two different partitions (0 and 1).

7. Go to the /DS_Fundamentals/Labs/ folder and explore the content of files, Selling_Group_Mapping.txt, TargetFile1.txt and TargetFile2.txt.
8. Steps on how to edit a file on the RHEL Linux VMWare image:
9. Login in as dsadm / inf0server to your RHEL VMWare server.
10. Once you are logged on, open a terminal window with the right mouse button over the desktop.



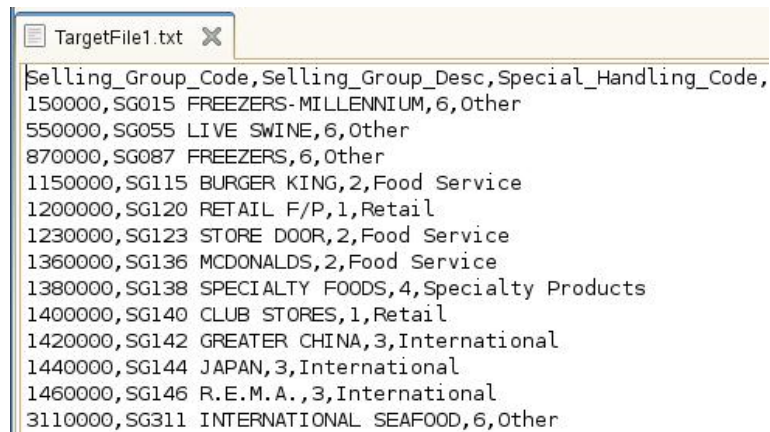
11. Type “gedit /DS_Fundamentals/Labs/Selling_Group_Mapping.txt” to open the text file editor.

Source file:

```

Selling_Group_Mapping.txt
Selling_Group_Code,Selling_Group_Desc,Special_Handling_Code,Distr_Chann_Desc
150000,SG015 FREEZERS-MILLENNIUM,6,Other
530000,SG053 TRUCKING,6,Other
550000,SG055 LIVE SWINE,6,Other
860000,SG086 TRUCKING,6,Other
870000,SG087 FREEZERS,6,Other
1120000,SG112 N.W.A. SWINE,6,Other
1150000,SG115 BURGER KING,2,Food Service
1190000,SG119 DISTRIBUTION CVP,2,Food Service
1200000,SG120 RETAIL F/P,1,Retail
1210000,SG121 RETAIL FRESH,1,Retail
1230000,SG123 STORE DOOR,2,Food Service
1250000,SG125 FOOD SERVICE,2,Food Service
1360000,SG136 MCDONALDS,2,Food Service
1370000,SG137 EQUITY,6,Other
1380000,SG138 SPECIALTY FOODS,4,Specialty Products
1390000,SG139 MILITARY,1,Retail
1400000,SG140 CLUB STORES,1,Retail
1410000,SG141 INTERNATIONAL,3,International
1420000,SG142 GREATER CHINA,3,International
1430000,SG143 AMERICAS,3,International
1440000,SG144 JAPAN,3,International
    
```

Target file 1:

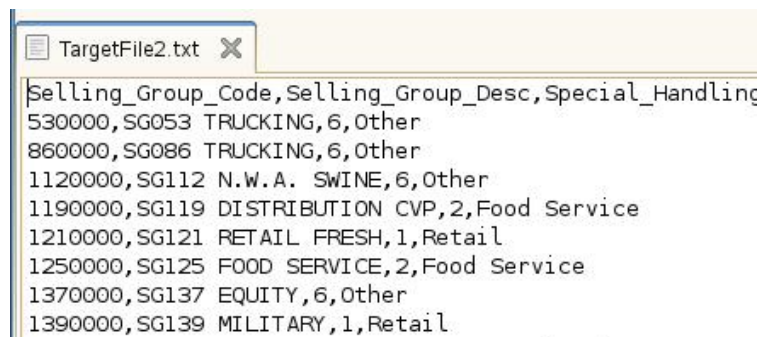


```

Selling_Group_Code,Selling_Group_Desc,Special_Handling_Code,
150000,SG015 FREEZERS-MILLENNIUM,6,Other
550000,SG055 LIVE SWINE,6,Other
870000,SG087 FREEZERS,6,Other
1150000,SG115 BURGER KING,2,Food Service
1200000,SG120 RETAIL F/P,1,Retail
1230000,SG123 STORE DOOR,2,Food Service
1360000,SG136 MCDONALDS,2,Food Service
1380000,SG138 SPECIALTY FOODS,4,Specialty Products
1400000,SG140 CLUB STORES,1,Retail
1420000,SG142 GREATER CHINA,3,International
1440000,SG144 JAPAN,3,International
1460000,SG146 R.E.M.A.,3,International
3110000,SG311 INTERNATIONAL SEAFOOD,6,Other

```

Target file 2:



```

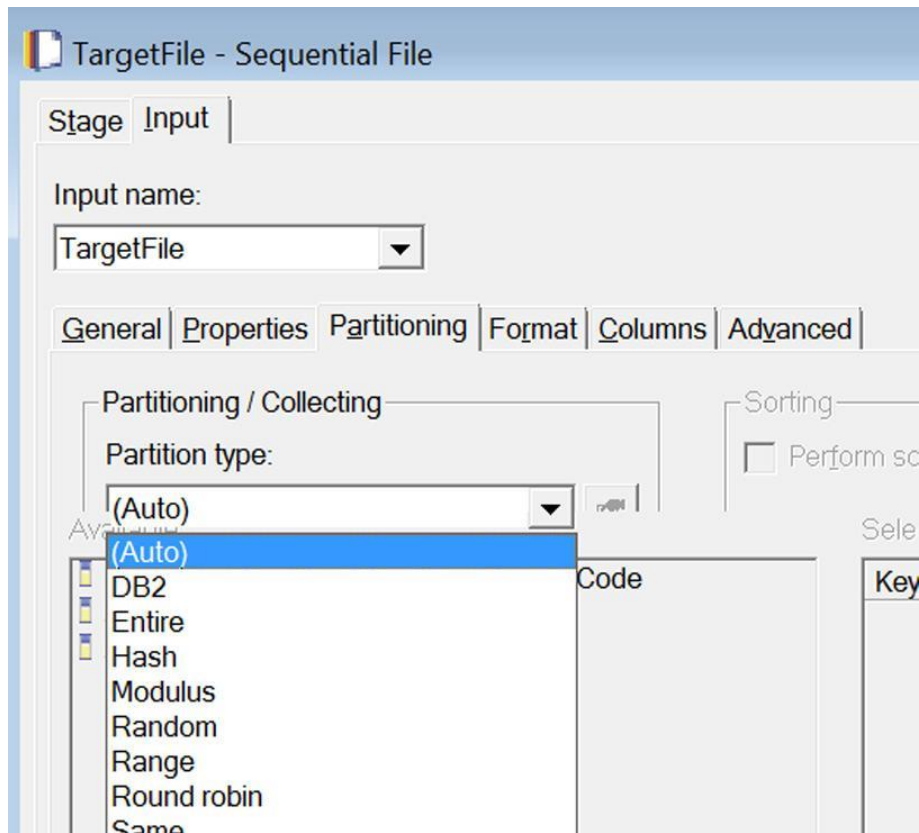
Selling_Group_Code,Selling_Group_Desc,Special_Handling_Code,
530000,SG053 TRUCKING,6,Other
860000,SG086 TRUCKING,6,Other
1120000,SG112 N.W.A. SWINE,6,Other
1190000,SG119 DISTRIBUTION CVP,2,Food Service
1210000,SG121 RETAIL FRESH,1,Retail
1250000,SG125 FOOD SERVICE,2,Food Service
1370000,SG137 EQUITY,6,Other
1390000,SG139 MILITARY,1,Retail

```

Notice how the data is partitioned. Here, we see that the 1st, 3rd, 5th, etc. go into one file and the 2nd, 4th, 6th, etc. go in the other file. This is because the default partitioning algorithm is Round Robin.

Task: Experiment with different partitioning methods

1. Open the target Sequential File stage. Go to the 'Partitioning' tab. Change the partitioning algorithm setting to various settings, e.g. ENTIRE, RANDOM, and HASH.



2. Compile and run the job again. Open the target files and examine. Notice how the data gets distributed. Experiment with different partitioning algorithms!

3. The following table shows the results for several partitioning algorithms.

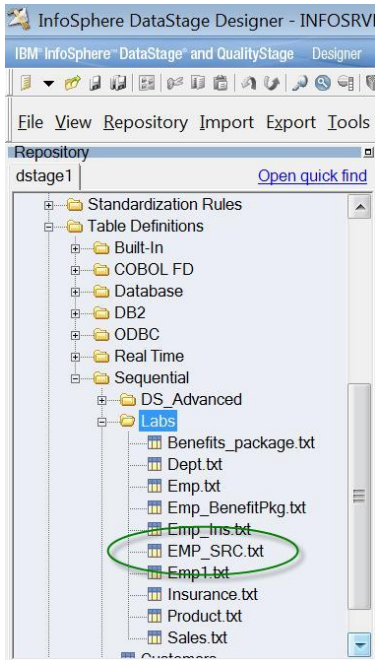
Partitioning Algorithm	Records in File1	Records in File2	Comments
Round-Robin (Auto)	23	24	
Entire	47	47	Each file contains all the records
Random	22	25	random distribution (record # results may vary)
Hash on column "Special_Handling_Code"	20	27	File 1 with Special_Handling_code 6; File 2 with other Special_Handling_codes

Lab 06: Sequential Data Access

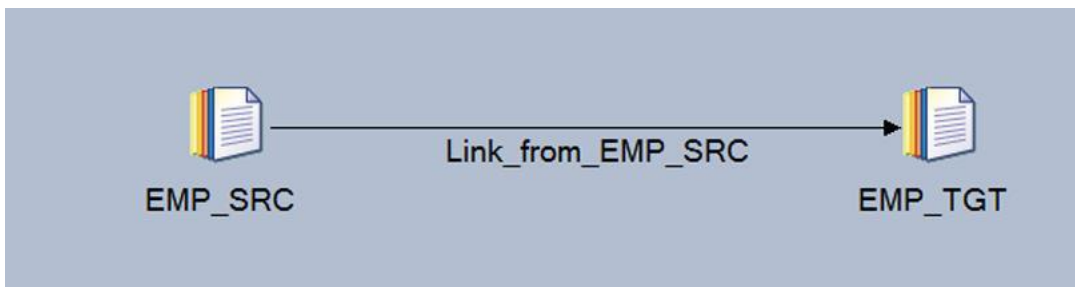
Task: Write data to and read data from a sequential file

In this task, we will create a job that will read data from a sequential file and write data to a sequential file.

1. Browse Repository window for a Table Definition called EMP_SRC.txt. The Table Definitions are located in the "Table Definitions" Folder Sequential Labs. It should be in the folder.

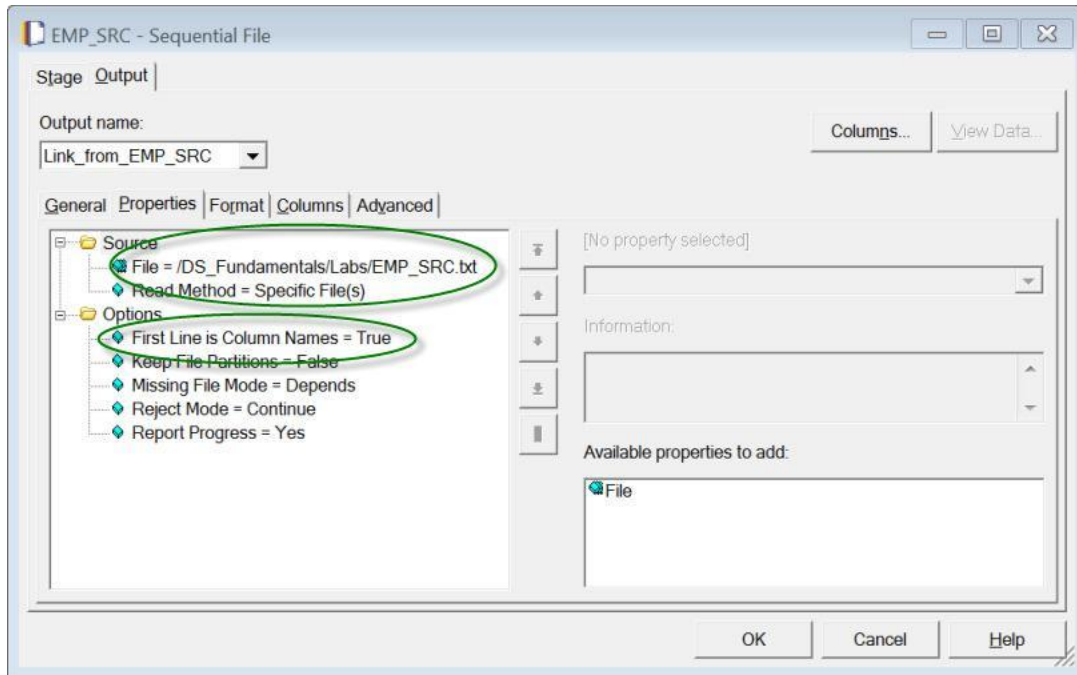


2. Create a new parallel job named SeqEmp as shown.

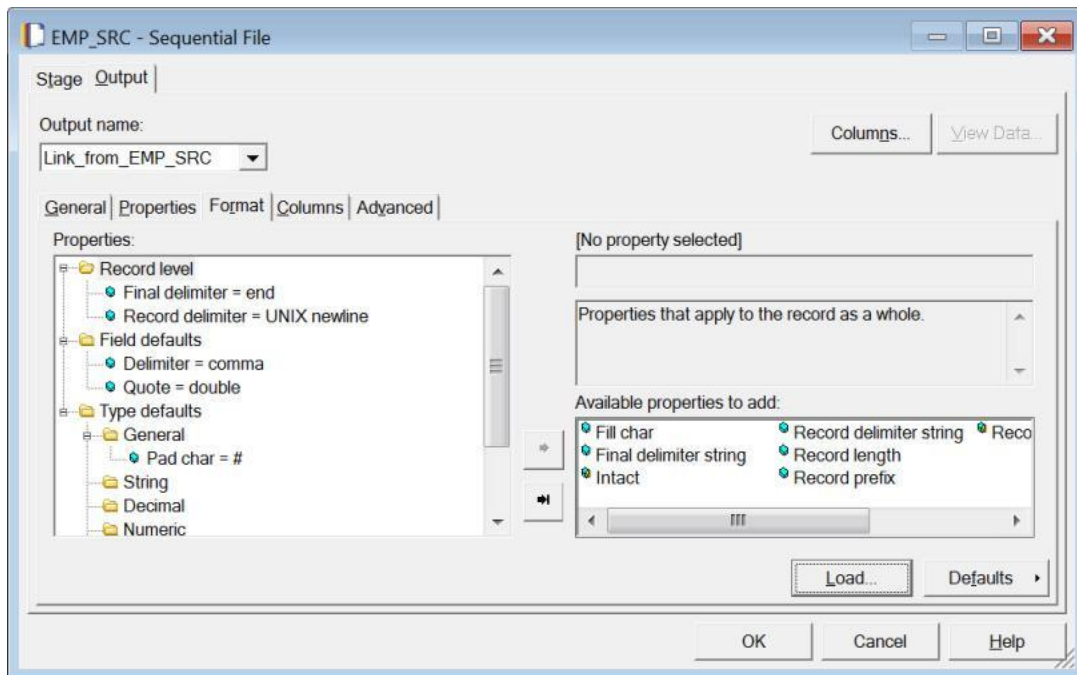


3. Rename the stage and link names as shown for good standard practice.

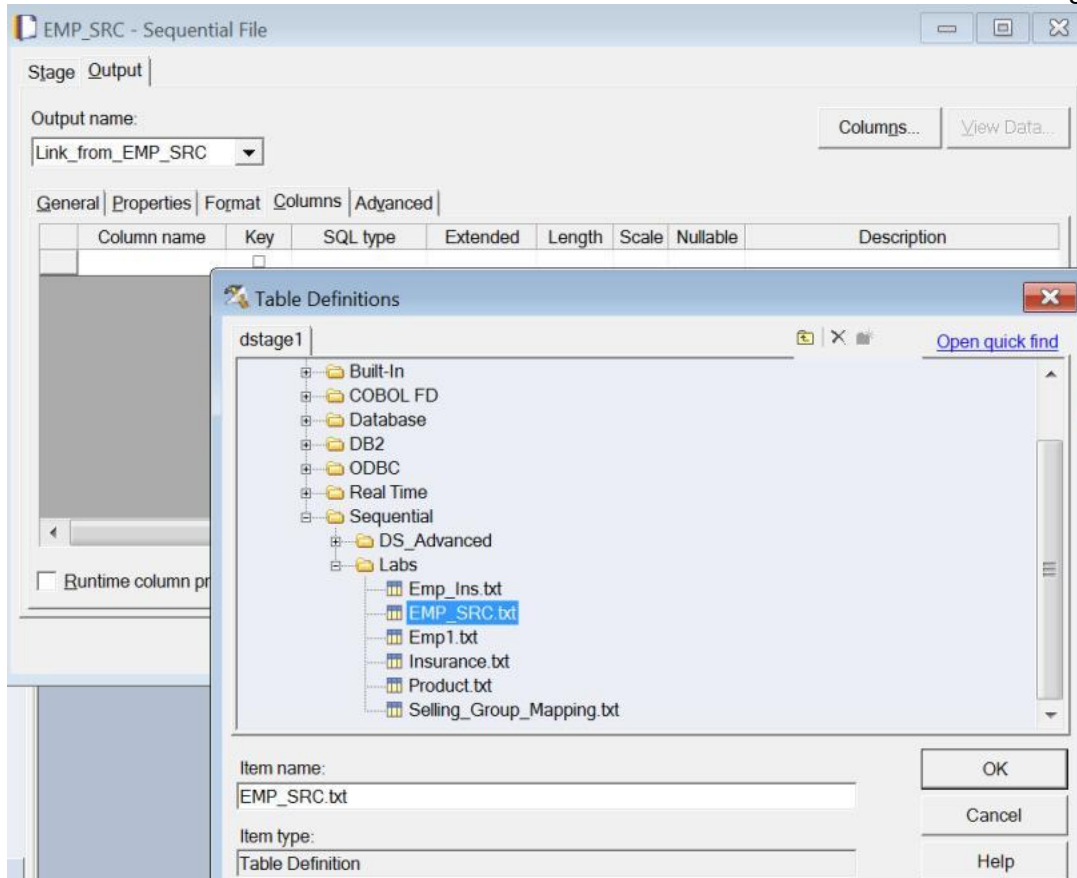
4. Edit the source Sequential File stage to enter the properties as shown below.



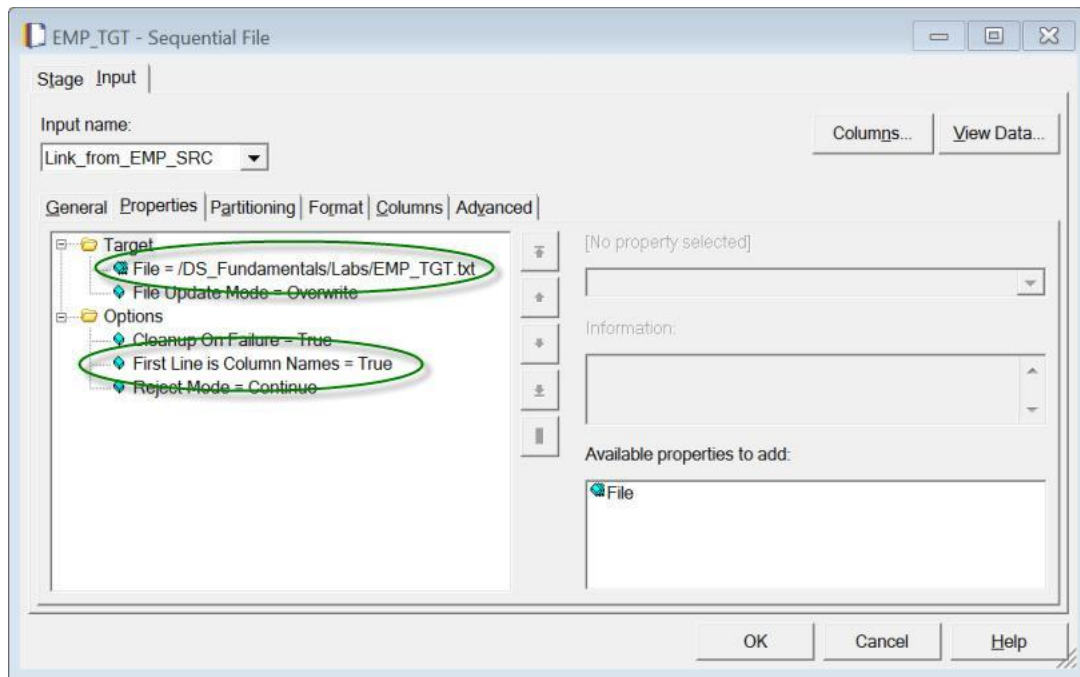
5. Click on the Format tab and click on the Load button and locate the table definition of the sequential file (EMP_SRC.txt) from the repository. Click OK to load.



6. In the columns tab, click on the Load button and locate the table definition of the sequential file (EMP_SRC.txt) from the repository. Click OK twice to load the columns into the columns tab. Click OK to close the stage.



7. In the target Sequential File stage, enter the values as shown in the properties tab.



8. Click OK.
9. Compile and run the job.
10. Verify the source and target data by right-clicking on the source and target stages and selecting View Lnk_frm_EMP_SRC data. They should be identical.

SeqEmp..EMP_TGT.Link_from_EMP_SRC - Data Browser

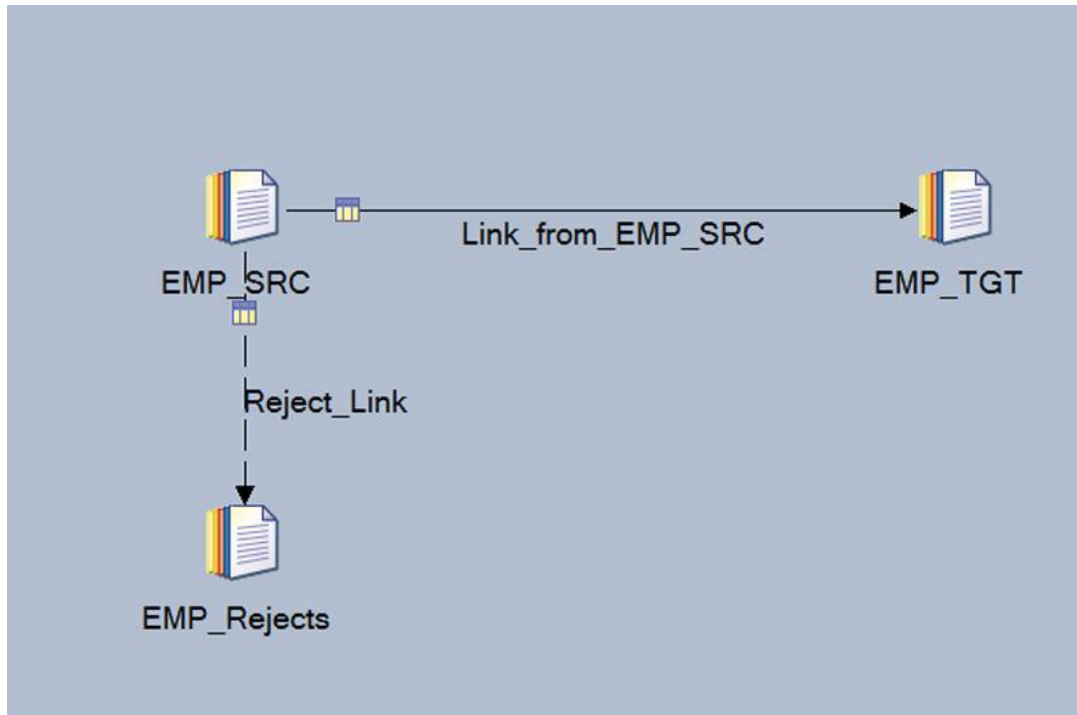
EMPNO	ENAME	JOB	MGR	HIREDATE	SAL	COMM	DEPTNO
7369	SMITH	CLERK	7902	1980-12-17	000000080	000000020	20
7499	ALLEN	CLERK	7698	1981-02-20	000000160	000000030	30
7521	WARD	SALESMAN	7698	1981-02-22	000000120	000000005	30
7566	JONES	ANALYST	7839	1981-04-02	000000290	000000020	20
7654	MARTIN	SALESMAN	7698	1981-09-28	000000120	000000014	30
7698	BLAKE	MANAGER	7839	1981-05-01	000000280	000000020	30
7782	CLARK	MANAGER	7839	1981-06-09	000000240	000000020	10
7788	SCOTT	ANALYST	7566	1982-12-09	000000300	000000020	20
7839	KING	PRESIDENT	7566	1981-11-17	000000500	000000020	10
7844	TURNER	SALESMAN	7698	1981-09-08	000000150	000000000	30
7876	ADAMS	CLERK	7788	1983-01-12	000000110	000000020	20
7900	JAMES	CLERK	7698	1981-12-03	000000090	000000020	30
7902	FORD	ANALYST	7566	1981-12-03	000000300	000000020	20
7934	MILLER	CLERK	7782	1982-01-23	000000130	000000020	10

Close Find... Display... Help

Task: Reject link of a Sequential File stage

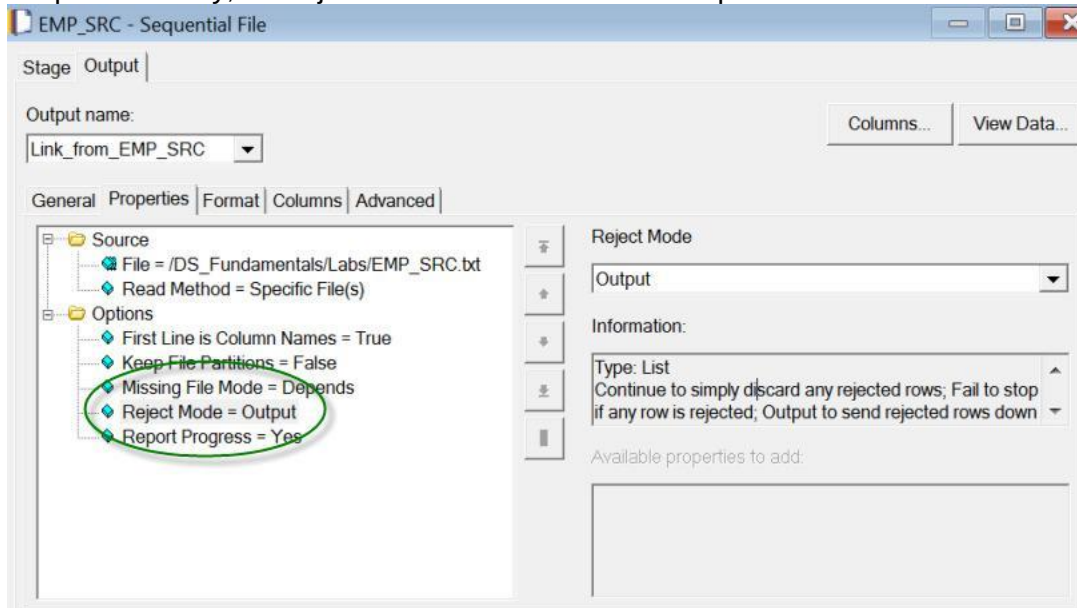
In this task, we will add a reject link to the source stage to capture the records which are rejected due to formatting error.

1. We will use the existing job SeqEmp and save it as RejectEmp. Add a reject link to the source stage as shown.



2. Rename the stage and link names as shown for good standard practice.

3. Edit the EMP_SRC Sequential File stage and set the property Reject Mode to Output. This way, the rejected records will flow to a sequential file.



4. Edit the source file EMP_SRC.txt to add some wrong data, such as additional column values 'abc' and 'pqr' in the rows with the keys 7369 and 7521.

```
*EMP_SRC.txt
EMPNO,ENAME,JOB,MGR,HIREDATE,SAL,COMM,DEPTNO
7369,abc,SMITH,CLERK,7902,1980-12-17,00800.00,200.00,20
7499,ALLEN,CLERK,7698,1981-02-20,01600.00,00300.00,30
7521,pqr,WARD,SALESMAN,7698,1981-02-22,01250.00,00500.00,30
7566,JONES,ANALYST,7839,1981-04-02,02975.00,200.00,20
```

5. Save the file after you have completed the changes.
6. Keep the gedit window open as we will use it to examine results and correct the file back after this lab is completed.
7. Modify the Sequential File stage EMP_Rejects to write the output to a file EMP_Reject.txt.



8. On the Format tab, change the Quote property to none. Click OK.

9. Save and compile the job. Run the job and view the job log. The result will be as shown below. In order to see the number of records on the links, don't forget to turn on the Show performance statistics for the job from the canvas if they are not there. .

The screenshot displays the IBM DataStage job execution interface. The top pane shows the job canvas with a Sequential File stage named EMP_SRC connected to a Sequential File stage named EMP_TGT via a link labeled 'Link_from_EMP_SRC: 12 rows, 0 rows/sec'. A 'Reject_Link: 2 rows, 0 rows/sec' connects EMP_SRC to a Sequential File stage named EMP_Rejects. The bottom pane shows the job log with various messages, including import warnings and completion messages for EMP_SRC, EMP_TGT, and EMP_Rejects. A green circle highlights the warning messages in the log.

Type	Occurred	Event
	2/16/2012 ...	main_program: Dump: ...
	2/16/2012 ...	main_program: APT configuration file: /opt/IBM/InformationServer/Server/Configurations/default...
	2/16/2012 ...	main_program: This step has 2 datasets: ...
	2/16/2012 ...	main_program: Schemas: ...
	2/16/2012 ...	EMP_SRC,0: Field "MGR" has import error and no default value; data: {S A L E S M A N}, at offset: 1...
	2/16/2012 ...	EMP_SRC,0: Import warning at record 2. [new-impexp/file_import.C:3452]
	2/16/2012 ...	EMP_SRC,0: Field "MGR" has import error and no default value; data: {M A N A G E R}, at offset: 15...
	2/16/2012 ...	EMP_SRC,0: Import warning at record 6. [new-impexp/file_import.C:3452]
	2/16/2012 ...	EMP_SRC,0: Import complete; 12 records imported successfully, 2 rejected.
	2/16/2012 ...	EMP_TGT,0: Export complete; 12 records exported successfully, 0 rejected.
	2/16/2012 ...	EMP_TGT,0: Input 0 consumed 12 records.
	2/16/2012 ...	EMP_Rejects,0: Export complete; 2 records exported successfully, 0 rejected.
	2/16/2012 ...	EMP_Rejects,0: Input 0 consumed 2 records.
	2/16/2012 ...	EMP_SRC,0: Output 0 produced 12 records. ...
	2/16/2012 ...	main_program: Step execution finished with status = OK

10. Open the EMP_Reject.txt file to view the rejected records. Use the gedit command in the VMWare image.

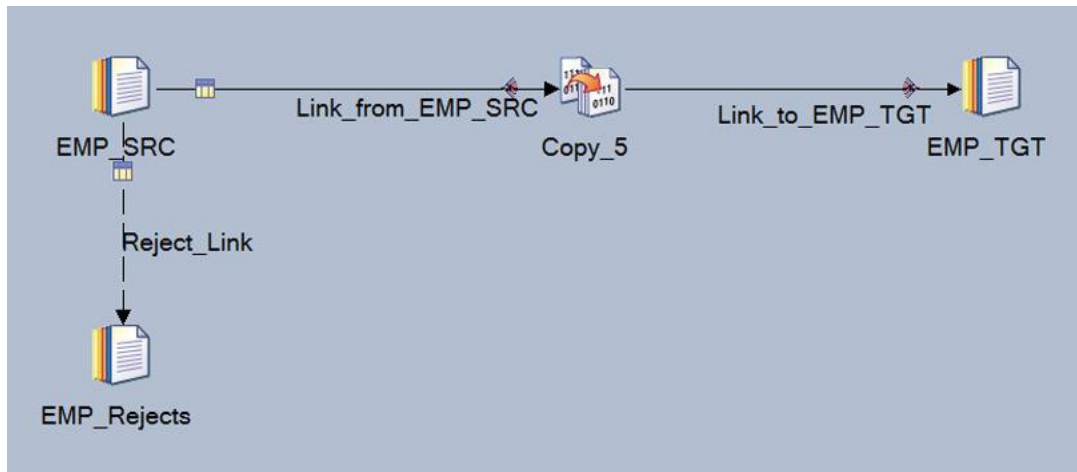
The screenshot shows the gedit text editor with the file EMP_Reject.txt open. The file contains two lines of rejected records in a comma-separated format.

```
7521,123,WARD,SALESMAN,7698,1981-02-22,01250.00,00500.00,30
7782,abc,CLARK,MANAGER,7839,1981-06-09,02450.00,200.00,10
```

Task: Handling NULL values in a Sequential File stage

In this task, we will create a job that will read data from a sequential file and write to another sequential file. We will also see how NULL values can be interpreted to be read as NULL from the source and written as assigned value to the target.

1. Save the job as NullEmp and add a Copy stage between the Sequential File stages as shown.

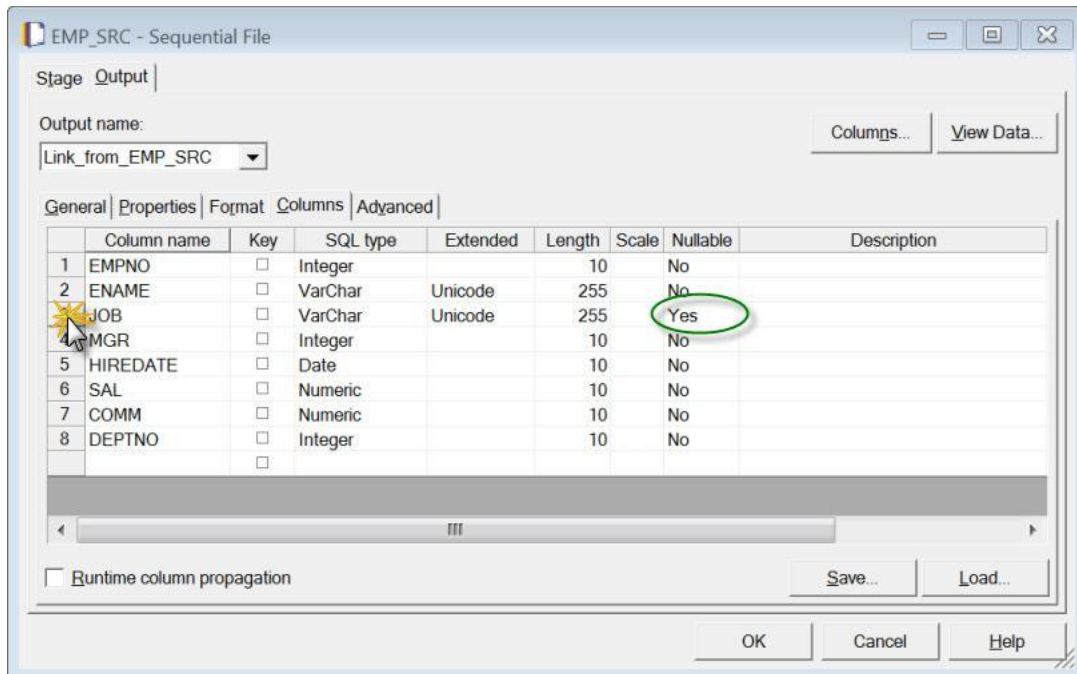


2. Edit the source file EMP_SRC.txt to add null values (empty string) to the JOB column in the second and fourth row. Also, correct the two rows that have the extra data inserted by removing them. Save the changes.

```

EMP_SRC.txt
EMPNO,ENAME,JOB,MGR,HIREDATE,SAL,COMM,DEPTNO
7369,SMITH,CLERK,7902,1980-12-17, 00800.00,200.00,20
7499,ALLEN,,7698,1981-02-20, 01600.00, 00300.00,30
7521,WARD,SALESMAN,7698,1981-02-22, 01250.00, 00500.00
7566,JONES,,7839,1981-04-02, 02975.00,200.00,20
7654,MARTIN,SALESMAN,7698,1981-09-28, 01250.00, 01400.
7698,BLAKE,MANAGER,7839,1981-05-01, 02850.00,200.00,30
  
```

- Click the Columns tab of the source Sequential File stage. In the row with Column name "JOB", change the field Nullable to Yes. Then, double-click the column number 3 (to the left of the column name) to open up the Edit Column Meta Data window.



- On the Properties section, click on Nullable and then add the Null field value property. Here, we will treat the empty string as meaning NULL. To do this specify "" (back-to-back double quotes). Click on Apply and then Close to close the window.

Edit Column Meta Data

Column name: Key:

SQL type: Length: Scale:

Nullable: Native type:

Description:

Server | COBOL | Parallel | Analytical information |

Field type: Extended (Unicode) ☒

Vector occurs: Variable ☐

Level number:

Properties:

- Field level
 - String type (Unicode)
 - Nullable
 - Null field value = ""

[No property selected]

Properties that apply to fields as a whole:

Available properties to add:

- Bytes to skip
- Delimiter
- Delimiter string
- Drop on input
- Prefix bytes
- Print field
- Quote
- Start position

< Previous Next > Close Apply Reset Help

- Map all the columns from input to output in the Copy stage.

6. In the properties window, make sure the File property is set to point to the /DS_Fundamentals/Labs/EMP_TGT.txt target file.
7. Click the Columns tab of the target Sequential File stage. In the row with Column name "JOB", change the field Nullable to 'Yes' in case it's set to 'No'. Then, double-click the column number 3 (to the left of the column name) to open up the Edit Column Meta Data window.
8. On the Properties section, click on Nullable and then add the Null field value property. Here, we will write the string "NO JOB" when a NULL is encountered. Click on Apply and then Close to close the window.

Edit Column Meta Data

Column name: Key:

SQL type: Length: Scale:

Nullable: Native type:

Description:

Server | COBOL | Parallel | Analytical information

Field type: ☒ Extended (Unicode)

Vector occurs: ☐ Variable

Level number:

Properties:

- Field level
 - String type (Unicode)
 - Nullable
 - Null field value = NO JOB

Null field value

On import, the value given to a field containing a null. On export, the value given to an exported field if the source field is set to null.

Available properties to add:

< Previous Next > Close Apply Reset Help

9. Compile and Run the job.

10. View the data at the source Sequential File stage by right-click on the stage and selecting View Link_from_EMP_SRC data. Notice the word “NULL” in those two records with the empty string. This is because you have told DataStage that the empty string represent a NULL value.

NullEmp..EMP_SRC.Link_from_EMP_SRC - Data Browser

EMPNO	ENAME	JOB	MGR	HIREDATE	SAL	COMM	DEPTNO
7369	SMITH	CLERK	7902	1980-12-17	000000080	000000020	20
7499	ALLEN	NULL	7698	1981-02-20	000000160	000000030	30
7521	WARD	SALESMAN	7698	1981-02-22	000000120	000000050	30
7566	JONES	NULL	7839	1981-04-02	000000290	000000020	20
7654	MARTIN	SALESMAN	7698	1981-09-28	000000120	000000140	30
7698	BLAKE	MANAGER	7839	1981-05-01	000000280	000000020	30
7782	CLARK	MANAGER	7839	1981-06-09	000000240	000000020	10
7788	SCOTT	ANALYST	7566	1982-12-09	000000300	000000020	20
7839	KING	PRESIDENT	7566	1981-11-17	000000500	000000020	10
7844	TURNER	SALESMAN	7698	1981-09-08	000000150	000000000	30
7876	ADAMS	CLERK	7788	1983-01-12	000000110	000000020	20
7900	JAMES	CLERK	7698	1981-12-03	000000090	000000020	30
7902	FORD	ANALYST	7566	1981-12-03	000000300	000000020	20
7934	MILLER	CLERK	7782	1982-01-23	000000130	000000020	10

Close Find... Display... Help

11. Now view the data at the target Sequential File stage by right-click on the stage and selecting View Link_to_EMP_TGT data. Notice the two records still have the word NULL. This is because we are still looking at the data from DataStage.

NullEmp..EMP_TGT.Link_to_EMP_TGT - Data Browser

EMPNO	ENAME	JOB	MGR	HIREDATE	SAL	COMM	DEPTNO
7499	ALLEN	NULL	7698	1981-02-20	000000160	000000030	30
7566	JONES	NULL	7839	1981-04-02	000000290	000000020	20
7698	BLAKE	MANAGER	7839	1981-05-01	000000280	000000020	30
7788	SCOTT	ANALYST	7566	1982-12-09	000000300	000000020	20
7844	TURNER	SALESMAN	7698	1981-09-08	000000150	000000000	30
7900	JAMES	CLERK	7698	1981-12-03	000000090	000000020	30
7934	MILLER	CLERK	7782	1982-01-23	000000130	000000020	10
7369	SMITH	CLERK	7902	1980-12-17	000000080	000000020	20
7521	WARD	SALESMAN	7698	1981-02-22	000000120	000000050	30
7654	MARTIN	SALESMAN	7698	1981-09-28	000000120	000000140	30
7782	CLARK	MANAGER	7839	1981-06-09	000000240	000000020	10
7839	KING	PRESIDENT	7566	1981-11-17	000000500	000000020	10
7876	ADAMS	CLERK	7788	1983-01-12	000000110	000000020	20
7902	FORD	ANALYST	7566	1981-12-03	000000300	000000020	20

Close Find... Display... Help

12. Now go to the VMWare image and view the actual file EMP_TGT.txt with gedit. You will see that the records contain the string that we assigned, "NO JOB", to represent a NULL value.

```

7369,"SMITH","CLERK",7902,"1980-12-17", 0000000800., 0000000200.,20
7499,"ALLEN","",7698,"1981-02-20", 0000001600., 0000000300.,30
7521,"WARD","SALESMAN",7698,"1981-02-22", 0000001250., 0000000500.,30
7566,"JONES","",7839,"1981-04-02", 0000002975., 0000000200.,20
7654,"MARTIN","SALESMAN",7698,"1981-09-28", 0000001250., 0000001400.,30
7698,"BLAKE","MANAGER",7839,"1981-05-01", 0000002850., 0000000200.,30
7782,"CLARK","MANAGER",7839,"1981-06-09", 0000002450., 0000000200.,10
7788,"SCOTT","ANALYST",7566,"1982-12-09", 0000003000., 0000000200.,20
7839,"KING","PRESIDENT",7566,"1981-11-17", 0000005000., 0000000200.,10
7844,"TURNER","SALESMAN",7698,"1981-09-08", 0000001500., 0000000000.,30
7876,"ADAMS","CLERK",7788,"1983-01-12", 0000001100., 0000000200.,20
7900,"JAMES","CLERK",7698,"1981-12-03", 0000000950., 0000000200.,30
7902,"FORD","ANALYST",7566,"1981-12-03", 0000003000., 0000000200.,20
7934,"MILLER","CLERK",7782,"1982-01-23", 0000001300., 0000000200.,10
7369,"SMITH","CLERK",7902,"1980-12-17", 0000000800., 0000000200.,20
7499,"ALLEN","",7698,"1981-02-20", 0000001600., 0000000300.,30
7521,"WARD","SALESMAN",7698,"1981-02-22", 0000001250., 0000000500.,30
7566,"JONES","",7839,"1981-04-02", 0000002975., 0000000200.,20
7654,"MARTIN","SALESMAN",7698,"1981-09-28", 0000001250., 0000001400.,30
7698,"BLAKE","MANAGER",7839,"1981-05-01", 0000002850., 0000000200.,30
7782,"CLARK","MANAGER",7839,"1981-06-09", 0000002450., 0000000200.,10

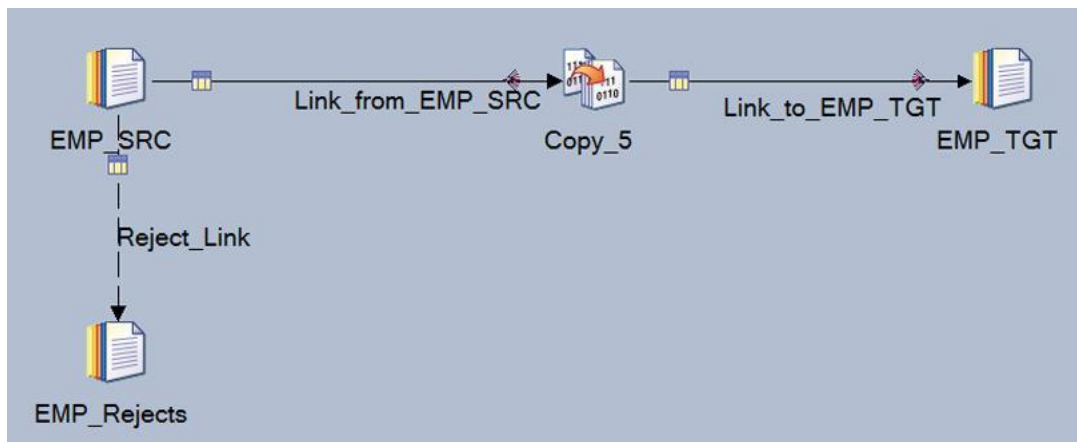
```

13. Go back to the EMP_SRC.txt file and add the missing job values again. For employee number 7499 with name ALLEN, add the job description value CLERK. For employee number 7566 with name JONES, add the job description value ANALYST.

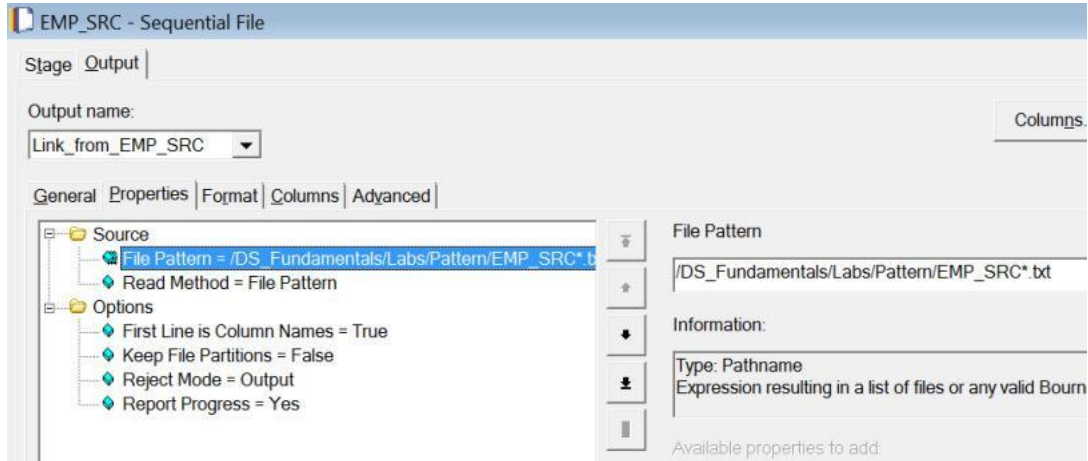
Task: Read data from multiple sequential files using File Pattern

In this task, we will create a job that will read data from multiple sequential files and write to a sequential file. We will use the File Pattern option to read multiple files in a Sequential File stage.

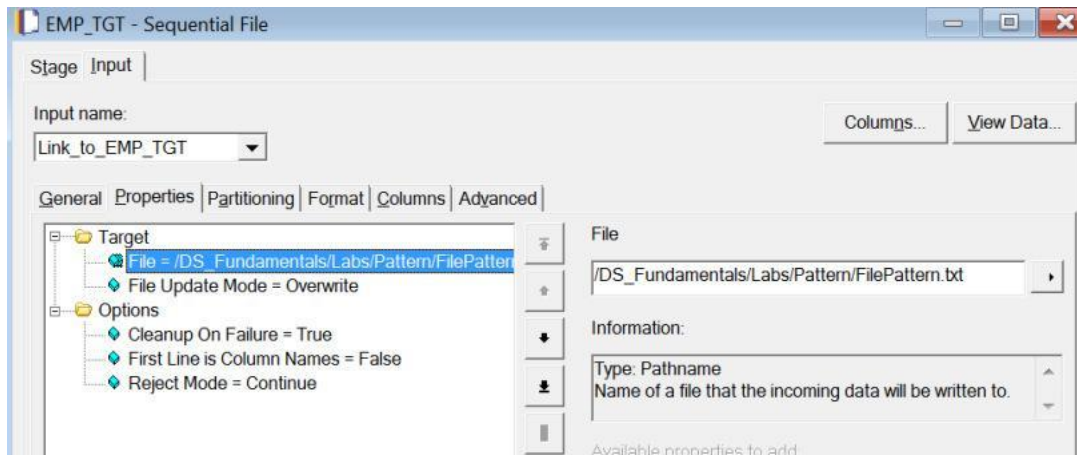
1. Save the previous job RejectEmp as FilePatternEmp.



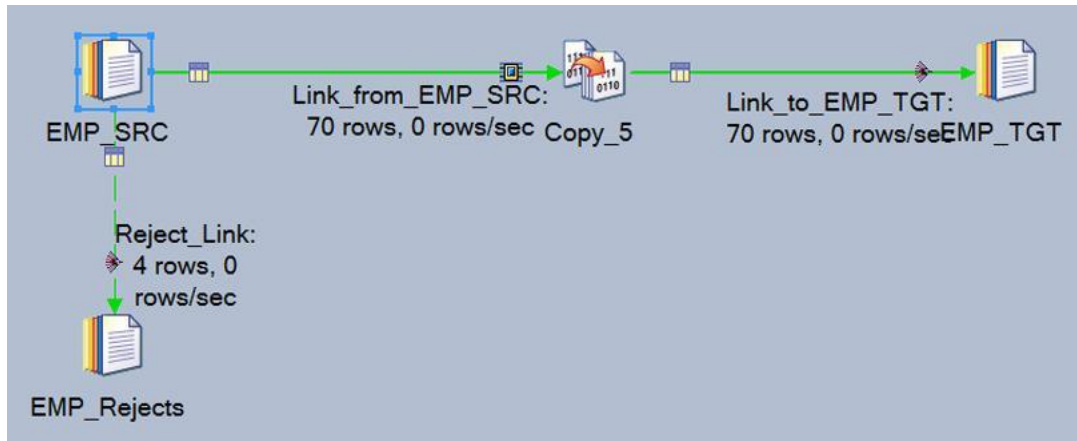
2. Edit the source Sequential File stage Read Method to File Pattern. Accept the warning message.
3. Specify the file path as shown (/DS_Fundamentals/Labs/Pattern/EMP_SRC*.txt). This will read all the files matching the file pattern in the specified directory.



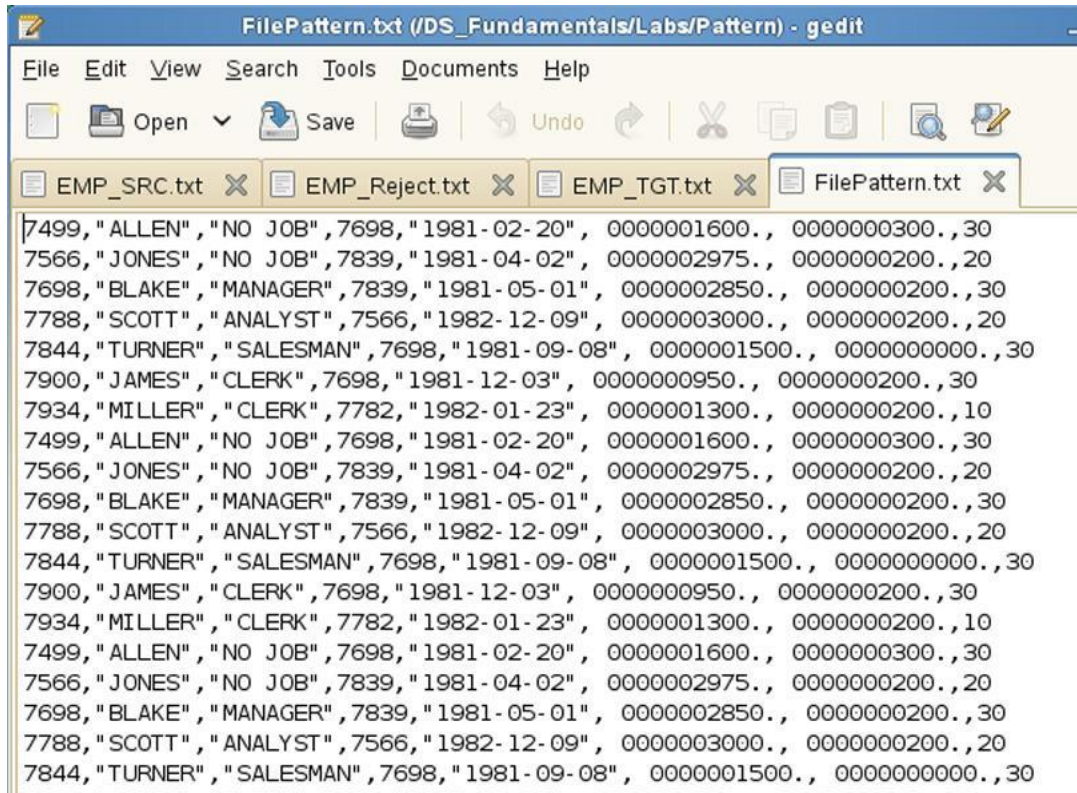
4. Edit the target Sequential File stage to write to the output file FilePattern.txt in directory Pattern. Close the stage editor.



5. Compile and run the job. As can be seen, the source stage reads data from all the source files matching the pattern and writes it to the output file.



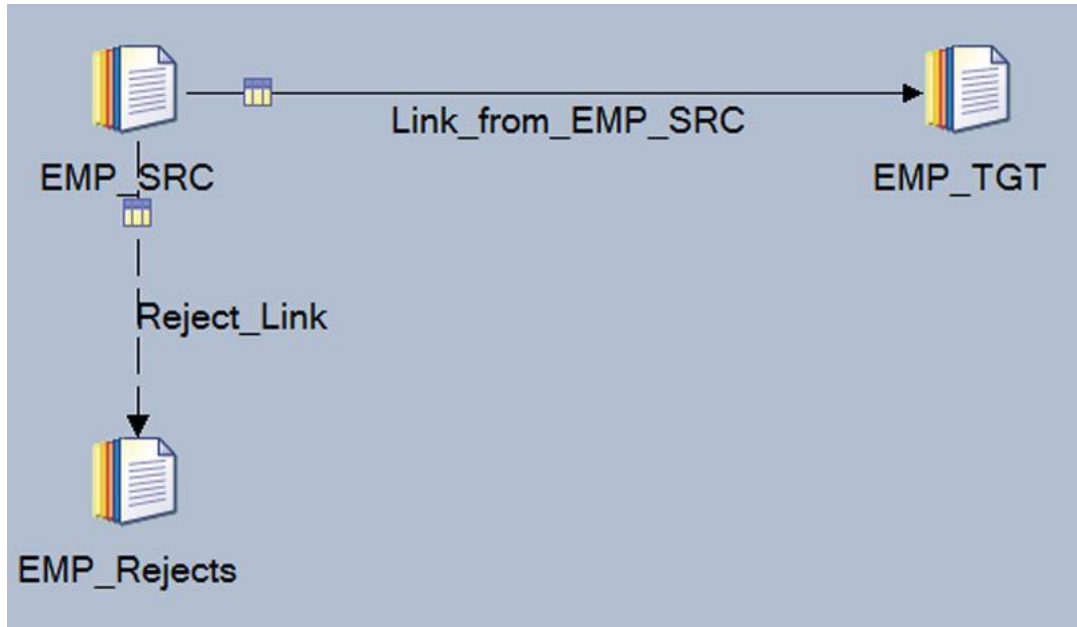
6. Check the results in output file and verify it has all the files that satisfy the file pattern.



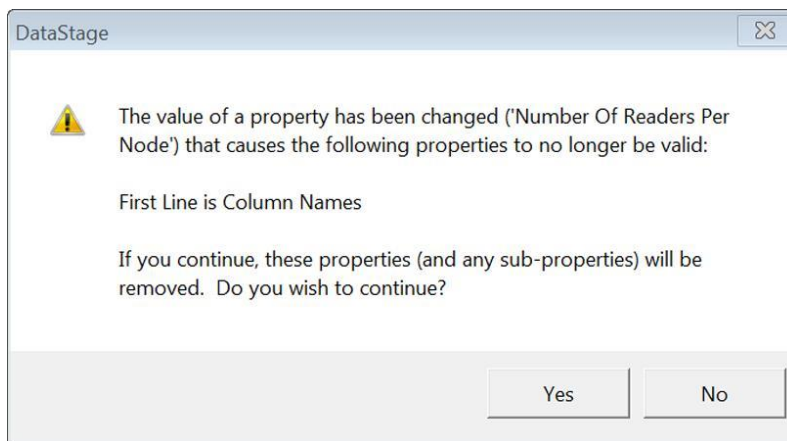
Task: Read data with multiple readers

In this task, we will create a job that will read data from a sequential file and write to another sequential file. We will see how to read a single sequential file in parallel.

1. Save the previous job RejectEmp as MultiReadEmp.



2. Open the Properties tab of the source Sequential File stage. Click the Options folder and add the “Number of Readers Per Node” property. You will get a warning about First line is column names property cannot be retained.

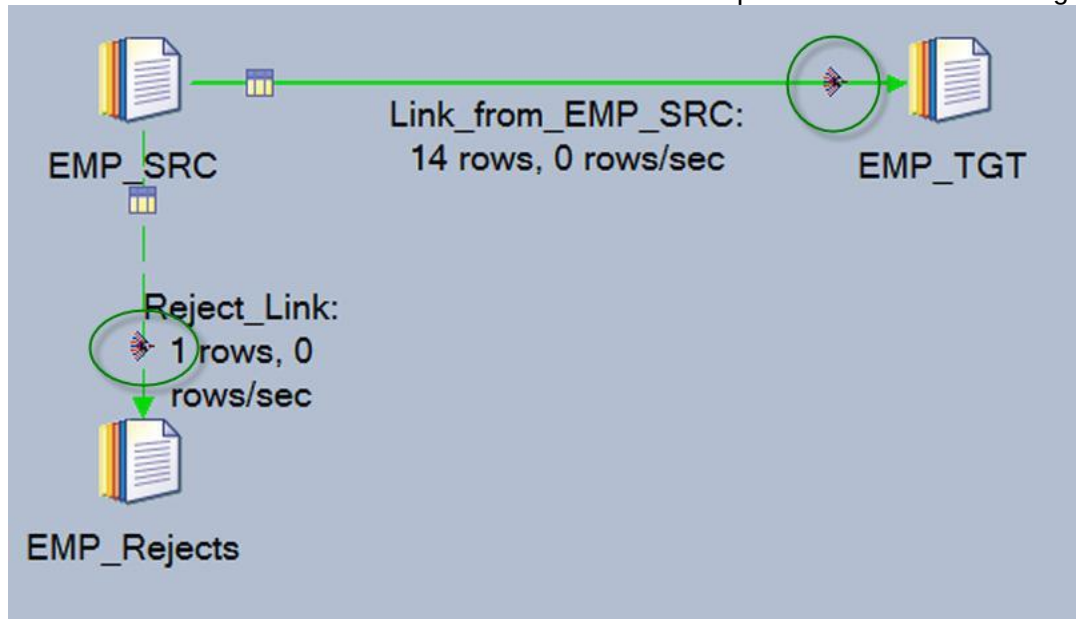


3. Click YES to accept. Set number of readers to 2. Close the stage editor.
4. Compile and run the job.

- View the results in the job log. As the “First Line is Column Names” property is no longer valid, some warning messages are displayed. Although, the first row will be rejected, you can ignore this warning since we know the first record is there and in this specific situation (multiple readers) the property is not valid.

Type	Occurred	Event
	2/17/2012 3:10:57 PM	main_program: conductor uname: -s=Linux; -r=2.6.32.12-0.7-pae; -v=#1 SMP 2010-05-20 11:14:20 +0200; -n=infosrvr; -m=i686
	2/17/2012 3:10:57 PM	main_program: orchgeneral: loaded ...
	2/17/2012 3:10:57 PM	main_program: Echo: ...
	2/17/2012 3:10:57 PM	main_program: Explanation: ...
	2/17/2012 3:10:57 PM	main_program: Dump: ...
	2/17/2012 3:10:57 PM	main_program: APT configuration file: /opt/IBM/InformationServer/Server/Configurations/default.apr ...
	2/17/2012 3:10:57 PM	main_program: This step has 2 datasets: ...
	2/17/2012 3:10:57 PM	main_program: Schemas: ...
	2/17/2012 3:10:57 PM	EMP_SRC,0: Import complete; 7 records imported successfully, 0 rejected.
	2/17/2012 3:10:57 PM	EMP_SRC,1: Output 0 produced 7 records. ...
	2/17/2012 3:10:57 PM	EMP_SRC,0: Field "EMPNO" has import error and no default value; data: {E M P N O}; at offset: 0 [impexp/group_comp.C:8144]
	2/17/2012 3:10:57 PM	EMP_SRC,0: Import warning at record 0. [new-impexp/file_import.C:3452]
	2/17/2012 3:10:57 PM	EMP_SRC,0: Import complete; 7 records imported successfully, 1 rejected.
	2/17/2012 3:10:57 PM	EMP_TGT,0: Export complete; 14 records exported successfully, 0 rejected.
	2/17/2012 3:10:57 PM	EMP_TGT,0: Input 0 consumed 14 records.
	2/17/2012 3:10:57 PM	EMP_Rejects,0: Export complete; 1 records exported successfully, 0 rejected.
	2/17/2012 3:10:57 PM	EMP_SRC,0: Output 0 produced 7 records.
	2/17/2012 3:10:57 PM	EMP_Rejects,0: Input 0 consumed 1 records.
	2/17/2012 3:10:57 PM	EMP_SRC,0: Output 1 produced 1 records.
	2/17/2012 3:10:57 PM	main_program: Step execution finished with status = OK.
	2/17/2012 3:10:57 PM	main_program: Startup time, 0:00; production run time, 0:00.
	2/17/2012 3:10:57 PM	Parallel job reports successful completion
	2/17/2012 3:10:57 PM	Finished Job MultiReadEmp.

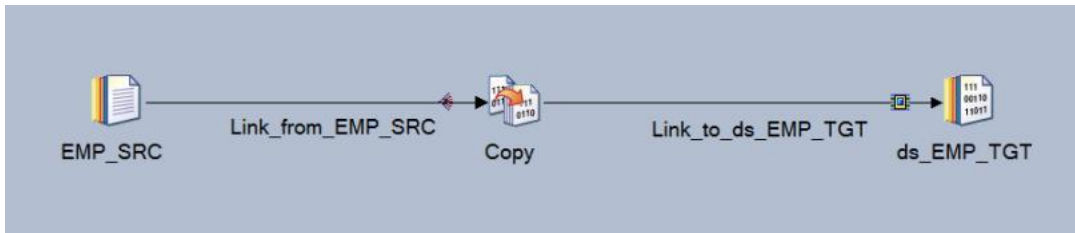
- In the job log, you will find log messages from Import EMP_SRC,0 and EMP_SRC,1. These messages are from reader 1 and reader 2.
- In addition, you can see that DataStage is now collecting the partitions before the data can be written to the Sequential File Stages.



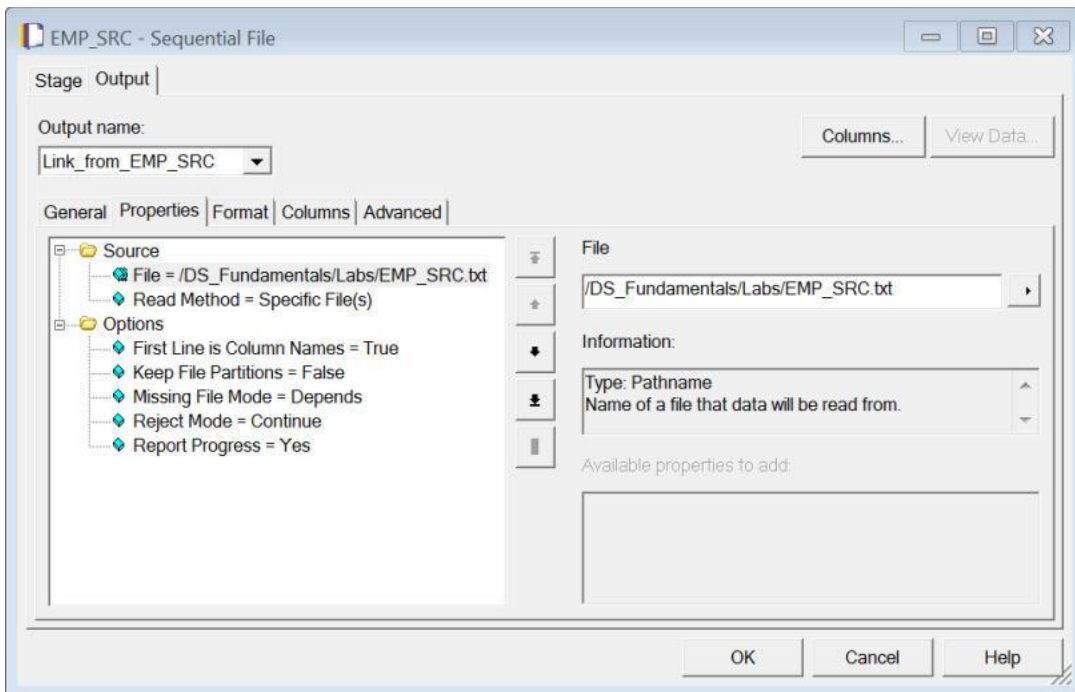
Task: Write data to a Data Set

In this task, we will create a job that will read data from a sequential file and write to a data set.

1. Create a new parallel job named DatasetEmp as shown.

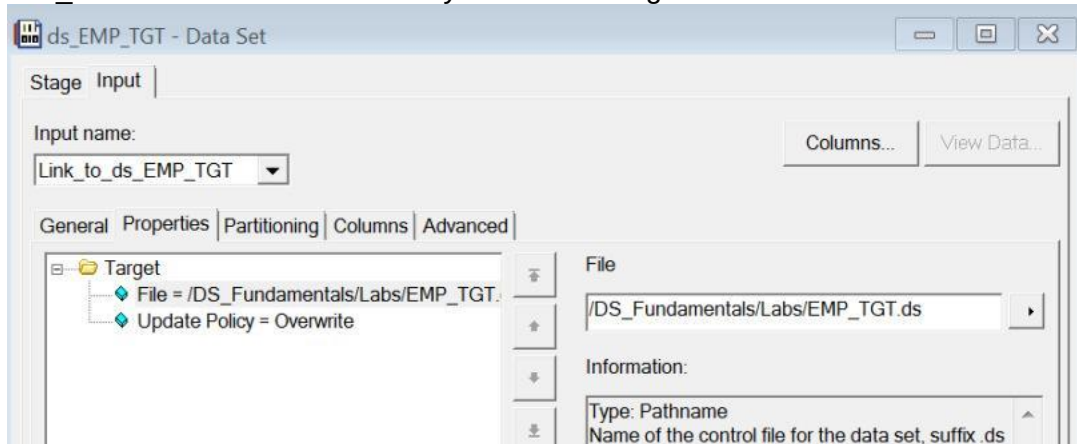


2. Rename the stage and link names as shown for good standard practice.
3. Click the Properties tab of the source Sequential File stage and edit the properties as shown.

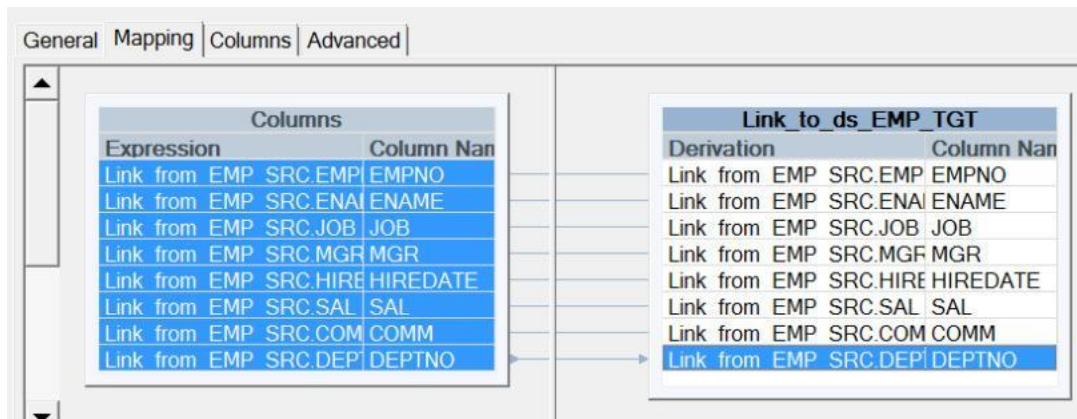


4. Go to both the Format and Columns tabs. On each tab, click Load to load the table definition EMP_SRC.txt from folder /Table Definitions/Sequential/Labs.

5. Edit the target Dataset stage properties. Write to a file named EMP_TGT.ds in the /DS_Fundamentals/Labs/ directory. Close the stage editor.



6. Map the input columns in the copy stage to the output.



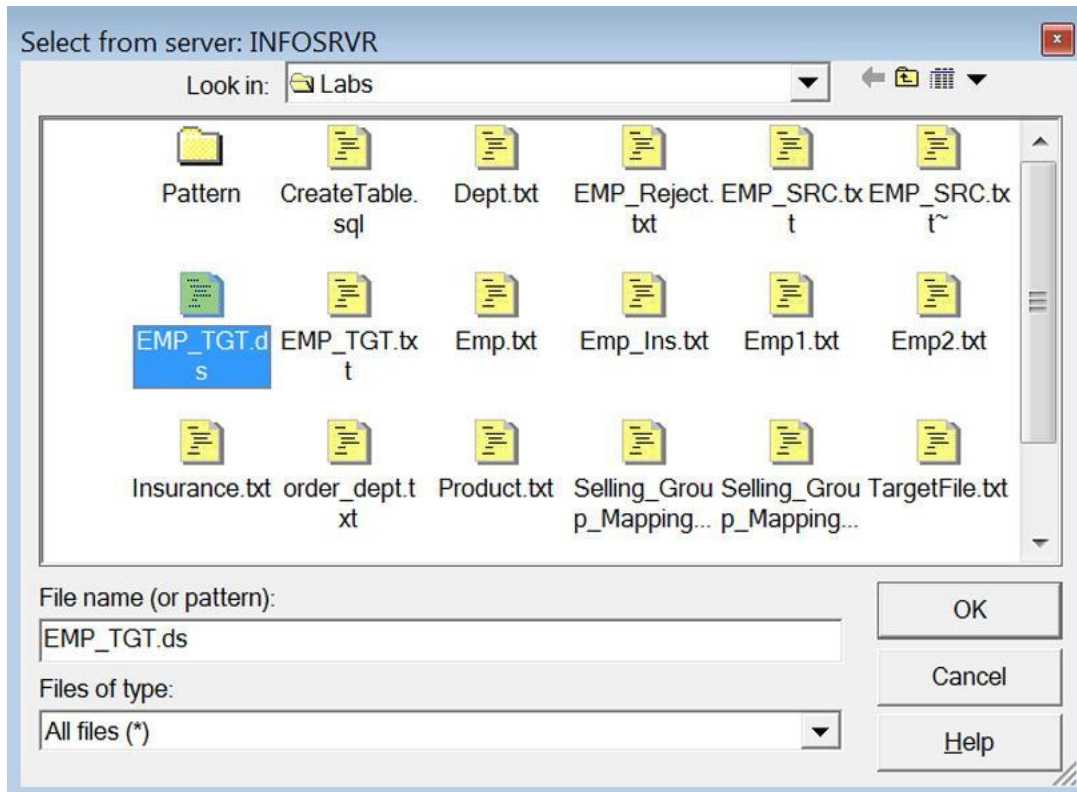
7. Compile and run the job.
8. View the output in the job log.

```

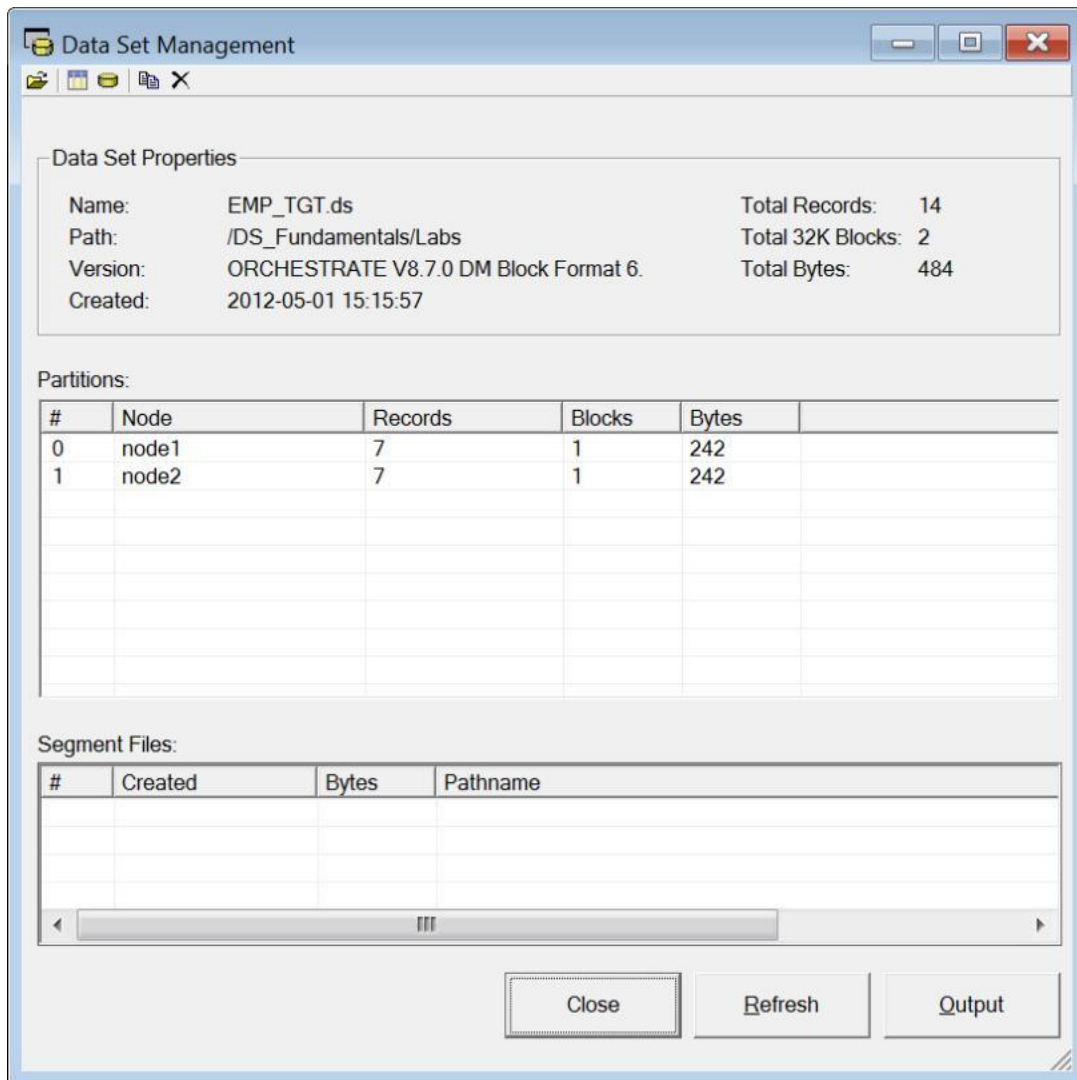
2/17/2012 ... main_program: APT configuration file: /opt/IBM/InformationServer/Server/Configurations/default.apt ...
2/17/2012 ... main_program: This step has 2 datasets: ...
2/17/2012 ... main_program: Schemas: ...
2/17/2012 ... EMP_SRC,0: Import complete; 14 records imported successfully, 0 rejected.
2/17/2012 ... Copy,0: Input 0 consumed 7 records.
2/17/2012 ... Copy,1: Input 0 consumed 7 records.
2/17/2012 ... Copy,1: Output 0 produced 7 records.
2/17/2012 ... EMP_SRC,0: Output 0 produced 14 records.
2/17/2012 ... Copy,0: Output 0 produced 7 records.
2/17/2012 ... ds_EMP_TGT,0: Input 0 consumed 7 records.
2/17/2012 ... ds_EMP_TGT,0: Output 0 produced 7 records.
2/17/2012 ... ds_EMP_TGT,1: Input 0 consumed 7 records.
2/17/2012 ... ds_EMP_TGT,1: Output 0 produced 7 records.
2/17/2012 ... main_program: Step execution finished with status = OK.
2/17/2012 ... main_program: Startup time, 0:03; production run time, 0:00.
2/17/2012 ... Parallel job reports successful completion
2/17/2012 ... Finished Job DatasetEmp.

```

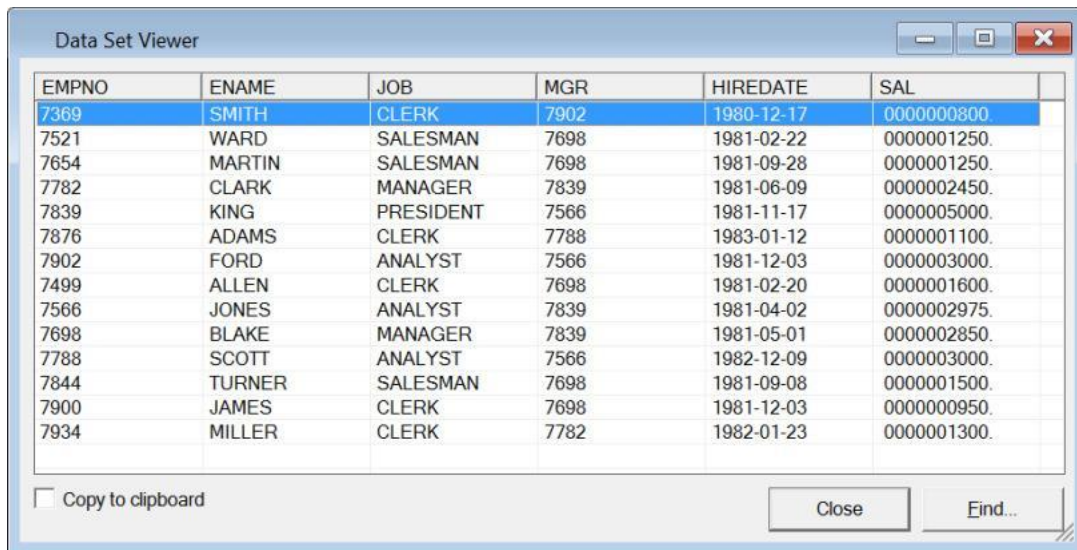
9. In Designer click on Tools > Data Set Management. Select the Data Set that was just created.



10. The Data Set Management window opens up as shown.



11. Click the Show Data at the top to view the data of the Data Set.

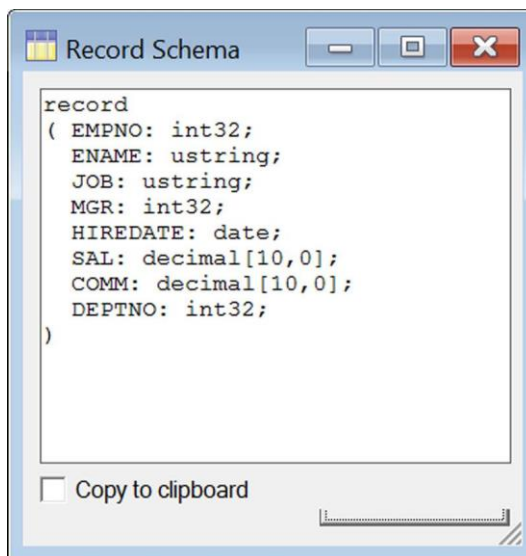


The Data Set Viewer window displays a table with the following columns: EMPNO, ENAME, JOB, MGR, HIREDATE, and SAL. The first row is highlighted in blue.

EMPNO	ENAME	JOB	MGR	HIREDATE	SAL
7369	SMITH	CLERK	7902	1980-12-17	0000000800
7521	WARD	SALESMAN	7698	1981-02-22	0000001250.
7654	MARTIN	SALESMAN	7698	1981-09-28	0000001250.
7782	CLARK	MANAGER	7839	1981-06-09	0000002450.
7839	KING	PRESIDENT	7566	1981-11-17	0000005000.
7876	ADAMS	CLERK	7788	1983-01-12	0000001100.
7902	FORD	ANALYST	7566	1981-12-03	0000003000.
7499	ALLEN	CLERK	7698	1981-02-20	0000001600.
7566	JONES	ANALYST	7839	1981-04-02	0000002975.
7698	BLAKE	MANAGER	7839	1981-05-01	0000002850.
7788	SCOTT	ANALYST	7566	1982-12-09	0000003000.
7844	TURNER	SALESMAN	7698	1981-09-08	0000001500.
7900	JAMES	CLERK	7698	1981-12-03	0000000950.
7934	MILLER	CLERK	7782	1982-01-23	0000001300.

At the bottom of the window, there is a checkbox labeled "Copy to clipboard" and buttons for "Close" and "Find..."

12. Click the Show Schema icon (2nd icon) to view the Data Set schema.



The Record Schema window displays the schema definition for the data set. The schema is defined as follows:

```
record
(
  EMPNO: int32;
  ENAME: ustring;
  JOB: ustring;
  MGR: int32;
  HIREDATE: date;
  SAL: decimal[10,0];
  COMM: decimal[10,0];
  DEPTNO: int32;
)
```

At the bottom of the window, there is a checkbox labeled "Copy to clipboard" and a scroll bar.

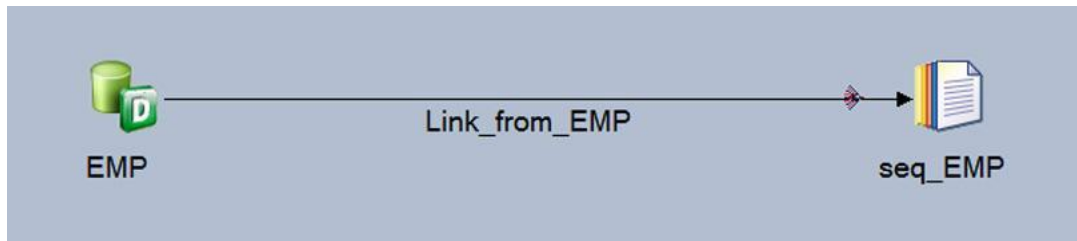
13. Close the Dataset Management Utility.

Lab 07: Relational (RDBMS) Data Access

Task: Read data from a DB2 UDB table using a DB2 Connector stage

In this task, we will create a job that reads data from a DB2 UDB table and loads them into a sequential file. We will use a DB2 Connector stage to read data from the DB2 database table.

1. Create a new parallel job named DB2ConnTableToSeqFile as shown.



2. Rename the stage and link names as shown for good standard practice.
3. Edit the DB2 Connector stage to enter the properties as shown below. User db2inst1/inf0server

EMP - DB2 Connector

Stage Output

Output name (downstream stage)
Link_from_EMP (seq_EMP)

General Properties Columns Advanced

Connection Test Load Save

Instance	db2inst1
Database *	SAMPLE
User name	db2inst1
Password	*****
DB2 client library file	
▶ Alternate conductor settings	No
Use direct connections	No
Keep conductor connection alive	Yes

Usage View Data

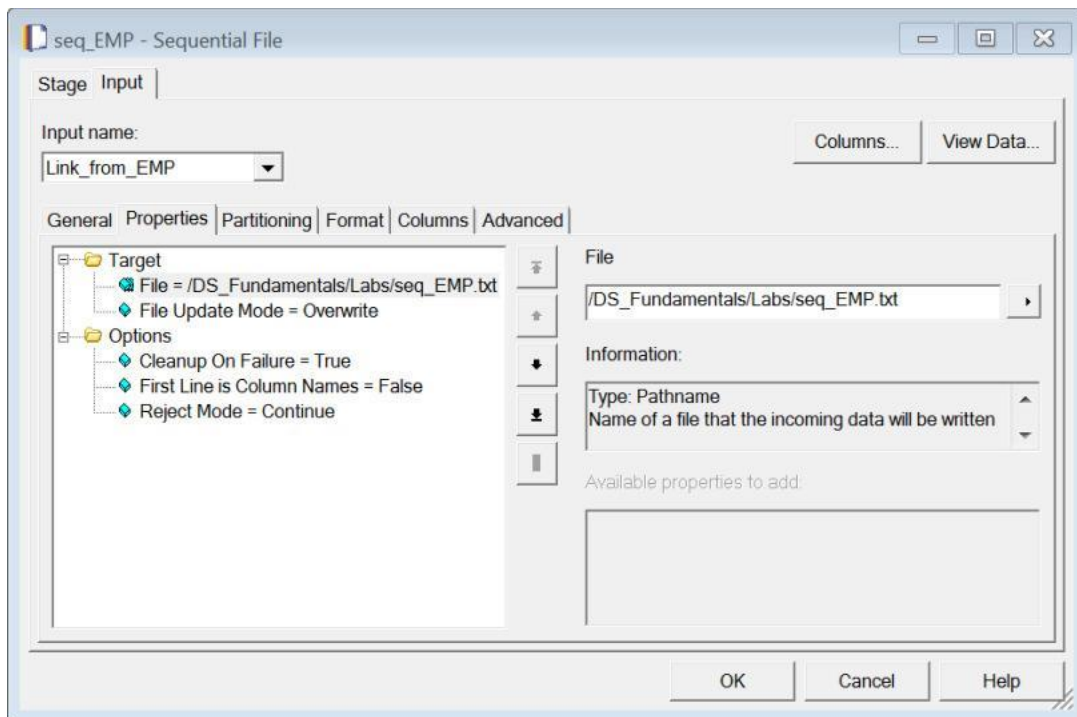
Generate SQL	Yes
Table name *	EMP
Enable quoted identifiers	No
Prefix for expression columns *	EXPR
▼ SQL	
▶ Select statement *	
▶ Enable partitioned reads	No
▼ Transaction	
Record count	2000
▶ End of wave	None
▼ Session	
Isolation level *	Cursor stability

OK Cancel Help

- Load the table definition in the Columns tab. Click on load and then select EMP under the Table Definitions/Database folder. Close the stage editor.

	Column name	Key	SQL type	Extended	Length	Scale	Nullable	Data element	Description
1	EMPNO	☒	Char	Unicode	6		No		
2	FIRSTNME	☐	VarChar	Unicode	12		No		
3	MIDINIT	☐	Char	Unicode	1		Yes		
4	LASTNAME	☐	VarChar	Unicode	15		No		
5	WORKDEPT	☐	Char	Unicode	3		Yes		
6	PHONENO	☐	Char	Unicode	4		Yes		
7	HIREDATE	☐	Date		10		Yes		
8	JOB	☐	Char	Unicode	8		Yes		
9	EDLEVEL	☐	SmallInt		2		No		
10	SEX	☐	Char	Unicode	1		Yes		
11	BIRTHDATE	☐	Date		10		Yes		
12	SALARY	☐	Decimal		9	2	Yes		
13	BONUS	☐	Decimal		9	2	Yes		
14	COMM	☐	Decimal		9	2	Yes		

- Edit the target Sequential File stage to write the data into the seq_EMP.txt file.



- On the Format tab, specify comma as the field delimiter and quote as none.
- Save and compile the job.

- Run the job and view the data by right-clicking on the target stage and select View Link_from_EMP data.

DB2ConnTableToSeqFile..seq_EMP.Link_from_EMP - Data Browser

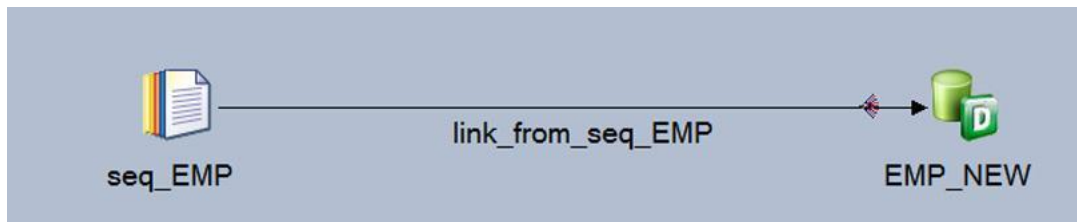
EMPNO	FIRSTNAME	MIDINIT	LASTNAME	WORKDEPT	PHONENO	HIREDATE	JOB	EDLEVEL	SEX	BIRTHDATE
000010	CHRISTINE	I	HAAS	A00	3978	1995-01-01	PRES	18	F	1963-08-
000020	MICHAEL	L	THOMPSON	B01	3476	2003-10-10	MANAGER	18	M	1978-02-
000030	SALLY	A	KWAN	C01	4738	2005-04-05	MANAGER	20	F	1971-05-
000050	JOHN	B	GEYER	E01	6789	1979-08-17	MANAGER	16	M	1955-09-
000060	IRVING	F	STERN	D11	6423	2003-09-14	MANAGER	16	M	1975-07-
000070	EVA	D	PULASKI	D21	7831	2005-09-30	MANAGER	16	F	2003-05-
000090	EILEEN	W	HENDERSON	E11	5498	2000-08-15	MANAGER	16	F	1971-05-
000100	THEODORE	Q	SPENSER	E21	0972	2000-06-19	MANAGER	14	M	1980-12-
000110	VINCENZO	G	LUCCHETTI	A00	3490	1988-05-16	SALESREP	19	M	1959-11-
000120	SEAN		O'CONNELL	A00	2167	1993-12-05	CLERK	14	M	1972-10-
000130	DELORES	M	QUINTANA	C01	4578	2001-07-28	ANALYST	16	F	1955-09-
000140	HEATHER	A	NICHOLLS	C01	1793	2006-12-15	ANALYST	18	F	1976-01-
000150	BRUCE		ADAMSON	D11	4510	2002-02-12	DESIGNER	16	M	1977-05-
000160	ELIZABETH	R	PIANKA	D11	3782	2006-10-11	DESIGNER	17	F	1980-04-
000170	MASATOSHI	J	YOSHIMURA	D11	2890	1999-09-15	DESIGNER	16	M	1981-01-
000180	MARILYN	S	SCOUTTEN	D11	1682	2003-07-07	DESIGNER	17	F	1979-02-
000190	JAMES	H	WALKER	D11	2886	2004-07-26	DESIGNER	16	M	1982-06-

Close Find... Display... Help

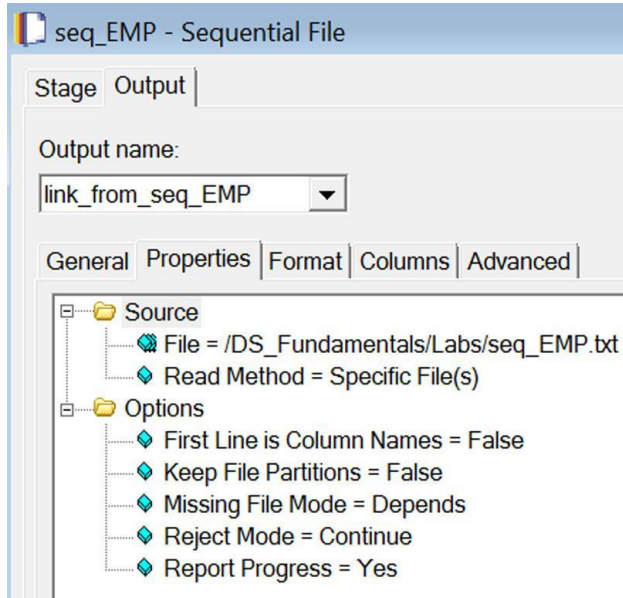
Task: Write data to a DB2 UDB Table using a DB2 Connector stage

In this task, we will create a job that reads data from a sequential file and then writes to a DB2 UDB table. We will use a DB2 Connector stage.

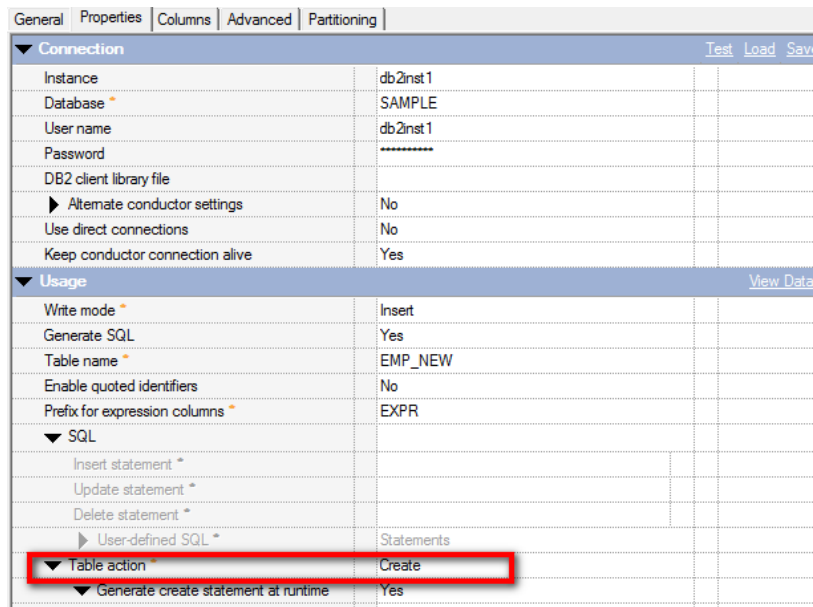
- Create a new parallel job named SeqFileToDB2ConnTable as shown.



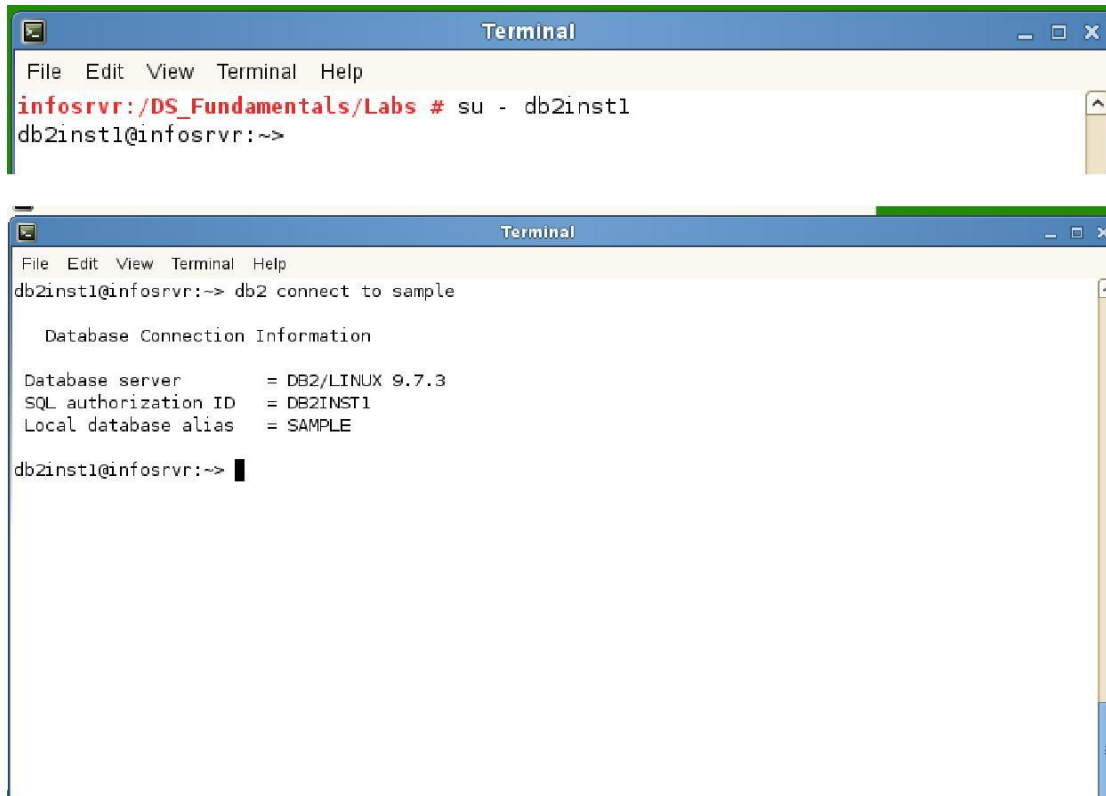
2. Edit the sequential file stage to read the same file (seq_EMP.txt) created in the previous job. You need to set the Format tab delimiter to comma and quote to none since this is how the file was created. Then you need to load the Columns tab using the one Table Definitions/Database/DB2INST1.EMP as that was the database table's metadata.



3. Edit the DB2 Connector stage and enter the values as shown below. Click OK to save changes. Please note that the Table name value is "EMP_NEW".



4. Save and compile the job
5. Go to the VMWare image. Log in as root if you haven't done that. Open a terminal. Switch the user to db2inst1 / inf0server. Connect to DB2 to view the contents of table EMP_NEW before running the job.



The image shows two terminal windows. The top window is titled 'Terminal' and shows the command 'su - db2inst1' being executed, resulting in a prompt change to 'db2inst1@infosrvr:~>'. The bottom window is also titled 'Terminal' and shows the command 'db2 connect to sample' being executed. This results in a 'Database Connection Information' section displaying details about the database server, SQL authorization ID, and local database alias.

```
Terminal
File Edit View Terminal Help
infosrvr:/DS_Fundamentals/Labs # su - db2inst1
db2inst1@infosrvr:~>

Terminal
File Edit View Terminal Help
db2inst1@infosrvr:~> db2 connect to sample

Database Connection Information

Database server      = DB2/LINUX 9.7.3
SQL authorization ID = DB2INST1
Local database alias = SAMPLE

db2inst1@infosrvr:~> █
```

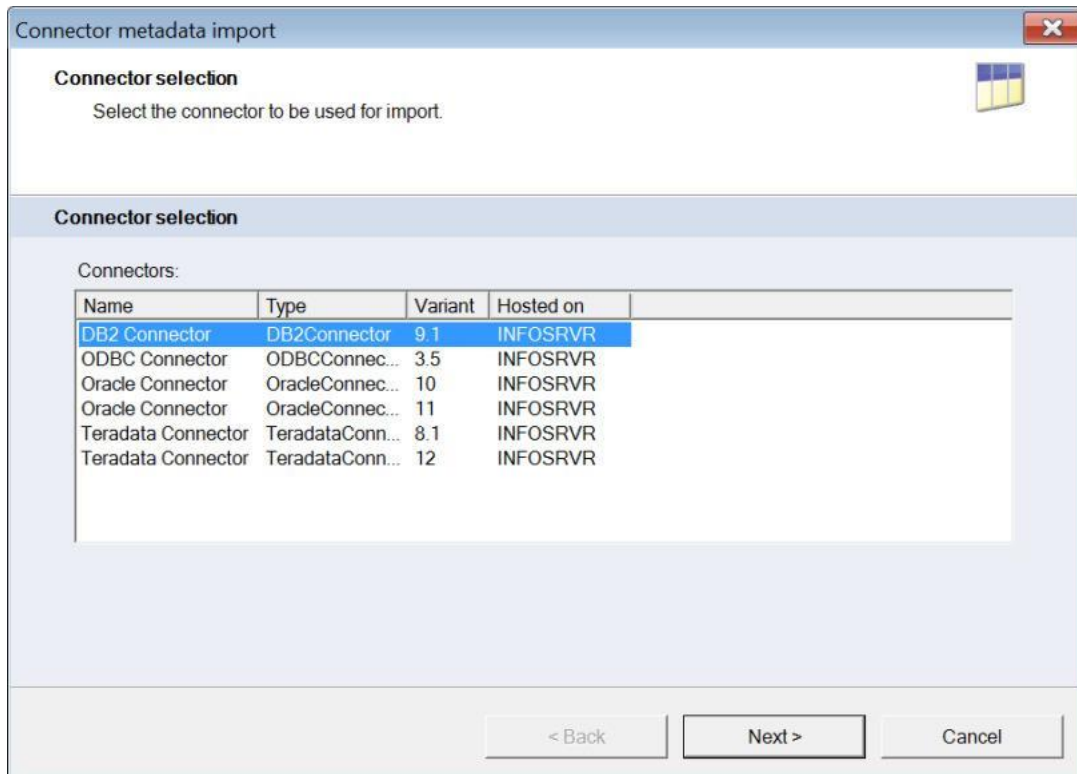
6. Now run the job.

Year	Value
1962	39950
1973	46500
1956	43840
1972	39250
1976	68420

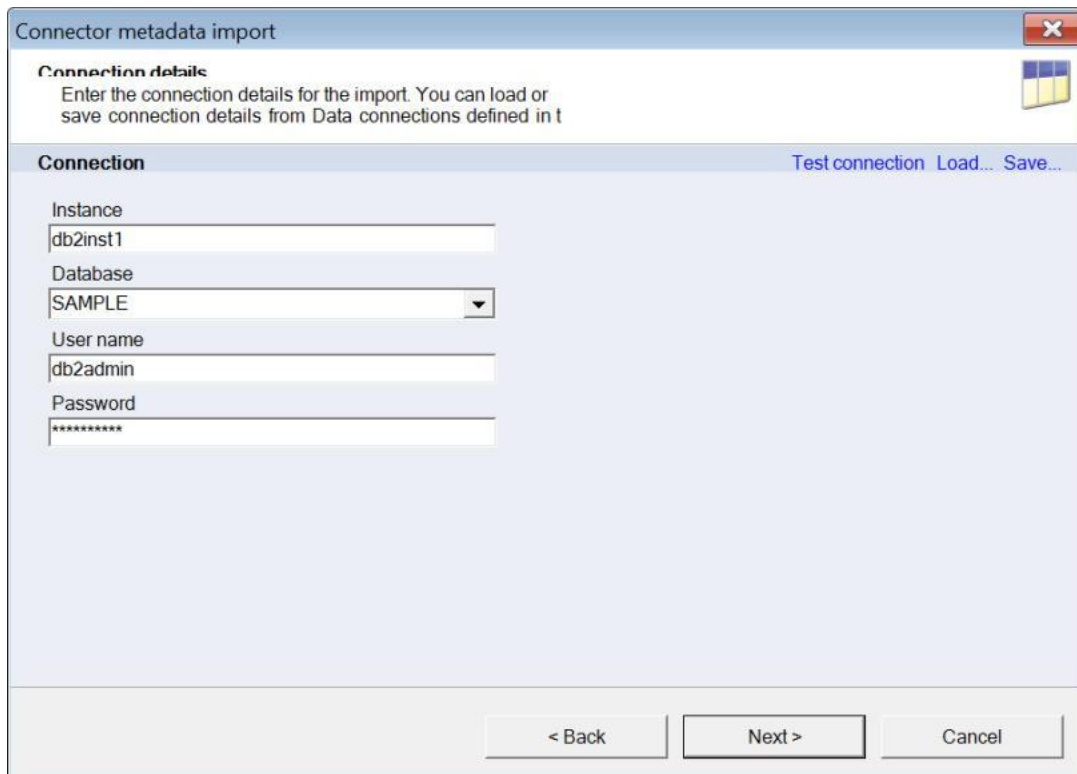
Task: Import table definition of a relational table using Connector Import Wizard

In this task, we will import the table definition of the table that we created in the last lab exercise, EMP_NEW, using the Connector Import Wizard.

1. Go to the menu and click Import > Table Definitions > Start Connector Import Wizard.
2. Select the DB2 Connector. Click Next.

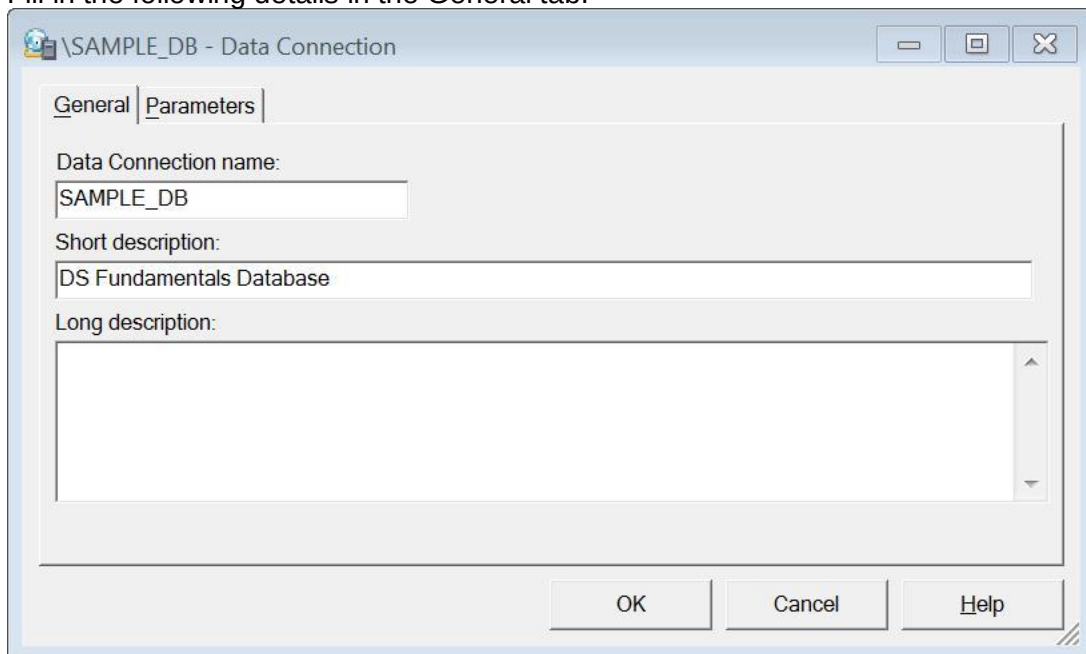


- Fill in the connection details as shown below, using 'inf0server' as a password.



The image shows a dialog box titled "Connector metadata import". It has a "Connection details" section with the instruction: "Enter the connection details for the import. You can load or save connection details from Data connections defined in t". Below this is a "Connection" section with the following fields: "Instance" (text box with "db2inst1"), "Database" (dropdown menu with "SAMPLE"), "User name" (text box with "db2admin"), and "Password" (text box with "*****"). To the right of the "Connection" section are links for "Test connection", "Load...", and "Save...". At the bottom are buttons for "< Back", "Next >", and "Cancel".

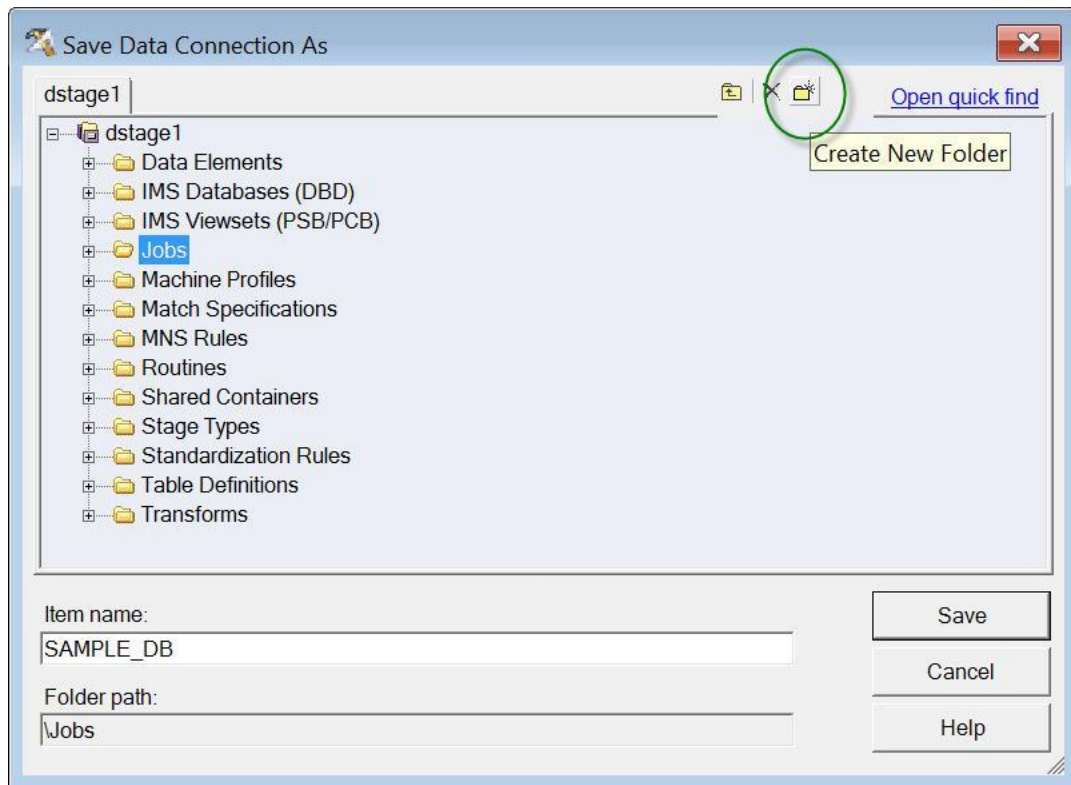
- Click 'Save...' to save this connection as a Data Connection object. The connection information can then be shared across connector stages in multiple jobs.
- Fill in the following details in the General tab.



The image shows a dialog box titled "SAMPLE_DB - Data Connection". It has two tabs: "General" and "Parameters". The "General" tab is active and contains the following fields: "Data Connection name:" (text box with "SAMPLE_DB"), "Short description:" (text box with "DS Fundamentals Database"), and "Long description:" (text area). At the bottom are buttons for "OK", "Cancel", and "Help".

- Review the Parameters tab and then click OK to save the Data Connection.

7. Create a new folder in Jobs called 'shared'.



8. Save the SAMPLE_DB Connection in Jobs > shared.
9. Click Next.
10. Fill in the Data source location information.

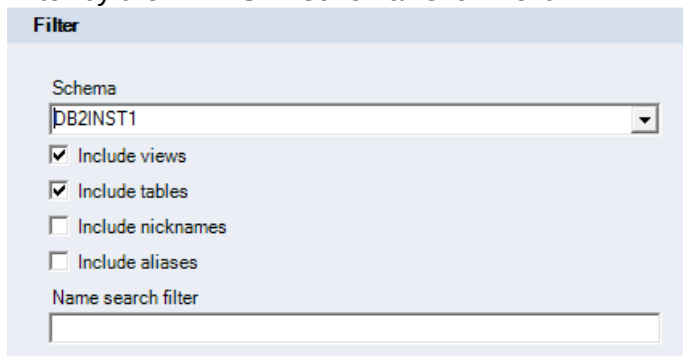
Connector metadata import

Data source location
Confirm, or choose the location details for your Import.

Data source location	New location...
Host name where database resides:	INFOSRVR
Database name:	SAMPLE (db2inst1)

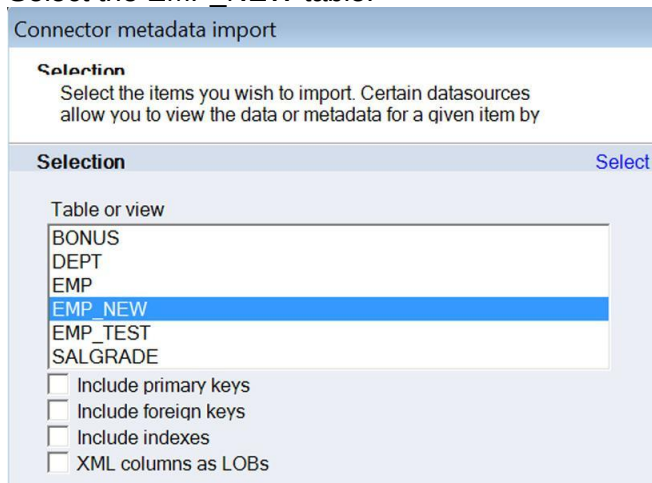
11. Click Next.

12. Filter by the DBINST1 schema. Click Next.



The Filter dialog box has a title bar labeled "Filter". Inside, there is a "Schema" dropdown menu with "DB2INST1" selected. Below the dropdown are four checkboxes: "Include views" (checked), "Include tables" (checked), "Include nicknames" (unchecked), and "Include aliases" (unchecked). At the bottom is a "Name search filter" text box.

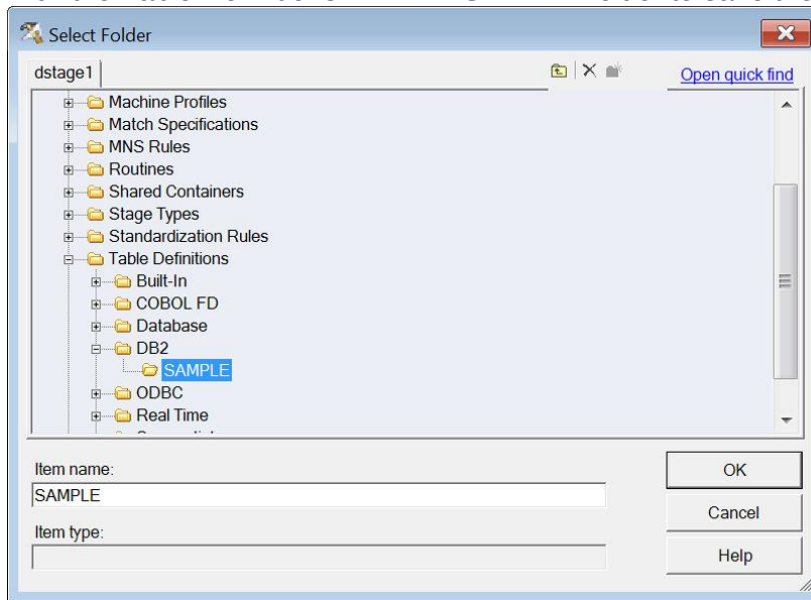
13. Select the EMP_NEW table.



The Connector metadata import Selection dialog box has a title bar labeled "Connector metadata import". Below the title bar is a "Selection" section with the text "Select the items you wish to import. Certain datasources allow you to view the data or metadata for a given item by". Below this is a "Selection" table with a "Select" button. The table lists "Table or view" and includes the following items: BONUS, DEPT, EMP, EMP_NEW (highlighted), EMP_TEST, and SALGRADE. Below the table are four checkboxes: "Include primary keys" (unchecked), "Include foreign keys" (unchecked), "Include indexes" (unchecked), and "XML columns as LOBs" (unchecked).

14. Click Next and then Import.

15. Pick the Table Definitions > DB2 > SAMPLE folder to save the Table Definition in.



Task: Read data from a DB2 UDB Table using an ODBC Connector stage

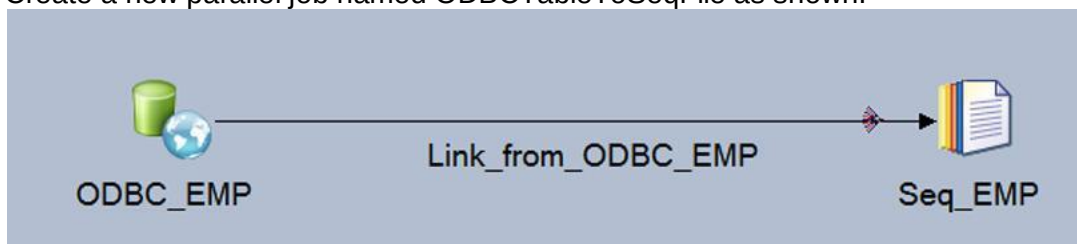
In this task, we will read data from a DB2 UDB table using an ODBC Connector stage and load the data into a sequential file.



The Open Database Connectivity (ODBC) standard accomplishes DBMS independence by using an ODBC driver.

An ODBC driver enables DataStage to use a data source, normally a DBMS or a CSV file. ODBC connectivity is relevant for data sources that do not have their own common connector, for example PostgreSQL or MySQL.

1. Create a new parallel job named ODBCTableToSeqFile as shown.



2. Rename the stage and link names as shown for good standard practice.

3. Edit the ODBC Connector stage to enter the properties as shown below.

ODBC_EMP - ODBC Connector

Stage | Output

Output name (downstream stage)
Link_From_ODBC_EMP (seq_EMP)

General | Properties | Columns | Advanced

Connection [Test](#) [Load](#) [Save](#)

Data source * SAMPLE

Username db2inst1

Password *****

Usage [View Data](#)

Generate SQL Yes

Table name * EMPLOYEE

Enable quoted identifiers No

SQL

Select statement *

Enable partitioned reads No

Transaction

Record count 2000

End of wave None

Session

Isolation level Read uncommitted

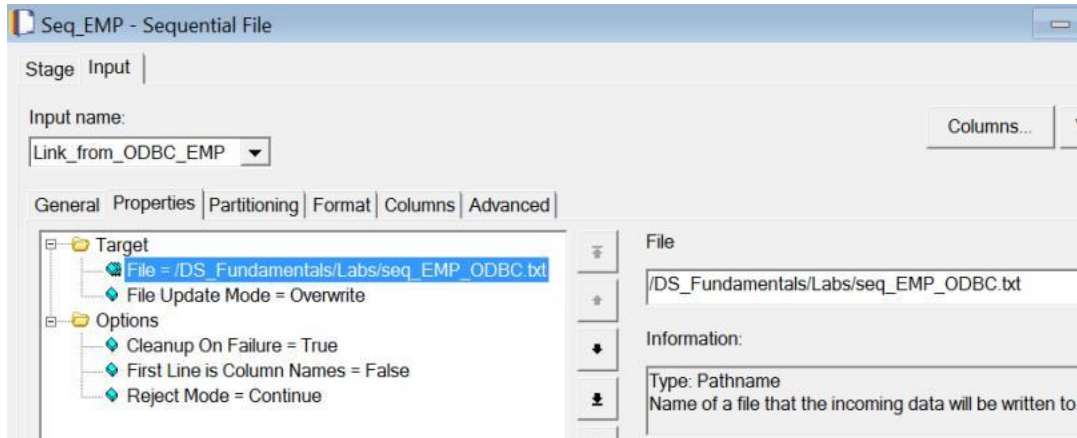
Autocommit mode Off

Array size 2000

4. Load the EMPLOYEE table definition from the Table Definitions > ODBC folder in the Columns tab and click on OK to close the stage editor.

	Column name	Key	SQL type	Extended	Length	Scale	Nullable
1	EMPNO	☒	Char	Unicode	6		No
2	FIRSTNME	☐	VarChar	Unicode	12		No
3	MIDINIT	☐	Char	Unicode	1		Yes
4	LASTNAME	☐	VarChar	Unicode	15		No
5	WORKDEPT	☐	Char	Unicode	3		Yes
6	PHONENO	☐	Char	Unicode	4		Yes
7	HIREDATE	☐	Date		10		Yes
8	JOB	☐	Char	Unicode	8		Yes
9	EDLEVEL	☐	SmallInt		5		No
10	SEX	☐	Char	Unicode	1		Yes
11	BIRTHDATE	☐	Date		10		Yes
12	SALARY	☐	Decimal		9	2	Yes
13	BONUS	☐	Decimal		9	2	Yes
14	COMM	☐	Decimal		9	2	Yes
		☐					

5. Edit the target Sequential File stage to write data into the seq_EMP_ODBC.txt file.

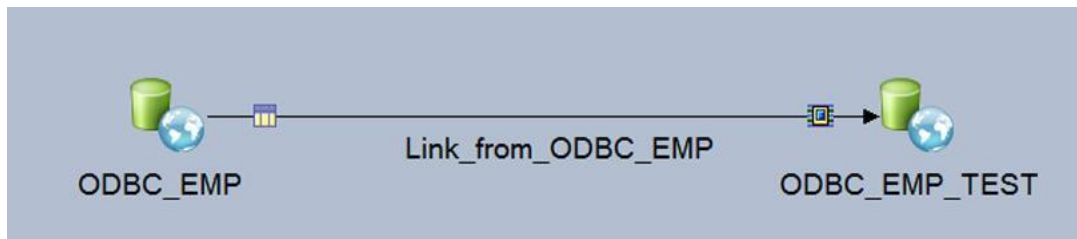


6. On the Format tab, specify comma as delimiter and quote as none.
7. Save and compile the job.
8. Run the job and view the target sequential file seq_EMP_ODBC.txt to verify.

Task: Using ODBC Connector stage and the SQL Query Builder

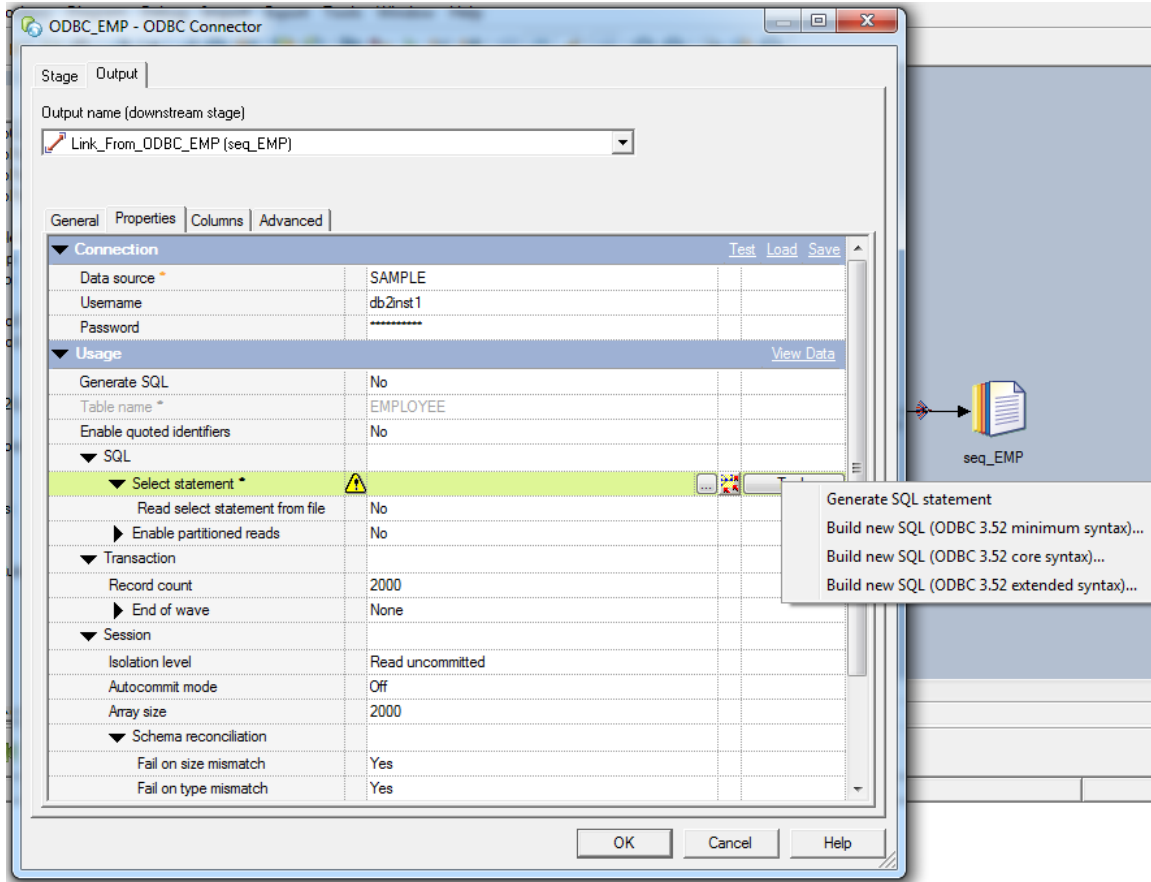
In this task, we will load data from one DB2 UDB table into another DB2 UDB table using the ODBC Connector stage. We will make use of the SQL query builder in the ODBC Connector stage.

1. Create a new parallel job named ODBCConnTableToODBCConnTable as shown.



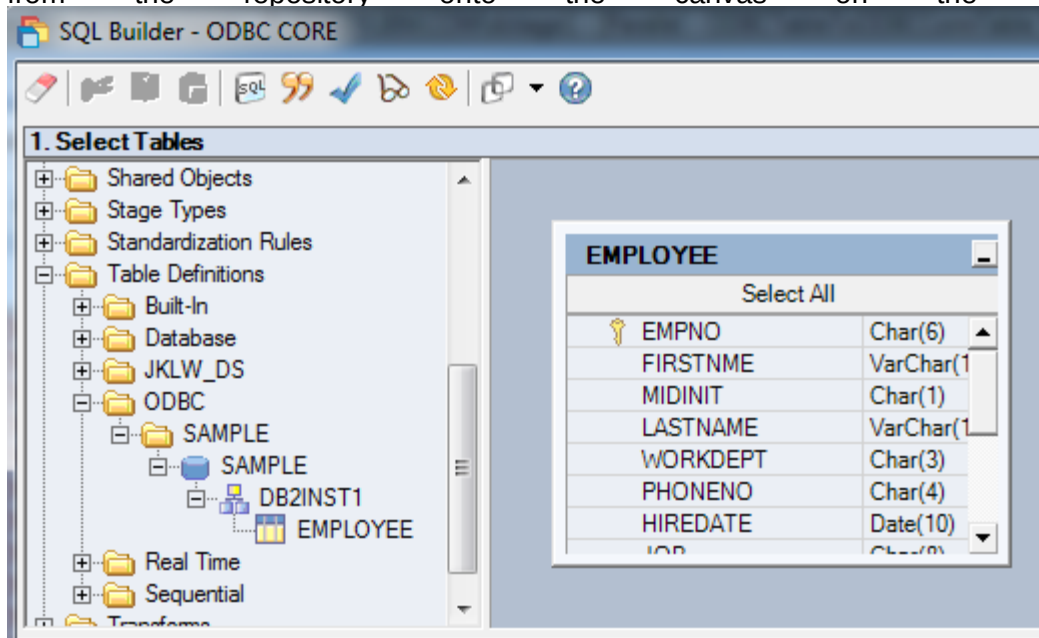
2. Rename the stage and link names as shown for good standard practice.

3. Edit the source ODBC Connector stage to enter the properties as shown below.

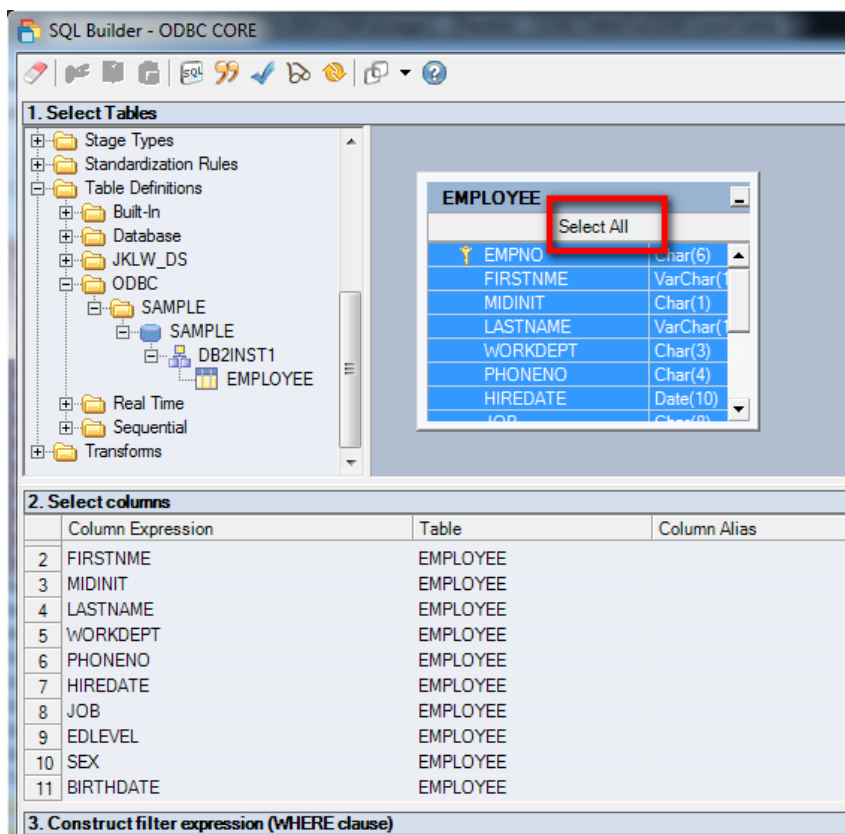


4. In the Usage section select Generate SQL as No. In the Select Statement field click on the Build button and select the Build new SQL option to open the SQL Builder window. You can use any of the three options.

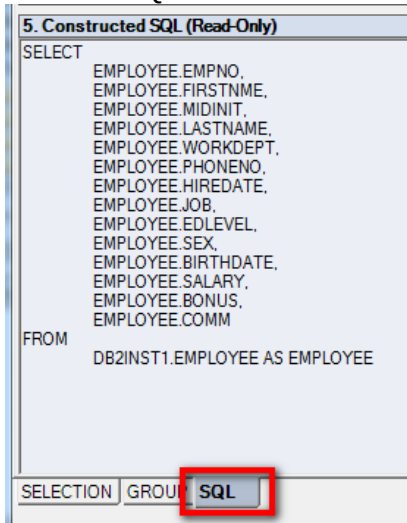
5. In the Select Tables window CLICK and DRAG the source table definition EMPLOYEE from the repository onto the canvas on the right.



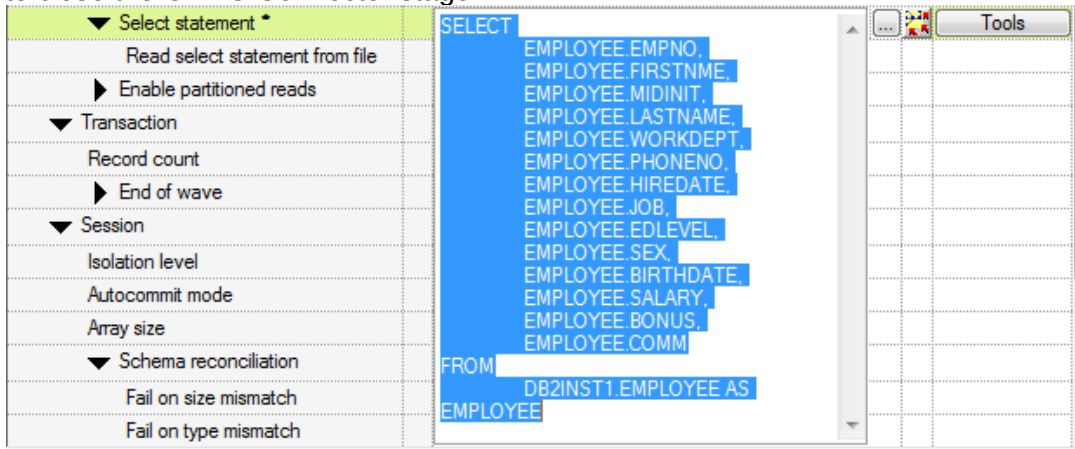
6. Click the Select All button to highlight all the columns. DRAG all the columns to the Select columns section.



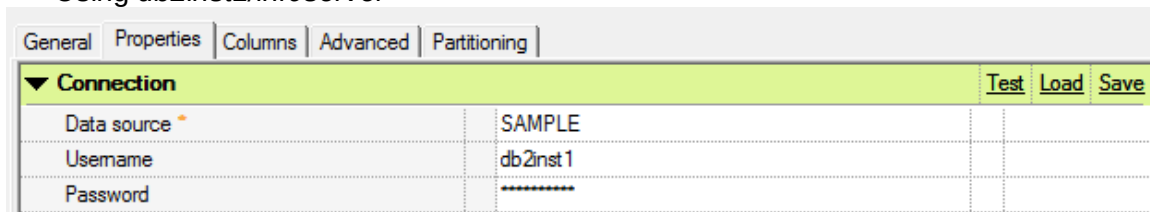
7. View the SQL in the Constructed SQL tab below and click OK.



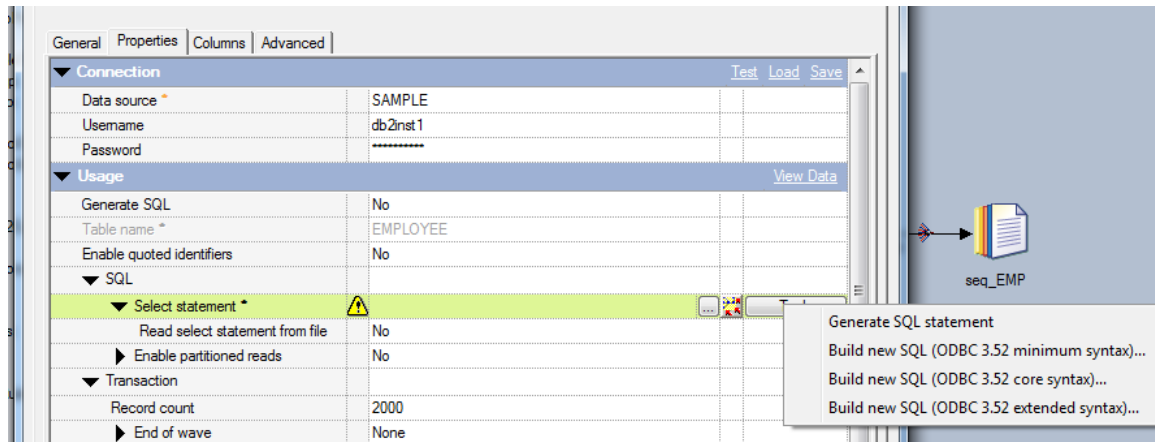
8. The constructed SQL then appears as shown in the ODBC connector stage. Click OK to close the ODBC Connector stage.



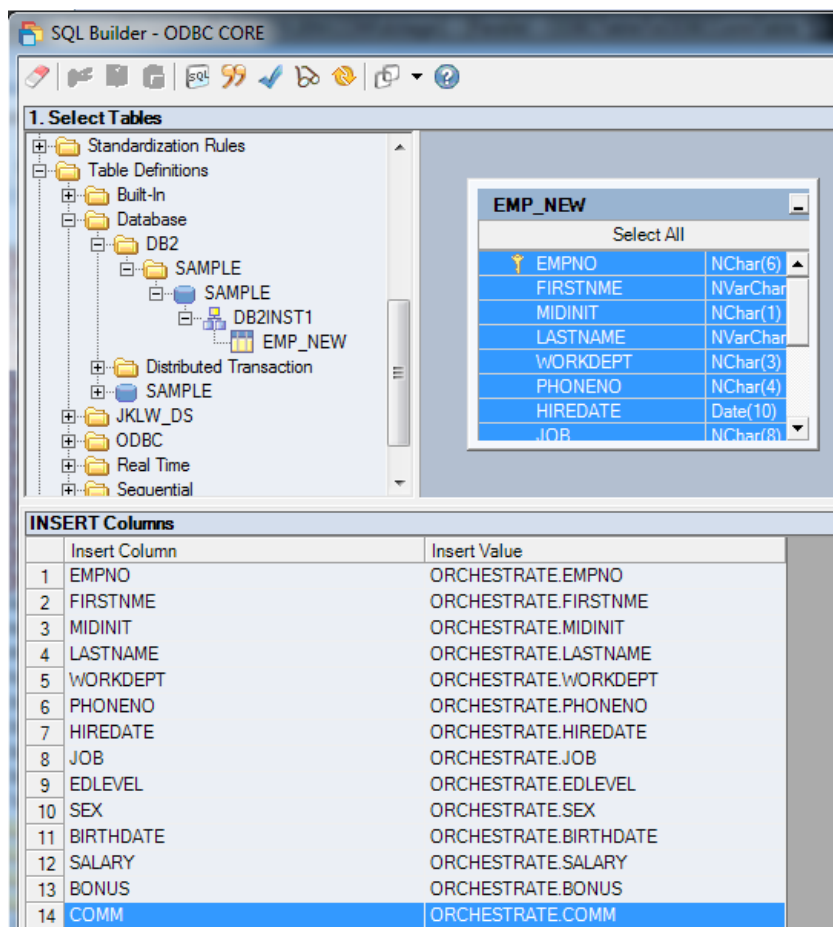
9. Edit the target ODBC Connector stage and enter the properties as shown.
Using *db2inst1/inf0server*



10. Here select Write Mode as Insert and Generate SQL as No. In the Insert Statement window select Build New SQL as shown.



11. In the Select Tables window drag the target table definition EMP_NEW from the repository onto the canvas on the right.
12. Click the Select All button to select all the columns. Drag the selected columns to the Insert Columns area.



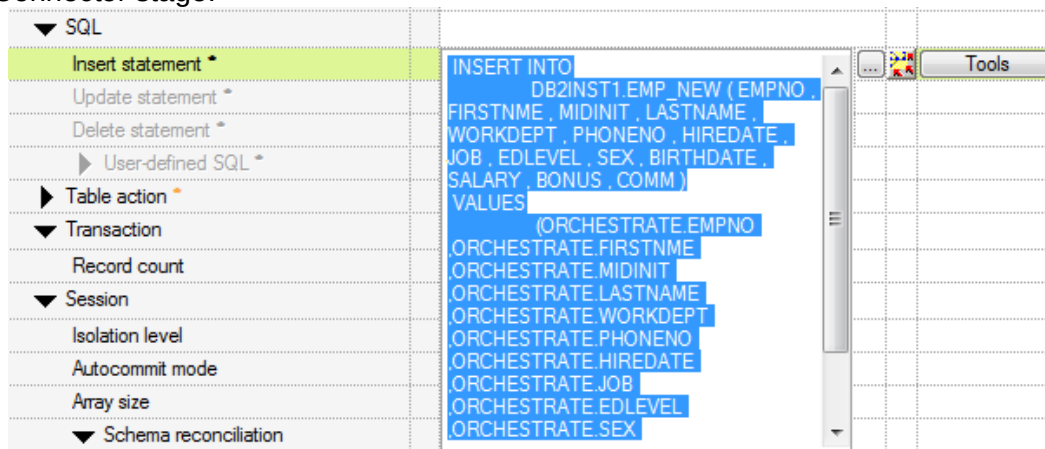
13. Notice in the Insert Value area, all the column values from each corresponding input columns are set with the special name in memory, ORCHESTRATE.XXX, correspondingly.
14. You can view the generated SQL by selecting the SQL tab below. Click OK to close.

4. Constructed SQL (Read-Only)

```

INSERT INTO
  DB2INST1.EMP_NEW ( EMPNO , FIRSTNME , MIDINIT , LASTNAME , WORKDEPT , PHONENO , HIREDATE , JOB , EDLEVEL , SEX , BIRTHDATE , SALARY , BONUS , COMM )
VALUES
  (ORCHESTRATE.EMPNO ,ORCHESTRATE.FIRSTNME ,ORCHESTRATE.MIDINIT ,ORCHESTRATE.LASTNAME ,ORCHESTRATE.WORKDEPT ,ORCHESTRATE.PHONENO ,ORCHESTRATE.HIREDATE
  ,ORCHESTRATE.JOB ,ORCHESTRATE.EDLEVEL ,ORCHESTRATE.SEX ,ORCHESTRATE.BIRTHDATE ,ORCHESTRATE.SALARY ,ORCHESTRATE.BONUS ,ORCHESTRATE.COMM )
  
```

15. The Insert statement now looks as shown below. Click OK to close the target ODBC Connector stage.



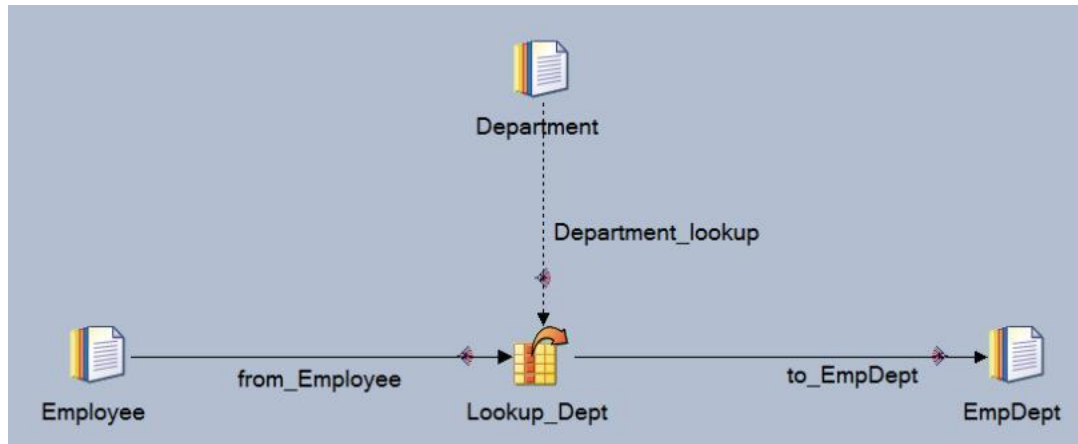
16. Save and compile the job. Run the job and view the output job log.
17. You can go to the VMWare image and as before use the db2inst1 user ID to view the data in the target table EMP_NEW by doing a *db2 "select * from EMP_NEW"*. Note that we specified the Table Action in the target ODBC Connector stage to Append. This means the total number of records in the table will be a multiple of 42 depending on how many times you have successfully executed the job.

Lab 08: Combining Data

Task: Lookup Stage with Equality Match

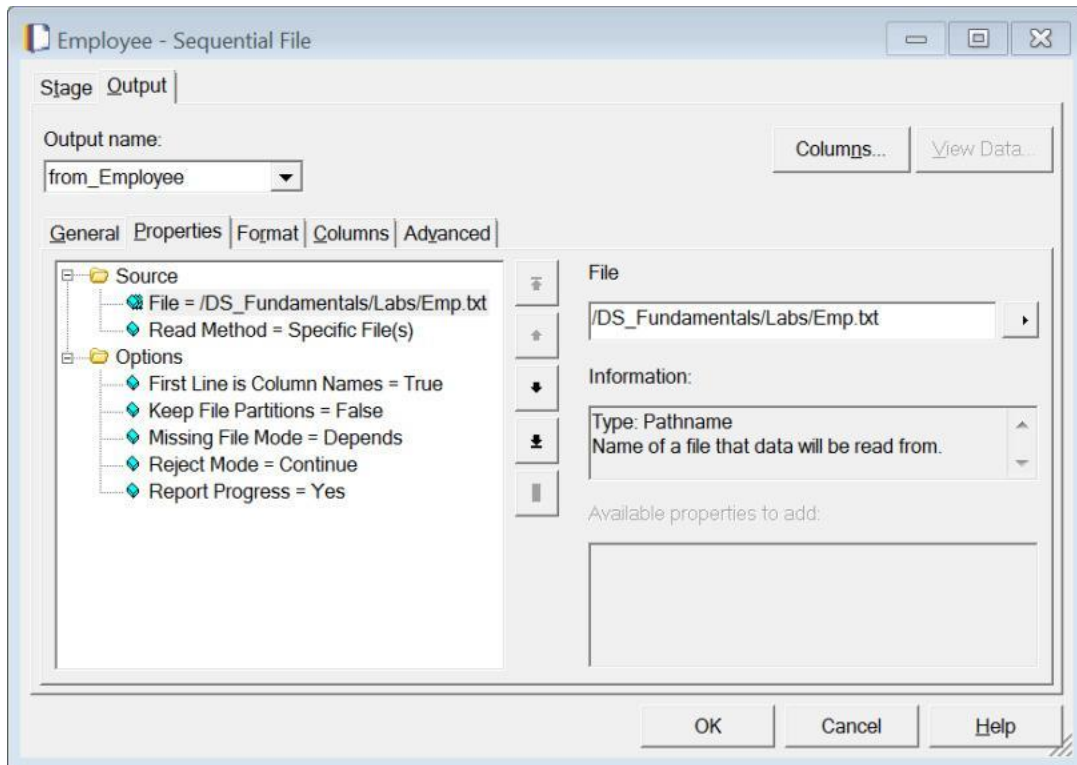
In this task, we will create a job that will read data from the Employee sequential file and lookup the Department file to fetch the department details and load it into a sequential file with all the employees' details.

1. Create the job as shown and save it as LKP_Equality.



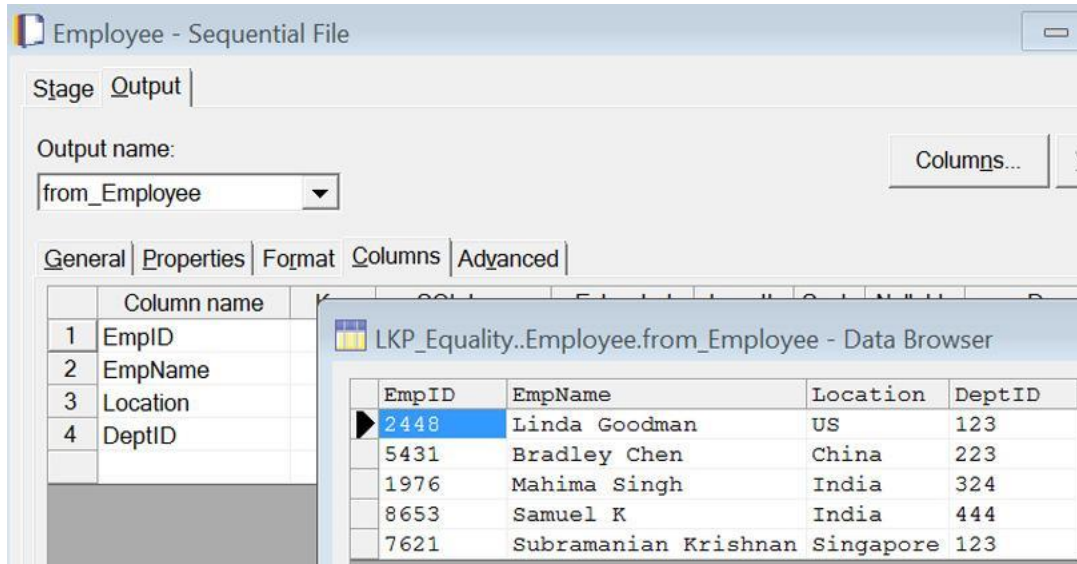
2. Rename the stage and link names as shown for good standard practice.

- Open the Employee Sequential File stage. On the Properties tab, specify the file Emp.txt to be read and other relevant properties. Remember to set the First Line is Column Names to True. If you don't, your job will have trouble reading the first row and issue a warning message in the log.

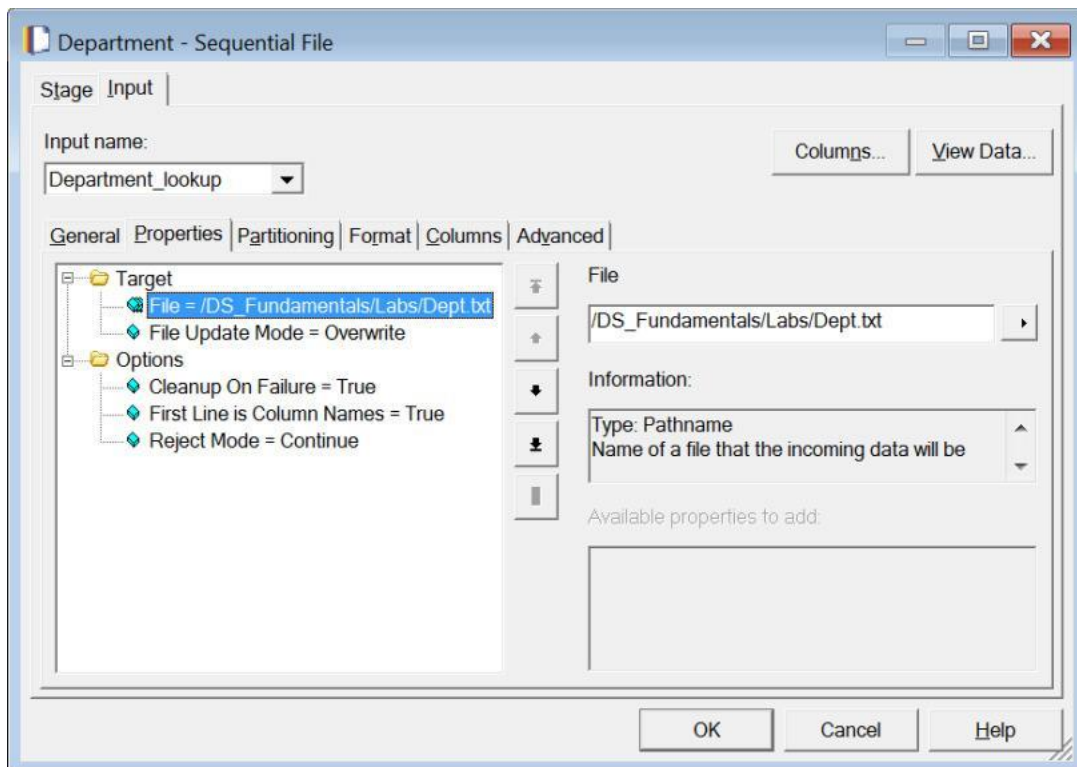


- Click on both the Format and Columns tab, click on the load button to load the format and column definitions from Emp.txt table definition under folder /Table Definitions/Sequential/Labs.
- In the Format tab, switch the Quote parameter to none.

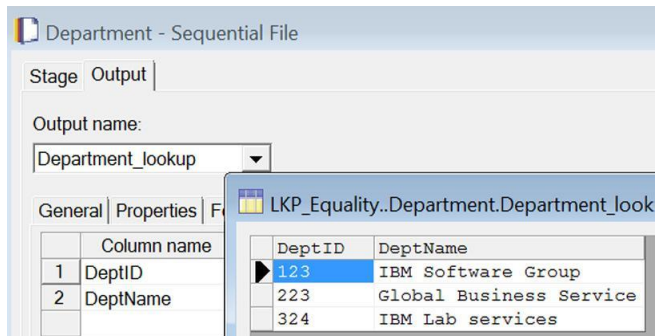
6. Click View Data to verify that the metadata has been specified properly.



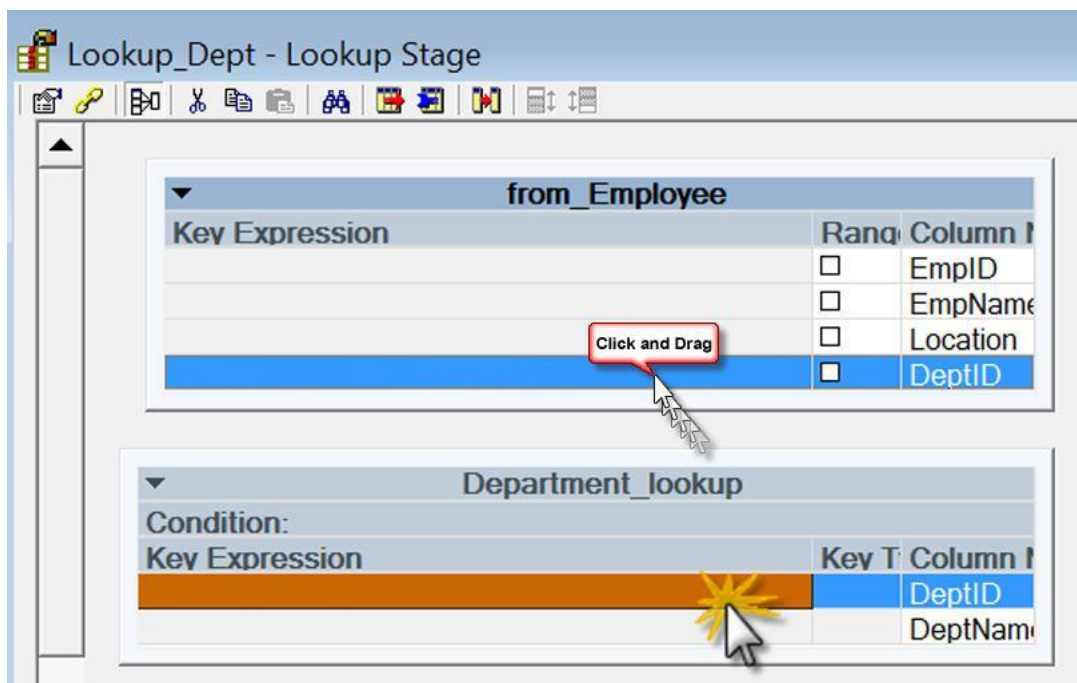
7. Open the Department Sequential File. On the Properties tab specify the file Dept.txt to be read and other relevant properties. Once again, remember to set the First Line is Column Names to True.



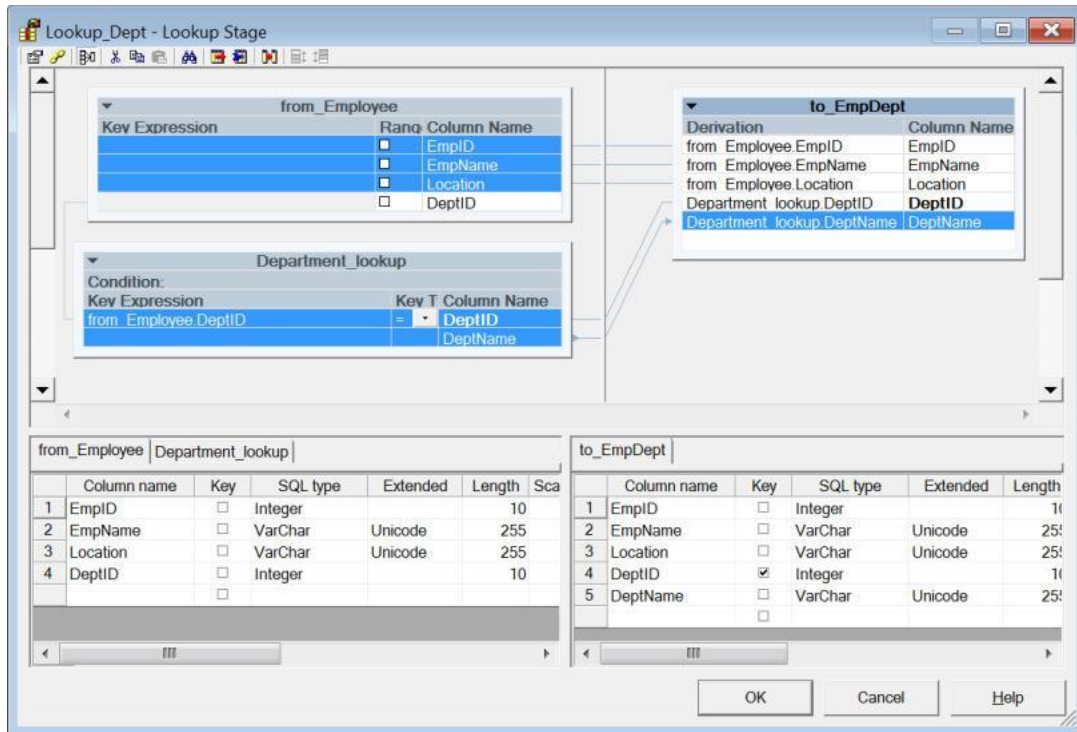
8. Load the format and columns from the table definition in the folder /Table Definitions/Sequential/Labs. Set the Quote property in Format to none.
9. Click View Data to verify that the metadata has been specified properly.



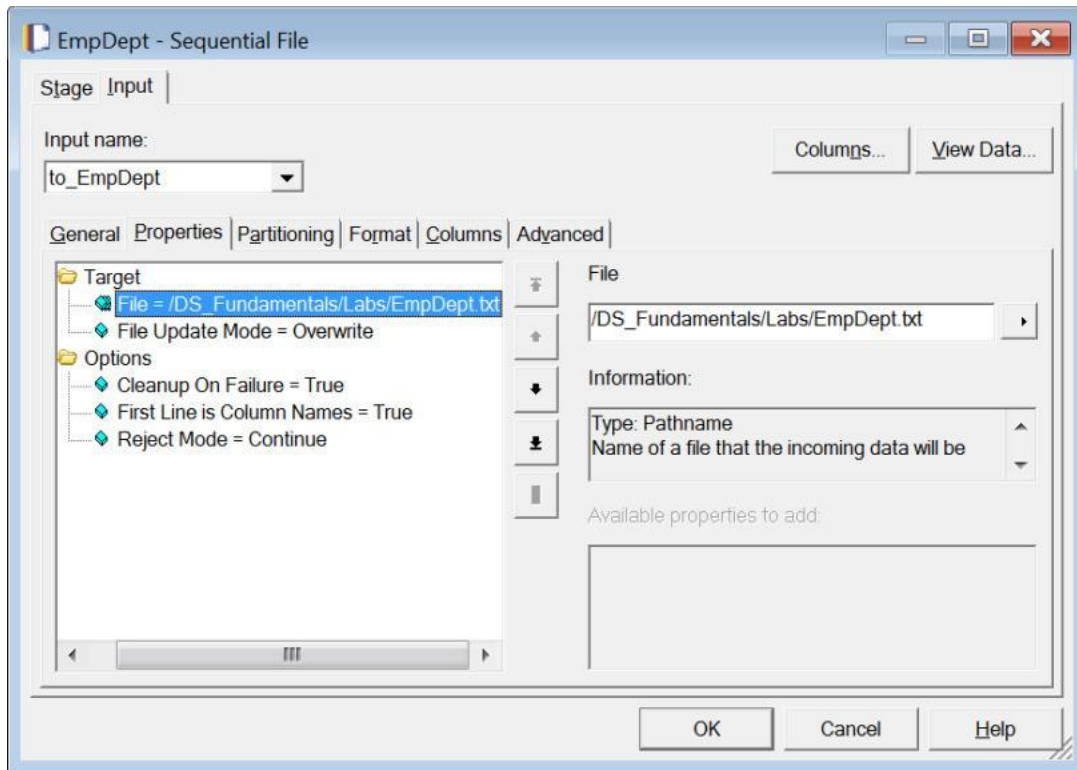
10. Edit the Lookup stage. Drag from_Employee.DeptID and drop it in Department_lookup.DeptID. There will be a Warning message displayed asking you to set DeptID as a key field. Simply select the Yes option to accept the message.



11. Map the fromEmployee.EmpID, EmpName and Location columns from the input link to the Output by dragging them across. Map both Department_lookup reference link columns to the output. Click OK to close the stage editor.



12. Open the EmpDept Sequential File stage. On the Properties tab, specify the path and file to write the output records to /DS_Fundamentals/Labs/EmpDept.txt. Remember to adjust 'First Line is Column Names' to True.



13. Save and compile the Job.
14. Run the job and check the results in the job log.

15. The job aborts due to a lookup failure as shown below, since there is a record in the input Emp.txt that does not have a match in the lookup table.

The screenshot displays the DataStage job execution interface. The top pane shows the job diagram with a flow from 'Employee' (5 rows, 5 rows/sec) to 'Lookup_Dept' (3 rows, 3 rows/sec) to 'EmpDept' (4 rows, 1 row/sec). The 'Lookup_Dept' stage is connected to a 'Department' table. The bottom pane shows the job execution logs. A green circle highlights the following error message:

```
2/21/2012 10:10:10 ... Lookup_Dept:1: Failed a key lookup for record 2 Key Values: DeptID: 444 [lookuptable/utprocessop.C:582]
```

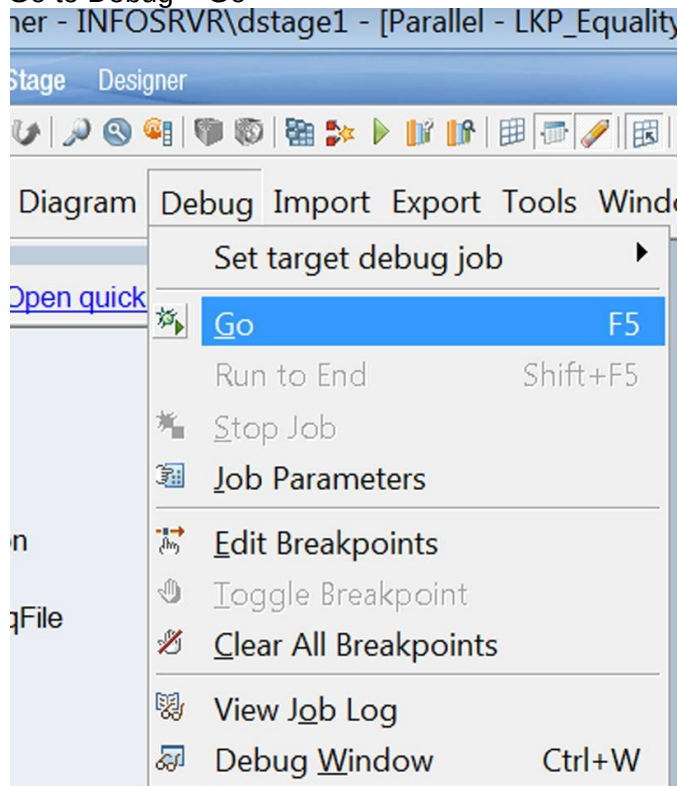
Task for extra credit: Examining the lookup failure in detail using the parallel debugger

1. Right click on the from_Employee link that goes into the Lookup stage and select Toggle Breakpoint. Then, right click on the Department_lookup link and create a breakpoint on this link as well.

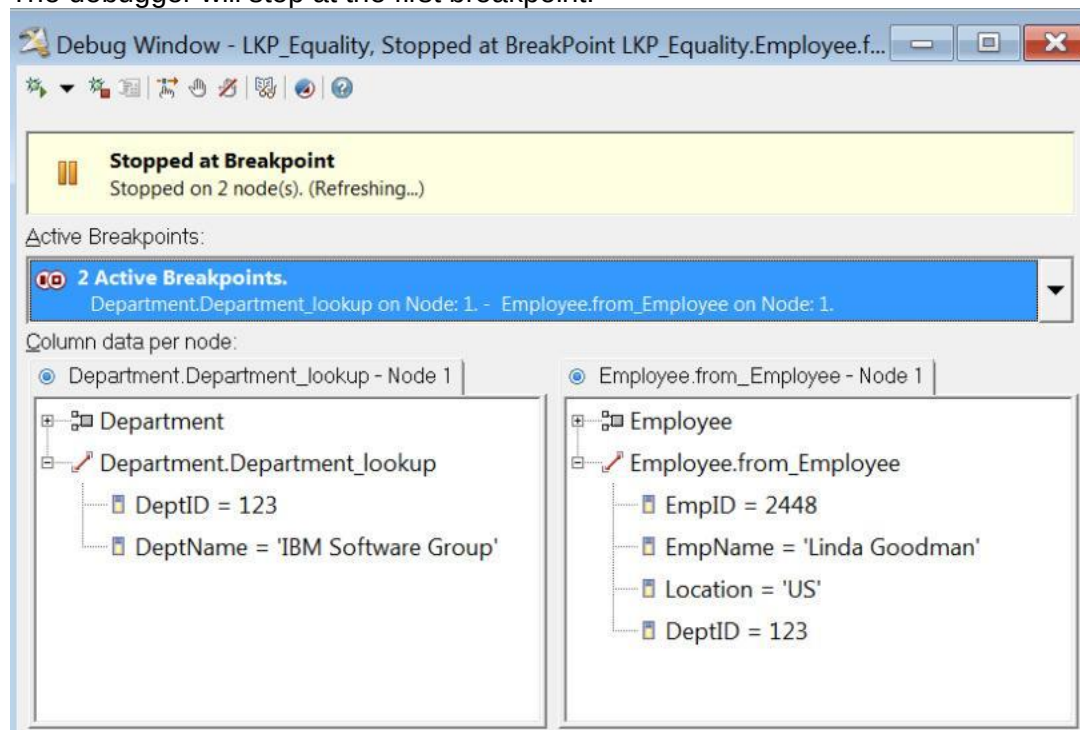
The screenshot shows the DataStage job diagram with the 'Lookup_Dept' stage. A context menu is open over the 'from_Employee' link, with 'Toggle Breakpoint' selected. The menu also shows options like 'Rename', 'Delete', 'Edit Breakpoints', 'Convert to Stream', and 'Convert to Reject'.

2. Compile the job.

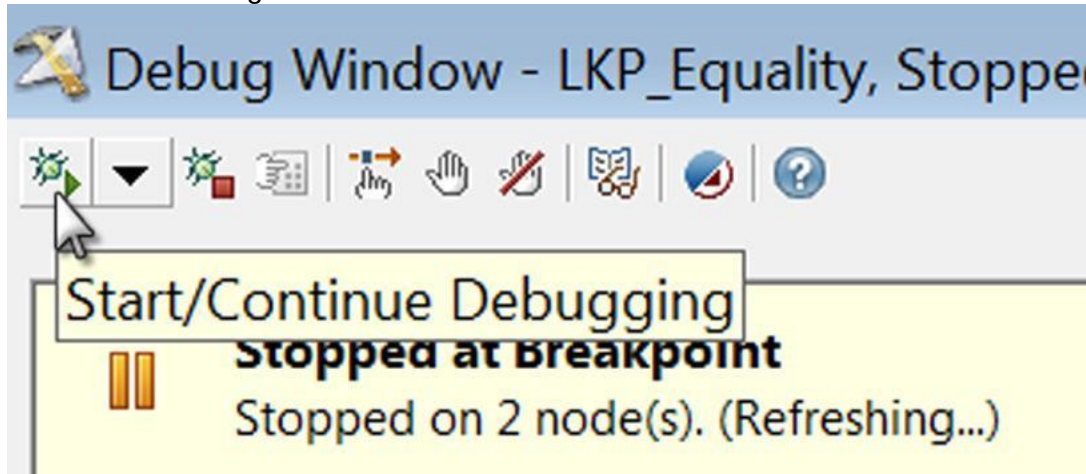
- Go to Debug > Go



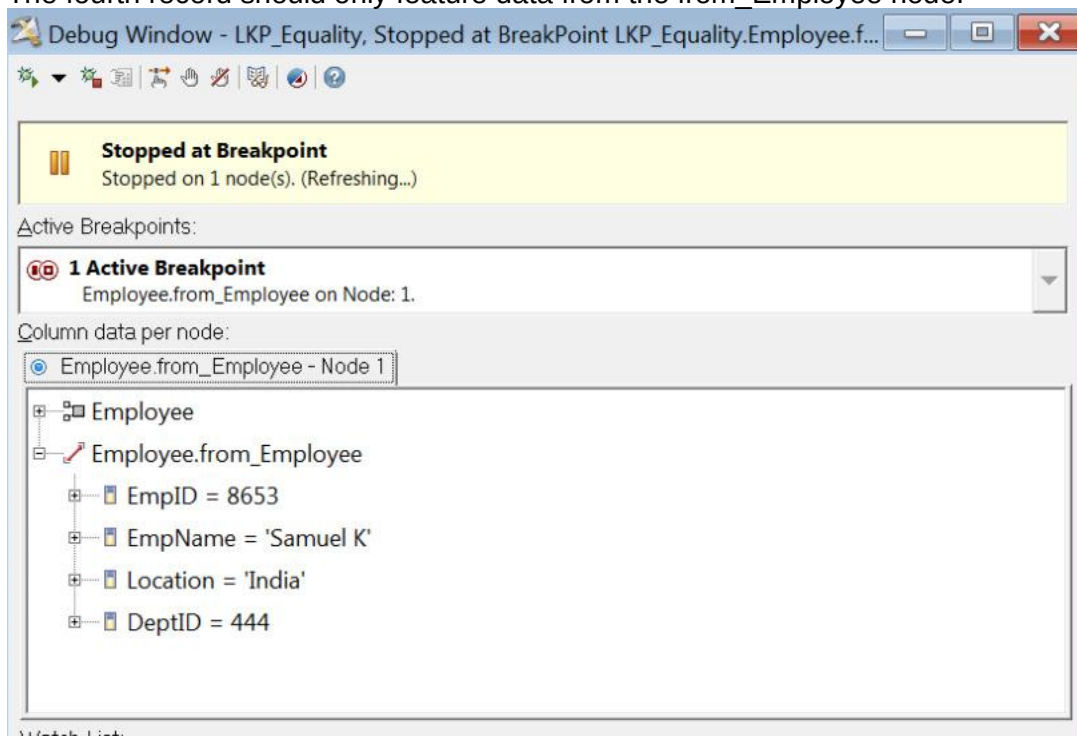
- Select OK to run the job.
- The debugger will stop at the first breakpoint.



6. You can see the Department Number 123 coming in from the from_Employee link. The lookup stage was able to find a match in the lookup table as shown in the Department_lookup link.
7. Continue the debug run.



8. Continue to debug. The debugger should find two more valid matches in the lookup table.
9. The fourth record should only feature data from the from_Employee node.

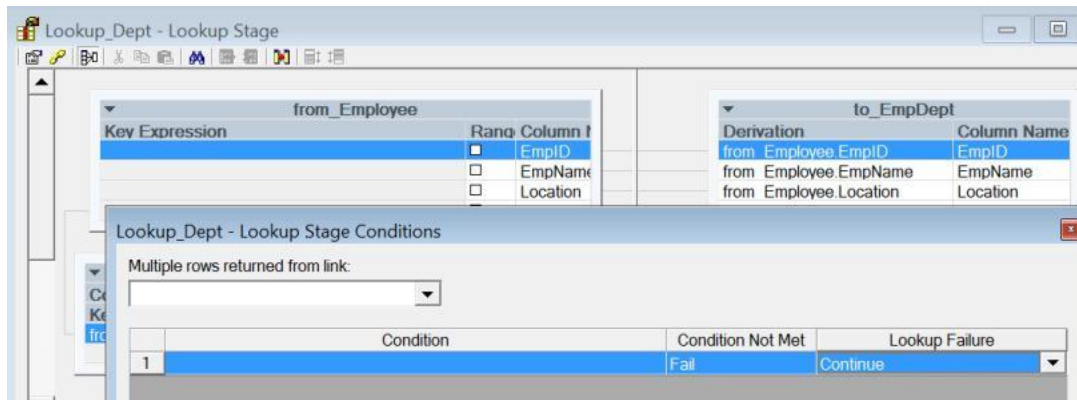


10. This is the point where the lookup stage is not able to find a corresponding match. This will abort the regular job run. You can now stop the job debugger and remove the breakpoints from the links.

Task: Handling lookup failure using lookup failure actions

Run the same Job created in the previous task with different Lookup Failure Options and observe the results – Continue, Drop, Fail, Reject. See the example below.

1. To Specify the Lookup condition, open the Lookup stage. Click the icon  (Constraints). Change the Lookup Failure option from Fail to Continue.



2. Compile and run the Job. Due to the new option, the job is not aborted.
3. Open the output file /DS_Fundamentals/Labs/EmpDept.txt. You can see that the record with an invalid DeptID have default values as DeptID and DeptName. Note: The default value is determined by DataStage depending on the column data type.

EmpID	EmpName	Location	DeptID	DeptName
5431	Bradley Chen	China	223	Global Business Service
8653	Samuel K	India	0	
2448	Linda Goodman	US	123	IBM Software Group
1976	Mahima Singh	India	324	IBM Lab services
7621	Subramanian Krishnan	Singapore	123	IBM Software Group

```

1 "EmpID","EmpName","Location","DeptID","DeptName"
2 5431,"Bradley Chen","China",223,"Global Business Service"
3 8653,"Samuel K","India",0,""
4 2448,"Linda Goodman","US",123,"IBM Software Group"
5 1976,"Mahima Singh","India",324,"IBM Lab services"
6 7621,"Subramanian Krishnan","Singapore",123,"IBM Software Group"
  
```

4. Go back to the job and open the Lookup stage.

5. Specify a different Lookup condition. Click the icon  (Constraints). Change the Lookup Failure option from Continue to Drop.
6. Compile and run your job.
7. As a result, the job runs successfully.
8. Open the output file /DS_Fundamentals/Labs/EmpDept.txt. You can see that the record with EmpID 8653 was dropped from the target file.

```

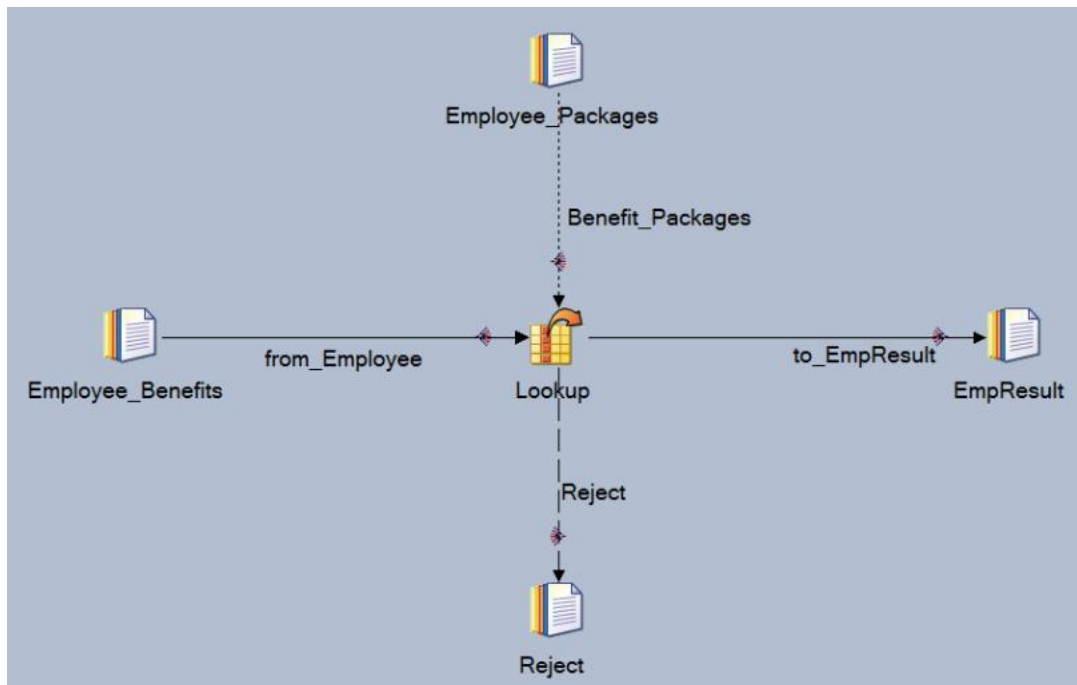
EmpDept.txt
1 "EmpID","EmpName","Location","DeptID","DeptName"
2 5431,"Bradley Chen","China",223,"Global Business Service"
3 2448,"Linda Goodman","US",123,"IBM Software Group"
4 1976,"Mahima Singh","India",324,"IBM Lab services"
5 7621,"Subramanian Krishnan","Singapore",123,"IBM Software Group"

```

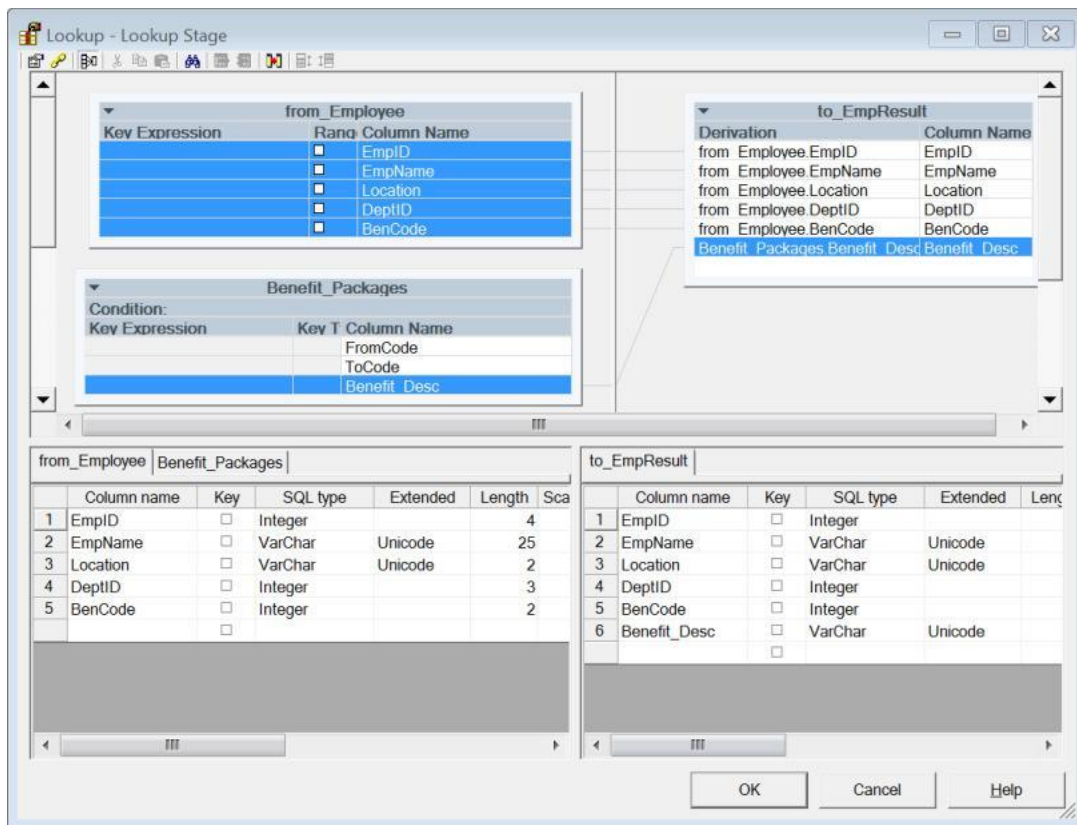
Task: Range lookup on reference link

In this task, we will verify to which class of benefits the employees' package belong to. This can be performed based on a range of benefit packages codes that refer to different classes. In order to do so, we will perform a Range lookup.

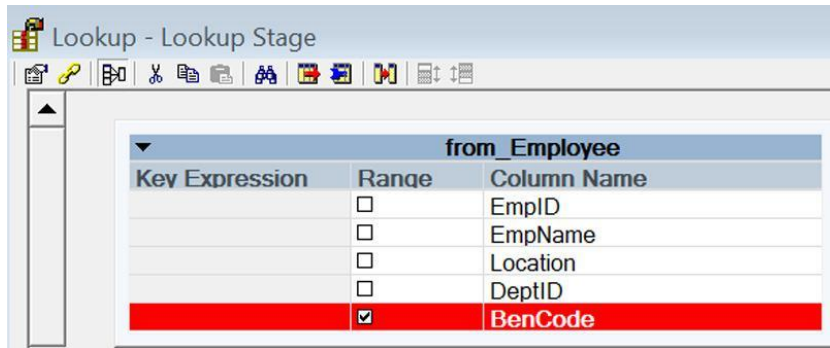
1. Create a job named LKP_Range containing a Sequential File stage as source named Employee_Benefits, a Sequential File stage as reference named Benefits_Package, a Lookup stage, a target Sequential File stage, and a Reject Sequential File stage.



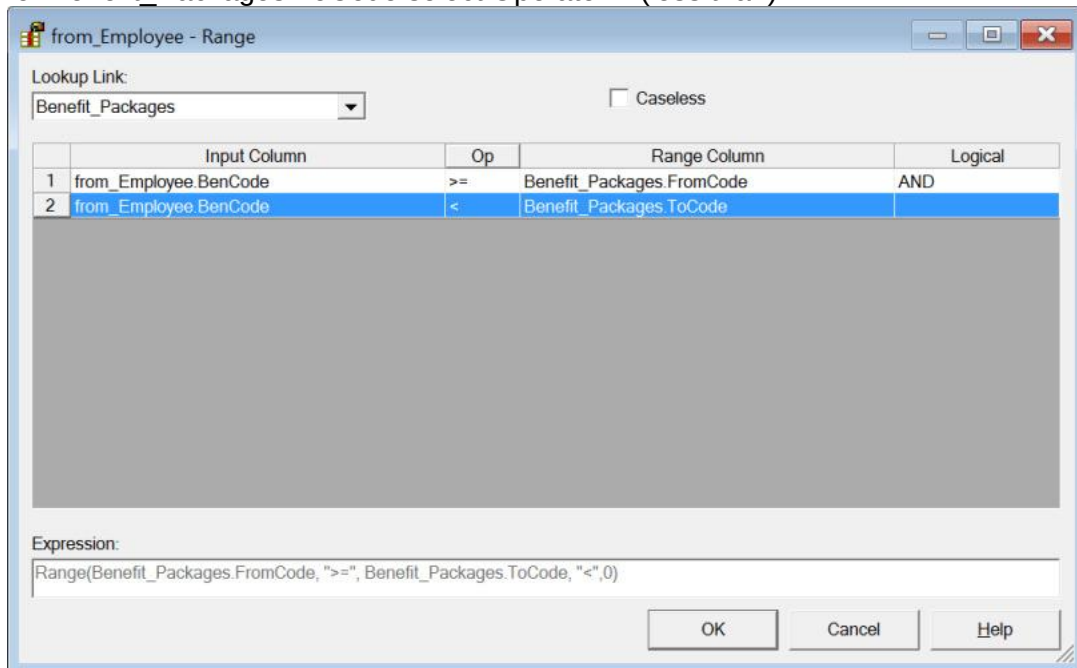
- Open the Employee_Benefits Sequential File stage. On the Properties tab, specify the file /DS_Fundamentals/Labs/Emp_BenefitPkg.txt to be read and other relevant properties.
- Load the table definition to the Format and Columns tab.
- Click View Data to verify that the metadata has been specified properly.
- Open the Employee_Packages Sequential File stage. Set up all the necessary properties and table definition information. The file path is /DS_Fundamentals/Labs/Benefits_package.txt
- Load the table definition to the Format and Columns tab.
- Click View Data to verify that the metadata has been specified properly.
- Edit the Lookup stage and map the input columns to the output as shown below.



9. Set the from_Employee.BenCode as a Range Key by checking the box in the 'Range' column.

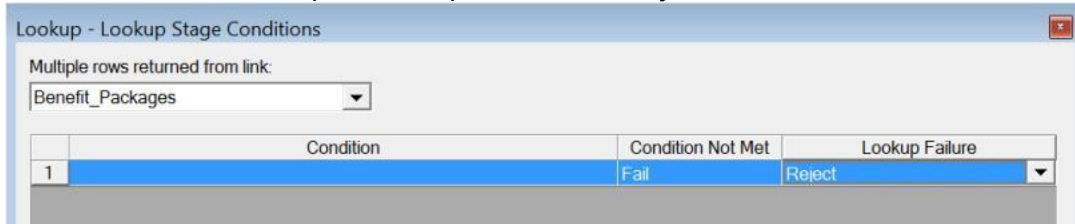


10. Then right-click on BenCode row and select "Edit Key Expression". The expression editor will be displayed.
11. The BenCode field should have a value between Benefit_Packages.FromCode and Benefit_Packages.ToCode. Select the Operators for each field from the drop down. For Benefit_Packages.FromCode, select Operator >= (greater and equal than), and for Benefit_Packages.ToCode select Operator < (less than).

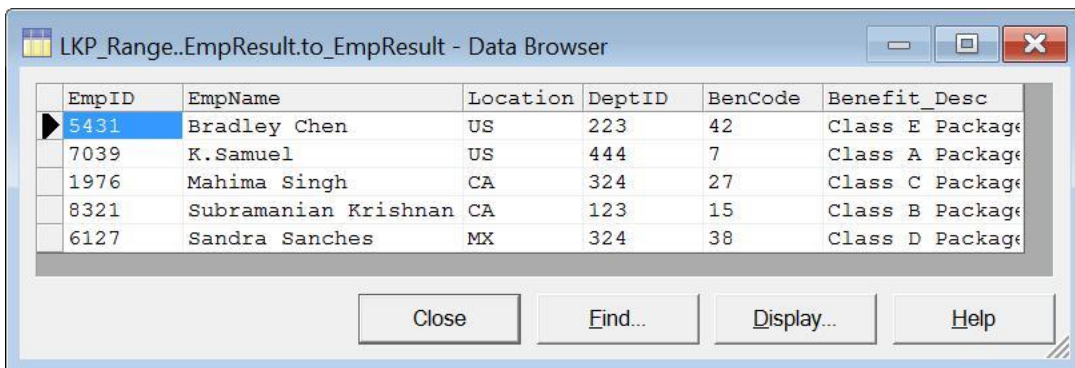


12. Click OK

13. Click on the Constraints icon . Make sure the Link “Benefit_Packages” is selected. For the Lookup Failure option, select “Reject” and click OK.

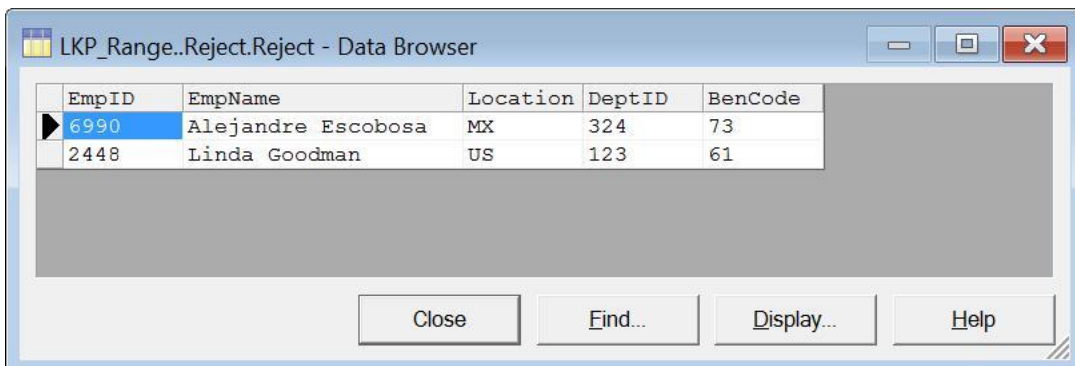


14. Open the EmpResult Sequential File. On the Properties tab specify the path and file to write the output records to file /DS_Fundamentals/Labs/Emp_Result.txt.
15. Open the Reject Sequential File. On the Properties tab specify the path and file to write the output records to file /DS_Fundamentals/Labs/Range_Rejects.txt.
16. Save and compile the job.
17. Run the job and after it's finished, validate the results in the Emp_Result.txt file. Out of 7 records, you can see that 5 met the range specified in the Lookup.



EmpID	EmpName	Location	DeptID	BenCode	Benefit_Desc
5431	Bradley Chen	US	223	42	Class E Package
7039	K.Samuel	US	444	7	Class A Package
1976	Mahima Singh	CA	324	27	Class C Package
8321	Subramanian Krishnan	CA	123	15	Class B Package
6127	Sandra Sanches	MX	324	38	Class D Package

18. Validate the results in the Range_Rejects.txt file. Two records were rejected as they did not meet the range criteria specified in the Lookup stage.

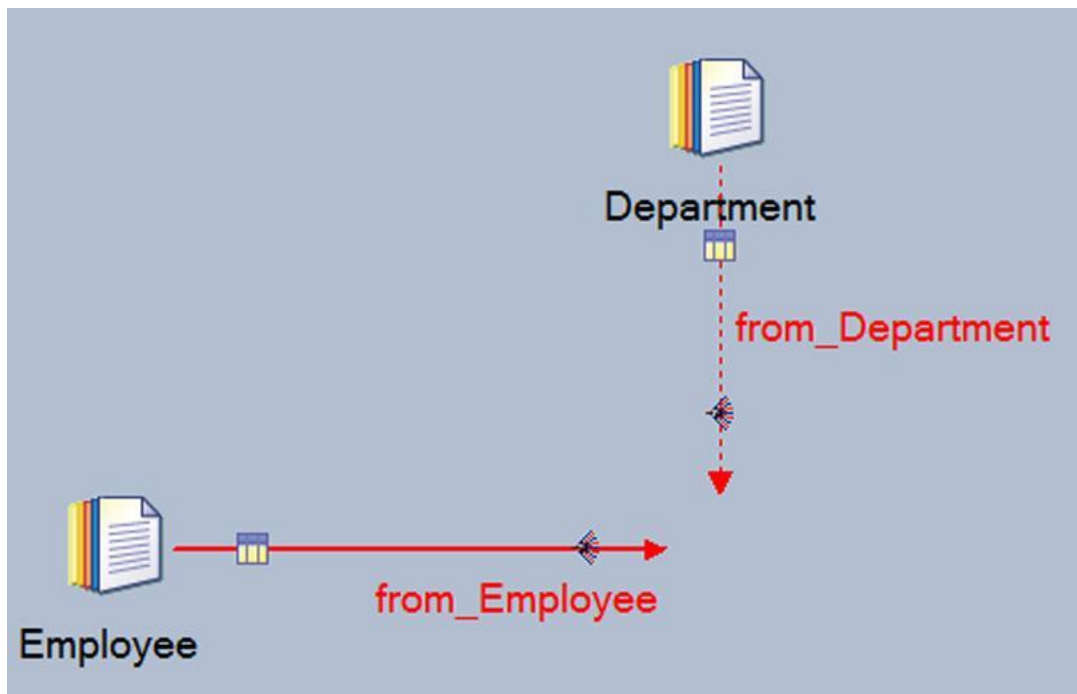


EmpID	EmpName	Location	DeptID	BenCode
6990	Alejandro Escobosa	MX	324	73
2448	Linda Goodman	US	123	61

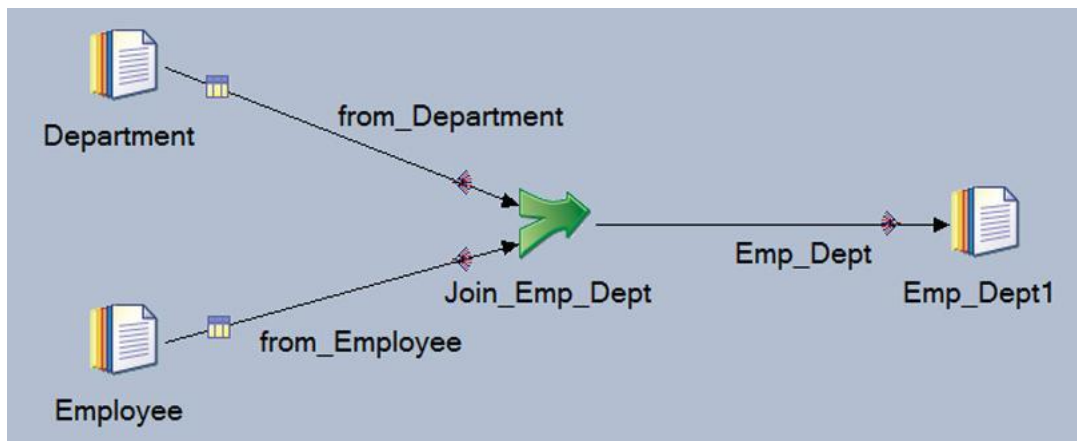
Task: Using the Join stage

In this task, we will join a file containing employee data with a file containing department information. We will use a Join stage and DeptID as our join key.

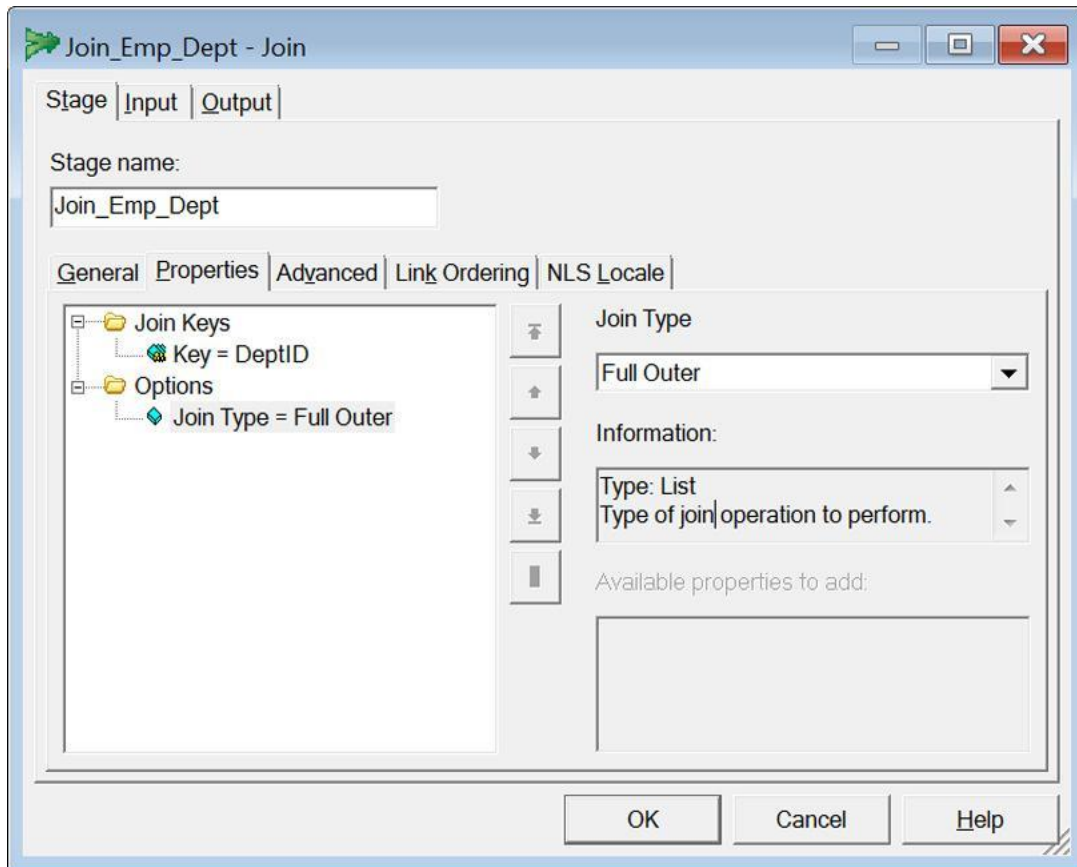
1. Open the LKP_Equality job and save it as JoinEmpDept.
2. Delete the Lookup stage, the Target sequential file, and the link between them. Rename the Department link to "from_Department".



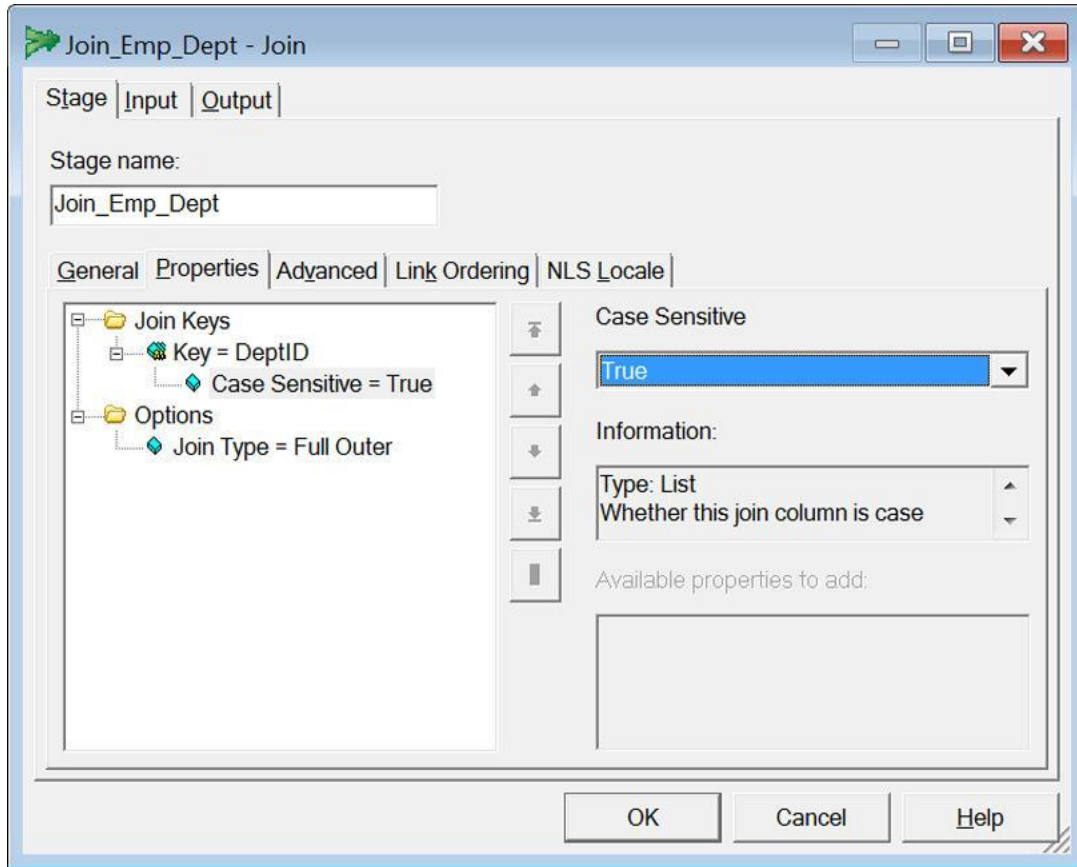
3. Add the stages and links below. Rename them for good standard practice and save your job.



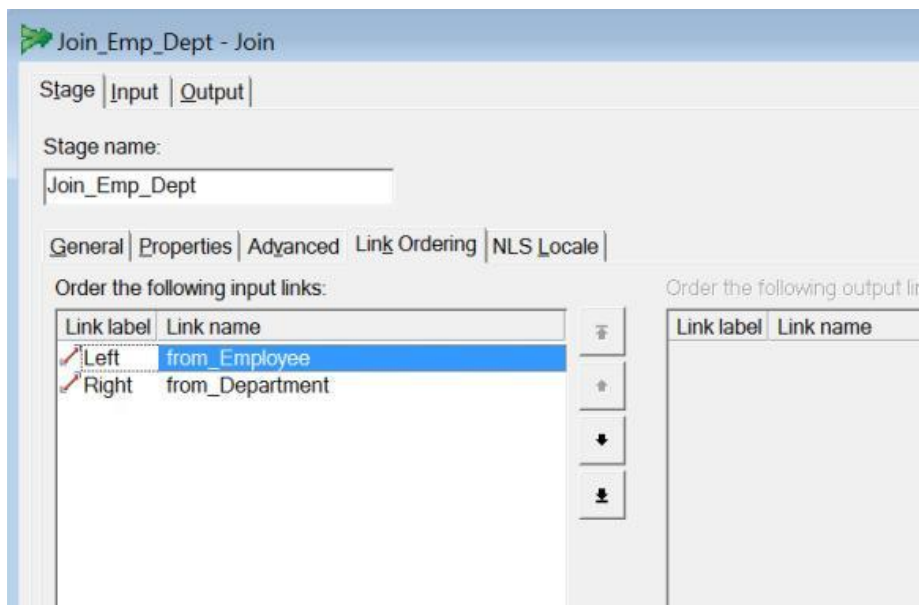
4. Open the Join Stage. Click on the Properties tab and specify the join key as DeptID and Join Type as Full Outer as below.



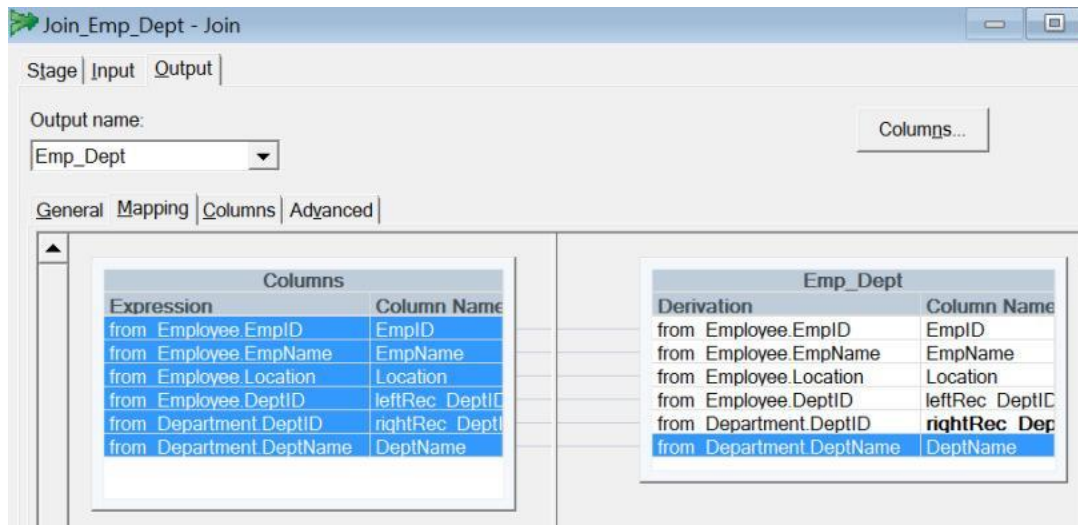
- Click on Key = DeptID to see the Case Sensitive property and set it to True.



- Check the Link Ordering tab. It is important to identify the correct left link and right link when doing either a left outer join or right outer join. Since we are doing a full outer join, it only serves to identify which link the key column is coming from. For this exercise purpose, set the links as shown.



7. Click on the Output > Mapping tab and map the columns to the target.



8. Open the Sequential File stage EmpDept1. On the Properties tab specify the path and file to write the output records /DS_Fundamentals/Labs/Emp_Dept1.txt. Remember to set the First Line is Column Names to True, so that the column names are added to the final file.
9. Save and compile the job.
10. Run the job. It will finish successfully, but with warnings. The case sensitive property has been set to true, but our key is integer and for this reason the property is not recognized.

```

2/21/2012 ... main_program: conductor uname: -s=Linux; -r=2.6.32.12-0.7-pae; -v=#1 SMP 2010-05-20 11:14:20 +0200; -n=infosrvr; -m=i686
2/21/2012 ... main_program: orchgeneral: loaded ...
2/21/2012 ... main_program: Echo: ...
2/21/2012 ... main_program: Explanation: ...
2/21/2012 ... main_program: Dump: ...
2/21/2012 ... main_program: APT configuration file: /opt/IBM/InformationServer/Server/Configurations/default.apt ...
2/21/2012 ... main_program: This step has 5 datasets: ...
2/21/2012 ... Join_Emp_Dept: When checking operator: Function for field "DeptID" of type int32 did not recognize parameters: (case=sensitive). [keygro
2/21/2012 ... main_program: When checking operator: When validating sort key schema: Unrecognized type-specific format properties: (case=sensitive)
2/21/2012 ... main_program: When checking operator: Function for field "DeptID" of type int32 did not recognize parameters: (case=sensitive). [keygro
2/21/2012 ... main_program: When checking operator: When validating sort key schema: Unrecognized type-specific format properties: (case=sensitive)
2/21/2012 ... main_program: When checking operator: Function for field "DeptID" of type int32 did not recognize parameters: (case=sensitive). [keygro
2/21/2012 ... main_program: Schemas: ...
2/21/2012 ... Emolovee.0: Import complete: 5 records imported successfully. 0 rejected.

```

11. Open the generated file in the specified path with the given name to check the data. Verify that two columns with new names were created for our key DeptID.

```

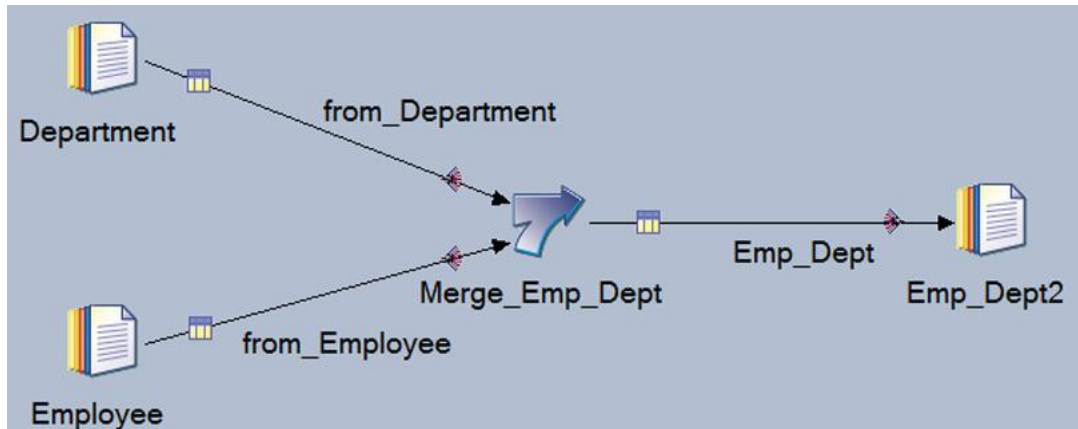
EmpID,EmpName,Location,leftRec_DeptID,rightRec_DeptID,DeptName
5431,Bradley Chen,China,223,223,Global Business Service
2448,Linda Goodman,US,123,123,IBM Software Group
7621,Subramanian Krishnan,Singapore,123,123,IBM Software Group
1976,Mahima Singh,India,324,324,IBM Lab services
8653,Samuel K,India,444,0,

```

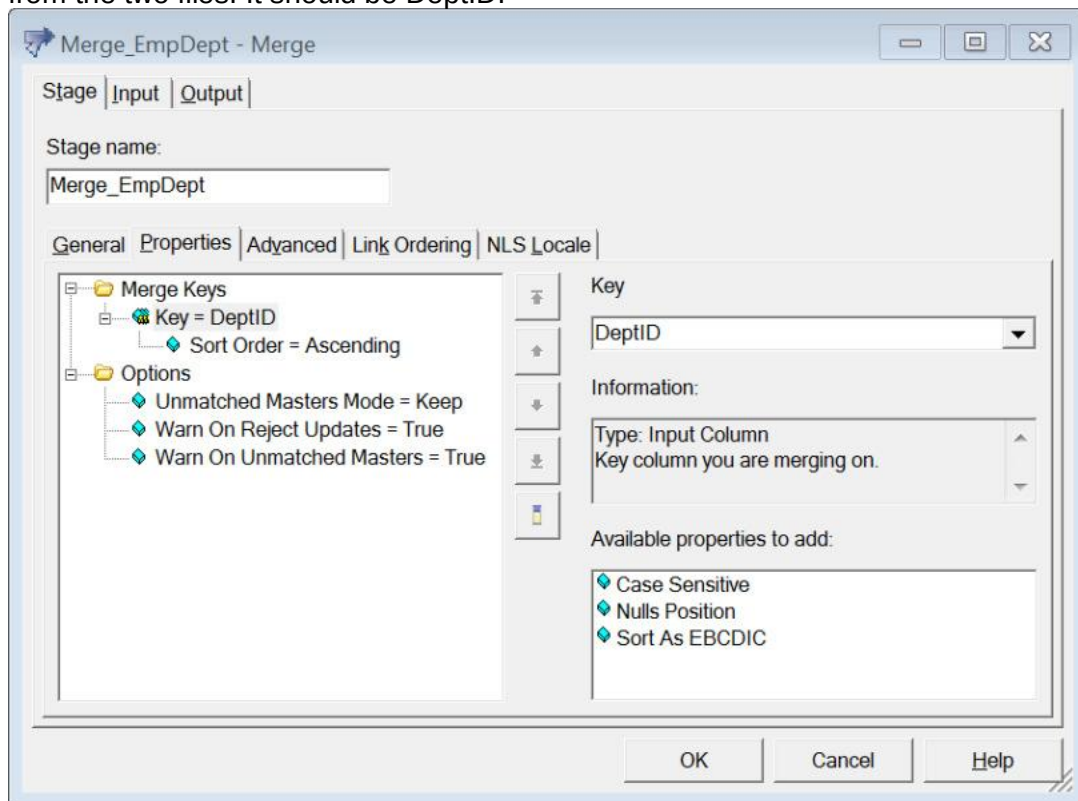
Task: Using the Merge stage

In this task, we will merge the Department details in sequential file Dept.txt with the Employee data in Emp.txt.

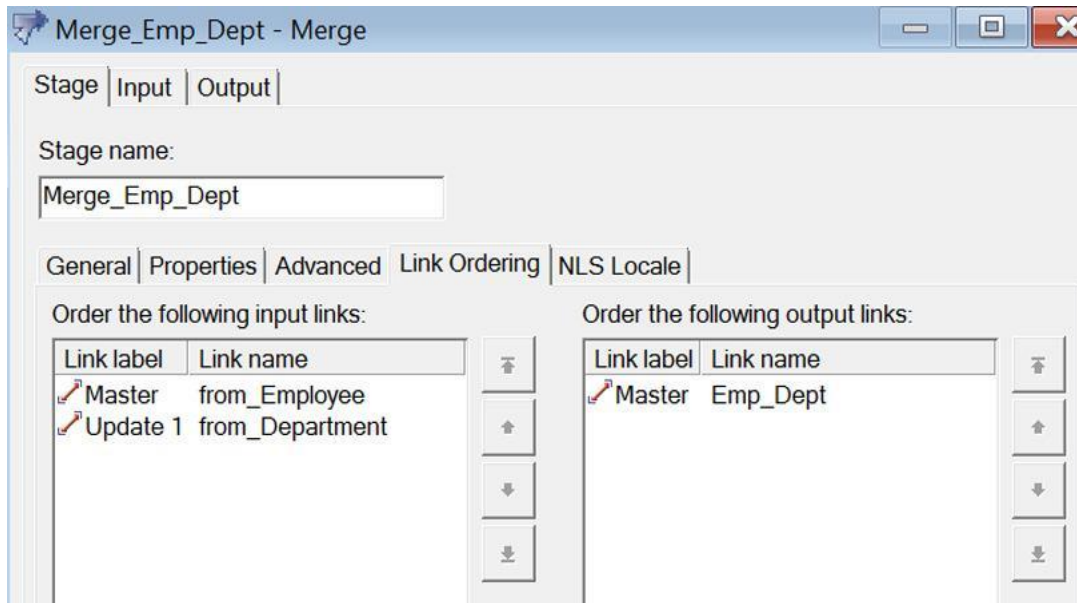
1. Open the JoinEmpDept job and save it as MergeEmpDept.
2. Delete the Join stage, the Target sequential file and the link between them.
3. Add a Merge stage and target Sequential file as below.



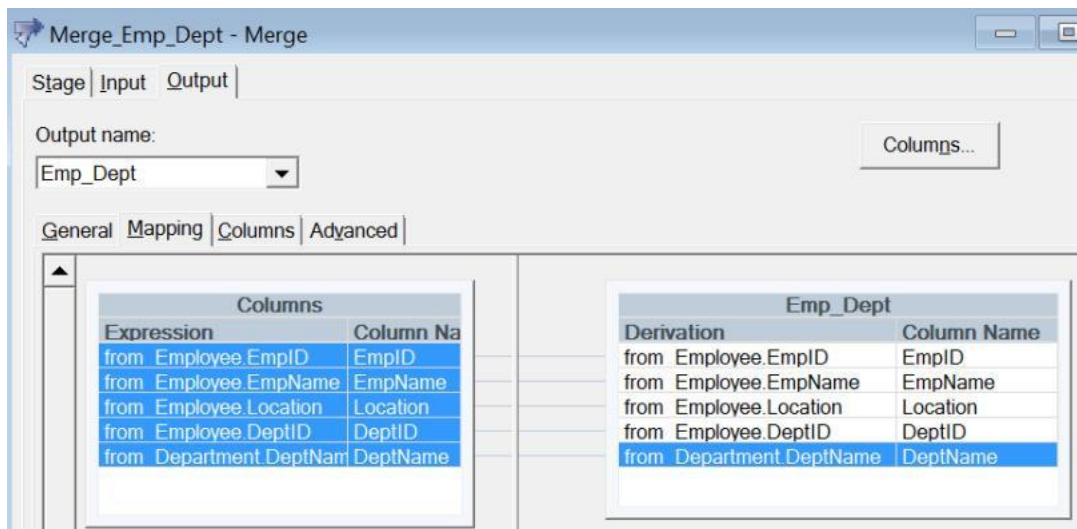
4. Open the Merge stage and specify the Key which will be used for matching records from the two files. It should be DeptID.



- Check the Link Ordering tab to make sure that you have the two input sources set correctly as Master and Update links. For this exercise, from_Employee should be the Master link and from_Department should be the Update link.



- Click on the Output > Mapping tab and map the columns to the target.



- Open Sequential File stage EmpDept2. On the Properties tab specify the path and file to write the output records, i.e. /DS_Fundamentals/Labs/Emp_Dept2.txt. Remember to set the First Line is Column Names to True, so that the column names are added to the final file.
- Save and compile the job.

9. Run the job and see the log. There is a warning for the duplicate key in the master records. And another warning for a master has no updates. Remember: **Links to Merge stage should not have duplicate data!**

Type	Occurred	Event
	2/21/2012 ...	subArgs={asc}}(0),0: Input 0 consumed 1 records.
	2/21/2012 ...	subArgs={asc}}(0),0: Output 0 produced 1 records.
	2/21/2012 ...	Merge_EmpDept,0: Input 0 consumed 1 records. ...
	2/21/2012 ...	Merge_EmpDept,0: Output 0 produced 1 records.
	2/21/2012 ...	subArgs={asc}}(1),1: Input 0 consumed 2 records.
	2/21/2012 ...	subArgs={asc}}(1),1: Output 0 produced 2 records.
⚠	2/21/2012 ...	Merge_EmpDept,1: Master record (1) is a duplicate [merge/merge.C:1075]
⚠	2/21/2012 ...	Merge_EmpDept,1: Master record (3) has no updates. [merge/merge.C:1253]
	2/21/2012 ...	subArgs={asc}}(0),1: Input 0 consumed 4 records.
	2/21/2012 ...	subArgs={asc}}(0),1: Output 0 produced 4 records.
	2/21/2012 ...	EmpDept2,0: Export complete; 5 records exported successfully, 0 rejected.
	2/21/2012 ...	EmpDept2,0: Input 0 consumed 5 records.
	2/21/2012 ...	Merge_EmpDept,1: Input 0 consumed 4 records. ...
	2/21/2012 ...	Merge_EmpDept,1: Output 0 produced 4 records.
	2/21/2012 ...	main_program: Step execution finished with status = OK.
	2/21/2012 ...	main_program: Startup time, 0:00; production run time, 0:00.
	2/21/2012 ...	Parallel job reports successful completion
■	2/21/2012 ...	Finished Job MergeEmpDept.

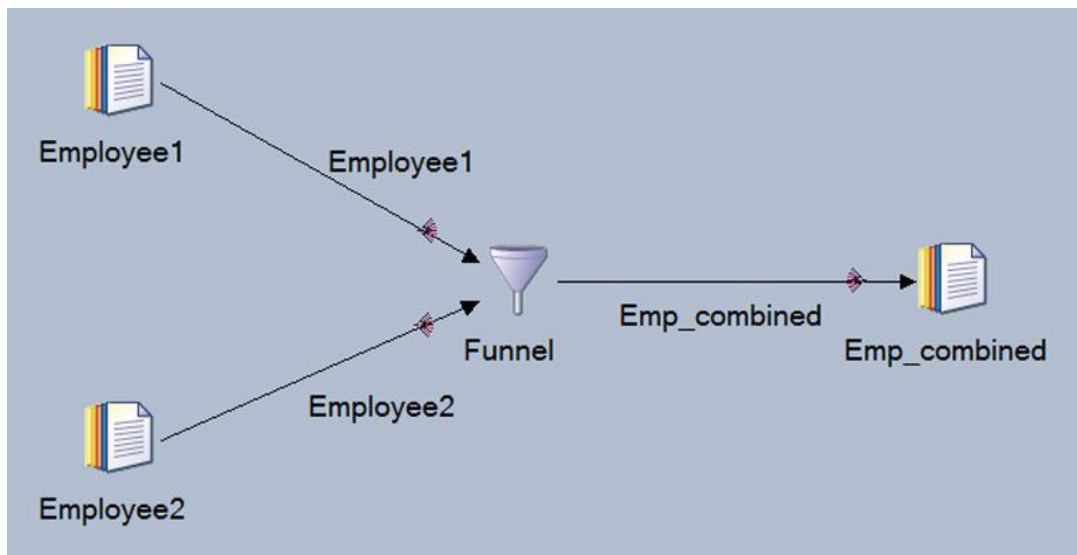
10. If you open the generated file, you will see the records with the duplicate key. This is because the first one used the Update record and the second one found no match. But since the Unmatched Master Record is set to Keep, you get the second record as well. But notice the first warning message is about the duplicate key.

MergeEmpDept..EmpDept2.LnkEmpDept - Data Browser				
EmpID	EmpName	Location	DeptID	DeptName
2448	Linda Goodman	US	123	IBM Software Group
7621	Subramanian Krishnan	Singapore	123	
5431	Bradley Chen	China	223	Global Business Service
1976	Mahima Singh	India	324	IBM Lab services
8653	Samuel K	India	444	

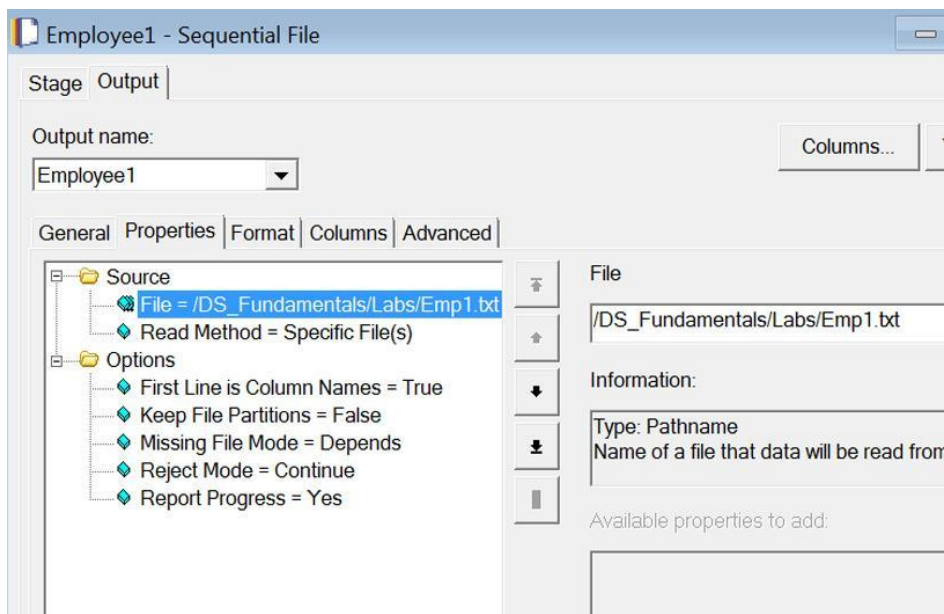
Task: Using Funnel stage

In this task, we will combine data from two different sequential files using Funnel stage. Funnel stage requires that both input files have the same metadata (table definition).

1. Create a new parallel job called FunnelEmp with two sequential file stages. Employee1 should read data from /DS_Fundamentals/Labs/Emp1.txt and Employee2 should read data from /DS_Fundamentals/Labs/Emp2.txt. Add a Funnel stage to combine the data and a Target Sequential file.



2. Open Sequential File stage Employee1. On the Properties tab specify the file to read as /DS_Fundamentals/Labs/Emp1.txt and other relevant properties. Remember to set the First Line is Column Names to True.



3. Click on Formats tab, set the delimiter to comma and quote to none.

- Click on Columns tab, click on the load button to add the column definitions from table definition Emp1.txt.

Employee1 - Sequential File

Stage Output

Output name: Employee1

General Properties Format Columns Advanced

	Column name	Key	SQL type	Extended	Length	Scale	Nullable
1	EmpID	☐	Integer		10		No
2	EmpName	☐	VarChar	Unicode	255		No
3	Company	☐	VarChar	Unicode	255		No
4	Role	☐	VarChar	Unicode	255		No
5	Location	☐	VarChar	Unicode	255		No
		☐					

- Click View Data to verify that the metadata has been specified properly.
- Open Sequential File stage Employee2. On the Properties tab specify the file to read as /DS_Fundamentals/Labs/Emp2.txt and other relevant properties. Don't forget to set the First Line is Column Names to True.

Employee2 - Sequential File

Stage Output

Output name: Employee2

Columns... View Data...

General Properties Format Columns Advanced

Source

- File = /DS_Fundamentals/Labs/Emp2.txt
- Read Method = Specific File(s)

Options

- First Line is Column Names = True
- Keep File Partitions = False
- Missing File Mode = Depends
- Reject Mode = Continue
- Report Progress = Yes

File

/DS_Fundamentals/Labs/Emp2.txt

Information:

Type: Pathname
Name of a file that data will be read from.

Available properties to add:

- Click on Formats tab, set the delimiter to comma and quote to none.

8. Click on Columns tab, click on the load button to add the column definitions from Emp1.txt table definition.

Employee2 - Sequential File

Stage Output

Output name: Employee2

General Properties Format Columns Advanced

	Column name	Key	SQL type	Extended	Length	Scale	Nullable
1	EmpID	☐	Integer		10		No
2	EmpName	☐	VarChar	Unicode	255		No
3	Company	☐	VarChar	Unicode	255		No
4	Role	☐	VarChar	Unicode	255		No
5	Location	☐	VarChar	Unicode	255		No

9. Click View Data to verify that the metadata has been specified properly.
10. Open the Funnel stage and edit the properties to specify the Funnel Type as "Sequence".

Funnel - Funnel

Stage Input Output

Stage name: Funnel

General Properties Advanced Link Ordering NLS Locale

Options

- Funnel Type = Sequence

Funnel Type

Sequence

Continuous Funnel

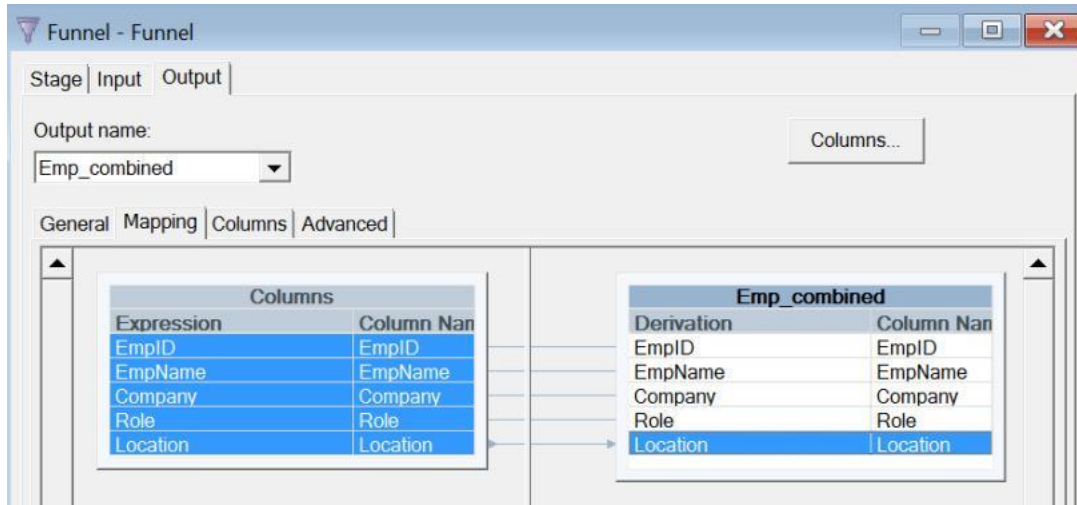
Sequence

Sort Funnel

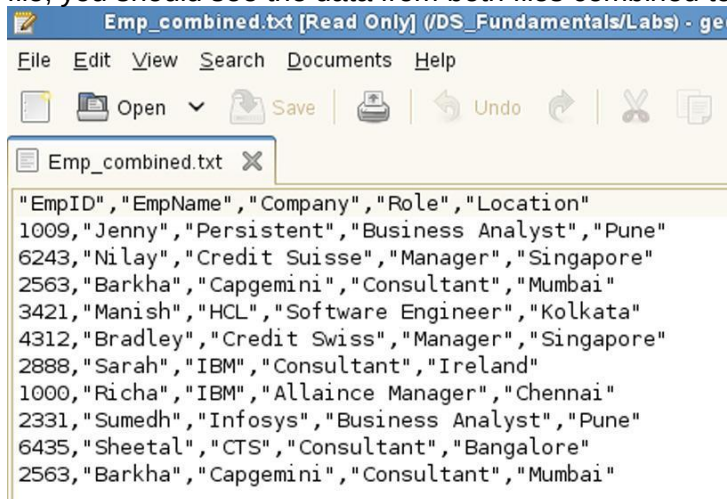
Type: List

Continuous Funnel combines records as they arrive (i.e. no particular order). Sort Funnel

11. Select the Output tab and map the input columns to the output columns.



12. Click "OK".
13. Open Sequential File stage Emp_combined. On the Properties tab specify the path and file to write the output records /DS_Fundamentals/Labs/Emp_combined.txt. Set "First line is column names" to true.
14. Save the job and compile it.
15. Run the job and check the output file Emp_combined for result. When opening the file, you should see the data from both files combined together:



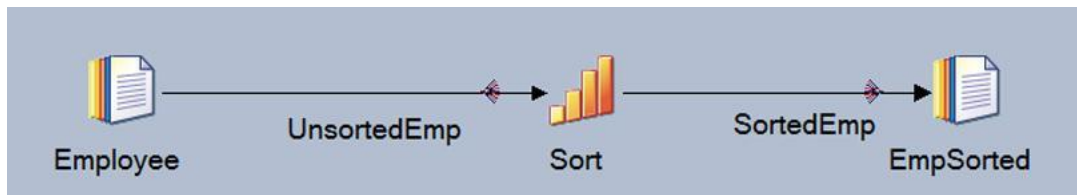
16. Open the /DS_Fundamentals/Labs/Emp1.txt and /DS_Fundamentals/Labs/Emp2.txt to verify they were both combined.

Lab 09: Sorting and Aggregating Data

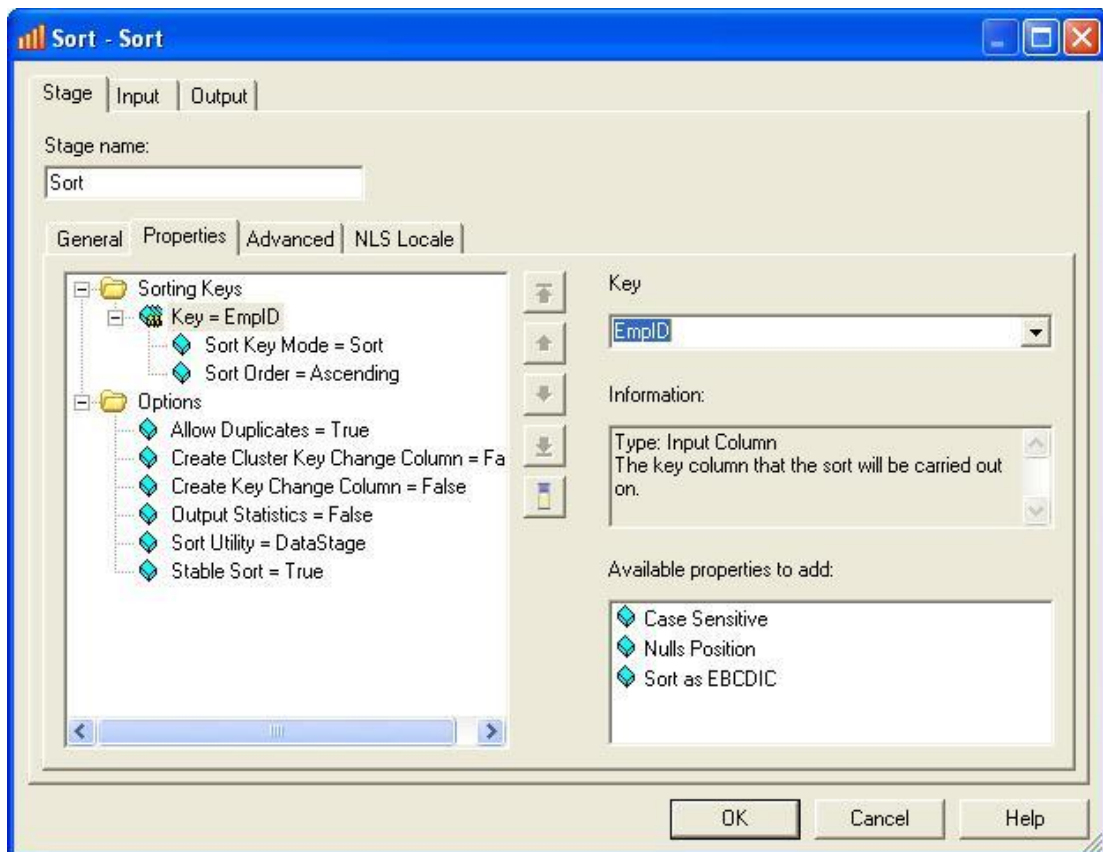
Task: Using Sort stage

In this task, we will build a job using the Sort stage which will sort the Employee data with EmpID as the key.

1. Create a simple Job called SortEmp. Configure the Employee stage to read the file /DS_Fundamentals/Labs/Emp1.txt and the EmpSorted stage to write the output file /DS_Fundamentals/Labs/EmpSorted.txt



2. Edit the Sort stage to specify the key as EmpID and Sort Order is ascending as shown in the snapshot below:



3. Don't forget to map all the input columns to the output of the Sort stage.
4. Save and compile the Job.

- Run the job and check the results. The output should contain data sorted by EmpID in ascending order.

SortEmp..EmpSorted.SortedEmp - Data Browser

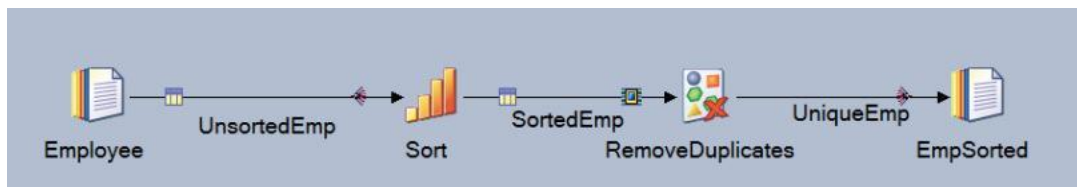
EmpID	EmpName	Company	Role	Location
2331	Sumedh	Infosys	Business Analyst	Pune
2563	Barkha	Capgemini	Consultant	Mumbai
2563	Barkha	Capgemini	Consultant	Mumbai
3421	Manish	HCL	Software Engineer	Kolkata
4312	Bradley	Credit Swiss	Manager	Singapore
6435	Sheetal	CTS	Consultant	Bangalore

Close Find... Display... Help

Task: Using Remove Duplicates stage

In this task, we will use the Remove Duplicate stage to remove the duplicate rows from the sorted data.

- Rename the created job, save it as RemoveDupEmp. Add a Remove Duplicate stage following the Sort stage.



- Edit the Remove Duplicate stage and specify the Key column as EmpID.

RemoveDuplicates - Remove Duplicates

Stage | Input | Output

Stage name: RemoveDuplicates

General | Properties | Advanced | NLS Locale

Keys that define Duplicates

- Key = EmpID

Options

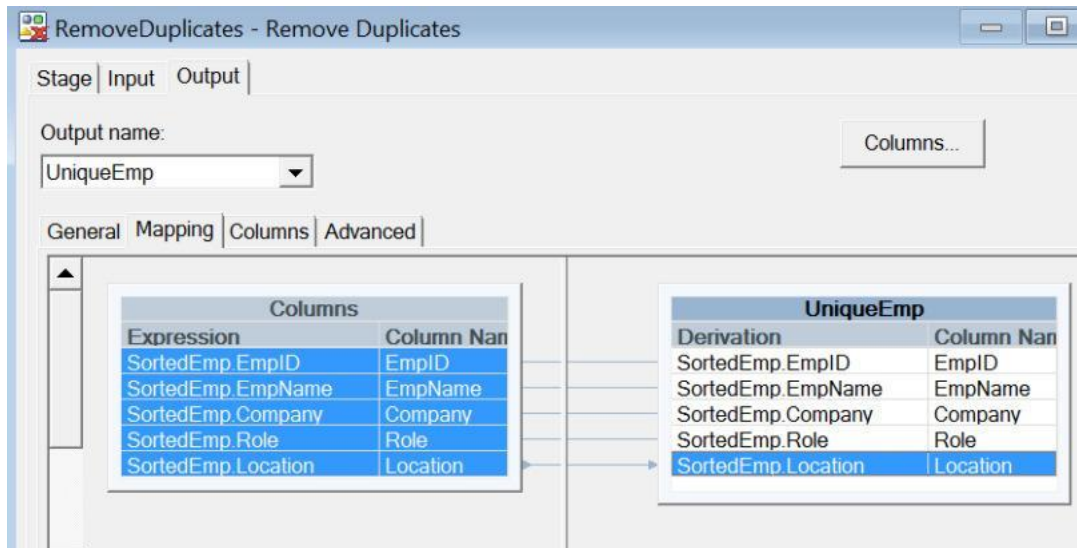
- Duplicate To Retain = First

Key: EmpID

Information:

Type: Input Column
Key column for the operation.

- From the Output tab, click the Mapping tab, and specify the mapping between input and output columns as shown below. Click Ok to close the stage.



- Open the target Sequential File stage and specify the output file as /DS_Fundamentals/Labs/EmpSorted.txt.
- Save and compile the job.
- Run the job and verify the results. Remember, as a result from the last job, EmpID = 2563 appeared twice. After Remove Duplicates was applied, there's only one distinct value for each key.

EmpID	EmpName	Company	Role	Location
2331	Sumedh	Infosys	Business Analyst	Pune
2563	Barkha	Capgemini	Consultant	Mumbai
3421	Manish	HCL	Software Engineer	Kolkata
4312	Bradley	Credit Swiss	Manager	Singapore
6435	Sheetal	CTS	Consultant	Bangalore



Preserve Partitioning Flag - A stage can request that the next stage in the job preserves whatever partitioning it has implemented. This is defined by the Preserve Partitioning flag. If the next stage ignores this request, a warning is displayed on the log to notify the developer.

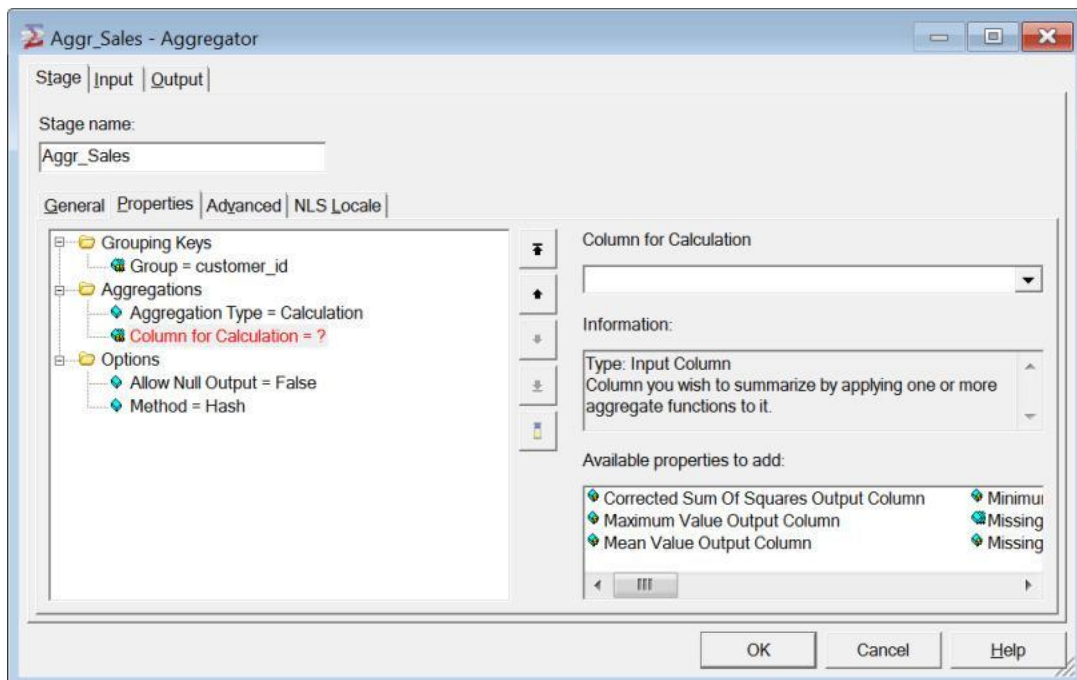
Task: Using Aggregator stage

In this task, we will calculate the revenue generated by customers of a specific Sales Department this month.

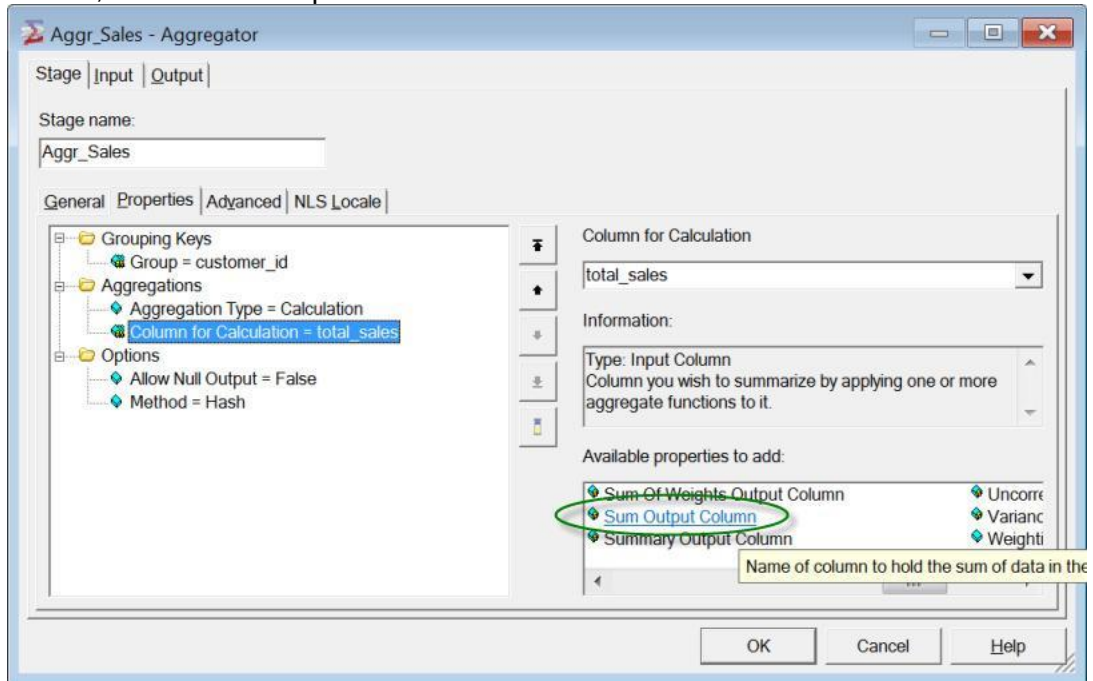
1. Create a simple job called AggrSales with a Sequential File as the input file. Use an Aggregator stage to sum up the sales per customer id and produce the output in a target sequential file with the result.



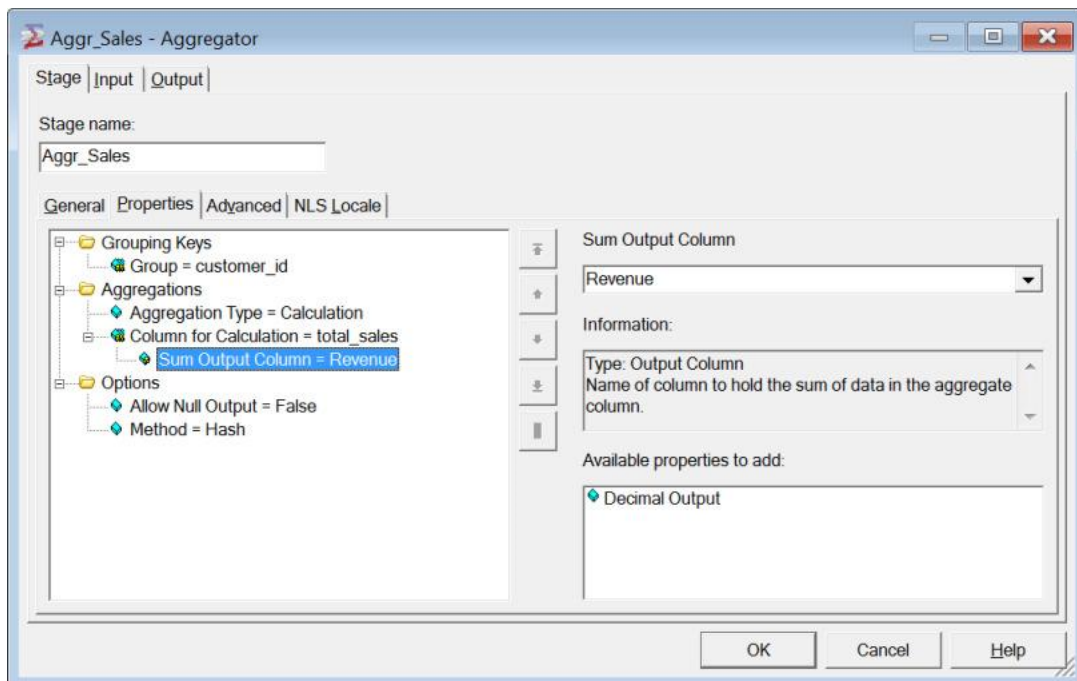
2. Open the Sequential File properties and set the file path to /DS_Fundamentals/Labs/Sales.txt. Don't forget to load the table definition in the Columns tab. It is located at \Table Definitions\Sequential\Labs.
3. Click on View data and make yourself familiar with the input columns before they are processed. We will use the Aggregator stage to calculate the total revenue for the sales data in this file.
4. Edit the Aggregator stage to add the grouping key, Customer_id. Also set the property Aggregation Type = Calculation, as shown below.



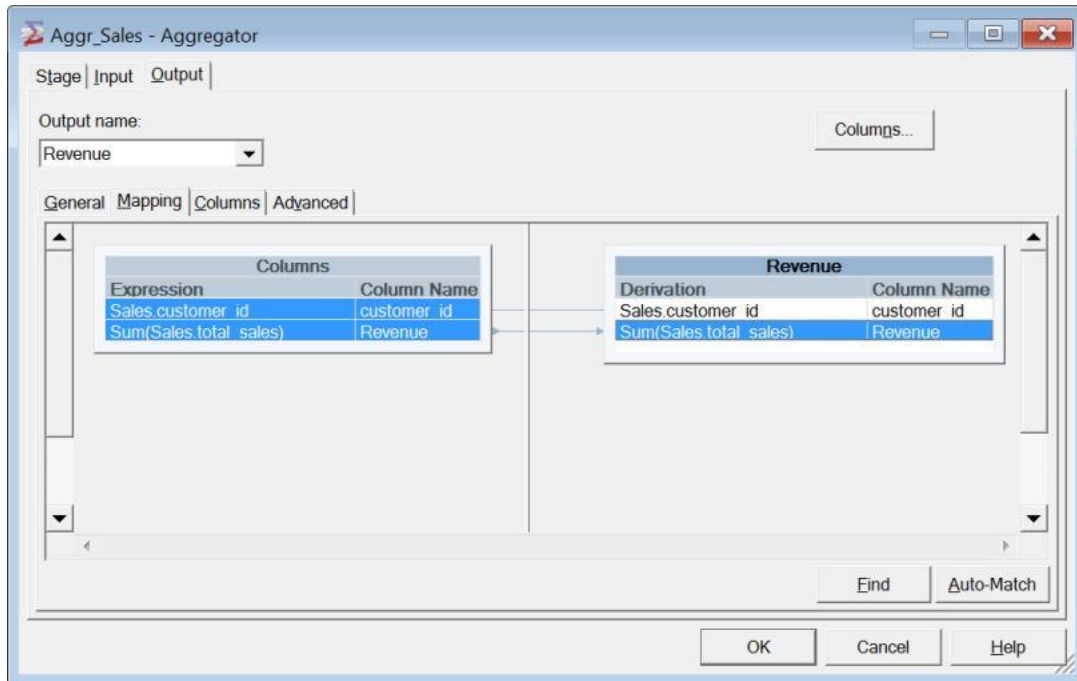
5. Select the Column for Calculation = total_sales and at the right bottom portion of the screen, select Sum Output Column.



6. A new column will be generated with the aggregation results. Type the new column name, Sum Output Column = Revenue.



- Click on the Output tab > Mapping sub-tab and map the input fields that should be in the target file.



- Click OK
- Open the Sales_Revenue Sequential File. On the Properties tab specify /DS_Fundamentals/Labs/Sales_Revenue.txt as the file to write and other relevant properties.
- On the Partitioning tab, select Sort Merge for collector type; check the Perform sort box, and select Revenue as the key with an option of Descending order.
- Save and compile.
- Run the job and verify the results. The final file should contain the Grouping Key = customer_id and the Revenue column with the results.

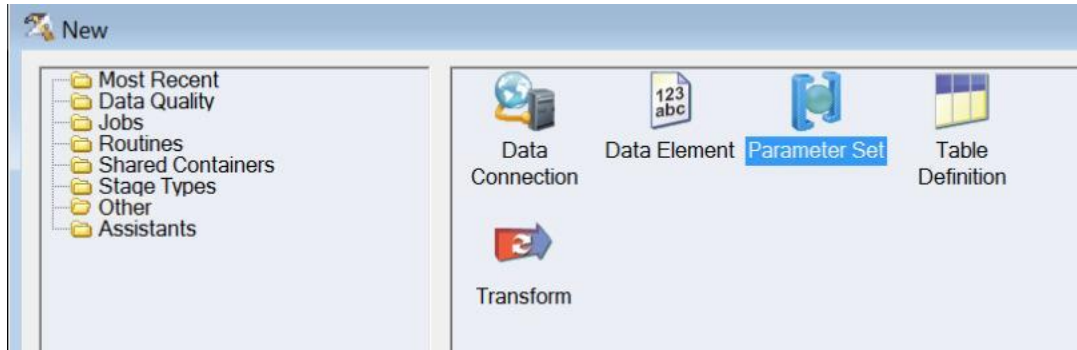
AggrSales..Sales_Revenue.Revenue

customer_id	Revenue
592	1774
109	1575
387	1481
740	1054
984	528
995	363
548	114

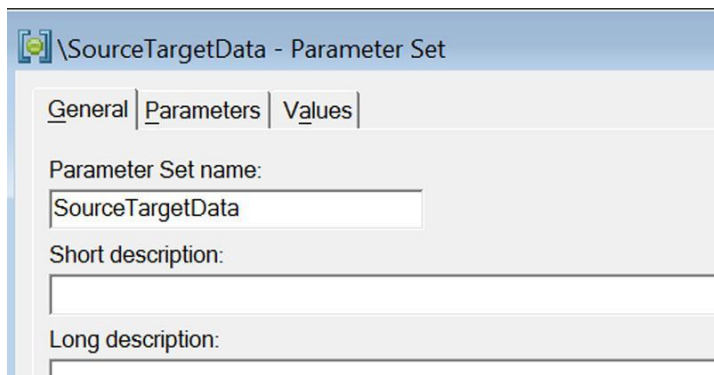
Lab 10: Transforming Data

Task: Create a parameter set

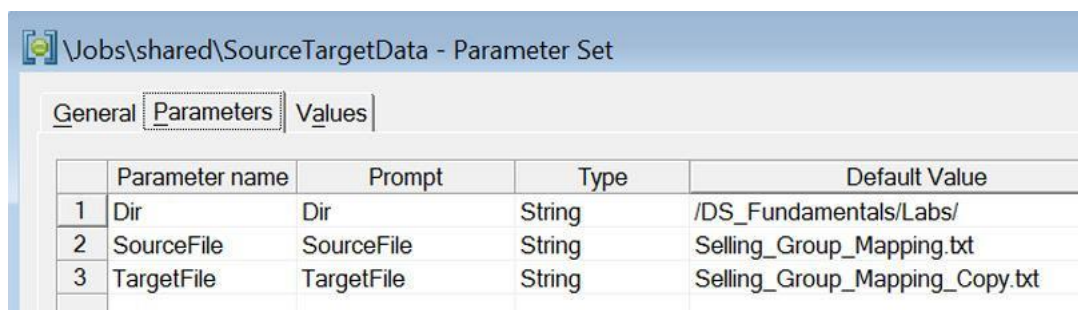
1. Click the New button on the Designer toolbar and then open the “Other” folder.



2. Double-click on the Parameter Set icon.
3. On the General tab, name your parameter set SourceTargetData.



4. On the Parameters tab, define the parameters as shown. Don't forget the last slash for the directory value.



5. On the Values tab, specify a name for the Value File that holds all the job parameters within this Parameter Set.



6. Save your new parameter set in Jobs > shared.

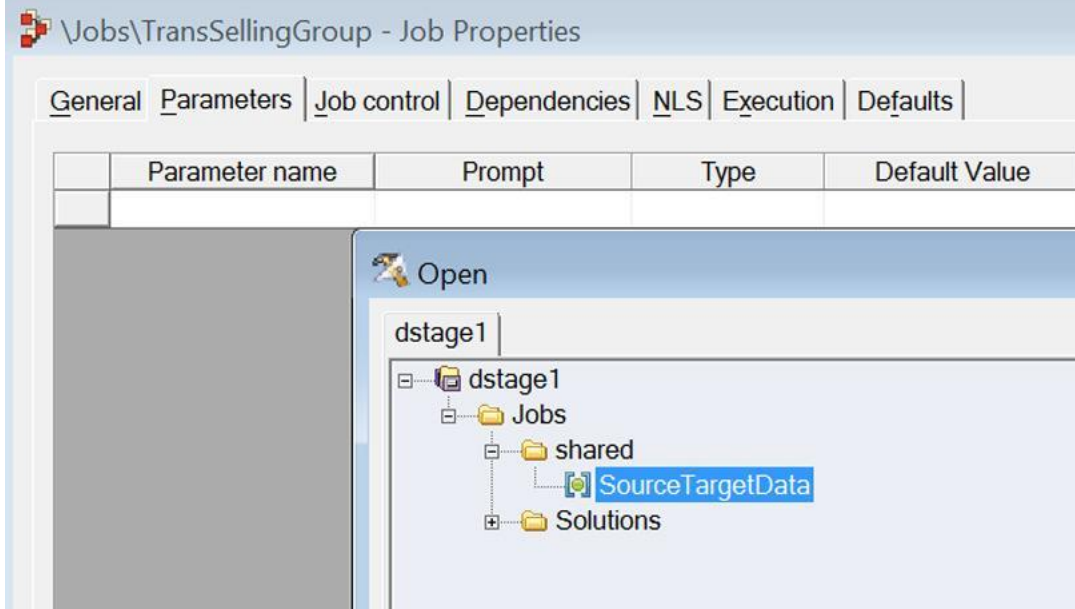


Task: Add a Transformer stage to a job and define a constraint

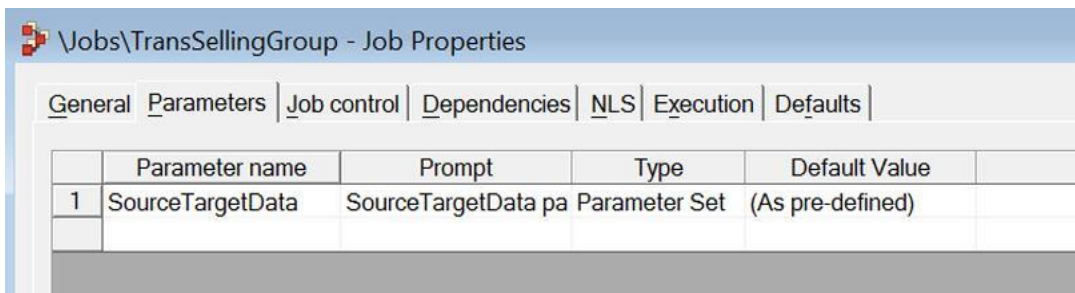
1. Create a parallel job TransSellingGroup as shown then save the job.



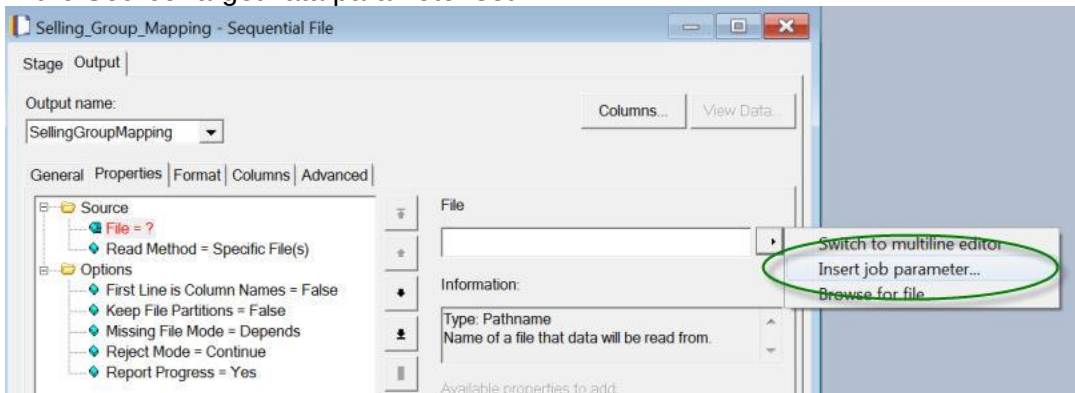
- In the menu bar, go to Edit and open up your Job Properties. Select the Parameters tab. Click Add Parameter Set. Select your SourceTargetData parameter set.



- Click OK.



- Configure the source Sequential File stage properties using the parameters included in the SourceTargetData parameter set.



File

#SourceTargetData.Dir##SourceTargetData.SourceFile#

5. Set the option “First Line is Column Names” as True.
6. Click Format tab, set Quote to none under Field defaults.
7. Load the Selling_Group_Mapping table definition previously imported in the Columns tab.

Selling_Group_Mapping - Sequential File

Stage Output

Output name:
SellingGroupMapping

General Properties Format Columns Advanced

	Column name	Key	SQL type	Extended	Length	Scale	Nullable
1	Selling_Group_Code	☐	Integer		10		No
2	Selling_Group_Desc	☐	VarChar	Unicode	255		No
3	Special_Handling_Code	☐	Integer		10		No
4	Distr_Chann_Desc	☐	VarChar	Unicode	255		No

8. In the Transformer stage, map all the columns from the source link to the target link selecting all the source columns and drag-dropping them to the output link. The transformer editor should appear as shown below:

Transform - Transformer Stage

SellingGroupMapping

Selling_Group_Code
Selling_Group_Desc
Special_Handling_Code
Distr_Chann_Desc

Stage Variables

Derivation Stage Variable

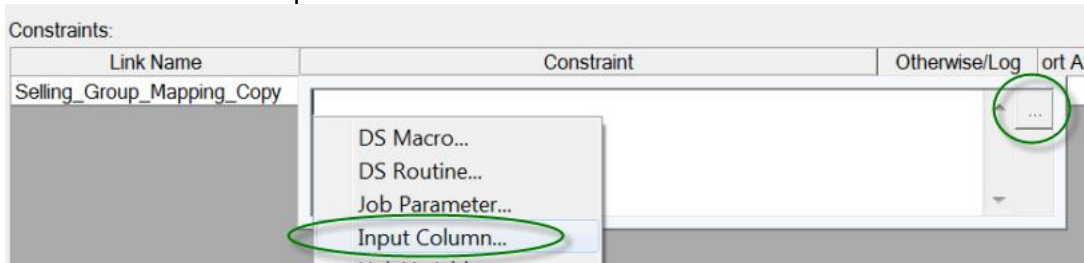
Loop Condition (No Loop)

Selling_Group_Mapping_Copy

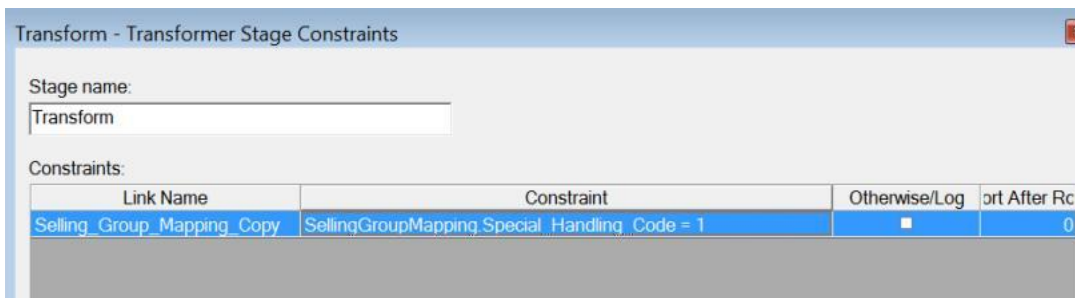
Constraint:

Derivation	Column Name
SellingGroupMapping.Selling_Group_Code	Selling_Group_Code
SellingGroupMapping.Selling_Group_Desc	Selling_Group_Desc
SellingGroupMapping.Special_Handling_Code	Special_Handling_Code
SellingGroupMapping.Distr_Chann_Desc	Distr_Chann_Desc

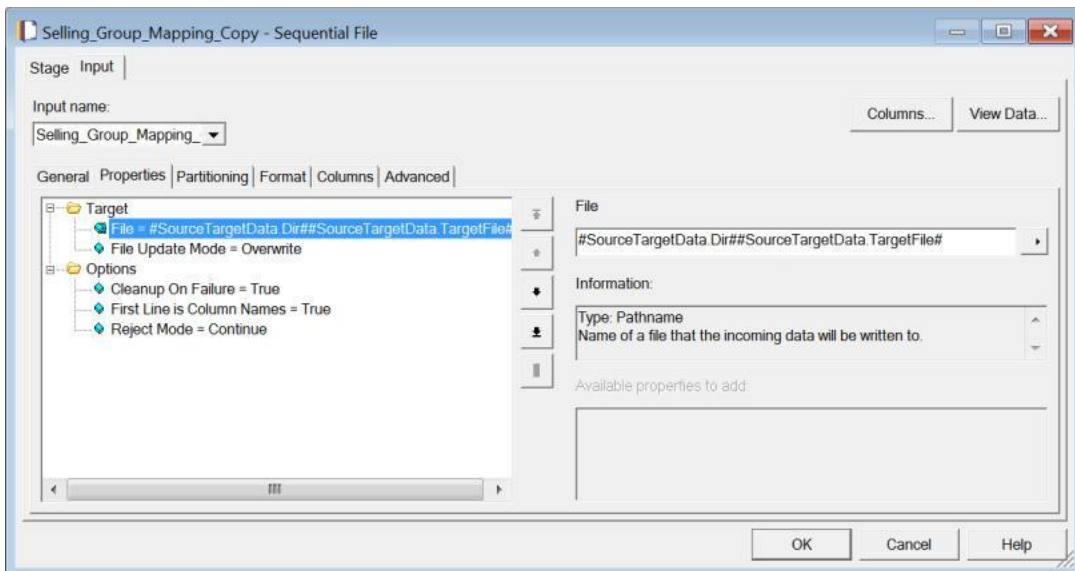
9. Open the transformer stage constraints by clicking on the chain icon and create a constraint that selects only records with a Special_Handling_Code = 1. To insert the column name without typing it, double click into the Constraint field, click on the '...' icon and select "Input Column"



10. Select SellingGroupMapping.Special_Handling_Code and complete the constraint. Hit Enter.



11. Select OK and close the stage editor.
12. Configure the properties for the target Sequential File stage. Use the TargetFile parameter included in the SourceTargetData parameter set to define the File property as shown. Also, set the option First Line is Column Names as True.

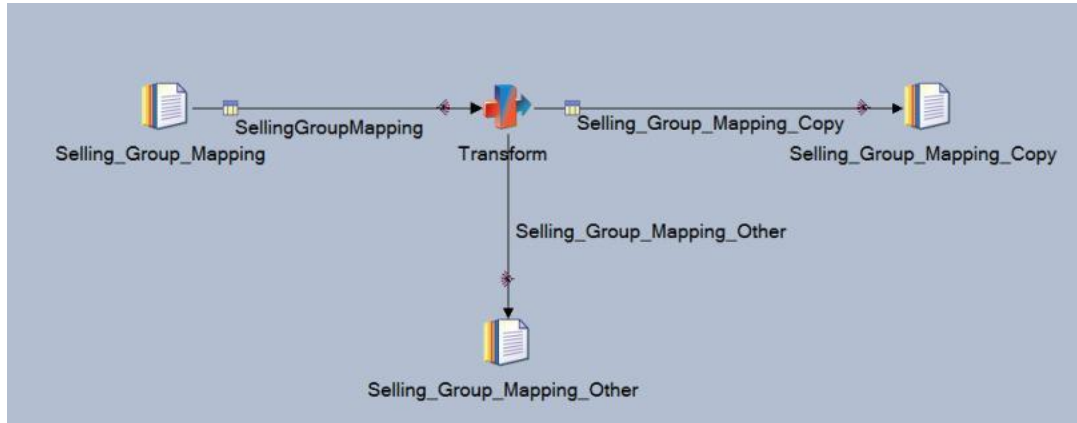


13. Compile and run your job.

14. View the data in the target and verify that there are only records having Special_Handling_Code = 1.

Task: Define an Otherwise link

1. Save the job TransSellingGroup as TransSellingGroupOtherwise.
2. Add a new Sequential File stage linked to the Transformer stage and name it as shown below.



3. In the Transformer, map all the input columns across to the new target link.

Transform - Transformer Stage

SellingGroupMapping

- Selling_Group_Code
- Selling_Group_Desc
- Special_Handling_Code
- Distr_Chann_Desc

Stage Variables

Derivation	Stage Variable

Loop Condition (No Loop)

Selling_Group_Mapping_Copy

Constraint: SellingGroupMapping.Special_Handling_Code = 1

Derivation	Column Name
SellingGroupMapping.Selling_Group_Code	Selling_Group_Code
SellingGroupMapping.Selling_Group_Desc	Selling_Group_Desc
SellingGroupMapping.Special_Handling_Code	Special_Handling_Code
SellingGroupMapping.Distr_Chann_Desc	Distr_Chann_Desc

Selling_Group_Mapping_Other

Constraint:

Derivation	Column Name
SellingGroupMapping.Selling_Group_Code	Selling_Group_Code
SellingGroupMapping.Selling_Group_Desc	Selling_Group_Desc
SellingGroupMapping.Special_Handling_Code	Special_Handling_Code
SellingGroupMapping.Distr_Chann_Desc	Distr_Chann_Desc

4. Open the Constraints window for the Selling_Group_Mapping_Other output link.
Note: You can also double-click the Constraint box.

Selling_Group_Mapping_Other	
Constraint:	
Derivation	Column Name
SellingGroupMapping.Selling_Group_Code	Selling_Group_Code
SellingGroupMapping.Selling_Group_Desc	Selling_Group_Desc
SellingGroupMapping.Special_Handling_Code	Special_Handling_Code
SellingGroupMapping.Distr_Chann_Desc	Distr_Chann_Desc

5. Check the Otherwise box for the Selling_Group_Mapping_Other link.

Transform - Transformer Stage Constraints			
Stage name: Transform			
Constraints:			
Link Name	Constraint	Otherwise/Log	Abort After Rows
Selling_Group_Mapping_Copy	SellingGroupMapping.Special_Handling_Code = 1	☐	0
Selling_Group_Mapping_Other		☒	0

6. Close the stage editor.
7. Edit the Selling_Group_Mapping_Other Sequential File stage as shown.

Selling_Group_Mapping_Other - Sequential File

Stage Input

Input name:
Selling_Group_Mapping_

General Properties Partitioning Format Columns Advanced

Target

- File = #SourceTargetData.Dir#Selling_Group_Mapping_Otherwise.txt
- File Update Mode = Overwrite

Options

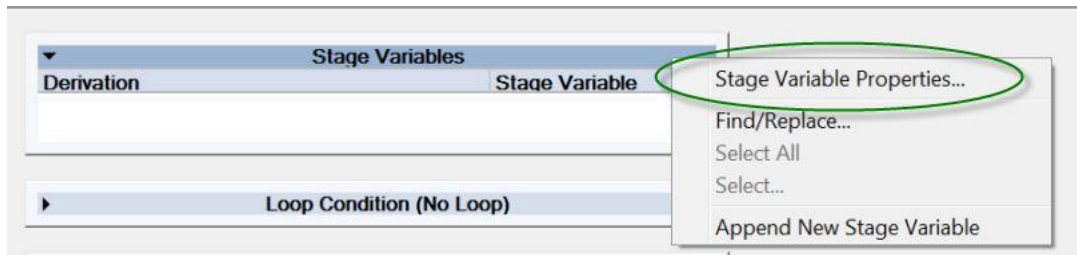
- Cleanup On Failure = True
- First Line is Column Names = True
- Reject Mode = Continue

8. Compile, run, and test your job. The rows going into the Selling_Group_Mapping_Other link should be all the rows that do not satisfy the constraint defined for the first link.

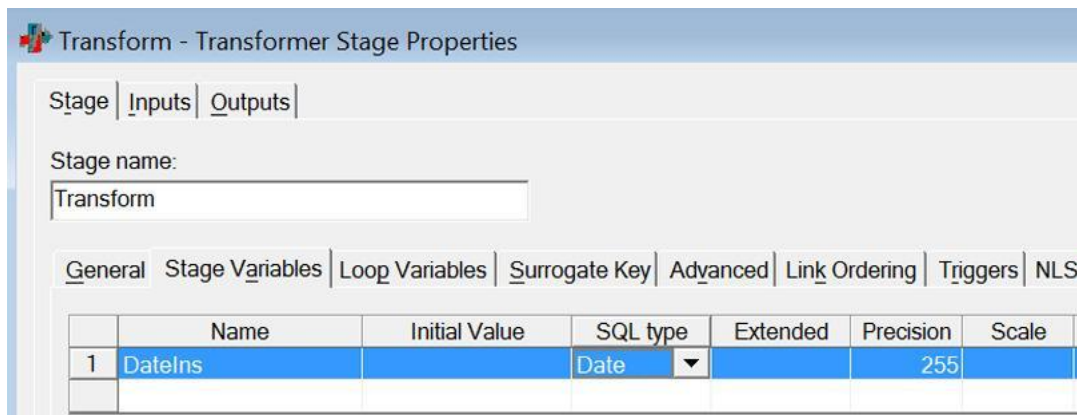
Task: Define derivations

In this task, you will define derivations. The first derivation constructs addresses from several input columns. The second defines the current date at runtime.

1. Save the job TransSellingGroupOtherwise as TransSellingGroupDerivations.
2. Open the Transformer. Right mouse click on the Stage Variables window, and click Stage Variable Properties,

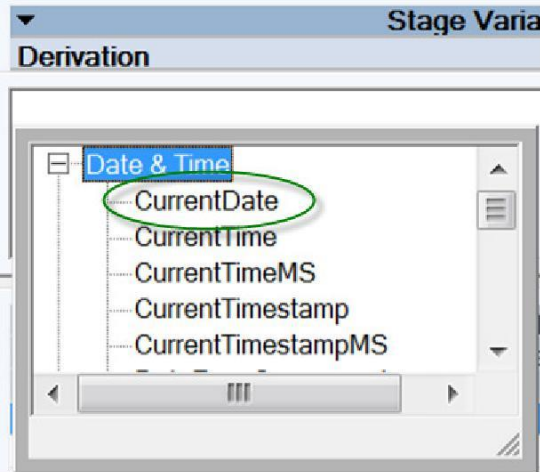
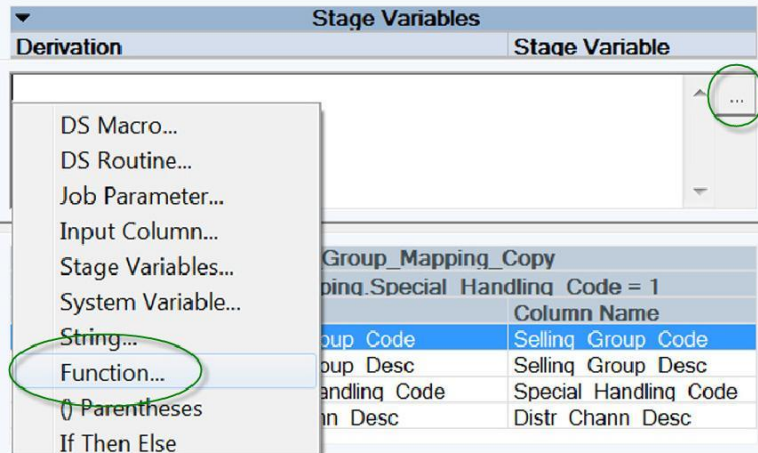


3. Under the Stage Variables tab, create a stage variable named DateIns with Date as the SQL type.



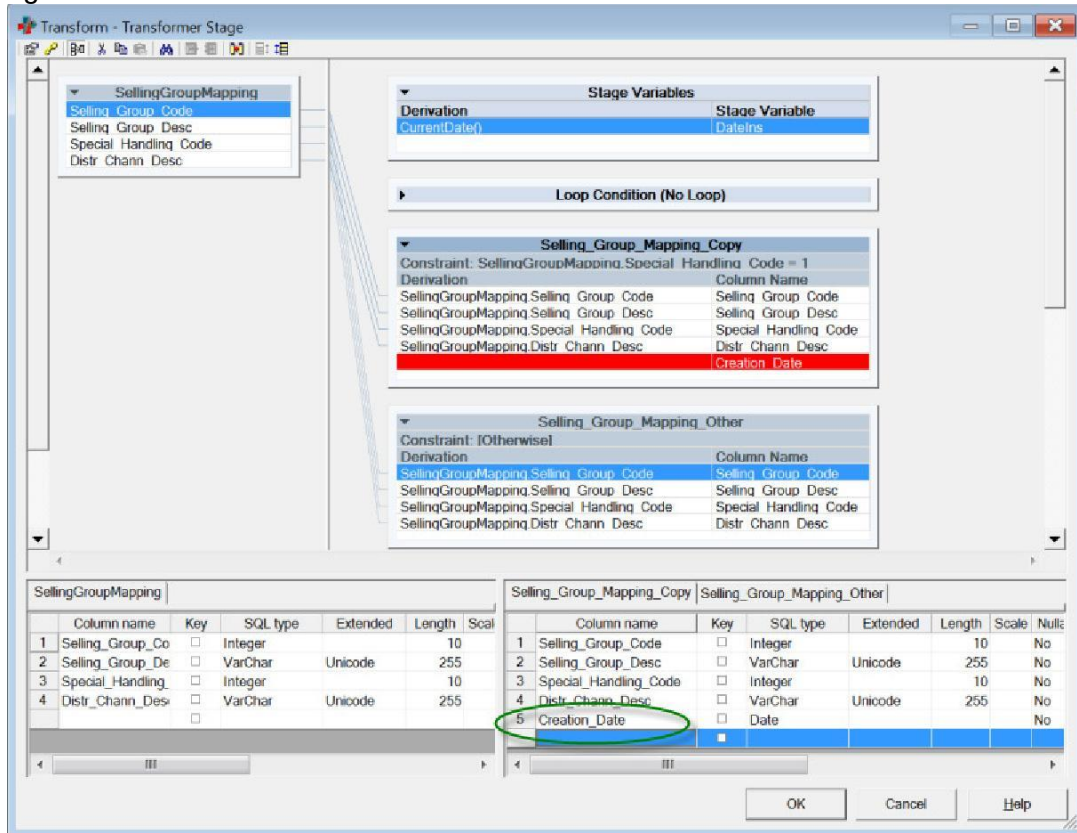
4. Close the Stage Variable Properties window.

5. Double-click in the derivation editor for the DateIns stage variable. Define a derivation that contains the current date using the function CurrentDate() for DateIns stage variable. You can either type or look the function up.



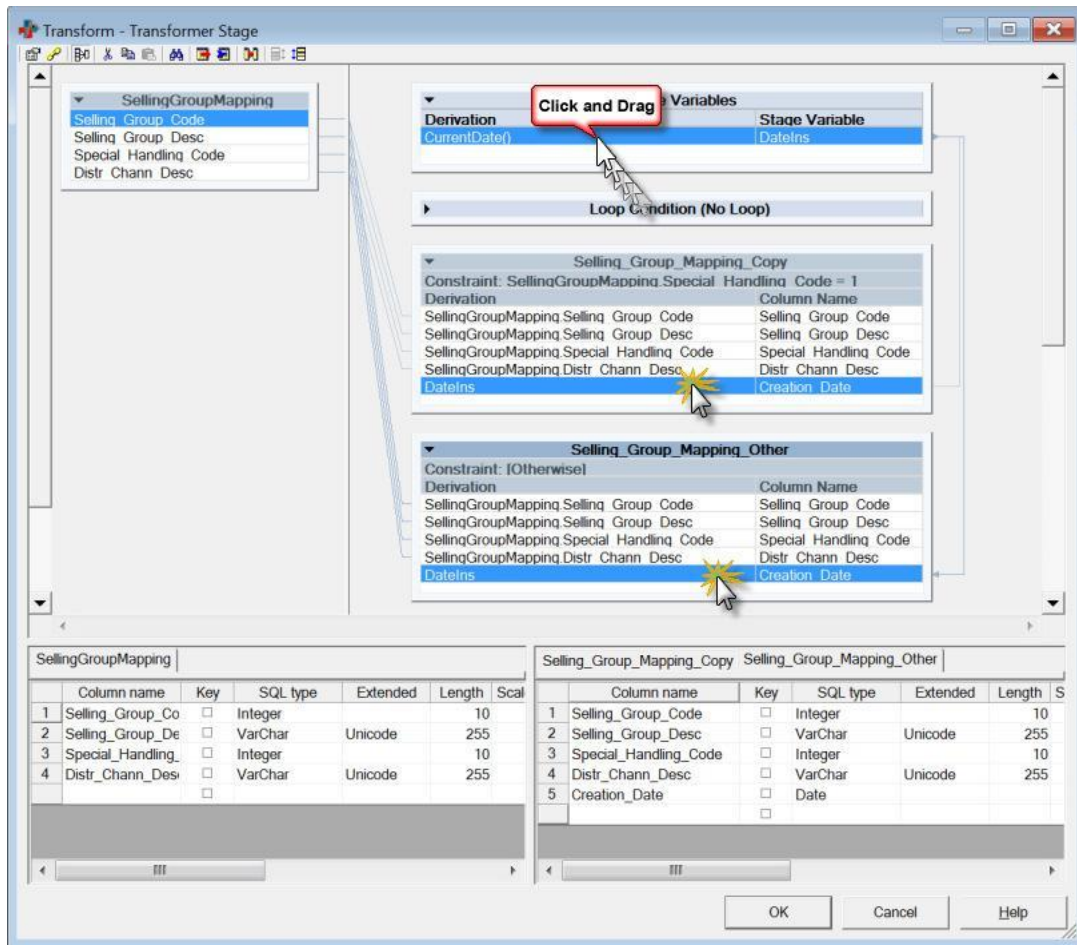
Stage Variables	
Derivation	Stage Variable
CurrentDate()	DateIns

6. Create a new column named `Creation_Date` with `Date` as the SQL type for each of the two output links by typing the new column name and its corresponding properties in the next empty row of the output column definition grid located at the right bottom as shown here.



Selling_Group_Mapping_Copy							
	Column name	Key	SQL type	Extended	Length	Scale	Null
1	Selling_Group_Code	☐	Integer		10		No
2	Selling_Group_Desc	☐	VarChar	Unicode	255		No
3	Special_Handling_Code	☐	Integer		10		No
4	Distr_Chann_Desc	☐	VarChar	Unicode	255		No
5	Creation_Date	☐	Date				No

- Define the derivations for these columns using the Stage Variable DateIns by dragging the DateIns variable and dropping it into the Derivation space of the Creation_Date fields. The Transformer editor should look like this:



- Exit the Stage Editor, save, compile and run the job.
- When you view the result data files, you will find the Creation_Date column filled.

TransSellingGroupDerivations..Selling_Group_Mapping_Copy.Selling_Group_Mapping_Copy - Data Browser

Selling_Group_Code	Selling_Group_Desc	Special_Handling_Code	Distr_Chann_Desc	Creation_Date
1210000	SG121 RETAIL FRESH	1	Retail	2012-02-22
1390000	SG139 MILITARY	1	Retail	2012-02-22
6120000	SG612 CONTINENTAL DELI	1	Retail	2012-02-22
6140000	SG614 RUSSER FOODS	1	Retail	2012-02-22
1200000	SG120 RETAIL F/P	1	Retail	2012-02-22
1400000	SG140 CLUB STORES	1	Retail	2012-02-22
5230000	SG523 ENTREE'	1	Retail	2012-02-22
6110000	SG611 REFRIGERATED PROCES	1	Retail	2012-02-22
6130000	SG613 IOWA HAM	1	Retail	2012-02-22

Extra Credit Exercise

10. Open the Transformer Stage Editor.
11. Write a derivation for the target Selling_Group_Desc column for the link of Selling_Group_Mapping_Copy that will replace "SG614" by "SH055", leaving the rest of the description as it is. In other words, "SG614 RUSSER FOODS", for example, becomes "SH055 RUSSER FOODS". Hint: use the IF THEN ELSE operator. Also, you will need to use the substring operator and the Len functions.

Selling_Group_Mapping_Copy	
Constraint: SellingGroupMapping.Special Handling Code = 1	
Derivation	Column Name
SellingGroupMapping.Selling_Group_Code	Selling_Group_Code
IF SellingGroupMapping.Selling_Group_Desc[1,5] = "SG614" THEN "SH055" : SellingGroupMapping.Selling_Group_Desc[6,Len (SellingGroupMapping.Selling_Group_Desc)-5] ELSE SellingGroupMapping.Selling_Group_Desc	Selling_Group_Desc
SellingGroupMapping.Special Handling Code	Special Handling Code
SellingGroupMapping.Distr_Chann_Desc	Distr_Chann_Desc
DateIns	Creation Date

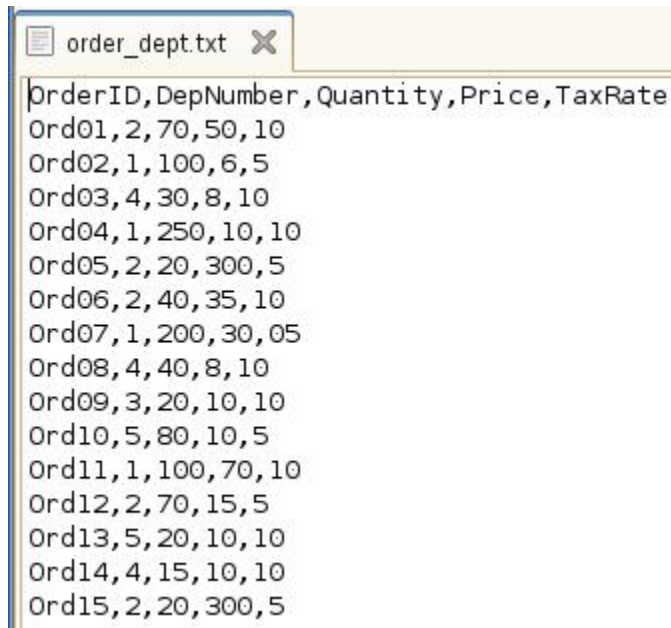
```
IF SellingGroupMapping.Selling_Group_Desc[1,5] = "SG614" THEN "SH055" :
SellingGroupMapping.Selling_Group_Desc[6,Len
(SellingGroupMapping.Selling_Group_Desc)-5] ELSE
SellingGroupMapping.Selling_Group_Desc
```

12. Compile, run, and test your job. Here is some of the output. Notice specifically, the row (614000), which shows the replacement of SG614 with SH055 in the second column.

TransSellingGroupDerivations..Selling_Group_Mapping_Copy.Selling_Group_Mapping_Copy - Data				
Selling_Group_Code	Selling_Group_Desc	Special_Handling_Code	Distr_Chann	
1200000	SG120 RETAIL F/P	1	Retail	
1400000	SG140 CLUB STORES	1	Retail	
5230000	SG523 ENTREE'	1	Retail	
6110000	SG611 REFRIGERATED PROCES	1	Retail	
6130000	SG613 IOWA HAM	1	Retail	
1210000	SG121 RETAIL FRESH	1	Retail	
1390000	SG139 MILITARY	1	Retail	
6120000	SG612 CONTINENTAL DELI	1	Retail	
6140000	SH055 RUSSER FOODS	1	Retail	

Task: Using a Transformer's loop function

1. Open the file /DS_Fundamentals/Labs/order_dept.txt. It includes information about orders made by five different departments. Each department is identified by a department number field called DepNumber.



```
order_dept.txt
OrderID,DepNumber,Quantity,Price,TaxRate
Ord01,2,70,50,10
Ord02,1,100,6,5
Ord03,4,30,8,10
Ord04,1,250,10,10
Ord05,2,20,300,5
Ord06,2,40,35,10
Ord07,1,200,30,05
Ord08,4,40,8,10
Ord09,3,20,10,10
Ord10,5,80,10,5
Ord11,1,100,70,10
Ord12,2,70,15,5
Ord13,5,20,10,10
Ord14,4,15,10,10
Ord15,2,20,300,5
```

2. Our goal is to generate a new column ValuePrc implementing the following rule:

$$\text{ValuePrc} = \text{Single Order Value} / \text{Total Department Orders} * 100$$

Where “Single Order Value” = Price * Quantity for each order and “Total Department Orders” is the cumulated value for all the orders made by a specific department.

3. Create a parallel job including two Sequential File stages, a Sort stage and a Transformer stage as shown. Save it as TransOrdersDept.



4. Import the table definition for the source Sequential File stage with the order_dept.txt file. Make sure you check the box of "First line is column names".

Define Sequential Meta Data

Filename: /DS_Fundamentals/Labs/order_dept.txt (Map=UTF8)

Format | Define

☐ Fixed-width columns

☒ First line is column names

Fixed-width

Column widths:

Spaces between columns:

Delimited

☐ Tab

☐ Space

☒ Comma

Other Delimiter: ,

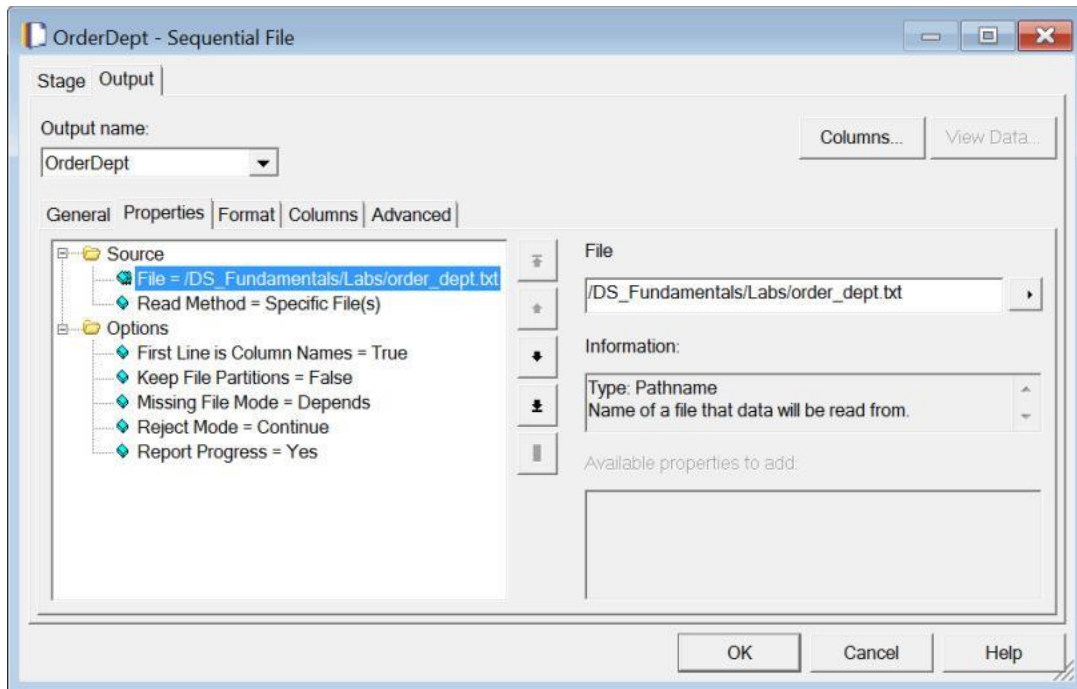
Quote Character: "

Data Preview

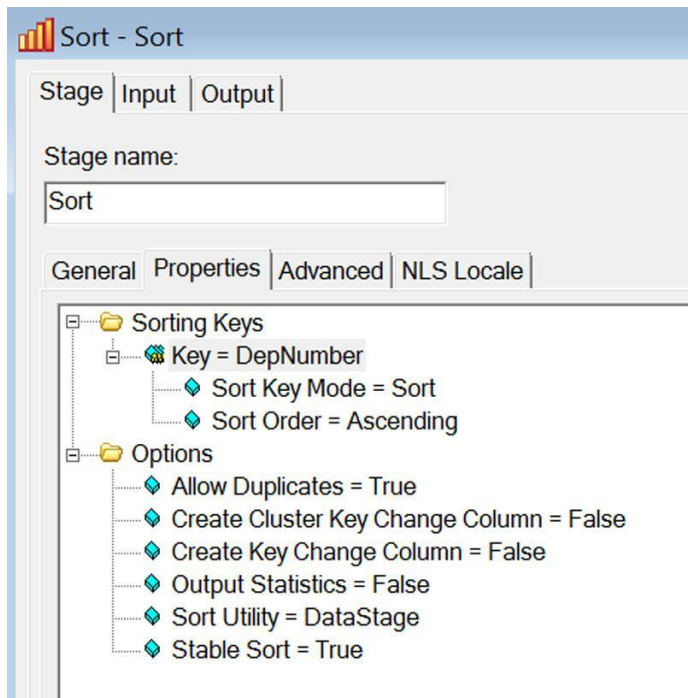
OrderID	DepNumber	Quantity	Price	TaxRate
Ord01	2	70	50	10
Ord02	1	100	6	5
Ord03	4	30	8	10
Ord04	1	250	10	10
Ord05	2	20	300	5
Ord06	2	40	35	10

Preview OK Cancel Help

5. Edit the source Sequential File stage to read file order_dept.txt using the imported table definition to define the Format and Column tabs. Also, set the option First Line is Column Names as True and the File properties.



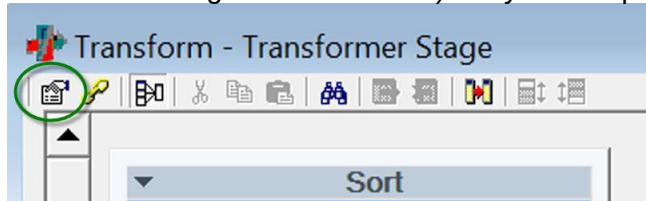
6. Configure the Sort stage specifying the DepNumber column as the Key with ascending order is necessary because the Transformer stage will process calculations using a key break detection mechanism based on the DepNumber column.



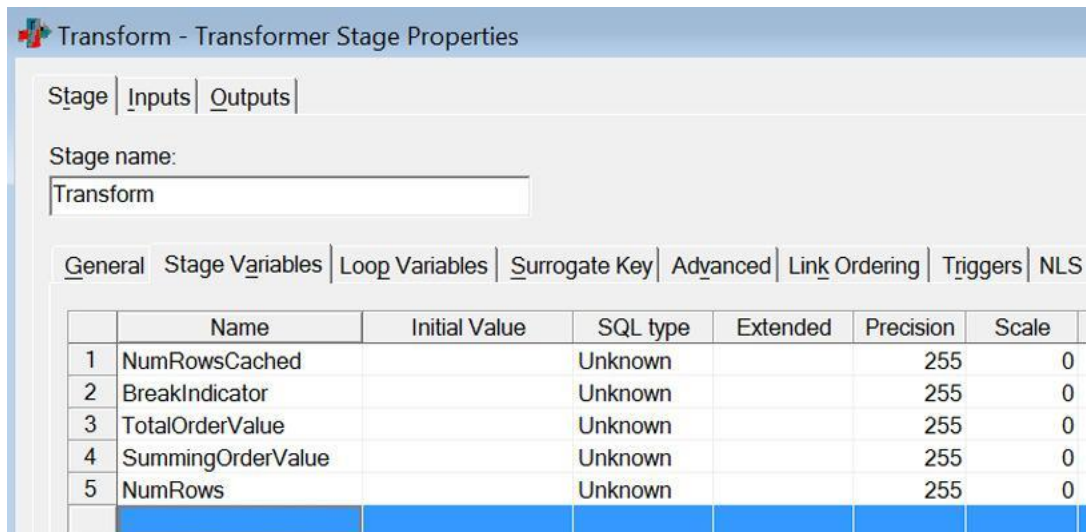
7. In the Output tab, propagate all the input columns to the output link.



8. Open the transformer stage editor and open the Stage Variable Properties (by right click on the Stage Variables area) or by the Properties icon from the task bar.



9. Define the Stage variables as shown:



10. Click OK.
11. Define the values for each stage variable as shown. We will need these variables to define both the loop variables and derivations.

Stage Variables	
Derivation	Stage Variable
SaveInputRecord()	NumRowsCached
LastRowInGroup(Sort.DepNumber)	BreakIndicator
IF BreakIndicator THEN SummingOrderValue + (Sort.Price * Sort.Quantity) ELSE 0	TotalOrderValue
IF BreakIndicator THEN 0 ELSE SummingOrderValue + (Sort.Price * Sort.Quantity)	SummingOrderValue
IF BreakIndicator THEN NumRowsCached ELSE 0	NumRows

12. Open the Loop Variable Properties (by right clicking on the Loop Variables area). Define the Loop Variables as shown:

Transform - Transformer Stage Properties

Stage | Inputs | Outputs

Stage name:
Transform

General | Stage Variables | **Loop Variables** | Surrogate Key | Advanced | Link Ordering | Triggers | NLS

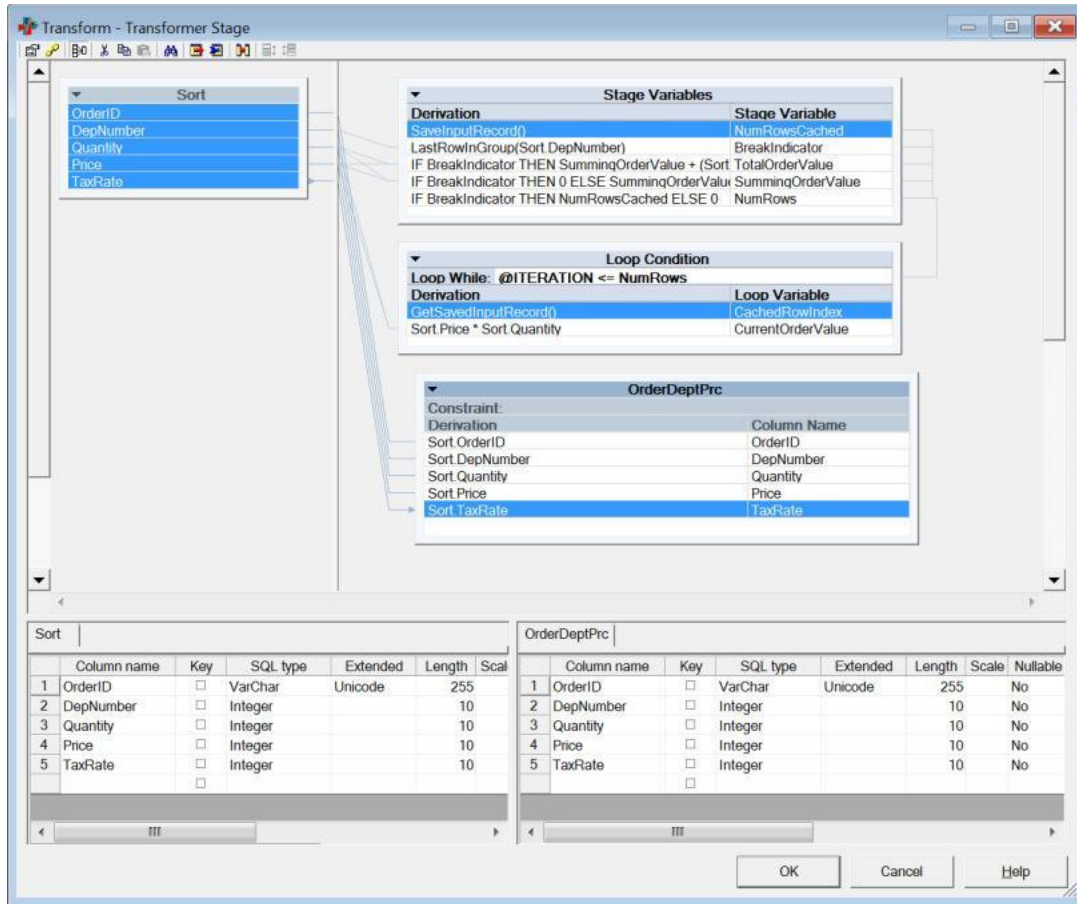
	Name	Initial Value	SQL type	Extended	Precision	Scale
1	CachedRowIndex		Unknown		255	0
2	CurrentOrderValue		VarChar		255	0

13. Define the loop condition and the derivations for both loop variables as shown:

Loop Condition	
Loop While: @ITERATION <= NumRows	
Derivation	Loop Variable
GetSavedInputRecord()	CachedRowIndex
Sort.Price * Sort.Quantity	CurrentOrderValue

Note: SaveInputRecord() saves the current input row in the cache, and returns the count of records currently in the cache. Each input row in a group of the same department is saved until the break value is reached. When the last row of the group is reached, NumRows is set to the number of rows stored in the input cache. The Loop Condition then loops through the records N times, where the number of times N is specified by NumRows. During each iteration of the loop, GetSavedInputRecord() is called to make the next saved input row current before re-processing each input row to create each output row. The usage of the inlink columns in the output link refers to their values in the currently retrieved input row so it will be updated on each output loop.

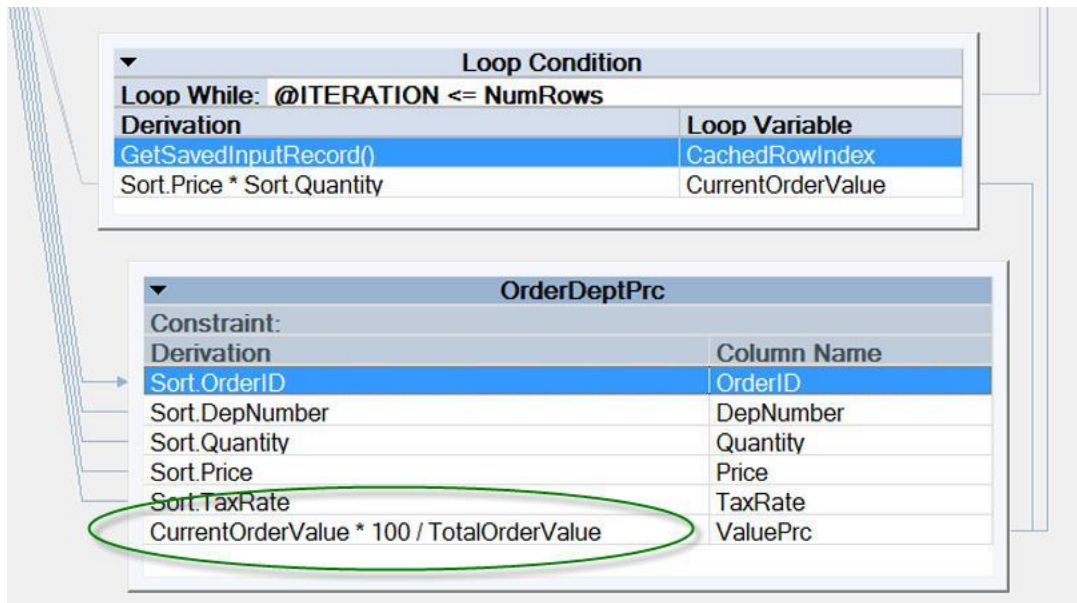
14. Drag and drop all the columns from the input link to the output link OrderDeptPrc.



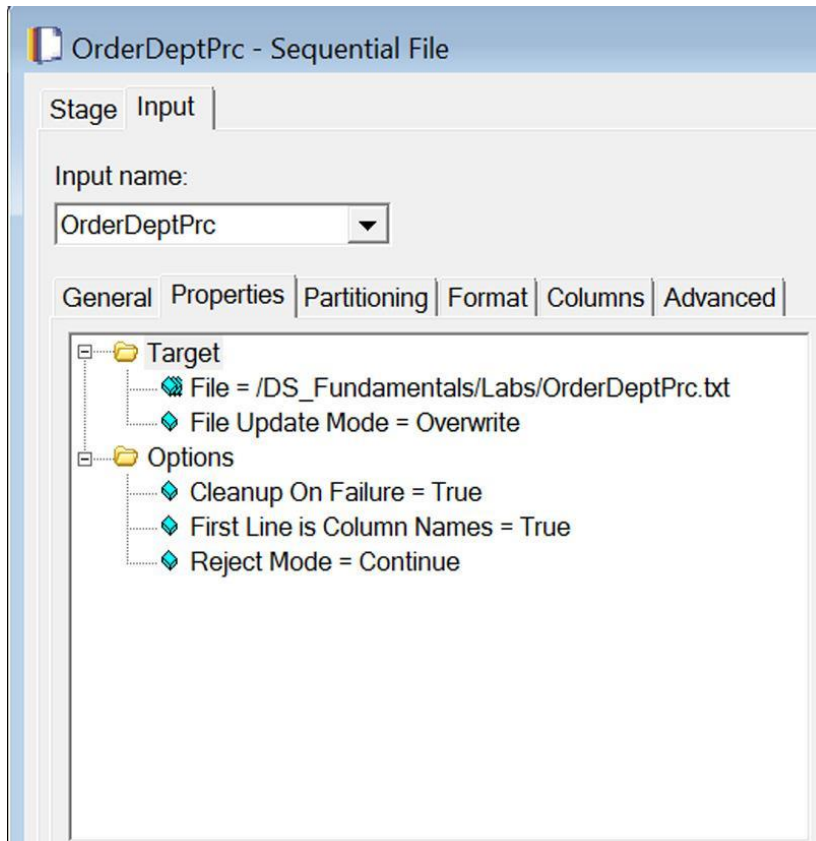
15. Create a new output column ValuePrc type numeric(5,2) in the output link metadata area.

OrderDeptPrc							
	Column name	Key	SQL type	Extended	Length	Scale	Nullable
1	OrderID	☐	VarChar	Unicode	255		No
2	DepNumber	☐	Integer		10		No
3	Quantity	☐	Integer		10		No
4	Price	☐	Integer		10		No
5	TaxRate	☐	Integer		10		No
6	ValuePrc	☐	Numeric		5	2	No

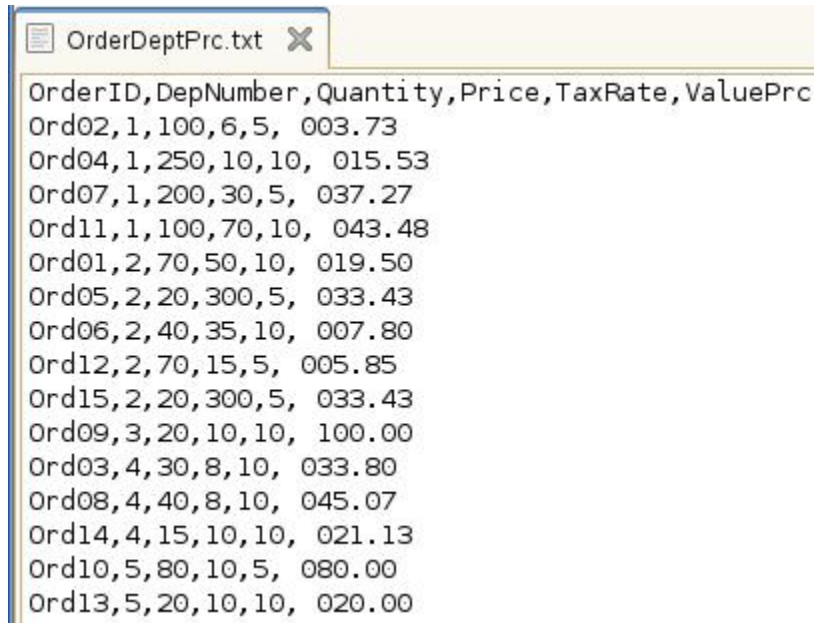
16. Define the derivation for the column as shown. Close the stage editor.



17. Configure the target Sequential File as shown.



18. Save, compile and run the job. Open and analyze the OrderDeptPrc.txt file and notice the ValuePrc values.



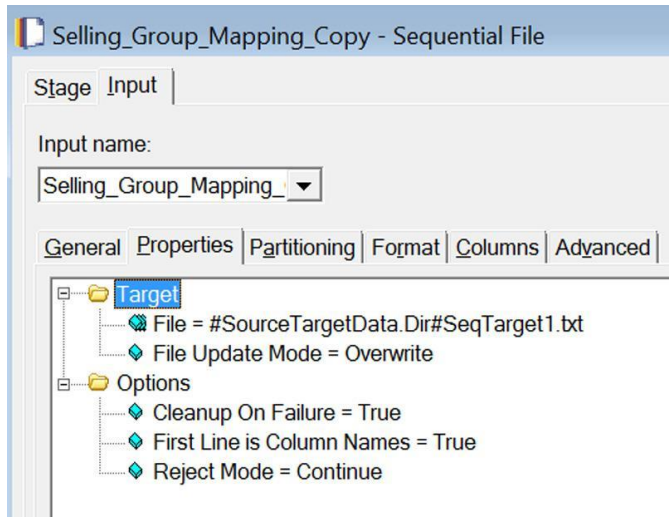
```
OrderID,DepNumber,Quantity,Price,TaxRate,ValuePrc
Ord02,1,100,6,5, 003.73
Ord04,1,250,10,10, 015.53
Ord07,1,200,30,5, 037.27
Ord11,1,100,70,10, 043.48
Ord01,2,70,50,10, 019.50
Ord05,2,20,300,5, 033.43
Ord06,2,40,35,10, 007.80
Ord12,2,70,15,5, 005.85
Ord15,2,20,300,5, 033.43
Ord09,3,20,10,10, 100.00
Ord03,4,30,8,10, 033.80
Ord08,4,40,8,10, 045.07
Ord14,4,15,10,10, 021.13
Ord10,5,80,10,5, 080.00
Ord13,5,20,10,10, 020.00
```

Lab 11: Job Control

In this lab, you will create a single job sequence that executes three jobs.

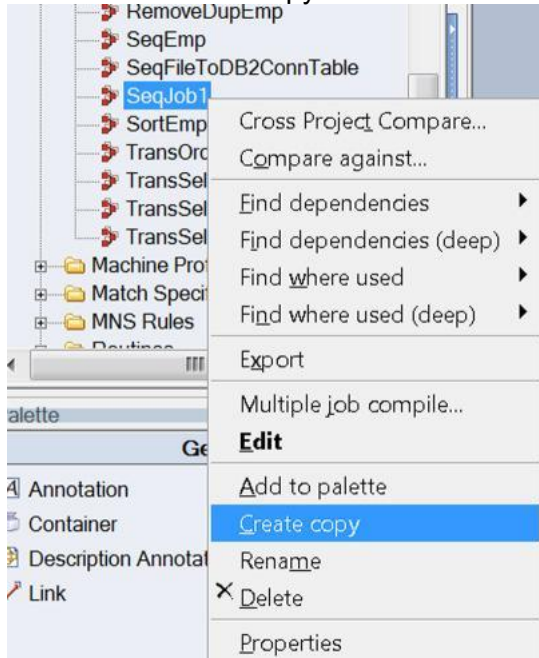
Task: Build a Job Sequence

1. Open the TransSellingGroup job, save it as SeqJob1 and edit the properties defining a target file named SeqTarget1.txt.



2. Specify Quote=none in the Format tab.
3. Save, compile and run SeqJob1. Make sure there are no errors and the target file contains the correct data.

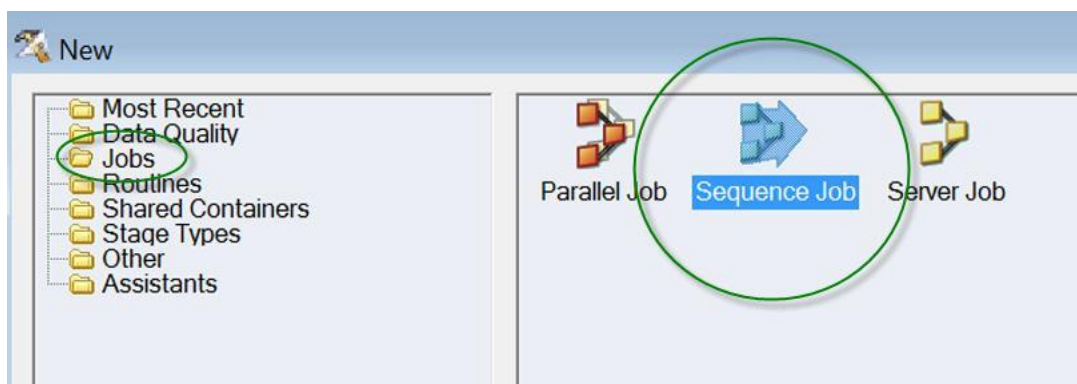
4. Create two copies of SeqJob1 called SeqJob2 and SeqJob3. Right click on the job and select “Create copy”



5. Configure the source and target sequential files to have following file from the same directory pointed by the job parameter SourceTargetData.Dir:

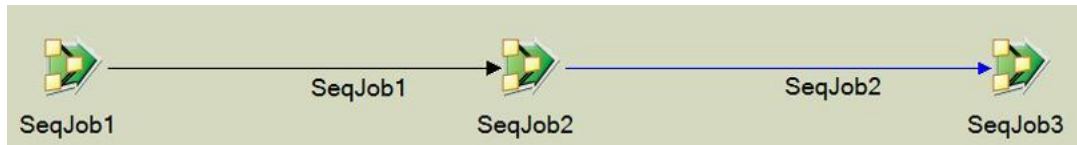
JOB	SOURCE FILE	TARGET FILE
SeqJob2	SeqTarget1.txt	SeqTarget2.txt
SeqJob3	SeqTarget2.txt	SeqTarget3.txt

6. Compile and run SeqJob2 and SeqJob3 to verify that all three target files have been created in the /DS_fundamentals/Labs folder.
7. In DataStage Designer, select “File” on the menu, and “New” on the popup window, then open the Jobs folder and select “Sequence Job” to create a new Job Sequence.

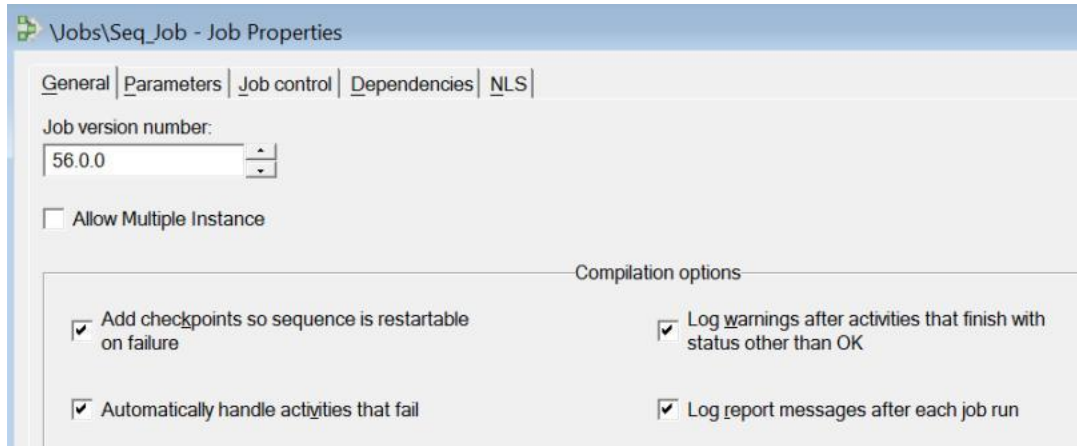


8. Save it as Seq_Job in the Jobs folder.

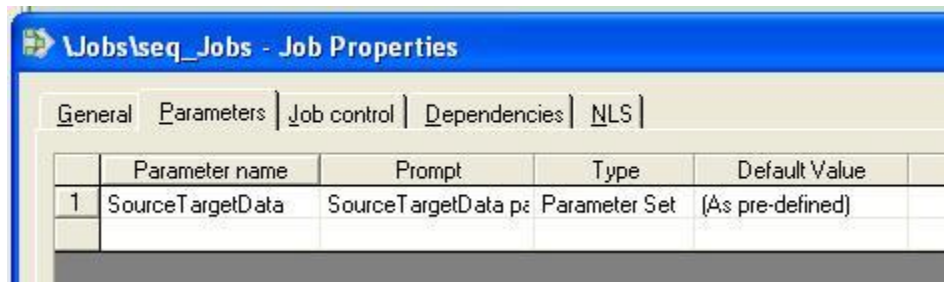
9. Drag and drop three Job Activity stages to the canvas, link them, and name the stages and links as shown.



10. Open the Job (Sequence) Properties. In the General tab, verify that all the compilation options are selected.



11. Click the Parameters tab and add the parameter set SourceTargetData as shown. These parameters will be available to all the stages within the job sequence during execution.



12. Select OK.
13. Open up each of the Job Activity stages and set the parallel job you want to be executed by each stage. That is, use SeqJob1 job for the SeqJob1 Activity, SeqJob2 for the SeqJob2 and so on. Also insert the parameter values for the corresponding job parameters in each Job Activity stage as shown. This way the Job Activity stages will use the values passed by the Job Sequence at runtime.

seqJob1 - Job Activity

General Job Triggers

Job name: SeqJob1

Invocation Id:

Execution action: Reset if required, then run

☐ Do not checkpoint run.

Parameters

Name	Value Expression
SourceTar...	"SourceTargetFile"

Type: Parameter Set
Prompt: SourceTargetData parameters

Insert Parameter
Clear
Clear All
Set to Default
All to Default

OK Cancel Help

14. For Job Activity stage SeqJob2 and SeqJob3, we want them to be executed only when the upstream job ran without any error, although possibly with warnings.

Note: This means that the DSJ.JOBSTATUS can be either DSJS.RUNOK or DSJS.RUNWARN. You can browse the Activity Variables and the DS Constant in the expression editor to compose the triggers. The result in the case of SeqJob1 (similarly for SeqJob2) should look like:

Name	Expression Type	Expression
SeqJob1	Custom - (Conditional)	SeqJob1.\$JobStatus = DSJS.RUNOK or SeqJob1.\$JobStatus = DSJS.RUNWARN

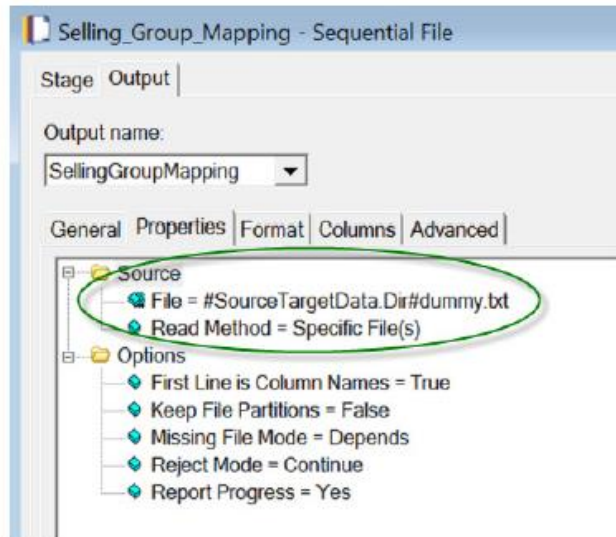
15. Compile and run your job sequence.
16. Open the job log for the job sequence. Verify that each job ran successfully. Locate and examine the job sequence summary.

```

2/23/2012 3:56:09 PM Seq_Job..JobControl (@Coordinator): Starting new run of checkpointed Sequence job
2/23/2012 3:56:10 PM Seq_Job -> (SeqJob1): Job run requested ...
2/23/2012 3:56:10 PM Seq_Job..JobControl (DSRunJob): Waiting for job SeqJob1 to start
2/23/2012 3:56:11 PM Seq_Job..JobControl (DSWaitForJob): Waiting for job SeqJob1 to finish
2/23/2012 3:56:19 PM Seq_Job..JobControl (DSWaitForJob): Job SeqJob1 has finished, status = 1 (Finished OK)
2/23/2012 3:56:19 PM Seq_Job..JobControl (@SeqJob1): Report on job: SeqJob1
2/23/2012 3:56:19 PM Seq_Job..JobControl (@SeqJob1): Checkpointed run of job 'SeqJob1'
2/23/2012 3:56:19 PM Seq_Job -> (SeqJob2): Job run requested ...
2/23/2012 3:56:19 PM Seq_Job..JobControl (DSRunJob): Waiting for job SeqJob2 to start
2/23/2012 3:56:20 PM Seq_Job..JobControl (DSWaitForJob): Waiting for job SeqJob2 to finish
2/23/2012 3:56:27 PM Seq_Job..JobControl (DSWaitForJob): Job SeqJob2 has finished, status = 1 (Finished OK)
2/23/2012 3:56:27 PM Seq_Job..JobControl (@SeqJob2): Report on job: SeqJob2 ...
2/23/2012 3:56:27 PM Seq_Job..JobControl (@SeqJob2): Checkpointed run of job 'SeqJob2'
2/23/2012 3:56:27 PM Seq_Job -> (SeqJob3): Job run requested ...
2/23/2012 3:56:27 PM Seq_Job..JobControl (DSRunJob): Waiting for job SeqJob3 to start
2/23/2012 3:56:29 PM Seq_Job..JobControl (DSWaitForJob): Waiting for job SeqJob3 to finish
2/23/2012 3:56:36 PM Seq_Job..JobControl (DSWaitForJob): Job SeqJob3 has finished, status = 1 (Finished OK)
2/23/2012 3:56:36 PM Seq_Job..JobControl (@SeqJob3): Report on job: SeqJob3 ...
2/23/2012 3:56:36 PM Seq_Job..JobControl (@SeqJob3): Checkpointed run of job 'SeqJob3'
2/23/2012 3:56:36 PM Seq_Job..JobControl (@Coordinator): Removed checkpoint record at successful completion of sequence
2/23/2012 3:56:36 PM Seq_Job..JobControl (@Coordinator): Summary of sequence run ...
2/23/2012 3:56:36 PM Finished Job Seq_Job.
2/23/2012 3:56:37 PM Project-level auto-purge, 25 log entries removed

```

17. Examine what happens if the second job aborts. To cause that, open up the job SeqJob2 and replace in the source Sequential File name SeqTarget1.txt with the non-existent dummy.txt as shown below. Save and compile SeqJob2.



18. Execute the job sequence Seq_Job and check the log showing the job is aborted. The first error message in the job log should contain the relevant error.

Note: you don't need to recompile the job sequence to execute it since nothing was changed in the job sequence.

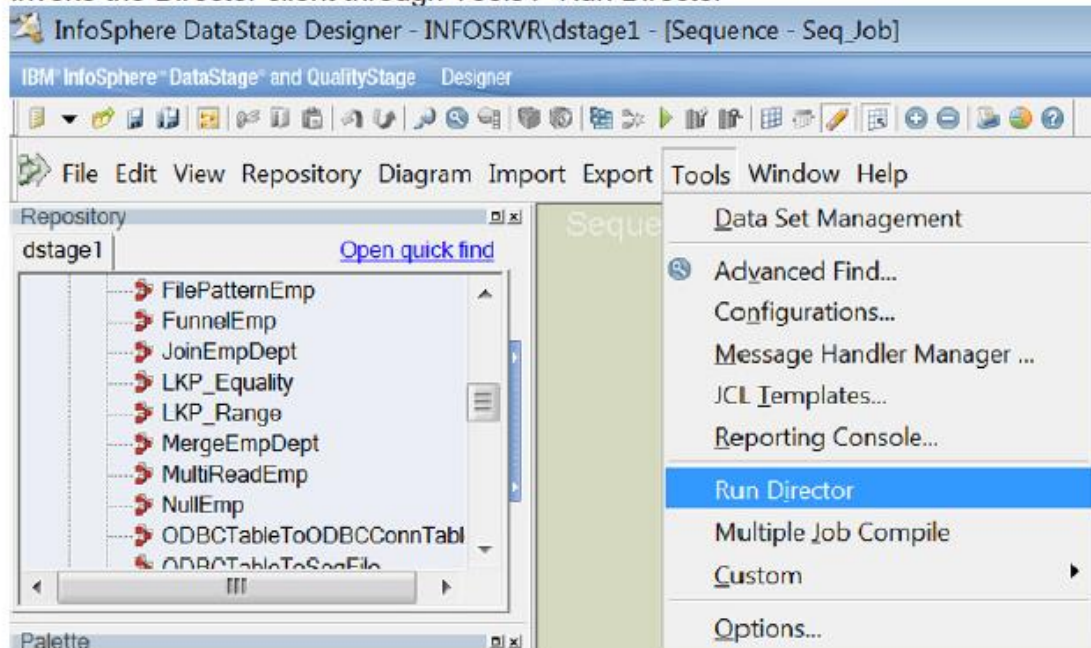
```

2/23/2012 3:59:39 PM Seq_Job -> (SeqJob2): Job run requested ...
2/23/2012 3:59:39 PM Seq_Job.JobControl (DSRunJob): Waiting for job SeqJob2 to start
2/23/2012 3:59:40 PM Seq_Job.JobControl (DSWaitForJob): Waiting for job SeqJob2 to finish
2/23/2012 3:59:52 PM Seq_Job.JobControl (DSWaitForJob): Job SeqJob2 has finished, status = 3 (Aborted)
2/23/2012 3:59:52 PM Seq_Job.JobControl (@SeqJob2): Job SeqJob2 did not finish OK, status = 'Aborted'
2/23/2012 3:59:52 PM Seq_Job.JobControl (@SeqJob2): Report on job: SeqJob2 ...
2/23/2012 3:59:52 PM Seq_Job.JobControl (@SeqJob2): Controller problem: Unhandled abort encountered in job SeqJob2
2/23/2012 3:59:52 PM Seq_Job.JobControl (@Coordinator): Summary of sequence run ...
2/23/2012 3:59:52 PM Seq_Job.JobControl (fatal error from @Coordinator): Sequence job (restartable) will abort due to previous unrecoverable errors
2/23/2012 3:59:52 PM Attempting to Cleanup after ABORT raised in job Seq_Job.JobControl
2/23/2012 3:59:52 PM Job Seq_Job aborted.

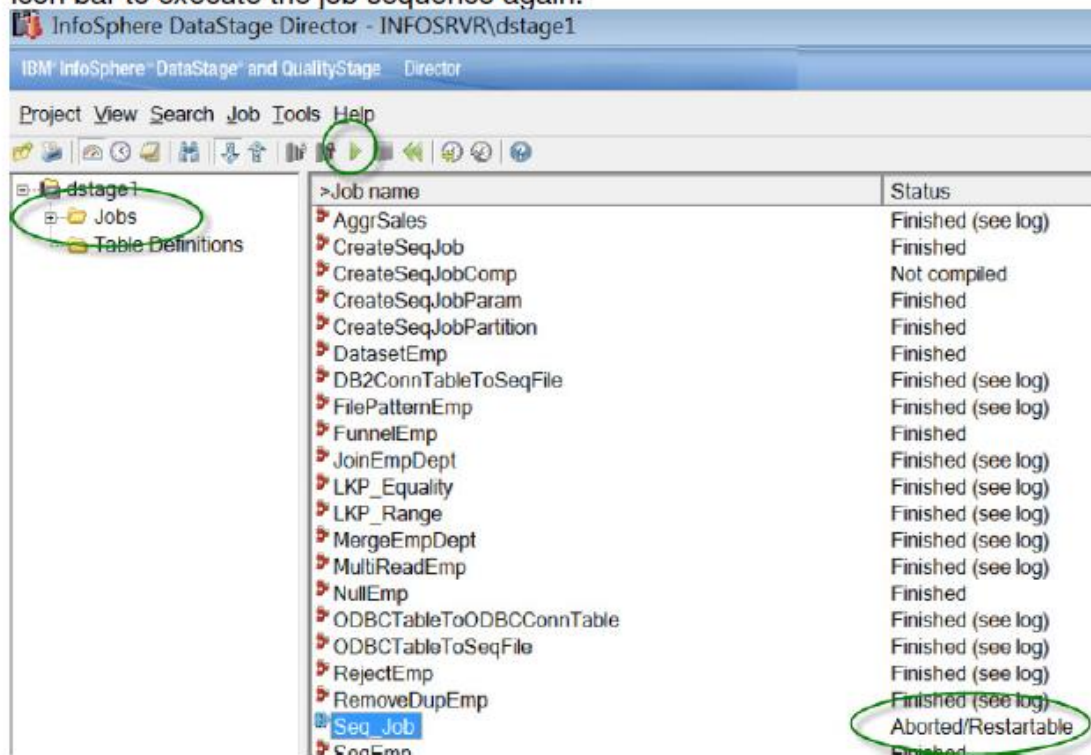
```

19. Open the job SeqJob2 replacing the dummy.txt source file with the original SeqTarget1.txt in the source sequential file name. Then save and compile the job.

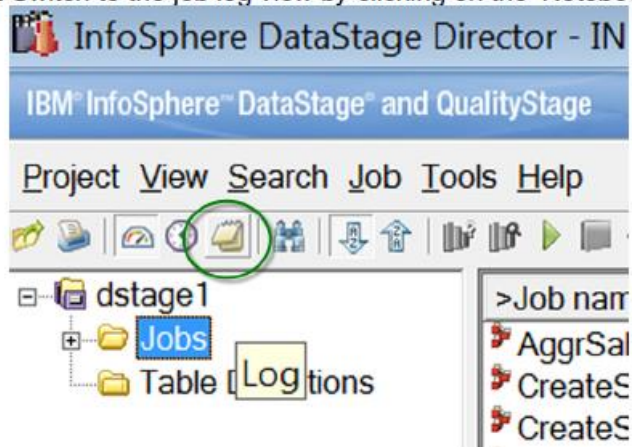
20. Restart the job using the DataStage and QualityStage Director client. The Director is the client component that validates, runs, schedules, and monitors jobs. You can invoke the Director client through Tools > Run Director



21. Switch into the Jobs folder, highlight Seq_Job and click on 'run now..' in the shortcut icon bar to execute the job sequence again.



22. Switch to the job log view by clicking on the 'Notebook' icon.

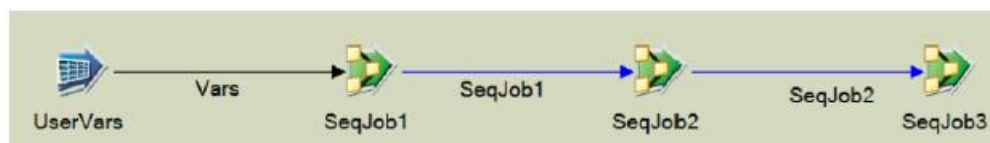


23. Notice that SeqJob1 is not executed because it ran successfully during the previous execution. This behavior is possible because the Job Sequence property "Add checkpoints so sequence is restartable on failure" is enabled.

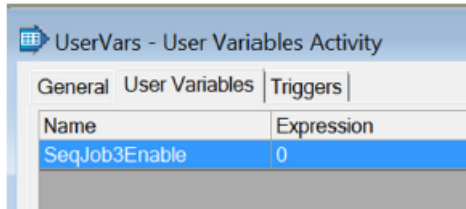
4:21:06 PM	2/23/2012	Control	Starting Job Seq_Job ()
4:21:06 PM	2/23/2012	Info	Environment variable settings: (...)
4:21:06 PM	2/23/2012	Info	Seq_Job: Set NLS locale to US-ENGLISH,US-ENGLISH,US-ENGLISH,US-ENGLISH,US-ENGLISH
4:21:06 PM	2/23/2012	Info	Seq_Job.JobControl (@Coordinator): Sequence job is being restarted after failure (...)
4:21:06 PM	2/23/2012	Info	Seq_Job.JobControl (@SeqJob1): Skipped run of job 'SeqJob1' on restart
4:21:06 PM	2/23/2012	RunJob	Seq_Job -> (SeqJob2) Job run requested (...)
4:21:06 PM	2/23/2012	Info	Seq_Job.JobControl (DSRunJob): Waiting for job SeqJob2 to start
4:21:07 PM	2/23/2012	Info	Seq_Job.JobControl (DSWaitForJob): Waiting for job SeqJob2 to finish
4:21:19 PM	2/23/2012	Info	Seq_Job.JobControl (DSWaitForJob): Job SeqJob2 has finished, status = 1 (Finished OK)
4:21:19 PM	2/23/2012	Info	Seq_Job.JobControl (@SeqJob2): Report on job: SeqJob2 (...)
4:21:19 PM	2/23/2012	Info	Seq_Job.JobControl (@SeqJob2): Checkpointed run of job 'SeqJob2'
4:21:19 PM	2/23/2012	RunJob	Seq_Job -> (SeqJob3) Job run requested (...)
4:21:19 PM	2/23/2012	Info	Seq_Job.JobControl (DSRunJob): Waiting for job SeqJob3 to start
4:21:20 PM	2/23/2012	Info	Seq_Job.JobControl (DSWaitForJob): Waiting for job SeqJob3 to finish
4:21:26 PM	2/23/2012	Info	Seq_Job.JobControl (DSWaitForJob): Job SeqJob3 has finished, status = 1 (Finished OK)
4:21:26 PM	2/23/2012	Info	Seq_Job.JobControl (@SeqJob3): Report on job: SeqJob3 (...)
4:21:26 PM	2/23/2012	Info	Seq_Job.JobControl (@SeqJob3): Checkpointed run of job 'SeqJob3'
4:21:26 PM	2/23/2012	Info	Seq_Job.JobControl (@Coordinator): Removed checkpoint record at successful completion of sequence
4:21:26 PM	2/23/2012	Info	Seq_Job.JobControl (@Coordinator): Summary of sequence run (...)
4:21:26 PM	2/23/2012	Control	Finished Job Seq_Job
4:21:26 PM	2/23/2012	Reset	Project-level auto-purge, 43 log entries removed

Task: Add a user variable

1. Save the job sequence Seq_Job as Seq_Job_var. Add a User Variable Activity stage as shown.



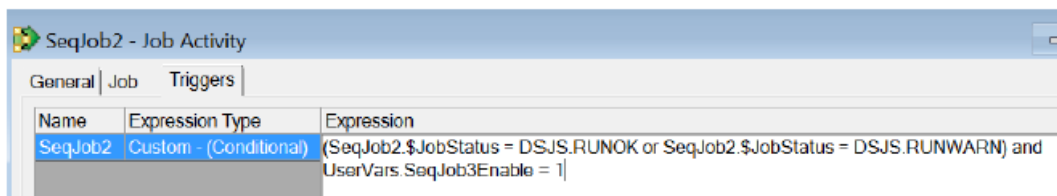
- Open the User Variables Activity stage and select the User Variables tab. Create a variable named SeqJob3Enable with value 0. Right click your mouse and select Add Row to create.



- We want to enable the execution of SeqJob3 only if the value of the SeqJob3Enable variable is 1. To specify this condition open the Trigger tab in the SeqJob2 Job Activity stage and modify the expression as shown.

Note: you can refer to the User Variable Activity stage variables within any stage in the job sequence using the syntax:

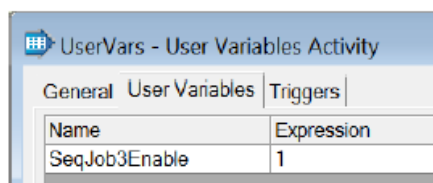
UserVariableActivityName.UservariableName



- Compile and run the job sequence Seq_Job_var. You should notice that SeqJob3 has not been executed because UserVars.SeqJob3Enable value is 0.

Type	Occurred	Event
	2/23/2012 6:21:36 PM	Seq_Job_var.JobControl (@SeqJob1): Report on job: SeqJob1
	2/23/2012 6:21:36 PM	Seq_Job_var.JobControl (@SeqJob1): Checkpointed run of job 'SeqJob1'
	2/23/2012 6:21:36 PM	Seq_Job_var -> (SeqJob2): Job run requested ...
	2/23/2012 6:21:36 PM	Seq_Job_var.JobControl (DSRunJob): Waiting for job SeqJob2 to start
	2/23/2012 6:21:38 PM	Seq_Job_var.JobControl (DSWaitForJob): Waiting for job SeqJob2 to finish
	2/23/2012 6:21:44 PM	Seq_Job_var.JobControl (DSWaitForJob): Job SeqJob2 has finished, status = 1 (Finished OK)
	2/23/2012 6:21:44 PM	Seq_Job_var.JobControl (@SeqJob2): Report on job: SeqJob2
	2/23/2012 6:21:44 PM	Seq_Job_var.JobControl (@SeqJob2): Checkpointed run of job 'SeqJob2'
	2/23/2012 6:21:44 PM	Seq_Job_var.JobControl (@Coordinator): Removed checkpoint record at successful completion of sequence
	2/23/2012 6:21:44 PM	Seq_Job_var.JobControl (@Coordinator): Summary of sequence run ...
	2/23/2012 6:21:45 PM	Finished Job Seq_Job_var.

- Edit the UserVars stage and change the SeqJob3Enable value to 1. This will cause SeqJob3 to execute.

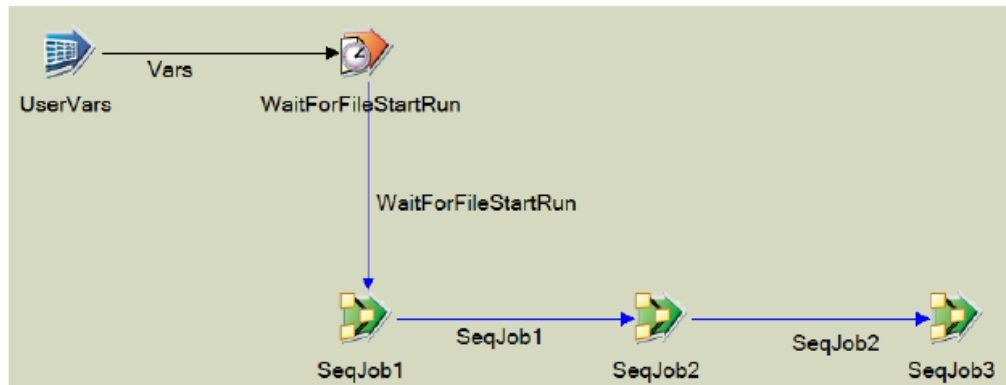


6. Compile and run the job sequence again and verify in the logs that SeqJob3 was executed.

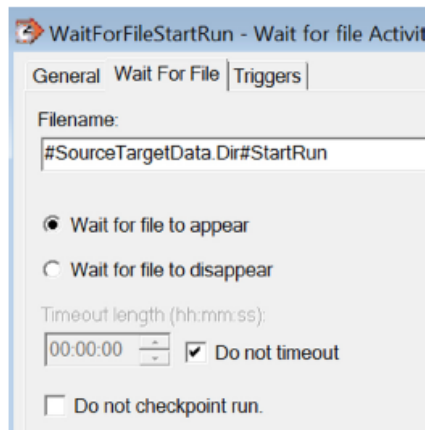
Task: Add a Wait For File stage

In this task, you will modify your design so that your job is not executed until a file called StartRun appears in directory /DS_Fundamentals/Labs.

1. Save your job from the last lab as Seq_Job_wait.
2. Add Wait for File stage as shown.



3. Open the Wait For File stage and set the filename of the file as shown below.



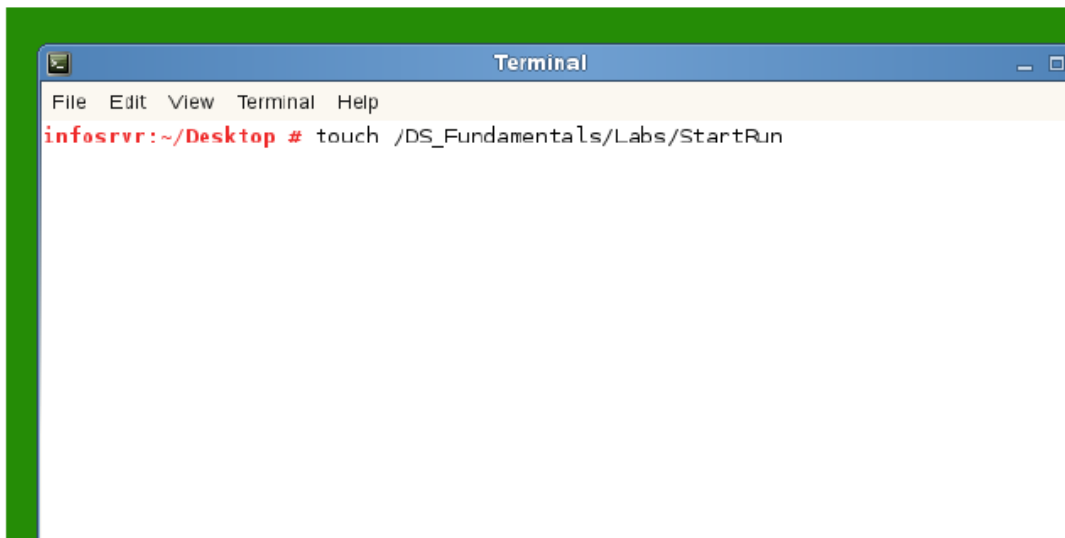
Note: the "Do not timeout" option makes the stage wait forever for the file StartRun until it appears in the specified location.

4. Define an unconditional trigger so the following Activity, SeqJob1, will be started as soon as the file StartRun appears in directory /DS_Fundamentals/Labs.

5. Compile and run your job. Notice that after the job starts it waits for the file StartRun to appear in the expected folder.

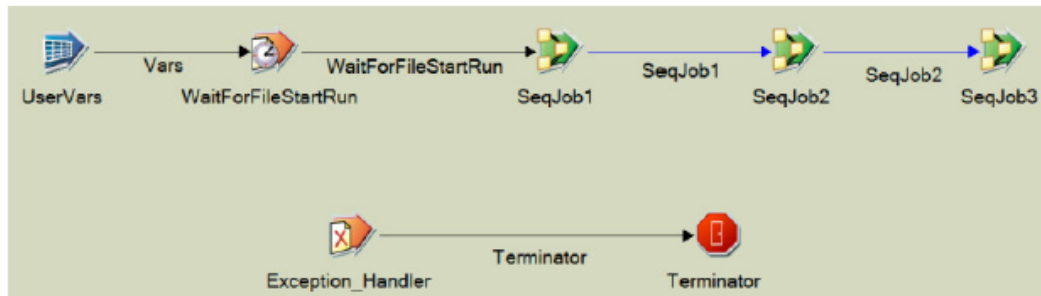
```
2/23/2012 6:29:57 PM Seq_Job_waitJobControl (@Coordinator): Starting new run of checkpointed Sequence job
2/23/2012 6:29:57 PM Seq_Job_waitJobControl (DSWaitForFile): Will wait until (forever, or job stopped) for file '/DS_Fundamentals/Labs/StartRun' to be present
```

6. Create a file named StartRun in the directory /DS_Fundamentals/Labs. You can use the command “touch StartRun” for this purpose. Notice the log messages and the job sequence execution should now continue by running the stage following the Wait For File Activity.

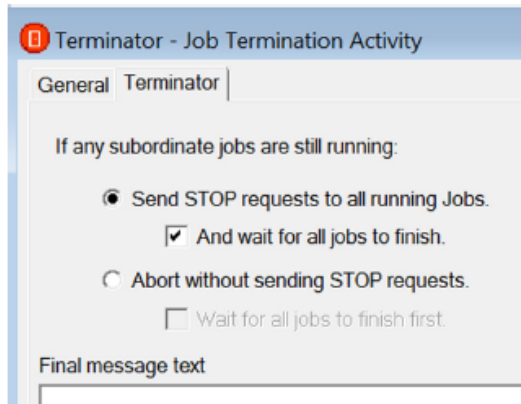


Task: Add exception handling

1. Save your job from the previous task as Seq_Job_exception.
2. Add the Exception Handler and Terminator stages as shown.



3. Edit the Terminator stage so that any running job is stopped when an exception occurs.



4. To see how the exception handling takes over control over the test the job sequence, you will have to make one of the jobs that are part of the Job Sequence fail. Modify the job SeqJob2 replacing the file SeqTarget1.txt with dummy.txt in the source Sequential File and compile the job. Run the job sequence again and check the log with the Director client. Note that as SeqJob2 did not finish successfully, the sequence is aborted.

```

2/23/2012 ... Seq_Job_exception.JobControl (@SeqJob1): Report on job: SeqJob1 ...
2/23/2012 ... Seq_Job_exception.JobControl (@SeqJob1): Checkpointed run of job 'SeqJob1'
2/23/2012 ... Seq_Job_exception -> (SeqJob2): Job run requested ...
2/23/2012 ... Seq_Job_exception.JobControl (DSRunJob): Waiting for job SeqJob2 to start
2/23/2012 ... Seq_Job_exception.JobControl (DSWaitForJob): Waiting for job SeqJob2 to finish
2/23/2012 ... Seq_Job_exception.JobControl (DSWaitForJob): Job SeqJob2 has finished, status = 3 (Aborted)
2/23/2012 ... Seq_Job_exception.JobControl (@SeqJob2): Job SeqJob2 did not finish OK, status = 'Aborted'
2/23/2012 ... Seq_Job_exception.JobControl (@SeqJob2): Report on job: SeqJob2 ...
2/23/2012 ... Seq_Job_exception.JobControl (@SeqJob2): Will execute error activity: Exception_Handler
2/23/2012 ... Seq_Job_exception.JobControl (@Coordinator): Summary of sequence run ...
2/23/2012 ... Seq_Job_exception.JobControl (fatal error from @Terminator): Sequence abort requested
2/23/2012 ... Attempting to Cleanup after ABORT raised in job Seq_Job_exception.JobControl
2/23/2012 ... Job Seq_Job_exception aborted.

```